

A Mermaid-Superset Diagram Compiler

Beautiful, Deterministic, Agent-Authorable
Architecture & Specification

Timeline Compiler Project

June 2026

This document is dogfooded. Every diagram in this specification is rendered by the compiler the document describes. Diagram sources live in `design/figures/src/` as `.mmd` files authored in our Mermaid-superset DSL; running `make figures` from the `design/` directory invokes the CLI (`timeline render <name>.mmd -format png -scale 3`) to produce high-DPI PNGs that are then included via `\ourdiagram`. This reusable recipe works for any $\text{\LaTeX}$ document: author diagrams in our DSL, render to PNG, include with `\includegraphics`. Vector PDF output is a future enhancement (see §0.11).

Contents

I	Thesis & Positioning	22
0.1	Problem Definition	23
0.1.1	The Communication Gap	23
0.1.2	Why Mermaid Is Necessary But Insufficient	23
0.1.2.1	Mermaid’s Strengths (Why We Build On It)	23
0.1.2.2	Mermaid’s Limitations (Why We Exist)	24
0.1.2.3	The Agent Blindspot	24
0.1.3	Who This Is For	24
0.1.3.1	Human Authors (Mermaid-Superset DSL)	24
0.1.3.2	AI Agents (Structured IR)	25
0.1.4	What Success Looks Like	25
0.1.4.1	Scenario: Drop-in Mermaid Upgrade	25
0.1.4.2	Scenario: Agent-Generated Architecture Diagram	25
0.1.4.3	Scenario: Multi-Diagram Poster	25
0.1.4.4	Success Criteria	25
0.1.5	What This Is Not	26
0.2	Central Thesis: A Mermaid-Superset Diagram Compiler	26
0.2.1	The Positioning	26
0.2.2	The Engine-as-Asset Insight	26
0.2.3	The Pipeline Model	27
0.2.4	Why “Diagram Compiler,” Not “Diagram Tool”	27
0.2.5	The Five Diagram Families	28
0.2.6	The Kernel/Grammar/Composition Architecture	28
0.2.7	Summary of the Strategic Positioning	28
0.3	Design Principles	29

0.3.1	Presentation-First	29
0.3.2	Deterministic Output	29
0.3.3	Human-Readable Source	30
0.3.4	Agent-Friendly Authoring	30
0.3.5	Theme-Driven Rendering	31
0.3.6	Separation of Planning and Rendering	31
0.3.7	Minimal Visual Clutter	31
0.3.8	Source-Control Friendly	32
0.3.9	Composability	32
0.3.10	Graceful Degradation	32
0.3.11	Extensibility Without Complexity	33
0.3.12	Principle Summary	33
0.4	Scope Boundaries	34
0.4.1	Governing Principle	34
0.4.2	In Scope	34
0.4.2.1	Diagram Coverage	34
0.4.2.2	Input Paths	34
0.4.2.3	Output Formats	34
0.4.2.4	Key Capabilities	34
0.4.3	Out of Scope	35
0.4.4	Scope Summary	35
0.5	Positioning: Why a Mermaid Superset	35
0.5.1	The Competitive Landscape	35
0.5.2	Why Full Mermaid Compatibility	35
0.5.3	Differentiation Pillars	36
0.5.4	vs. PlantUML	37
0.5.5	vs. D2	37
0.5.6	vs. Vega-Lite	37

II	The Front-End	38
0.6	Dual Front-End Architecture	39
0.6.1	The Two Input Paths	39
0.6.2	Convergence at Domain IR	39
0.6.3	The Agent IR as First-Class API	40
0.6.4	Parser Architecture	40
0.6.5	Implementation Status	40
0.7	Full Mermaid Compatibility	41
0.7.1	The 22 Diagram Types	41
0.7.2	Compatibility Strategy	41
0.7.2.1	Syntax Fidelity	41
0.7.2.2	Semantic Mapping	42
0.7.2.3	Directive and Frontmatter Hooks	42
0.7.3	Reuse of Existing Kernels	42
0.8	Extended Timeline Syntax	43
0.8.1	Two-Tier Portability Model	43
0.8.2	Frontmatter Configuration	43
0.8.3	Grammar Sketch	44
0.8.4	Construct-to-IR Mapping	46
0.8.5	One IR, Many Layouts	47
0.8.6	Degradation and Portability Contract	48
0.8.7	Worked Examples	48
0.8.7.1	Full Extended Timeline	48
0.8.7.2	Same Data, Multiple Layouts	49
0.8.8	Known IR Gaps and Deferred Questions	50
0.9	Superset Extensions Beyond Mermaid	51
0.9.1	Extension Mechanisms	51
0.9.2	Composition / Posters	51
0.9.3	Rich Theming	52
0.9.4	Animation	52

0.9.5	Cross-Diagram References	53
0.9.6	Icons and Images in Nodes	53
0.9.7	The Structured IR as Agent API	53
0.9.8	Graceful Degradation	53
III	The Kernel	55
0.10	Scene IR: The Universal Render Contract	56
0.10.1	Design Rationale	56
0.10.2	Scene Model	56
0.10.2.1	Canvas Descriptor	56
0.10.2.2	Drawing Primitives	57
0.10.2.3	Effect Definitions	58
0.10.3	Animation Hints	58
0.10.4	Scene Determinism Contract	59
0.10.5	Scene Serialization	60
0.10.6	Scene IR as the Integration Point	60
0.11	Rendering Backends: SVG as Source of Truth	60
0.11.1	The Backend Architecture	60
0.11.2	SVG as Canonical Source of Truth	61
0.11.3	The Skia Backend: Art Effects	61
0.11.4	The PPTX Backend: Native Shapes	62
0.11.5	Rejecting HTML/CSS-First Architecture	62
0.11.6	Backend Selection Logic	63
0.11.7	Fidelity Tiers	63
0.11.8	Determinism Verification	63
0.12	Determinism Contract	64
0.12.1	The Fundamental Guarantee	64
0.12.2	Why Determinism Matters	64
0.12.3	Sources of Non-Determinism (and How They're Eliminated)	65
0.12.3.1	System Clock	65

0.12.3.2	Random Number Generators	65
0.12.3.3	Floating-Point Rounding	65
0.12.3.4	Collection Ordering	65
0.12.3.5	Font Metrics	66
0.12.3.6	Environment Variables and Locale	66
0.12.3.7	File System State	66
0.12.4	The Determinism Verification Protocol	66
0.12.4.1	Scene-Level Verification	66
0.12.4.2	SVG-Level Verification	67
0.12.4.3	Raster-Level Verification	67
0.12.4.4	Cross-Platform Verification	67
0.12.5	Determinism Across Backend Versions	67
0.12.6	The Exception: Cross-Backend Differences	67
0.13	Animation: Additive and Backend-Conditional	68
0.13.1	Design Philosophy	68
0.13.2	Animation Patterns from the Corpus	68
0.13.3	Animation Hint Types	69
0.13.3.1	FlowingDashes	69
0.13.3.2	DrawOn	69
0.13.3.3	DotAlongPath	70
0.13.3.4	Pulse	70
0.13.3.5	FadeIn	70
0.13.4	Theme-Level Animation Defaults	70
0.13.5	Backend Behavior	71
0.13.6	Scope Discipline: What's Out	71
IV	The Diagram Families	72
0.14	The Grammar Concept: Two-IR-Layer Model	73
0.14.1	What Is a Grammar?	73
0.14.2	Grounding in the Grammar of Graphics	73

0.14.3	Agent-Authoring Reliability: Constrained DSL	74
0.14.4	The Two-IR-Layer Model	74
0.14.5	Rejecting the God-IR Approach	75
0.14.6	Composition IR: Above Domain IRs	76
0.14.7	Grammar Interface Contract	77
0.14.8	Current and Planned Grammars	77
0.14.9	The Compounding Payoff	78
0.15	The Five Diagram Families	78
0.15.1	Family Overview	78
0.15.2	Family 1: Node-Link / Graph	78
0.15.3	Family 2: UML / Software (Tier-1 Priority)	80
0.15.4	Family 3: Charts (Data $\rightarrow$ Geometry)	80
0.15.5	Family 4: Timeline / Project	81
0.15.6	Family 5: Tree / Hierarchy	81
0.15.7	Coverage Roadmap Tiers	82
0.16	Timeline Grammar: Domain IR	82
0.16.1	Design Principles	82
0.16.2	Type Vocabulary	83
0.16.3	Document Structure	83
0.16.3.1	Root Fields	83
0.16.4	Metadata	83
0.16.4.1	TimeRange	83
0.16.4.2	AxisUnit	83
0.16.4.3	Date Anchor and Fiscal Calendar	84
0.16.5	Date Model	84
0.16.5.1	Date Representations	85
0.16.5.2	Uncertain and Open Dates	85
0.16.5.3	DateRange	85
0.16.6	Tracks (Swimlanes)	86
0.16.7	Groups	86

0.16.8 Activities	86
0.16.8.1 Status Enum	87
0.16.8.2 Progress	87
0.16.9 Milestones	87
0.16.10 Annotations	88
0.16.10.1 Annotation Types	88
0.16.11 Sections	89
0.16.12 Legend	89
0.16.13 ID and Reference System	89
0.16.13.1 Identifier Format	89
0.16.13.2 References	90
0.16.13.3 Reference Namespacing	90
0.16.14 Well-Formedness Rules	90
0.16.14.1 Structural Invariants	91
0.16.14.2 Referential Invariants	91
0.16.14.3 Temporal Invariants	91
0.16.14.4 Value Invariants	91
0.16.14.5 Soft Warnings (Valid but Suspicious)	91
0.16.15 Complete Examples	92
0.16.15.1 Example 1: Product Roadmap	92
0.16.15.2 Example 2: Executive Program Timeline	93
0.16.15.3 Example 3: Architecture Evolution Timeline	96
0.16.16 Extensibility	98
0.16.16.1 Metadata Extension	98
0.16.16.2 Custom Categories	99
0.16.16.3 Version Compatibility	99
0.16.17 Serialization and Syntax	99
0.16.17.1 Canonical Format	99
0.16.17.2 Alternative Formats	99
0.16.17.3 Git Friendliness	100

0.16.18 Validation	100
0.16.18.1 Error Reporting	100
0.16.18.2 Strict vs. Lenient Mode	100
0.16.19 Relationship to Other Components	101
0.16.20 Build vs. Adopt: A Survey of Existing Representations	101
0.16.20.1 Timeline Text Languages	101
0.16.20.2 Visualization Grammars	102
0.16.20.3 Timeline Data Models	103
0.16.20.4 Temporal and Calendar Standards	103
0.16.20.5 Scheduling and PM Formats	104
0.16.20.6 Analogous Document IRs	104
0.16.20.7 Comparison Table	104
0.16.20.8 Recommendation: BUILD with Borrowed Vocabulary	104
0.16.20.9 Alignment of Mark's IR with Prior Art	106
0.17 Timeline Layout Engine	106
0.17.1 Layout Pipeline Overview	106
0.17.2 Determinism Contract	106
0.17.3 Coordinate System	107
0.17.4 Timeline Axis	108
0.17.4.1 Axis Unit and Inference	108
0.17.4.2 Date-to-Coordinate Function	108
0.17.4.3 Date Coercion for Bar Edges	109
0.17.4.4 Tick and Gridline Placement	109
0.17.5 The Six-Phase Layout Algorithm	109
0.17.5.1 Phase 1: Axis Computation	110
0.17.5.2 Phase 2: Track Placement	110
0.17.5.3 Phase 3: Activity Geometry	111
0.17.5.4 Phase 4: Milestone Geometry	112
0.17.5.5 Phase 5: Sections and Annotations	113
0.17.5.6 Phase 6: Label Collision Resolution	113

0.17.6	Scene / Render IR — Output of the Pipeline	114
0.17.7	Edge Cases	114
0.17.8	Known IR Gaps	116
0.18	Flow Grammar: Domain IR	117
0.18.1	Scope & Intent	117
0.18.1.1	What a Flow Diagram IS	117
0.18.1.2	What a Flow Diagram is NOT	118
0.18.1.3	Modeling Boundary	118
0.18.2	Flow Domain IR	118
0.18.2.1	Document Structure	118
0.18.2.2	Root Fields	119
0.18.2.3	FlowNode	119
0.18.2.4	FlowEdge	119
0.18.2.5	FlowGroup	120
0.18.2.6	Layout Intent	120
0.18.3	Layout Contract	120
0.18.3.1	Layout Family Selection	120
0.18.3.2	Sugiyama Layered Layout (DAGs and Cyclic Flows)	121
0.18.3.3	Linear Sequence Layout (Pipelines)	121
0.18.3.4	Determinism Mandate	121
0.18.4	Lowering to the Scene IR	122
0.18.4.1	Node Lowering	122
0.18.4.2	Edge Lowering	123
0.18.4.3	Group Lowering	123
0.18.5	Edge Cases & Total Semantics	123
0.18.5.1	Cycles	124
0.18.5.2	Self-Loops	124
0.18.5.3	Multi-Edges	124
0.18.5.4	Disconnected / Orphan Nodes	124
0.18.5.5	Missing Target/Source ID	124

0.18.5.6	Empty Flow	125
0.18.5.7	Duplicate Node IDs	125
0.18.5.8	Edge Referencing Same Source and Target (Self-Loop)	125
0.18.6	Worked Example: RAG Pipeline	125
0.18.6.1	Flow Domain IR	125
0.18.6.2	Lowering to Scene IR (Prose Description)	126
0.18.7	Determinism & Opt-In Discipline	127
0.18.8	Open Questions (Deferred to Mark/Barbara)	127
0.19	Sequence Grammar: Domain IR	128
0.19.1	Scope & Intent	128
0.19.1.1	What a Sequence Diagram IS	128
0.19.1.2	What a Sequence Diagram is NOT	129
0.19.1.3	Modeling Boundary	129
0.19.1.4	Why De-Risked	129
0.19.2	Sequence Domain IR	130
0.19.2.1	Document Structure	130
0.19.2.2	Root Fields	130
0.19.2.3	Participant	130
0.19.2.4	Message	131
0.19.2.5	Activation	132
0.19.2.6	Fragment	132
0.19.3	Layout Contract (Deterministic-by-Construction)	132
0.19.3.1	Participant Placement (x-axis)	132
0.19.3.2	Lifelines (y-axis skeleton)	133
0.19.3.3	Message Placement (y-axis)	133
0.19.3.4	Activation Bars	133
0.19.3.5	Fragment Rectangles	134
0.19.3.6	Contrast with Flow Grammar Layout	134
0.19.4	Lowering to the Scene IR	134
0.19.4.1	Participant Headers	134

0.19.4.2	Lifelines	134
0.19.4.3	Messages	135
0.19.4.4	Activation Bars	135
0.19.4.5	Fragments	136
0.19.5	Edge Cases & Total Semantics	136
0.19.5.1	Self-Message	136
0.19.5.2	Message Referencing an Undeclared Participant	136
0.19.5.3	Duplicate Order Indices	136
0.19.5.4	Participant with No Messages	137
0.19.5.5	Empty Diagram	137
0.19.5.6	Nested Fragments	137
0.19.5.7	Async Message with No Corresponding Reply	137
0.19.5.8	Activation Referencing Non-Existent Order	137
0.19.6	Worked Example: Token-Based REST API Authentication	137
0.19.6.1	Sequence Domain IR	138
0.19.6.2	Lowering to Scene IR (Detailed)	138
0.19.7	Determinism & Opt-In Discipline	139
0.19.8	Open Questions (Deferred to Mark/Barbara)	140
0.20	Tree Grammar: Domain IR	140
0.20.1	Scope & Intent	141
0.20.1.1	What a Tree Diagram IS	141
0.20.1.2	What a Tree Diagram is NOT	141
0.20.1.3	Modeling Boundary	141
0.20.1.4	Why De-Risked	142
0.20.2	Tree Domain IR	142
0.20.2.1	Document Structure	142
0.20.2.2	Root Fields	143
0.20.2.3	TreeNode	143
0.20.3	Layout Contract (Deterministic Tidy-Tree)	144
0.20.3.1	Algorithm: Buchheim–Jünger–Leipert (2002)	144

0.20.3.2	Orientation	144
0.20.3.3	Node Sizing	145
0.20.3.4	Spacing Parameters (Theme Tokens)	145
0.20.3.5	Contrast with Flow Grammar’s Sugiyama	145
0.20.4	Lowering to the Scene IR	145
0.20.4.1	Node Lowering	146
0.20.4.2	Edge Lowering	146
0.20.4.3	Scene IR Mapping Summary	147
0.20.5	Edge Cases & Total Semantics	147
0.20.5.1	Multiple Roots (Forest)	147
0.20.5.2	Cycles	147
0.20.5.3	Orphan Nodes (Unresolved Parent)	148
0.20.5.4	Single-Node Tree	148
0.20.5.5	Very Wide Trees (Many Siblings)	148
0.20.5.6	Very Deep Trees	148
0.20.5.7	Duplicate IDs	148
0.20.5.8	Empty Children List	148
0.20.6	Worked Example: Document Structure Hierarchy	148
0.20.6.1	Tree Domain IR	149
0.20.6.2	Layout Computation	149
0.20.6.3	Lowering to Scene IR	150
0.20.7	Determinism & Theme/Semantics Split	150
0.20.8	Open Questions (Deferred to Mark/Barbara)	151
0.21	The Chart Family: A Grammar-of-Graphics Layer	152
0.21.1	Design Principles for Charts	152
0.21.2	Chart Domain IR	152
0.21.3	Chart Types	153
0.21.3.1	Pie Chart	153
0.21.3.2	XY Chart (Bar/Line)	153
0.21.3.3	Quadrant Chart	154

0.21.3.4 Radar Chart	154
0.21.4 Layout Engine	154
0.21.5 Relation to Vega-Lite	154
0.21.6 Implementation Status	155
V Aesthetics & Theming	168
0.22 Theme Architecture	169
0.22.1 Current Reality: Per-Component Fragmentation	169
0.22.2 The Themes $\times$ Components Matrix	170
0.22.3 The General Theme Contract	170
0.22.3.1 Palette	172
0.22.3.2 Typography	173
0.22.3.3 Spacing / Rhythm Scale	174
0.22.3.4 Density	174
0.22.3.5 Shape Language	175
0.22.3.6 Effects and Motion	175
0.22.3.7 The Timeline ResolvedTheme as the Generalisation Seed	175
0.22.4 Theme Drives Geometry, Layout, and Routing	176
0.22.5 Reach: Theme, Layout, and New Components	176
0.22.6 Mermaid Configuration Surface	177
0.22.7 Determinism Under Theme-Driven Layout	178
0.22.8 Fidelity Tiers and Backend Effect Profiles	178
0.22.9 Theme Inheritance and Extension	179
0.22.10 Built-in Named Themes	180
0.22.11 The Three-Tier Token Architecture	181
0.22.11.1 Tier-3 Example: serpentine turn radius	181
0.22.12 Contract Vocabulary Summary	182
0.22.13 Contract Adoption Plan	183
0.22.13.1 Proof Set: Validate the Contract First	183
0.22.13.2 Migration Order	183

0.22.14 Implementation Status (2026)	184
0.23 The Aesthetic Bar: Beating Mermaid Visually	184
0.23.1 Why Aesthetics Is Architecture	185
0.23.2 The Design System	185
0.23.3 Default Theme Craft	185
0.23.4 Brand Themes and Coherence	186
0.23.5 Grammar = Semantics; Theme = Style	186
0.23.6 Aesthetic Verification	186
0.23.7 Implementation Status	187
VI Composition	188
0.24 Composition: Multi-Panel Posters	189
0.24.1 Design Philosophy	189
0.24.2 Definitions	189
0.24.3 Composition IR — Formal Shape	190
0.24.4 The Sub-Scene Embed Mechanism	192
0.24.4.1 Step 1: Compile Each Cell's Content to a Sub-Scene . .	192
0.24.4.2 Step 2: Compute Grid Layout (Cell Rectangles)	192
0.24.4.3 Step 3: Translate and Scale Sub-Scenes into Cell Rect- angles	192
0.24.4.4 Step 4: Render Panel Chrome	193
0.24.4.5 Step 5: Merge into Final Scene	194
0.24.5 Layout Contract — Deterministic Grid Sizing	194
0.24.5.1 Inputs	195
0.24.5.2 Column Width Computation	195
0.24.5.3 Row Height Computation	195
0.24.5.4 Cell Rectangle Computation	195
0.24.5.5 Canvas Height (auto)	196
0.24.6 Determinism and Theme/Semantics Split	196
0.24.6.1 What the IR Carries (Semantics)	196

0.24.6.2	What the Theme Carries (Style)	196
0.24.7	Edge Cases	197
0.24.7.1	Sub-Scene Larger Than Cell (Scale-to-Fit)	197
0.24.7.2	Empty Cell	197
0.24.7.3	Single-Cell Composition	198
0.24.7.4	Ragged Grid (Missing Cells)	198
0.24.7.5	Nested Compositions	198
0.24.8	Worked Example: Multi-Grammar Poster	198
0.24.8.1	Composition IR	198
0.24.8.2	Compilation Walkthrough	200
0.24.9	Panel References	201
0.24.10	Compilation Pipeline Summary	202
0.24.11	Limitations	202
0.24.12	Open Questions	202
0.25	Cross-Diagram Node Linking	203
0.25.1	Concept and Motivation	203
0.25.2	Link Syntax	204
0.25.3	The Node-Anchor Registry	205
0.25.3.1	Candidate Implementations	205
0.25.3.2	Recommended Implementation: Sidecar Anchors (Candidate b)	207
0.25.4	Overlay Routing	208
0.25.5	Linkable-Type Rules	210
0.25.6	Semantics and Degradation Contract	211
0.25.7	Engineering Prerequisites	212
0.25.8	The <code>trace</code> Abstraction: Named, Typed, Ordered Paths	213
0.25.8.1	Concept	213
0.25.8.2	Syntax	214
0.25.8.3	Trace-Type Vocabulary	215
0.25.8.4	Rationale: Why Traces Beat Loose Links	216

0.25.8.5	Presentation Semantics	216
0.25.8.6	Worked Examples	218
0.25.8.7	Agent Integration	221
VII	Architecture & Packaging	223
0.26	Layered Architecture	224
0.26.1	The Three Layers	224
0.26.2	Layer 1: The Kernel	224
0.26.2.1	Kernel Components	225
0.26.2.2	Kernel Contract	225
0.26.3	Layer 2: Grammars	225
0.26.3.1	Grammar Independence	226
0.26.3.2	Grammar-Kernel Dependency	226
0.26.4	Layer 3: Composition	226
0.26.4.1	Composition Dependencies	226
0.26.5	Dependency Rules	227
0.26.6	Module Boundaries	227
0.26.6.1	What Lives in the Kernel	227
0.26.6.2	What Lives in Grammars	227
0.26.6.3	What Lives in Composition	227
0.26.7	API Surface	228
0.26.7.1	Kernel API	228
0.26.7.2	Grammar API	228
0.26.7.3	Composition API	228
0.27	Packaging & Repository Strategy	229
0.27.1	Strategic Principle	229
0.27.2	Compounding Payoff	229
0.27.3	The Incremental Path	230
0.27.3.1	Phase 0: Draw the Seam (Current)	230
0.27.3.2	Phase 1: Prove Grammar-Agnosticism	230

0.27.3.3	Phase 2: Extract Packages	230
0.27.3.4	Phase 3: Grammar Marketplace (Future)	231
0.27.4	Package Naming	231
0.27.5	Versioning Strategy	231
0.27.6	Monorepo Tooling	232
0.27.7	CI/CD Considerations	232
0.28	Graph Auto-Layout: The Hard Problem	232
0.28.1	Why Graph Layout Is Hard	232
0.28.2	The Constraint as a Feature	233
0.28.3	Algorithm Details	233
0.28.3.1	Tidy-Tree (Reingold-Tilford)	233
0.28.3.2	DAG (Sugiyama Layered Layout)	234
0.28.3.3	Hub-and-Spoke (Radial)	234
0.28.3.4	Force-Directed (General Graphs)	234
0.28.3.5	Orthogonal Layout (Architecture Diagrams)	235
0.28.3.6	Constraint-Based Layout (Swimlanes, Alignment)	236
0.28.4	Practical Layout Engines	236
0.28.4.1	ELK (Eclipse Layout Kernel)	236
0.28.4.2	dagre	236
0.28.5	Build vs. Adopt Decision	237
0.28.6	Risk Assessment	237
0.28.7	Layout Hints (Escape Hatch)	237
0.29	Agent Integration	238
0.29.1	The Ingestion Contract	238
0.29.1.1	The Prompt as First-Class Input	238
0.29.1.2	What Ingestion May Not Do	238
0.29.2	Ingestion Workflows	239
0.29.2.1	Azure DevOps Work Items	239
0.29.2.2	Natural Language Prose	241
0.29.2.3	GitHub Projects	243

0.29.2.4	Mermaid Timeline Syntax	243
0.29.3	Reliable IR Generation by Agents	245
0.29.3.1	JSON Schema Publication	245
0.29.3.2	Structured Output Constraints	247
0.29.3.3	IR Design Choices That Help Agents	247
0.29.4	Validation Pipeline	248
0.29.4.1	Errors vs. Warnings	249
0.29.5	Error Handling and Agent Repair Cycle	249
0.29.5.1	Error Message Format	249
0.29.5.2	Agent Repair Cycle	250
0.29.6	MCP Server	251
0.29.6.1	Tool Contracts	251
0.29.6.2	MCP Server Deployment	253
0.29.7	Round-Trip and Iterative Editing	254
0.29.7.1	Source Provenance via Metadata	254
0.29.7.2	Re-Sync Strategy	255
0.29.7.3	Stable IDs and Git-Friendly YAML	255
0.29.7.4	Human Edits and Incremental Refinement	256
0.29.8	IR Gaps and Open Questions	256
VIII	Roadmap & MVP	258
0.30	Coverage Roadmap	259
0.30.1	Strategy: Mermaid-Complete First, Net-New After	259
0.30.2	Tier 0: Wire Existing Grammars to Mermaid Syntax	259
0.30.3	Tier 1: The UML/Software Line	259
0.30.4	Tier 2: The Chart Family	260
0.30.5	Tier 3: Remaining Types	260
0.30.6	MVP Definition (Reframed)	260
0.30.7	Post-MVP: Net-New Diagram Types	261
0.30.8	Implementation Status Summary	261

0.31	Distribution Strategy	262
0.31.1	Distribution Requirements	262
0.31.2	Packaging Options	262
0.31.2.1	Core Library	262
0.31.2.2	Command-Line Interface	263
0.31.2.3	VS Code Extension	264
0.31.2.4	MCP Server (Model Context Protocol)	264
0.31.2.5	NPM Package for Embedding	265
0.31.2.6	Standalone Go Binary	265
0.31.3	Implementation Language Trade-offs	266
0.31.4	Recommended Architecture	266
0.31.5	Versioning and Compatibility	267
0.31.6	Distribution Priorities	267
0.32	Open-Source Strategy	268
0.32.1	Is This a Viable Open-Source Project?	268
0.32.2	Closest Existing Projects and the Gap They Leave	268
0.32.3	Potential Adopters	269
0.32.4	Ecosystem Fit	269
0.32.4.1	Mermaid / Markdown / CI Ecosystem	269
0.32.4.2	MCP / Agent Ecosystem	270
0.32.4.3	PPTX Ecosystem	270
0.32.5	Extension Opportunities	270
0.32.6	Strongest Differentiators	271
0.32.7	Community Contribution Model	271
0.32.8	Adoption Risk Assessment	271
0.33	Target Outputs: Worked Examples and Coverage Analysis	272
0.33.1	T1 — Vertical Spine, Dark Theme, Icon Badges	273
0.33.2	T2 — Horizontal Linear, Light, Numbered Nodes	276
0.33.3	T3 — Dense Vertical Infographic, AI Timeline	278
0.33.4	T4 — Serpentine Glowing Path	280

0.33.5	T5 — Gitline: Card Timeline, Dark Textured, Data-Driven . . .	281
0.33.6	Synthesis	284
0.33.6.1	A: Layout Families Finding	284
0.33.6.2	B: Consolidated Coverage Table	284
0.33.6.3	C: Gap List	284
0.33.6.4	D: Verdict	287

Part I

Thesis & Positioning

0.1 Problem Definition

0.1.1 The Communication Gap

Mermaid [33] has become ubiquitous. With 88,000+ GitHub stars, native rendering on GitHub, GitLab, Notion, and Obsidian, and 22 diagram types covering flowcharts to sequence diagrams to Gantt charts, it is the de facto standard for diagrams-as-code. Developers, architects, and technical writers reach for Mermaid daily.

Yet Mermaid's output is *ugly*. Functional, yes—but never presentation-grade. The diagrams look like documentation artifacts: fixed styles, poor typography, no coherent design system, no theming beyond color overrides. Meanwhile, AI agents generating diagrams have no structured, deterministic target—they must emit raw text DSL and hope for the best.

The gap is now two-dimensional:

Aesthetics gap

Mermaid is ubiquitous but visually mediocre. Teams paste Mermaid output into READMEs but would never put it in a board deck, a technical blog, or a client deliverable.

Agent gap

AI agents can generate structured plans, architectures, and specifications—but have no widely-adopted deterministic compiler target that accepts structured IR and guarantees byte-identical, beautiful output.

This compiler closes both gaps simultaneously: a **full Mermaid superset** that parses every existing Mermaid file, produces dramatically better-looking output, and provides a structured IR path for agents.

0.1.2 Why Mermaid Is Necessary But Insufficient

0.1.2.1 Mermaid's Strengths (Why We Build On It)

Mermaid [33] earned its dominance through genuine strengths:

- **Zero-install rendering** on GitHub, GitLab, Notion, Obsidian, and hundreds of platforms
- **Simple, memorable syntax** that developers learn in minutes
- **Broad coverage** — 22 diagram types spanning flowcharts, UML, charts, and more
- **Git-friendly plain text** that diffs cleanly and lives alongside code
- **Ecosystem momentum** — AI tools already generate Mermaid syntax

These strengths make Mermaid the right *starting point*. We do not compete with Mermaid's ecosystem; we **subsume** it.

0.1.2.2 Mermaid's Limitations (Why We Exist)

- **Ugly, fixed output.** No design system. Poor typography. Diagrams look like 2015 documentation tools.
- **No real theming.** Color overrides only; no typographic, geometric, or spatial control.
- **No structured IR.** Agents must generate text DSL; no JSON/YAML path for constrained generation.
- **No composition.** Cannot combine diagrams into multi-panel posters or referenced layouts.
- **Non-deterministic edge cases.** Layout varies across versions and renderers.
- **No animation.** Static-only output; no declarative motion.
- **Limited UML.** Sequence and class exist but look dated; no modern software-diagram aesthetic.

0.1.2.3 The Agent Blindspot

Modern AI agents (LLMs with tool use, planning agents, copilots) can generate structured specifications—architectures, state machines, data models, workflows. But their options for visual output are:

- Generate raw Mermaid text (fragile; no constrained generation; ugly output)
- Generate SVG directly (unreliable; no semantics; no theming)
- Call a design tool API (proprietary; non-deterministic; heavyweight)

What agents need: a **structured IR** with a published JSON Schema, grammar-constrained generation support, deterministic rendering, and beautiful output. That is the structured IR path of this compiler (Part II).

0.1.3 Who This Is For

This compiler serves two complementary audiences through two input paths:

0.1.3.1 Human Authors (Mermaid-Superset DSL)

Software engineers and architects

— Use Mermaid daily for flowcharts, sequence diagrams, and architecture docs. Want better-looking output without changing workflow.

Technical writers and DevRel

— Need diagrams for blogs, documentation, and presentations that look professional, not like auto-generated documentation artifacts.

Product managers and consultants

— Need roadmaps and architecture visuals suitable for client deliverables and executive communication.

0.1.3.2 AI Agents (Structured IR)

AI agents require:

- A well-defined intermediate representation (IR) they can generate via constrained decoding
- Deterministic rendering — the same IR always produces the same visual
- Theming separation — agents produce structure, not style
- Schema validation — agents can verify their output conforms before rendering
- Embeddability — agents can invoke rendering as a tool (MCP server)

0.1.4 What Success Looks Like

0.1.4.1 Scenario: Drop-in Mermaid Upgrade

A developer has 50 Mermaid diagrams in their documentation repository. They switch the renderer to this compiler. Every diagram parses correctly and produces output that is visually superior—better typography, more coherent layout, proper spacing—with zero file changes.

0.1.4.2 Scenario: Agent-Generated Architecture Diagram

An AI coding agent analyzes a codebase and generates a C4 component diagram as structured YAML. The compiler renders it to a presentation-grade SVG with proper stereotypes, boundaries, and relationship labels—suitable for a design review or architecture decision record.

0.1.4.3 Scenario: Multi-Diagram Poster

A technical writer composes a “system overview” poster: a flowchart, sequence diagram, and deployment tree in a 2×2 grid with a title bar. A single `poster` file produces a cohesive multi-panel visualization.

0.1.4.4 Success Criteria

The compiler succeeds when:

1. Every valid Mermaid file parses and renders correctly (Mermaid-complete)
2. Output is demonstrably more beautiful than Mermaid’s for every diagram type
3. AI agents can generate valid structured IR via constrained decoding
4. The same IR, rendered with the same theme, produces byte-identical output across runs
5. Users need zero migration effort—existing Mermaid files just work
6. The UML/software diagram line rivals or exceeds PlantUML’s coverage with modern aesthetics

0.1.5 What This Is Not

To prevent scope creep and misunderstanding:

- **Not a Mermaid fork** — we do not modify Mermaid’s codebase; we are an independent compiler that accepts Mermaid syntax as input
- **Not a project management tool** — does not track work, assign resources, or compute schedules
- **Not a design tool** — does not provide a canvas, drag-and-drop, or WYSIWYG editing
- **Not interactive** — output is static (SVG, PNG, PDF); interactivity is out of scope
- **Not a charting library** — though we render charts, we are not competing with D3/Plotly for general data visualization

This is a *diagram compiler*: text or structured input in, beautiful deterministic visual output out. The value is in the combination of Mermaid compatibility + superior aesthetics + determinism + agent IR + composition.

0.2 Central Thesis: A Mermaid-Superset Diagram Compiler

This document specifies a **Mermaid-superset diagram compiler**: a system that accepts every valid Mermaid diagram, produces dramatically better-looking output, provides a structured IR for agent authoring, and extends Mermaid with composition, rich theming, animation, and a dedicated UML/software diagram line.

0.2.1 The Positioning

Central thesis: All of Mermaid’s 22 diagram types work out of the box—parsed faithfully from Mermaid syntax—but compile through our deterministic Scene IR to produce *beautiful, themeable, coherent* output. Differentiation comes from three pillars: (1) explicitly beating Mermaid’s aesthetics, (2) a Tier-1 UML/software diagram line, and (3) an agent-authorable structured IR that enables dual front-end authoring.

The strategy is: **Mermaid-complete first, craft net-new diagrams after.**

0.2.2 The Engine-as-Asset Insight

The strategic insight underlying this specification: **the real asset is the compiler architecture**—a shared deterministic Scene IR kernel that hosts a family of diagram grammars, each with a Mermaid-compatible parser and a structured IR API.

The compounding payoff: animation, themes, new backends, and quality-gates added once in the kernel are inherited by every grammar. A new grammar (e.g., class diagrams) gains determinism, theming, SVG/PNG/PDF output, composition support, and validate-before-render for free.

0.2.3 The Pipeline Model

Every diagram in the compiler follows the same pipeline, regardless of whether it enters as Mermaid text or structured IR:

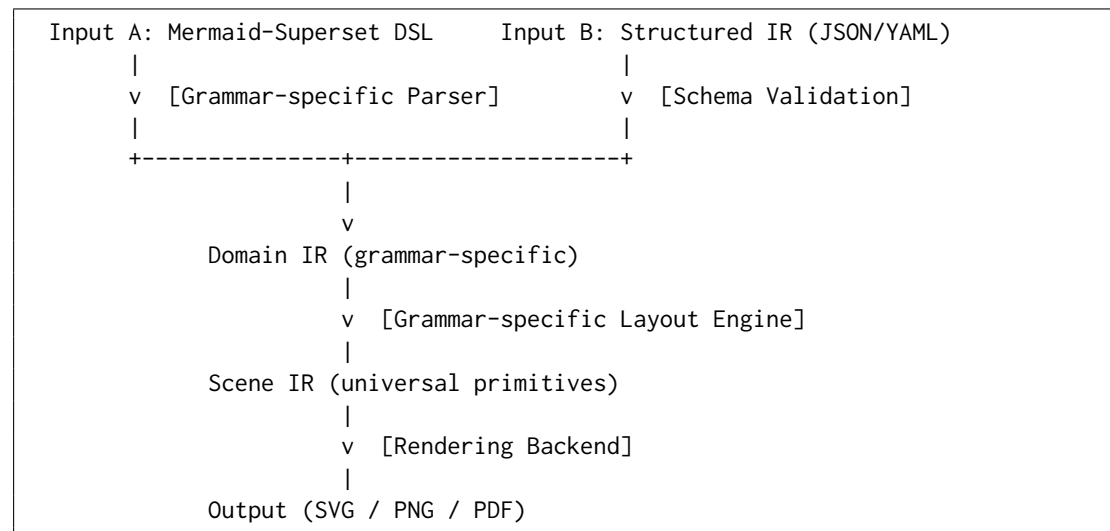


Figure 1: The dual-input pipeline: DSL and structured IR converge at Domain IR

Validate-before-render. The pipeline enforces a strict contract: no diagram reaches the renderer until it passes schema validation and semantic checks. This is not defensive coding; it is a *specification-level guarantee*. An invalid IR produces a diagnostic, never a corrupt visual.

Byte-identical golden determinism. The same IR plus the same theme produces byte-identical output across runs, platforms, and time. This property enables CI/CD integration, golden-image testing, and reliable agent round-trips.

0.2.4 Why “Diagram Compiler,” Not “Diagram Tool”

The term “compiler” is precise and intentional:

Structured input, structured output

The system accepts Mermaid text or structured IR and produces a deterministic artifact. No WYSIWYG, no freeform drawing.

Separation of concerns

Content (IR), styling (theme), and rendering (backend) are cleanly separated. The IR

describes *what* to communicate; the theme describes *how* it looks; the backend describes *where* it goes.

Typed intermediate representations

Like a programming-language compiler, this system uses typed IRs at multiple levels (Domain IR, Scene IR) with well-defined transformations between them.

Determinism by construction

The compiler is a pure function from (input, theme, backend) to output. No random state, no user interaction, no system clock.

0.2.5 The Five Diagram Families

The 22 Mermaid types plus our extensions organize into five families (Part IV):

1. **Node-link / graph**: flowchart, C4, architecture, block, requirement, gitGraph, sankey
2. **UML / software** (Tier-1 priority): sequence, class, state, ER
3. **Charts** (data→geometry): pie, xychart, quadrant, radar
4. **Timeline / project**: gantt, timeline, journey, kanban
5. **Tree / hierarchy**: mindmap, treemap

0.2.6 The Kernel/Grammar/Composition Architecture

The architecture has four layers (detailed in Part VII):

Front-End (Part II)

The dual-input layer: Mermaid-superset parsers and structured IR validation. Converts all inputs to Domain IR.

Grammars (Part IV)

Peer modules organized into five families. Each grammar defines a Domain IR and layout engine(s) that compile Domain IR to Scene IR.

Kernel (Part III)

The shared infrastructure: Scene IR, rendering backends, determinism contract, animation. The kernel knows nothing about specific diagram types—it renders primitives.

Composition (Part VI)

Arranges multiple diagrams into posters/grids. Each cell references a grammar’s output.

0.2.7 Summary of the Strategic Positioning

This specification captures the product’s positioning:

1. **Full Mermaid superset.** All 22 diagram types parse and render. Zero migration cost for existing Mermaid users.

2. **Beautiful output as primary differentiator.** Aesthetics is not polish; it is the reason users switch (Part V).
3. **Dual front-end.** Humans write Mermaid-superset text; agents emit structured IR. Both converge at the Domain IR.
4. **UML/software as Tier-1 priority.** Class, state, ER, C4—the diagrams software teams need daily, rendered with modern aesthetics.
5. **Deterministic, composable, animated.** Byte-identical output; multi-diagram posters; declarative animation. No other tool combines all three.
6. **Mermaid-complete first, net-new after.** Cover the 22 types before investing in novel diagram kinds.

The following parts specify each layer in detail.

0.3 Design Principles

The following principles govern Timeline Compiler’s architecture. They are listed in priority order; when principles conflict, earlier principles take precedence.

0.3.1 Presentation-First

Principle: Every design decision optimizes for presentation-quality visual output.

Rationale: The distinguishing feature of Timeline Compiler is producing visuals suitable for board presentations, investor decks, and executive communication. This is not a documentation tool (Mermaid serves that purpose) nor a project management tool (many exist). If a feature does not improve presentation quality, it should not exist.

Implications:

- Typography, spacing, and color receive first-class treatment in the theme engine
- The IR includes presentation hints (emphasis, annotations, callouts) that have no project-management semantics
- Output formats prioritize fidelity (PDF, SVG) over convenience

0.3.2 Deterministic Output

Principle: The same IR plus the same theme produces byte-identical output across all runs.

Rationale: Determinism enables:

- Version control — changes to the IR produce predictable visual diffs
- CI/CD integration — automated rendering in pipelines
- Agent reliability — agents can trust their output

- Reproducibility — artifacts can be regenerated exactly

Implications:

- No random layout algorithms; all positioning is computed deterministically
- Timestamps, UUIDs, and other entropy sources are excluded from output
- Font metrics must be stable (system fonts are problematic; embedded or specified fonts are required)
- Floating-point operations must be reproducible (or avoided)

0.3.3 Human-Readable Source

Principle: The IR must be readable and writable by humans without tooling.

Rationale: The IR serves dual audiences (humans and agents). Human readability ensures:

- Onboarding friction is low — a PM can write a roadmap without learning a complex DSL
- Debugging is possible — when output is wrong, the IR is inspectable
- Manual overrides are feasible — humans can adjust agent-generated IR
- Version control diffs are meaningful

Implications:

- YAML or YAML-like syntax preferred over JSON (less visual noise)
- Field names are descriptive English, not abbreviations
- Nesting depth is minimized
- Dates use ISO 8601; durations use human-readable formats where possible

0.3.4 Agent-Friendly Authoring

Principle: AI agents must be able to generate valid IR without special prompting or examples.

Rationale: A primary use case is agent-generated timelines. If agents struggle to produce valid IR, the system fails its purpose.

Implications:

- The IR has a formal schema (JSON Schema or equivalent) that agents can reference
- Field names and structure align with how LLMs naturally express plans
- The IR avoids footguns — common mistakes should be invalid or auto-corrected
- Validation provides clear, actionable error messages
- The schema is small enough to fit in a context window

0.3.5 Theme-Driven Rendering

Principle: Visual styling is entirely separated from content; all presentation decisions flow from the theme.

Rationale: Separation of content and presentation enables:

- Rebranding without changing content — swap themes, regenerate
- Consistent organizational aesthetics across timelines
- Agents authoring content without making design decisions
- Themes as a distribution mechanism (theme marketplaces, branded themes)

Implications:

- The IR contains no colors, fonts, or sizes — only semantic hints
- Themes define all visual properties
- The rendering model maps semantic IR elements to themed visual primitives
- Inline style overrides, if permitted, are explicit escapes from theme control

0.3.6 Separation of Planning and Rendering

Principle: The IR represents *what to communicate*, not *how to compute it*.

Rationale: Timeline Compiler is a rendering compiler, not a scheduling engine. The IR is a communication artifact, not a project plan. This separation:

- Keeps the IR minimal and focused
- Avoids reimplementing project-management semantics
- Allows integration with any upstream planning tool
- Clarifies scope boundaries

Implications:

- No dependency resolution, critical path calculation, or resource leveling
- Dates are provided, not computed
- Progress is asserted, not measured
- The IR describes the *output* of planning, not the *inputs* to planning

0.3.7 Minimal Visual Clutter

Principle: Default output favors clarity over information density.

Rationale: Presentation-quality visuals communicate one message clearly rather than many messages densely. Executive audiences prefer clean visuals over comprehensive ones.

Implications:

- Default themes use generous whitespace
- Labels are concise; detail lives in supporting materials
- Gridlines, axes, and chrome are minimized by default
- Information hierarchy is clear — not all elements receive equal visual weight

0.3.8 Source-Control Friendly

Principle: The IR is designed for version control workflows.

Rationale: Modern teams version everything. If the IR produces noisy diffs or merge conflicts, adoption suffers.

Implications:

- Text-based format (YAML preferred)
- Stable field ordering (or sorted keys)
- No generated IDs unless explicitly authored
- Changes to one element do not cascade spuriously to others
- Line-oriented structure where possible (each item on its own line)

0.3.9 Composability

Principle: Timeline definitions can reference, include, and compose other definitions.

Rationale: Real-world use involves:

- Shared track definitions across timelines
- Theme inheritance and overrides
- Milestone libraries
- Multi-file organization for large timelines

Implications:

- Support for file includes or references
- Namespacing to avoid collisions
- Clear resolution rules for overrides
- Themes can extend other themes

0.3.10 Graceful Degradation

Principle: Invalid or incomplete IR produces useful output or clear errors, not silent failures.

Rationale: Authors (especially agents) make mistakes. The system should:

- Validate early and report clearly
- Render partial output when possible (with warnings)
- Never produce corrupt or misleading visuals silently

Implications:

- Schema validation runs before rendering
- Semantic validation catches contradictions (e.g., end before start)
- Warnings are visible but do not block output when the issue is cosmetic
- Errors specify exactly what is wrong and where

0.3.11 Extensibility Without Complexity

Principle: The IR and theme systems support extension without requiring core changes.

Rationale: Long-term success requires community contribution — new themes, new semantic elements, integration with new upstream tools. But extension points must not compromise core simplicity.

Implications:

- Unknown IR fields are ignored (forward compatibility)
- Themes support custom properties via namespacing
- Plugin or extension architecture for renderers (future)
- Versioned schemas with clear migration paths

0.3.12 Principle Summary

Table 1: Design Principles Summary

Principle	Core Commitment
Presentation-First	Optimize for board-ready visuals
Deterministic Output	Same input, same output, always
Human-Readable Source	Writable without tooling
Agent-Friendly Authoring	Agents generate valid IR naturally
Theme-Driven Rendering	Content and style fully separated
Separation of Planning and Rendering	Describe output, not compute schedule
Minimal Visual Clutter	Clarity over density
Source-Control Friendly	Clean diffs, easy merges
Composability	Reference, include, extend
Graceful Degradation	Clear errors, useful partial output
Extensibility Without Complexity	Extend without core changes

0.4 Scope Boundaries

This section defines explicit boundaries for the Mermaid-superset diagram compiler.

0.4.1 Governing Principle

The scope boundary is defined by the *Mermaid-superset contract*: this compiler accepts all valid Mermaid diagrams and our extensions, and produces deterministic visual output. It does not execute, simulate, schedule, or interactively manipulate diagrams.

If Mermaid renders it, we render it (better). Beyond Mermaid, we add composition, rich theming, animation, and agent IR. We do not add interactivity, data management, or operational semantics.

0.4.2 In Scope

0.4.2.1 Diagram Coverage

All 22 Mermaid diagram types (Section 0.7), organized into five families, plus our superset extensions (composition, animation, cross-references).

0.4.2.2 Input Paths

Mermaid-superset DSL

Text syntax compatible with Mermaid plus our extensions.

Structured IR

JSON/YAML conforming to per-grammar Domain IR schemas.

0.4.2.3 Output Formats

SVG

Primary vector output (source-of-truth backend).

PNG

Rasterized output via resvg/Skia.

PDF

Print-quality vector output for documents and decks.

0.4.2.4 Key Capabilities

Theming

Complete visual customization via theme files.

Determinism

Byte-identical output guarantee.

Composition

Multi-diagram posters and grid layouts.

Animation

Declarative SMIL animation in SVG output.

Agent integration

MCP server, CLI, library API.

Schema validation

Per-grammar JSON Schema validation with clear diagnostics.

0.4.3 Out of Scope

Interactive output

No clickable regions, tooltips, or drill-down. Output is static.

WYSIWYG editing

No drag-and-drop authoring. This is a compiler, not an editor.

Data management

No persistent storage, collaboration, or version history.

Operational semantics

No dependency scheduling, resource leveling, critical-path analysis, or simulation.

General data visualization

Not competing with D3/Plotly for arbitrary charts. The chart family covers Mermaid's four chart types only.

Native integrations

Does not natively read from Jira, Azure DevOps, or other systems. Consumes only its IR format or Mermaid text.

Live collaboration

No real-time multi-user editing.

0.4.4 Scope Summary

0.5 Positioning: Why a Mermaid Superset

This compiler is positioned as a **full Mermaid superset**: every valid Mermaid diagram parses and renders correctly, but the output is dramatically better-looking, the architecture is deterministic and composable, and agents get a first-class structured IR path alongside the text DSL.

0.5.1 The Competitive Landscape

0.5.2 Why Full Mermaid Compatibility

Mermaid [33] is the de facto standard for diagrams-as-code. It renders natively on GitHub, GitLab, Notion, Obsidian, and hundreds of platforms. Over 88,000 GitHub

Table 2: Scope Boundaries Summary

Category	Status
All 22 Mermaid diagram types	In Scope
Mermaid-superset DSL parsing	In Scope
Structured IR (agent path)	In Scope
Beautiful themed output (SVG, PNG, PDF)	In Scope
Deterministic byte-identical rendering	In Scope
Multi-diagram composition/posters	In Scope
Declarative animation	In Scope
CLI + library + MCP server	In Scope
Interactive/clickable output	Out of Scope
WYSIWYG/canvas editing	Out of Scope
Operational/scheduling semantics	Out of Scope
Data storage and collaboration	Out of Scope
Native data source integrations	Out of Scope
General-purpose data visualization	Out of Scope

stars and a \$7.5M raise confirm its ubiquity. But Mermaid has clear limitations:

- **Ugly output.** Mermaid diagrams look like documentation artifacts, not presentation-grade visuals. Fixed styles, poor typography, no coherent design system.
- **No structured IR.** Agents must generate raw text DSL; no JSON/YAML path for reliable constrained generation.
- **No composition.** Cannot combine multiple diagrams into posters or multi-panel layouts.
- **Non-deterministic edge cases.** Layout can vary across versions and renderers.
- **Limited theming.** Color overrides only; no typographic or geometric control.

Our strategy: **parse every Mermaid grammar faithfully**, then differentiate on aesthetics, determinism, composition, and the agent IR. Users can drop in existing Mermaid files and immediately get better output with zero migration cost.

0.5.3 Differentiation Pillars

Beautiful, coherent output

Explicitly beating Mermaid’s visual quality. Professional themes, proper typography, consistent spacing, dark/light modes. Aesthetics is a first-class architectural pillar (Part V).

UML/software diagram line

Tier-1 investment in class, state, ER, and C4 diagrams—the diagrams software teams use daily. PlantUML owns this niche with dated visuals; we deliver modern, beautiful UML.

Agent-authorable structured IR

Dual front-end: humans write Mermaid-superset text; agents emit/consume a structured

Table 3: Positioning matrix: diagram-as-code tools

Axis	Mermaid	PlantUML	D2	Vega-Lite	Ours
Diagram types	22 types	UML-centric	General graph	Charts only	22+ (super-set)
Visual quality	Documentation grade	Dated	Modern, limited	Good charts	Presentation-grade
Theming	Basic color overrides	Skins (limited)	Themes	Spec-level	Full theme system
Determinism	Mostly	No (Graphviz)	No	Yes	Byte-identical
Agent IR	Text only	Text only	Text only	JSON spec	Structured IR + DSL
Composition	None	None	None	Concat/layer	Grid posters
UML/software focus	Partial	Primary	No	No	Tier-1 priority

IR (JSON/YAML). Both compile to the same domain IR, then to Scene IR, then to backends (Part II).

Deterministic composition

Byte-identical output; multi-diagram posters; cross-diagram references. No other diagram-as-code tool offers this.

0.5.4 vs. PlantUML

PlantUML [42] is Java-based, GPL-licensed, Graphviz-dependent (non-deterministic), and produces visually dated output. Its UML coverage is comprehensive but its rendering is a liability in 2026. We capture PlantUML’s UML breadth with modern visuals and deterministic output.

0.5.5 vs. D2

D2 [55] is a modern diagram language with good aesthetics but no Mermaid compatibility, no structured IR for agents, limited diagram types, and non-deterministic layout (ELK/dagre). We offer broader coverage, Mermaid migration path, and byte-identical determinism.

0.5.6 vs. Vega-Lite

Vega-Lite [48] is the gold standard for declarative statistical charts—but it has no concept of node-link graphs, sequence diagrams, or flowcharts. Our chart family (Section 0.21) draws on Vega-Lite’s grammar-of-graphics principles for the quantitative subset, while covering the full diagram-type space Vega-Lite cannot reach.

Part II

The Front-End

0.6 Dual Front-End Architecture

The compiler exposes two input paths that converge on the same internal pipeline. This dual front-end is the architectural key to serving both human authors (who prefer text DSLs) and AI agents (who prefer structured data).

0.6.1 The Two Input Paths

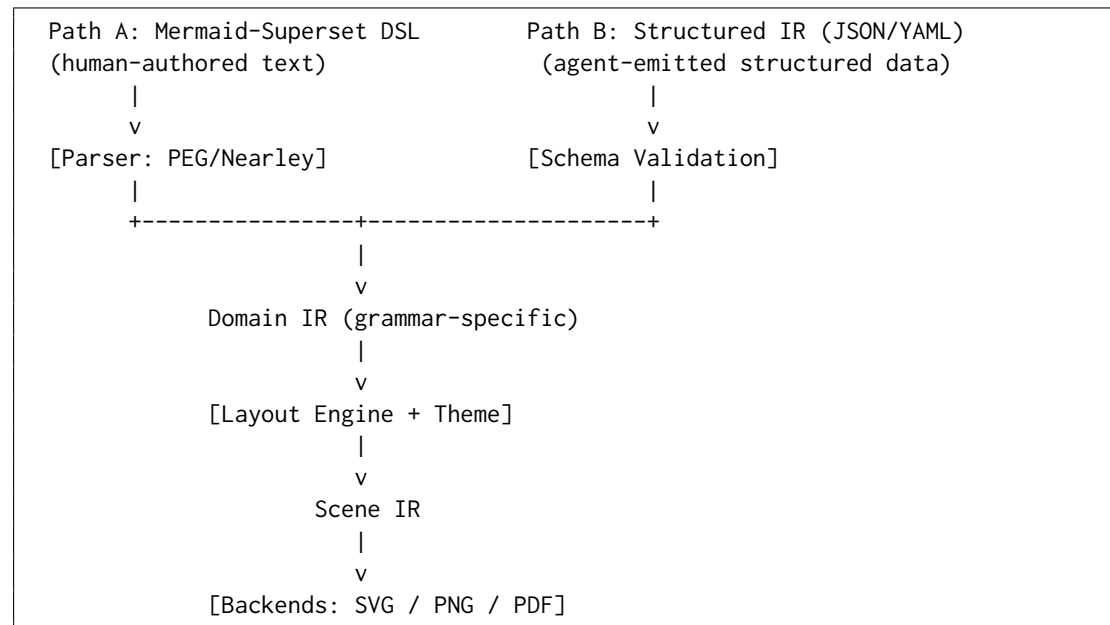


Figure 2: Dual front-end: text DSL and structured IR converge at the Domain IR layer

Path A: Mermaid-Superset DSL. Human authors write diagrams in Mermaid syntax (or our extensions). A parser converts the text to the grammar’s Domain IR. This path preserves the “diagrams-as-code” ergonomics that made Mermaid popular.

Path B: Structured IR. Agents emit JSON or YAML conforming to the Domain IR’s JSON Schema. No parsing step is needed—only schema validation. This path enables grammar-constrained generation, round-trip editing, and tool integration.

0.6.2 Convergence at Domain IR

Both paths produce the same Domain IR. From that point forward, the pipeline is identical: Domain IR → Layout Engine → Scene IR → Backend → Output.

This convergence guarantees:

- A Mermaid file and its equivalent YAML produce **byte-identical output**.

- Agents can consume human-authored diagrams (parse DSL $\rightarrow$ IR $\rightarrow$ emit structured).
- Humans can inspect agent-generated diagrams (structured IR $\rightarrow$ pretty-print DSL).
- Tooling (validation, linting, diffing) works on the common Domain IR regardless of input path.

0.6.3 The Agent IR as First-Class API

The structured IR is not a secondary or internal representation—it is a **first-class public API**. Design commitments:

Published JSON Schema

Every grammar’s Domain IR has a versioned JSON Schema, suitable for OpenAI Structured Outputs, XGrammar [9], or llama.cpp GBNF constrained generation.

Stable contract

The IR schema follows semver. Breaking changes require a major version bump.

Round-trip fidelity

DSL $\rightarrow$ IR $\rightarrow$ DSL preserves semantics (formatting may differ).

Diff-friendly

Changes to one entity do not cascade to unrelated IR fields.

Small schemas

Each grammar’s schema fits comfortably in an LLM context window (typically under 200 lines of JSON Schema).

0.6.4 Parser Architecture

The DSL parser is grammar-specific. Each grammar defines:

- A PEG or Nearley grammar specification for its Mermaid-compatible syntax
- A CST-to-Domain-IR lowering pass
- Extension hooks for superset syntax (new keywords, extended directives)

The parser layer is the **new architectural component** introduced by the Mermaid-superset positioning. It sits above the existing Domain IR / Layout Engine / Scene IR pipeline and does not affect the proven kernel or grammar internals.

0.6.5 Implementation Status

Built: The structured IR path (Path B) is fully implemented for all four shipped grammars (timeline, flow, sequence, tree). Schema validation, layout, and rendering work end-to-end.

Planned: The Mermaid-superset DSL parsers (Path A) are the primary Tier-0 deliverable. Parsers for flowchart, sequence, gantt/timeline, and mindmap syntax will wire the existing Domain IRs to Mermaid text input.

0.7 Full Mermaid Compatibility

The compiler targets **complete compatibility with all 22 Mermaid diagram types**. Every valid Mermaid file must parse, validate, and render—producing output that is semantically equivalent to Mermaid’s but visually superior.

0.7.1 The 22 Diagram Types

Mermaid supports 22 diagram types as of 2026. We organize them into five families (Section 0.15) and assign coverage tiers:

Table 4: Mermaid diagram types: families and coverage tiers

Mermaid Type	Family	Tier	Status
flowchart	Node-link / graph	0	Built (Flow grammar)
sequence	UML / software	0	Built (Sequence grammar)
gantt	Timeline / project	0	Built (Timeline grammar)
timeline	Timeline / project	0	Built (Timeline grammar)
mindmap	Tree / hierarchy	0	Built (Tree grammar)
class	UML / software	1	Planned
state	UML / software	1	Planned
erDiagram	UML / software	1	Planned
C4Context	Node-link / graph	1	Planned
C4Container	Node-link / graph	1	Planned
C4Component	Node-link / graph	1	Planned
C4Dynamic	Node-link / graph	1	Planned
pie	Charts	2	Planned
xychart-beta	Charts	2	Planned
quadrantChart	Charts	2	Planned
radar	Charts	2	Planned
sankey-beta	Node-link / graph	3	Planned
requirement	Node-link / graph	3	Planned
gitGraph	Node-link / graph	3	Planned
block-beta	Node-link / graph	3	Planned
architecture	Node-link / graph	3	Planned
treemap	Tree / hierarchy	3	Planned
kanban	Timeline / project	3	Planned
journey	Timeline / project	3	Planned
packet	Node-link / graph	3	Planned

0.7.2 Compatibility Strategy

0.7.2.1 Syntax Fidelity

Each Mermaid grammar is parsed faithfully. Our parsers accept the exact syntax documented at mermaid.js.org [31], including:

- Indentation-sensitive formats (timeline, mindmap, gantt)

- Arrow syntax variants (->, -.->, ==>, etc. in flowchart)
- Section/participant declarations
- Frontmatter (-.-) and init directives (%%{init}%%)
- All node shape syntaxes (round, diamond, hexagon, etc.)

0.7.2.2 Semantic Mapping

Mermaid's syntax maps to our Domain IRs via grammar-specific lowering rules:

flowchart → **Flow IR**

Nodes become Flow nodes; edges become Flow edges; subgraphs become lanes/clusters.

sequence → **Sequence IR**

Participants, messages, activations, and fragments map directly.

gantt/timeline → **Timeline IR**

Sections become tracks; tasks become activities; milestones map directly.

mindmap → **Tree IR**

Indentation hierarchy becomes parent-child tree structure.

0.7.2.3 Directive and Frontmatter Hooks

Mermaid's configuration mechanisms become our extension points:

```

1  ---
2  title: Architecture Overview
3  theme: professional-dark
4  config:
5    flowchart:
6      nodeSpacing: 60
7      rankSpacing: 80
8  ---
9  flowchart LR
10     A[Client] --> B[API Gateway]
11     B --> C[Service Mesh]
```

Listing 1: Mermaid frontmatter as theme/extension hook

The `theme` field selects our theme system. The `config` block maps to grammar-specific theme extensions. Init directives (%%{init: {...}}%%) provide inline configuration.

0.7.3 Reuse of Existing Kernels

Most new diagram types reuse existing layout kernels:

Node-link kernel

(Sugiyama/layered layout) serves: flowchart, C4 variants, architecture, block, requirement, gitGraph, sankey.

Tree kernel

(Buchheim–Jünger–Leipert) serves: mindmap, treemap.

Timeline kernel

(track-based layout) serves: gantt, timeline, journey, kanban.

Sequence kernel

(order-deterministic columns) serves: sequence diagrams.

Chart kernel

(NEW grammar-of-graphics layer) serves: pie, xychart, quadrant, radar.

The chart family is the only genuinely new kernel required (Section 0.21). All other new types are primarily new *parsers* wiring Mermaid syntax to existing Domain IR shapes and layout engines.

0.8 Extended Timeline Syntax

Mermaid’s `timeline` keyword is intentionally minimal: a sequence of `section` groupings each containing `period : event` lines. That simplicity makes it portable but leaves the full power of the Timeline Domain IR (Section 0.16) unreachable from the text surface.

The *Extended Timeline* is a strict superset of that syntax. It uses the same `timeline` header keyword and accepts every legal Mermaid timeline file unchanged; files that use any extended construct are superset-only and carry a compiler warning (see Section 0.8.6). The design principle is *one IR, many diagrams*: a single extended-timeline source compiles to the same `IRDocument` regardless of which of the six layout families the author selects (Section 0.8.5), and it inherits all seven contract themes (Section 0.22) without modification.

0.8.1 Two-Tier Portability Model

Tier 1 — Mermaid-faithful timeline

(unchanged, portable). Uses only `section` and `period : event` syntax. Any valid Mermaid timeline is valid here and renders identically in vanilla Mermaid. The compiler maps these files to the Timeline IR via the existing `parseTimeline` path and defaults the layout to `timeline-columns` to match Mermaid’s visual presentation.

Tier 2 — Extended Timeline

(superset-only, not renderable in vanilla Mermaid). Uses any construct defined in this section: named tracks, duration activities, `@`-directives, sections with explicit time ranges, annotations, legend, or axis breaks. The compiler emits a diagnostic warning and proceeds normally; the same `IRDocument` is the output regardless of which layout is chosen.

The two tiers share *exactly* the same IR target. There is no new IR introduced by the Extended Timeline — every construct maps to an existing `IRDocument` field defined in `packages/core/src/types.ts` and validated by `packages/core/src/schema.ts`.

0.8.2 Frontmatter Configuration

The YAML frontmatter block (`-...-`) configures the document at parse time. All keys are optional; unknown keys are warned and ignored. The full config surface (Section 0.9,

§17.1) is available; the timeline-relevant subset is:

```

1 ---
2 title: "Platform Roadmap 2026"
3 subtitle: "Engineering org - Q1..Q4"
4 theme: executive          # any of 7 contract themes
5 layout: roadmap          # horizontal | vertical-spine | serpentine
6                           # roadmap | gantt | timeline-columns
7 density: compact         # compact | normal | comfortable
8 spineSpacing: even       # time | even (vertical-spine only)
9 ---

```

Listing 2: Extended timeline frontmatter

Table 5: Frontmatter keys and their IR targets

Key	IR field	Notes
title	metadata.title	Already supported in Tier 1
subtitle	metadata.subtitle	Already supported in Tier 1
theme	metadata.theme	7 contract theme names valid
layout	metadata.layout	6 values; see Table 7
density	Theme-resolution config	Not an IR Metadata field;
		passed to <code>resolveContractTheme</code>
spineSpacing	RenderOptions.spineSpacing	time (proportional) or even (infographic); only affects vertical-spine

Note on layout schema gap. As of 2026-06-15, `metadata.layout` in `schema.ts` accepts only four values (`horizontal`, `vertical-spine`, `serpentine`, `roadmap`); the TypeScript type in `types.ts` correctly includes all six. The Zod schema must be extended to add `gantt` and `timeline-columns` before those values can be round-tripped through JSON Schema validation. This is a known schema gap, not a design gap.

0.8.3 Grammar Sketch

The following EBNF-style grammar covers the Extended Timeline body (after frontmatter extraction). Whitespace between tokens is insignificant except that indented lines nest under the nearest enclosing construct. Two-space indentation is conventional; the parser accepts any consistent depth ≥ 1 .

```

1 extended-timeline ::=
2   'timeline' [ 'extended' ] EOL
3   stmt*
4
5 stmt ::=
6   title-stmt
7   | track-stmt
8   | section-stmt
9   | annotation-stmt
10  | legend-stmt
11  | break-stmt
12
13 (* ---- top-level declarations ---- *)
14 title-stmt      ::= 'title' text EOL

```

```

15
16 track-stmt      ::= 'track' track-name EOL
17                  INDENT track-item*
18
19 track-item      ::= activity-stmt | milestone-stmt
20
21 (* ---- activities ---- *)
22 activity-stmt ::= label ':' date-expr EOL
23               ( INDENT directive )*
24
25 date-expr      ::= irdate '..' irdate    (* duration form *)
26                  | irdate                (* single-date (span) form *)
27
28 (* ---- milestones ---- *)
29 milestone-stmt ::= '@milestone' label ':' irdate EOL
30               ( INDENT directive )*
31
32 (* ---- @-directives (sub-lines under activity or milestone) ---- *)
33 directive      ::= '@status'      status-value EOL
34                  | '@progress'   integer      EOL    (* 0-100 *)
35                  | '@shape'      shape-token  EOL
36                  | '@icon'       icon-name    EOL
37                  | '@color'      css-color    EOL
38                  | '@description' text        EOL
39
40 (* ---- sections ---- *)
41 section-stmt   ::= 'section' label
42                  ( '[' irdate '..' irdate ']' )? EOL
43
44 (* ---- annotations ---- *)
45 annotation-stmt ::= 'annotation' DQUOTED-TEXT
46                  '@' annotation-target EOL
47
48 annotation-target ::= irdate '..' irdate    (* period annotation *)
49                  | irdate                (* point annotation *)
50
51 (* ---- legend ---- *)
52 legend-stmt    ::= 'legend' ( 'show' | 'hide' )? EOL
53
54 (* ---- axis breaks ---- *)
55 break-stmt     ::= 'break' irdate '..' irdate EOL
56
57 (* ---- primitives ---- *)
58 irdate         ::= (* IR date string per §21 §4 date model;
59                   examples: 2026-Q2  2026-06-01  FY26-Q3  now  tbd *)
60
61 status-value  ::=
62   'planned' | 'in-progress' | 'done' |
63   'at-risk' | 'blocked'     | 'cancelled' | 'tentative'
64
65 shape-token   ::= (* theme-resolved icon name, e.g. 'diamond' 'star'
66                  'flag' *)
67
68 label         ::= (* non-empty text, no leading colon *)
69 track-name    ::= (* non-empty text *)

```

Listing 3: Extended Timeline EBNF sketch

Keyword extended. The optional `extended` modifier on the opening line is a forward hint. When present, the compiler suppresses the “extended features detected” warning (the author has explicitly opted in). When absent, the warning is emitted on the first use of any Tier-2 construct.

Indentation and nesting. A track header owns all activity and milestone lines at the next indentation level until a dedent or another `track/section/` annotation at the same level appears. `@`-directives attach to the *immediately preceding* activity or milestone and must be indented one level deeper than that activity.

0.8.4 Construct-to-IR Mapping

Every Extended Timeline construct maps to an existing IRDocument field. Table 6 gives the authoritative mapping using the canonical TypeScript field names from `types.ts` in `packages/core/src/`.

Table 6: Extended Timeline syntax $\rightarrow$ IRDocument field mapping

DSL construct	IR field	Notes
<code>track Name</code>	<code>Track { id, label }</code>	<code>id</code> = sanitized kebab-case slug of <code>Name</code>
<code>Label: start..end</code>	<code>Activity { track, label, start, end }</code>	Both <code>start</code> and <code>end</code> are <code>IRDate</code> strings
<code>Label: date</code>	<code>Activity { track, label, span }</code>	Single-date form; <code>Activity.span</code> is the period identifier
<code>@status value</code>	<code>Activity.status</code> <code>Milestone.status</code>	or Must be one of the 7 <code>Status</code> enum values
<code>@progress N</code>	<code>Activity.progress</code>	Author writes integer 0–100; parser stores $N \div 100 \in [0, 1]$
<code>@milestone Label: date</code>	<code>Milestone { id, label, date, track }</code>	<code>track</code> is the enclosing track block’s ID
<code>@shape token</code>	<code>Milestone.icon</code>	No <code>shape</code> field in IR; <code>icon</code> is the closest proxy. Exact marker geometry is theme-resolved. (<i>Gap: see §0.8.8</i>)
<code>section Name [s..e]</code>	<code>Section { id, label, time_range }</code>	<code>time_range.start</code> and <code>time_range.end</code> from bracket
<code>annotation "text" @ date</code>	<code>Annotation { type: 'callout', text, date }</code>	Point annotation; <code>type = 'callout'</code>
<code>annotation "text" @ s..e</code>	<code>Annotation { type: 'period', text, start, end }</code>	Range annotation; <code>type = 'period'</code>
<code>legend show</code>	<code>Legend { show: true }</code>	Renderer auto-builds entries from <code>Status/category</code> data
<code>break s..e</code>	<code>Metadata.axis_breaks: [{ from, to }]</code>	Collapses dead calendar gaps to a <code>//</code> notch

The Status enum. The seven legal `@status` values (from `types.ts`) are: `planned`, `in-progress`, `done`, `at-risk`, `blocked`, `cancelled`, `tentative`. The theme contract maps each to a role-palette color (Section 0.22.3.1).

The IRDate format. Both `start..end` operands and standalone date values must conform to the date model defined in Section 0.16.5 and the regex in `schema.ts`. Legal examples: `2026-06-01`, `2026-Q2`, `2026-H1`, `FY26-Q3`, `now`, `tbd`, `~2027-Q1`.

@progress integer-to-float mapping. The IR stores `Activity.progress` as a decimal in $[0, 1]$ (type `number[0..1]`). The DSL accepts an author-friendly integer percent: `@progress N` $\mapsto$ `progress = N/100`. The parser clamps the value to $[0, 1]$; values outside $[0, 100]$ emit a warning.

0.8.5 One IR, Many Layouts

The canonical claim of the Extended Timeline is the *reach principle* (Section 0.22.5): one `IRDocument` can be rendered under any of the six layout families without changing the source. The author sets `layout:` in the frontmatter; switching layouts requires editing *only that line*.

Table 7: The six layout families and their extended-timeline affordances

layout value	Character	Best-fit use case
horizontal	Time axis left→right, track rows	Dense multi-track engineering schedules; high information density
vertical-spine	Central spine, L/R alternating entries	Narrative timelines; investor decks; few milestones
serpentine	Boustrophedon winding path	Journey maps; annual reviews with a clear chronological story
roadmap	Band-per-track, month/quarter columns	Product roadmaps; portfolio views; the <i>executive</i> sweet spot
gantt	Mermaid-faithful Gantt: task bars, date axis	Project management; dependency-heavy schedules
timeline-columns	Section-colored header bands, evenly-spaced columns	Mermaid-compatible presentation; events stacked under periods

The $7 \times 6 = 42$ look matrix. Any of the seven contract themes (`executive`, `midnight`, `blueprint`, `editorial`, `terminal`, `pastel`, `mono`) combined with any of the six layouts produces a distinct visual presentation from a single source file. Themes drive typography, palette, and connector style (Section 0.22.4); layouts drive geometry and axis orientation (Section 0.17). This *combination matrix* is the primary differentiator of the Extended Timeline over Mermaid’s fixed rendering.

The poster bridge. A poster (Section 0.9) can reference the same extended-timeline source under multiple layouts in adjacent cells, producing a side-by-side showcase of the 42-look matrix in a single document.

Dimension guard. The layout engine enforces a dimension guard (height ≤ 5000 px; aspect ratio $\leq 4 : 1$; see `gallery-dimensions.test.ts`). Pathological combinations — for example, `vertical-spine` over a multi-decade span with default `spineSpacing: time` — trigger a render-time warning and a concrete suggestion:

“vertical-spine with a time-proportional spine exceeds safe render dimensions. Consider `spineSpacing: even` or switching to `layout: roadmap / timeline-columns` for multi-decade spans.”

This is implemented in `packages/core/src/grammars/timeline/vertical-spine.ts` and was validated against the full gallery in the dimension-guard decision (2026-06-15).

0.8.6 Degradation and Portability Contract

Extended Timeline Portability Contract (2026-06-15).

1. A file using only Tier-1 constructs (`section + period : event`) is byte-compatible with vanilla Mermaid and produces no warnings from this compiler.
2. A file using any Tier-2 construct is superset-only. The compiler emits a **WARNING** at parse time: *“Extended timeline features detected — output is not renderable by vanilla Mermaid. Use Tier-1 syntax for portable files.”* and continues compilation normally.
3. The warning is *suppressed* when the opening line reads `timeline extended`, signalling explicit opt-in.
4. All Tier-2 constructs map to existing `IRDocument` fields. No new IR is introduced. Switching from a Tier-2 file back to Tier-1 syntax produces a valid (reduced) document with no schema changes.
5. Vanilla Mermaid renderers encountering a Tier-2 file will fail to parse it predictably (they do not silently corrupt output); the failure is a clear signal to switch to this compiler.

0.8.7 Worked Examples

0.8.7.1 Full Extended Timeline

The following example exercises every Tier-2 construct and shows the frontmatter configuration.

```

1 ---
2 title: "Platform Engineering Roadmap"
3 subtitle: "2026 - all tracks"
4 theme: executive
5 layout: roadmap
6 density: normal
7 ---
8 timeline extended
9
```

```

10 section Infrastructure [2026-Q1 .. 2026-Q2]
11 section Product [2026-Q2 .. 2026-Q4]
12
13 track Platform
14   Kubernetes Upgrade: 2026-Q1 .. 2026-Q2
15     @status done
16     @progress 100
17     @description "Migrate all clusters to k8s 1.32"
18   Service Mesh Rollout: 2026-Q2 .. 2026-Q3
19     @status in-progress
20     @progress 40
21   @milestone Hard Freeze: 2026-06-30
22     @status planned
23     @shape diamond
24
25 track API
26   GraphQL Gateway: 2026-Q1 .. 2026-Q3
27     @status in-progress
28     @progress 60
29   @milestone v3 Launch: 2026-09-01
30     @status planned
31     @shape star
32
33 track Mobile
34   iOS Redesign: 2026-Q2
35     @status planned
36   Android Parity: 2026-Q3 .. 2026-Q4
37     @status tentative
38
39 annotation "PCI-DSS audit window" @ 2026-06-01 .. 2026-06-30
40 annotation "Board review" @ 2026-Q3
41
42 break 2026-12-20 .. 2027-01-05
43
44 legend show

```

Listing 4: Full extended timeline source

0.8.7.2 Same Data, Multiple Layouts

Switching layout: in the frontmatter (one line) re-renders the identical IRDocument under a different presentation family.

```

1 # Layout 1: Gantt – dependency-aware task bars
2 layout: gantt
3
4 # Layout 2: Roadmap – band-per-track, quarter columns
5 layout: roadmap
6
7 # Layout 3: Serpentine – boustrophedon journey narrative
8 layout: serpentine
9
10 # Layout 4: Timeline-columns – Mermaid-compatible column bands
11 layout: timeline-columns

```

Listing 5: Same source, four layout variants

Combined with seven contract themes, this yields 42 distinct visual presentations from one source. A poster cell grid can render selected layout-theme combinations side by side:

```

1 ---
2 theme: executive
3 layout: grid 1x2
4 ---
5 poster "Roadmap two ways"
6   cell [0,0]: timeline extended
7     # ... (roadmap layout via frontmatter override)
8   cell [0,1]: timeline extended
9     # ... (gantt layout via frontmatter override)

```

Listing 6: Poster showcasing two layout variants of the same source

0.8.8 Known IR Gaps and Deferred Questions

The following items are gaps between what the Extended Timeline syntax expresses and what the current IR (2026-06-15) can cleanly carry. They are flagged here for future IR revision, not as blockers for the syntax definition.

@shape has no IR field.

Milestone in `types.ts` carries `icon?: string` and `category?: string` but no `shape` field. The parser maps `@shape diamond` to `Milestone.icon = 'diamond'`, conflating pictogram identity with marker geometry. Future IR revision should add `Milestone.shape?: enum{diamond, circle, square, star, flag}` and separate it from `icon`.

gantt and timeline-columns missing from Zod schema.

`metadata.layout` in `schema.ts` lists only four values (`horizontal`, `vertical-spine`, `serpentine`, `roadmap`). The TypeScript type in `types.ts` correctly includes all six. The schema must be extended to add `gantt` and `timeline-columns` for these layouts to survive JSON Schema round-trip validation.

density is not an IRDocument field.

Density is resolved at theme time via `resolveContractTheme` and never written into the IR. A document authored at `density: compact` is indistinguishable from one at `density: normal` after IR round-trip. Whether density should be promoted to `Metadata` is an open question for the Tier-2 contract evolution (Section 0.22.3).

legend auto-entry generation is unspecified.

The IR supports `Legend.entries` (explicit `LegendEntry` objects), but the parser-level behaviour of `legend show` without explicit entries (auto-build from `status/category` data) is a rendering convention, not an IR invariant. The spec defers the auto-entry algorithm to the layout engine documentation (Section 0.17).

@milestone without an enclosing track.

If `@milestone` appears outside any `track` block, the parser should assign `Milestone.track = undefined` (the IR permits this). The layout engine then renders the milestone on the global axis rather than a swimlane. This behaviour should be explicitly validated.

0.9 Superset Extensions Beyond Mermaid

The compiler is not merely Mermaid-compatible—it is a **superset** that extends Mermaid’s capabilities through well-defined extension mechanisms. Extensions use Mermaid’s own frontmatter/directive hooks plus new top-level keywords, ensuring that the base Mermaid grammar remains parseable by standard Mermaid renderers (with graceful degradation of extended features).

0.9.1 Extension Mechanisms

Frontmatter extensions

The YAML frontmatter block (`---`) carries our theme selection, animation directives, and composition metadata. Standard Mermaid renderers ignore unknown frontmatter fields.

Init directive extensions

The `%%{init: {...}}%%` block accepts grammar-specific configuration that maps to our theme extensions.

New top-level keywords

For capabilities with no Mermaid equivalent (e.g., `poster`, `compose`), we introduce new diagram-type keywords. These are clearly outside Mermaid’s grammar and signal superset-only features.

Inline annotations

Extended comment syntax (`%% @directive`) for per-element hints (animation targets, cross-diagram references).

0.9.2 Composition / Posters

A new `poster` keyword enables multi-diagram compositions directly in the DSL:

```

1  ---
2  theme: professional-dark
3  layout: grid 2x2
4  ---
5  poster "RAG Architecture"
6    cell [0,0]: flowchart LR
7      A[Query] --> B[Retriever] --> C[Generator]
8    cell [0,1]: sequence
9      User->>API: request
10     API->>DB: lookup
11    cell [1,0]: mindmap
12      root(Components)
13        Retriever
14        Generator
15        Router
16    cell [1,1]: stat
17      value: 99.2%
18      label: Accuracy

```

Listing 7: Poster composition in superset DSL

This maps to the Composition IR (Section 0.24) and produces grid-based multi-panel posters.

Dual Cell Addressing

Cell positions may be specified in either of two equivalent forms, which may be mixed freely within a single poster:

Bracket form

`cell [row, col]`: — explicit 0-indexed row and column. `cell [0,0]`: addresses the top-left cell; `cell [1,2]`: addresses row 1, column 2.

Excel/spreadsheet form

`cell A1`: — column letter(s) followed by a 1-indexed row number, matching spreadsheet convention. Column letters use bijective base-26: A=0, B=1, ..., Z=25, AA=26, AB=27, ... Row numbers are 1-indexed (row 1 → internal row 0).

The two forms are strictly equivalent: `cell A1` ≡ `cell [0,0]`; `cell B1` ≡ `cell [0,1]`; `cell A2` ≡ `cell [1,0]`; `cell AA1` ≡ `cell [0,26]`. Both forms are case-insensitive. A malformed address emits a warning and the cell is skipped.

0.9.3 Rich Theming

Superset theming goes beyond Mermaid's color overrides:

```

1  ---
2  theme: corporate-blue
3  themeVariables:
4    fontFamily: "Inter, system-ui"
5    fontSize: 14
6    borderRadius: 8
7    nodePadding: 16
8    lineWidth: 2
9  darkMode: true
10 ---

```

Listing 8: Rich theme configuration via frontmatter

Our theme system (Section 0.22) provides full typographic, geometric, and chromatic control—not just color token overrides.

0.9.4 Animation

Declarative animation hints attached to elements:

```

1  ---
2  animate:
3    edges: dashflow
4    duration: 2s
5  ---
6  flowchart LR

```

```
7 A --> B --> C
```

Listing 9: Animation extension

Animation compiles to SVG SMIL (Section 0.13) with raster backends rendering static resting frames.

0.9.5 Cross-Diagram References

Cross-diagram node linking connects nodes that live in *different* cells of a poster, turning a set of side-by-side panels into a linked multi-view system diagram. A `link` statement in the poster DSL names both endpoints using the existing dual cell-addressing scheme (Section 0.9.2) and a Mermaid-mirrored edge style:

```
1 link A1.pay --> B1.PaymentGateway : "handled by"
2 link A1.pick -.-> A2.Processing : "triggers"
3 link B1.OrderService --- A2.Pending
```

Listing 10: Cross-diagram link statement (superset-only)

Links are resolved at composition time, after each cell’s diagram has compiled independently, and rendered as overlay connectors on a synthetic layer above all cell primitives. They are *presentation-layer assertions on the composition* — they do not modify any diagram’s Domain IR or Scene IR. An unresolvable link (unknown cell, unknown node, or non-linkable diagram type) emits a WARNING and is skipped; the poster renders in full. The full specification — link grammar, the node-anchor registry prerequisite, overlay routing, linkable-type rules, and the degradation contract — is given in Section 0.25.

0.9.6 Icons and Images in Nodes

Nodes can embed icons from a registry or external images:

```
1 flowchart LR
2   A["{icon:aws-lambda}" Lambda] --> B["{icon:database}" Store]
```

Listing 11: Icon extension in flowchart

The icon registry is part of the kernel; themes control icon styling (color, size).

0.9.7 The Structured IR as Agent API

The superset’s most significant extension is invisible in the DSL: the structured IR path. Agents bypass the text parser entirely and emit Domain IR directly as JSON/YAML. This is not an “export format”—it is a first-class input API (Section 0.6).

0.9.8 Graceful Degradation

When a superset file is rendered by a standard Mermaid renderer:

- Frontmatter extensions are silently ignored (Mermaid skips unknown fields).
- New keywords (**poster**, etc.) produce a parse error—clearly signaling that the file requires our renderer.
- Inline annotations in comments (`%% @...`) are ignored as comments.

This ensures that the base diagram content remains portable; only superset features require our compiler.

Part III

The Kernel

0.10 Scene IR: The Universal Render Contract

The Scene IR is the central abstraction of the kernel—the single handoff point between any grammar’s layout engine and every rendering backend. It is a backend-agnostic, byte-deterministic description of every drawing primitive, visual treatment, and effect request.

0.10.1 Design Rationale

Why a Scene IR? A Scene IR (sometimes called a “Render IR” or “Display List”) decouples the concerns of layout from the concerns of output format. This separation enables:

- **Grammar-agnosticism:** Timeline, flow, and graph grammars all compile to the same Scene IR. The backends need not know which grammar produced the scene.
- **Backend-agnosticism:** SVG, PNG, PDF, and future backends consume the same input. Adding a new backend requires no changes to any grammar.
- **Determinism verification:** The Scene IR is hashable; golden-testing can verify that the same IR+theme produces byte-identical scenes.
- **Effect negotiation:** Effects (glow, drop-shadow, animation) are *requested* in the Scene IR with fallback policies. Backends fulfill or degrade gracefully.

0.10.2 Scene Model

The Scene consists of four top-level structures:

```
Scene {
  canvas:    { width, height, background }
  elements:  [ DrawingPrimitive ] -- ordered; painting order preserved
  effects:   { effect_id -> EffectDefinition }
  meta:      { theme_id, fidelity_tier, scene_hash(SHA-256), version }
}
```

0.10.2.1 Canvas Descriptor

```
Canvas {
  width:      number      -- logical pixels
  height:     number      -- computed from content
  background: Background   -- solid color, gradient, or pattern
}

Background := one of:
  SolidColor   { color: "#RRGGBB" | "#RRGGBBAA" }
  LinearGradient { angle_deg, stops: [(offset, color)...] }
  RadialGradient { cx, cy, radius, stops: [(offset, color)...] }
```

0.10.2.2 Drawing Primitives

The Scene IR defines a minimal set of drawing primitives—the “assembly language” of visual output:

```
DrawingPrimitive := one of:

  Rect {
    id, x, y, width, height,
    fill, stroke, stroke_width, corner_radius,
    opacity, clip_id?, effect_refs[]
  }

  Circle {
    id, cx, cy, radius,
    fill, stroke, stroke_width, opacity, effect_refs[]
  }

  Line {
    id, x1, y1, x2, y2,
    stroke, stroke_width, stroke_style,  -- solid | dashed | dotted
    opacity, effect_refs[], animation_ref?
  }

  Path {
    id, d_commands: [(op, args)...],      -- SVG path mini-language
    fill, stroke, stroke_width, opacity, effect_refs[]
  }

  Text {
    id, x, y, content,
    font_family, font_size, font_weight, color,
    anchor: 'start' | 'middle' | 'end',
    clip_id?, effect_refs[]
  }

  MultiText {
    id, x, y, lines: [TextSpan...],
    line_height, clip_id?
  }

  Image {
    id, x, y, width, height,
    data_uri: string                      -- embedded raster or SVG
  }

  Group {
    id, children: [DrawingPrimitive...],
    clip_rect?, transform?, opacity?
  }
```

No semantic primitives. The Scene IR contains *no* semantic types (Activity, Milestone, Track, Node, Edge). By the time content reaches the Scene, it has been reduced to pure geometry. This is the key to grammar-agnosticism.

0.10.2.3 Effect Definitions

Effects are visual treatments that may not be uniformly supported across backends. The Scene IR *requests* effects and specifies fallback policies:

```
EffectDefinition := one of:

  Glow {
    color, radius_px, intensity,
    fallback_policy: 'approximate' | 'omit' | 'embed-raster' | 'error'
  }

  DropShadow {
    offset_x, offset_y, blur_radius, color, opacity,
    fallback_policy
  }

  GradientFade {
    direction: 'left' | 'right' | 'top' | 'bottom',
    color_stops: [(offset, color)...],
    fade_px,
    fallback_policy
  }

  NoiseTexture {
    seed, scale, turbulence_type, opacity, color,
    fallback_policy
  }

  Bloom {
    radius_px, threshold, intensity,
    fallback_policy
  }
```

Fallback policies. Each effect carries a `fallback_policy` instructing backends what to do when they cannot render the effect natively:

Policy	Meaning
approximate	Use the closest available native mechanism
omit	Silently omit the effect; geometry unchanged
embed-raster	Rasterize the affected layer; embed as Image
error	Abort with a descriptive diagnostic

0.10.3 Animation Hints

Animation is modeled as *optional, declarative hints* attached to primitives. Backends that support animation (SVG, HTML) honor the hints; backends that produce static output (PNG, PDF) render the resting frame.

```
AnimationHint := one of:
```

```

FlowingDashes {
  target_id,                                -- Line or Path id
  dash_length, gap_length,
  speed_px_per_sec,
  direction: 'forward' | 'reverse'
}

DrawOn {
  target_id,                                -- Path id
  duration_ms,
  easing: 'linear' | 'ease-in-out' | 'ease-out'
}

Pulse {
  target_id,
  property: 'opacity' | 'scale' | 'fill',
  from, to, duration_ms, repeat: number | 'infinite'
}

DotAlongPath {
  target_id,                                -- Path id
  dot_radius, dot_fill,
  duration_ms, repeat
}

```

Determinism is preserved. An animated SVG is still byte-identical markup—the animation is declarative SMIL or CSS, not frame-by-frame. Golden tests can compare the SVG text. A rasterized frame is a static snapshot of the resting state.

0.10.4 Scene Determinism Contract

The Scene is byte-deterministic: two executions of any layout engine on identical IR and theme produce bitwise-identical Scene records.

The `scene_hash`. The Scene’s metadata includes a SHA-256 hash of the serialized element list and effect registry. This hash is reproducible across runs and is used for:

- Golden-image testing (compare hashes before comparing pixels)
- Cache invalidation (skip re-render if hash matches)
- Audit trails (record which scene produced which artifact)

Layered determinism. Determinism applies at three levels:

1. **Scene geometry—always byte-deterministic.** All coordinates, dimensions, and element ordering are bitwise-reproducible.
2. **Per-backend output—deterministic given pinned backend version.** The SVG backend with version X produces identical output from identical scenes. Effect seeds for procedural effects (noise, cloud) derive from `scene_hash + effect_id`.

3. **Cross-backend pixel identity—explicitly not promised.** The SVG and Skia backends produce visually different outputs (Skia renders art effects; SVG approximates or omits). This is correct behavior, not a defect.

0.10.5 Scene Serialization

The Scene IR has a canonical JSON serialization for debugging, testing, and tooling interoperability. The serialization is:

- **Sorted keys** at every level (for diff-friendliness)
- **No whitespace-only differences** (normalized indentation)
- **Floating-point precision** capped at two decimal places

This serialization is the basis for the `scene_hash` computation.

0.10.6 Scene IR as the Integration Point

The Scene IR is the *only* integration point between the upper layers (grammars, composition) and the lower layers (backends, output formats). This architectural choice has several consequences:

Adding a grammar

requires only implementing a layout engine that emits Scene IR. No backend changes are needed.

Adding a backend

requires only consuming Scene IR. No grammar changes are needed.

Testing

can happen at the Scene level: grammar tests verify Scene output; backend tests verify rendering from canonical scenes.

Themes

operate entirely within the Scene—they determine fill colors, stroke widths, and effect parameters, but they do not affect layout (which is already baked into coordinates).

0.11 Rendering Backends: SVG as Source of Truth

This section defines the rendering backends that consume the Scene IR and produce final output artifacts. The key architectural decision is that **SVG is the canonical source of truth**—not one backend among equals, but the primary representation from which other formats are derived or specialized.

0.11.1 The Backend Architecture

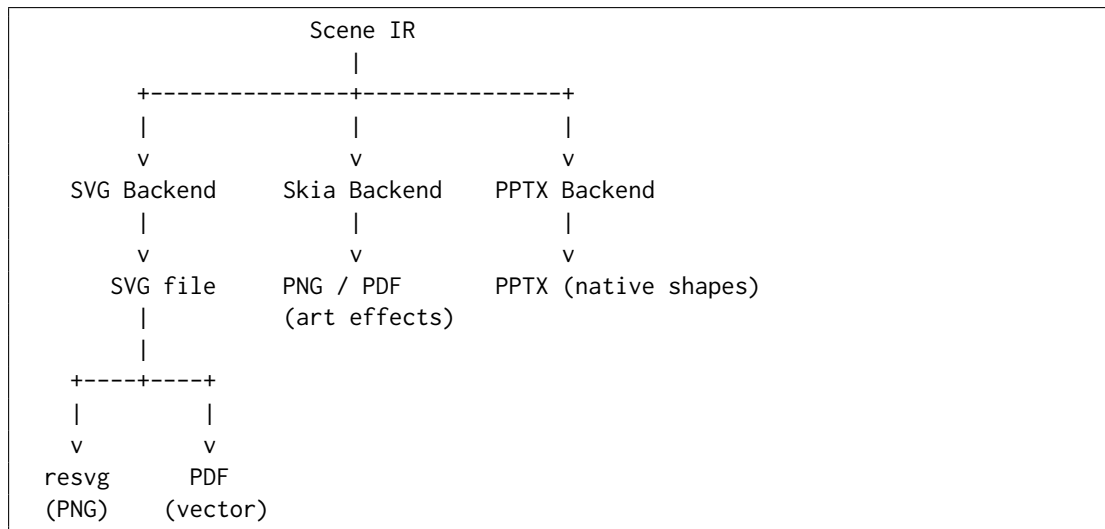


Figure 3: Backend architecture: SVG as the canonical path; Skia and PPTX as specialized backends

0.11.2 SVG as Canonical Source of Truth

Rationale. SVG is the canonical representation because:

1. **Text-based and diff-friendly.** SVG is human-readable XML. Golden tests can compare SVG text, not just pixel hashes.
2. **Resolution-independent.** SVG scales to any resolution without quality loss.
3. **Deterministic.** The same Scene produces byte-identical SVG. No font rendering variance, no anti-aliasing differences.
4. **Animation-capable.** SVG supports SMIL animation and CSS animations natively. An animated SVG is still a static text file—the animation is declarative.
5. **Universal embedding.** SVG embeds in web pages, Markdown, documentation, and most design tools.
6. **Superset relationship.** A static SVG is an animated SVG with no animation. An animated SVG degrades gracefully to still in print contexts.

PNG and PDF as exports from SVG. PNG output is produced by rasterizing SVG through resvg (a Rust-based SVG renderer with deterministic output). PDF output is produced by converting SVG to PDF vectors. Both are *exports* of the SVG truth, not parallel primary targets.

0.11.3 The Skia Backend: Art Effects

For diagrams requiring art effects beyond SVG’s native capabilities (high-quality glow, noise textures, bloom), the Skia backend [23] renders directly from Scene IR to raster (PNG) or PDF.

When to use Skia.

- Effects with `fallback_policy`: `embed-raster` that SVG cannot approximate
- Themes designated as “Tier 2” or “Tier 3” fidelity (see Section 0.22.8)
- Explicit user request for Skia rendering

Skia determinism. Skia output is deterministic given:

- Pinned Skia version
- Fixed effect seeds (derived from `scene_hash` + `effect_id`)
- No system font fallback (fonts embedded or specified)

0.11.4 The PPTX Backend: Native Shapes

PowerPoint output is a specialized backend that maps Scene IR primitives to PPTX’s native DrawingML shapes [13]. This produces editable slides, not embedded images.

Supported mappings.

- Rect → `<p:sp>` with `<a:prstGeom prst="rect">`
- Circle → `<a:prstGeom prst="ellipse">`
- Line → `<p:cnxSp>` connection shape
- Text → `<p:txBody>` with `<a:p>/<a:r>`
- Group → `<p:grpSp>`
- Effects → DrawingML effects (`<a:effectLst>`, `<a:glow>`, `<a:outerShdw>`)

Limitations. PPTX cannot represent arbitrary SVG paths. Complex paths are rasterized and embedded as images. Animation hints are ignored (PPTX animations are a separate, complex system not in scope).

0.11.5 Rejecting HTML/CSS-First Architecture

An alternative architecture would use HTML/CSS as the primary representation, leveraging browser layout (flexbox, grid) and CSS animations. This approach was explicitly considered and rejected:

Browser variance breaks determinism.

Different browsers render the same HTML/CSS differently. Font metrics, sub-pixel anti-aliasing, and layout rounding vary. This fundamentally violates the determinism contract.

No single-file portability.

HTML requires external CSS, fonts, and assets. SVG is self-contained.

Headless browser dependency.

Rasterizing HTML requires Puppeteer or Playwright—heavy, slow, and non-deterministic.

Layout leaks into rendering.

CSS layout (flexbox, grid) would compute positions. But determinism requires that layout happen in the Scene IR, not in the renderer. Mixing layout engines creates irreproducibility.

The honest caveat. The real robustness risk is the *layout engine*, not the backend. Computing node positions, label placements, and connector routing is hard. CSS auto-layout is tempting precisely because it solves layout. The architectural discipline is: keep layout computed deterministically in the grammar’s layout engine; the Scene IR records the result; the backend only draws.

If CSS auto-layout is ever desired (e.g., for a future HTML-interactive backend), it must be quarantined *after* the Scene IR seam, with acceptance that HTML output is not golden-testable in the same way as SVG.

0.11.6 Backend Selection Logic

The compiler selects a backend based on:

1. **Output format requested.** `.svg` → SVG backend; `.png` → SVG+resvg (default) or Skia (if effects require); `.pdf` → SVG+PDF conversion or Skia; `.pptx` → PPTX backend.
2. **Theme fidelity tier.** Tier 1 (clean) → SVG; Tier 2/3 (effects) → Skia fallback for PNG/PDF.
3. **Explicit override.** `-backend=skia` forces Skia for any format.

0.11.7 Fidelity Tiers

Themes declare a fidelity tier that guides backend selection:

Tier	Effects	SVG	Raster/PDF
Tier 1	None (clean vectors)	Full fidelity	resvg
Tier 2	Drop shadows, gradients	Approximate	Skia (recommended)
Tier 3	Glow, noise, bloom	Omit/rasterize	Skia (required)

The default theme (“consulting”) is Tier 1. Art-effect themes (“dark-executive”, “neon”) are Tier 2 or 3.

0.11.8 Determinism Verification

Each backend provides determinism verification:

SVG

Byte-identical file comparison. Golden SVG files are committed and compared character-by-character.

PNG (resvg)

Pixel-identical comparison. resvg is deterministic given the same SVG input and version.

PNG (Skia)

Pixel-identical comparison with tolerance. Skia’s anti-aliasing may vary slightly across platforms; a small per-pixel tolerance (e.g., 1 in each channel) is permitted.

PDF

Text-layer comparison (ignore binary streams). PDF metadata (timestamps) is stripped before comparison.

PPTX

XML comparison of shape definitions. Binary image blobs compared by hash.

0.12 Determinism Contract

Determinism is not a feature of the diagram compiler—it is the *central architectural invariant*. This section consolidates the determinism contract across all layers of the system.

0.12.1 The Fundamental Guarantee

Determinism Contract: The same IR document plus the same theme plus the same backend version produces byte-identical output across all runs, platforms, and time.

This is a *specification-level* requirement, not an implementation detail. If two conforming implementations produce different outputs from identical inputs, one of them has a defect.

0.12.2 Why Determinism Matters

Version control.

Changes to the IR produce predictable visual diffs. Teams can review diagram changes in pull requests.

CI/CD integration.

Automated pipelines can render diagrams without human inspection. Build artifacts are reproducible.

Agent reliability.

AI agents can trust that their generated IR produces the expected visual. Round-trip editing (generate → render → inspect → regenerate) works. This property interacts with LLM-side grammar constraints [65]: the compiler guarantees that a valid IR produces a specific visual, while grammar-constrained decoding guarantees that the LLM emits a valid IR. Together, they close the agent-authoring loop end-to-end.

Golden testing.

Reference images committed to the repository can be compared byte-for-byte against new renders. Regressions are detectable.

Audit trails.

Given an IR and theme, the exact output can be reconstructed at any future date.

0.12.3 Sources of Non-Determinism (and How They’re Eliminated)**0.12.3.1 System Clock**

Problem: Reading `Date.now()` or the system clock introduces time-dependent variability.

Solution: The compiler never consults the system clock. The “today” marker uses `metadata.today` from the IR. Timestamps in output metadata are either omitted or taken from IR fields.

0.12.3.2 Random Number Generators

Problem: Random values for layout jitter, effect seeds, or ID generation break determinism.

Solution: No random state is consumed. Effect seeds for procedural effects (noise textures, cloud layers) are derived deterministically from `scene_hash + effect_id`.

For layout engines: force-directed algorithms (Fruchterman-Reingold [16], Eades [11]) require a fixed initial placement. The recommended mitigation is to use stress majorization [18] with a deterministic initial layout (nodes sorted by ID on a circle), which makes the SMACOF iteration a pure function of the distance matrix with no random state. Alternatively, Sugiyama-style layered layout (dagre [7], ELK [12]) is deterministic by construction for a given node and edge ordering.

0.12.3.3 Floating-Point Rounding

Problem: Different CPUs may produce different floating-point results for the same operations due to extended precision, FMA instructions, or rounding mode differences.

Solution: All intermediate values are rounded using round-half-up ($\lfloor v + 0.5 \rfloor$) to two decimal places. Scene coordinates are expressed as fixed-precision decimals, not arbitrary floats.

0.12.3.4 Collection Ordering

Problem: Hash maps and sets have undefined iteration order. Processing elements in arbitrary order produces arbitrary output order.

Solution: Every ordered collection is sorted by an explicit, unambiguous key sequence:

- Tracks: by index ascending

- Activities: by (`start_ordinal`, `id`) ascending
- Milestones: by (`date_ordinal`, `id`) ascending
- All other entities: by `id` ascending (alphabetically)

ID uniqueness is guaranteed by schema validation; IDs break all ties.

0.12.3.5 Font Metrics

Problem: System fonts vary across platforms. Text measurement with system fonts produces different label widths on macOS vs. Linux vs. Windows.

Solution: Themes must specify embedded font files (WOFF2 or equivalent) for all layout-critical fonts. The compiler ships font metrics tables at build time. System fonts are never consulted for text measurement.

0.12.3.6 Environment Variables and Locale

Problem: Environment variables (`TZ`, `LANG`, `LC_*`) affect date formatting and text processing.

Solution: The compiler uses explicit locale from `metadata.locale`. Environment variables are ignored. Date formatting uses ISO 8601 internally; display formatting is locale-aware but deterministic given the locale.

0.12.3.7 File System State

Problem: Reading external files (images, fonts) that may change between runs.

Solution: External assets are resolved at IR load time and embedded (base64 data URIs) or checksummed. The Scene IR contains no file paths—only embedded data.

0.12.4 The Determinism Verification Protocol

Determinism is verified at multiple levels:

0.12.4.1 Scene-Level Verification

The `scene_hash` (SHA-256 of canonical Scene JSON) is computed after layout. Tests verify:

```

1 const scene1 = layoutEngine.compile(ir, theme);
2 const scene2 = layoutEngine.compile(ir, theme);
3 expect(scene1.meta.scene_hash).toBe(scene2.meta.scene_hash);

```

Listing 12: Scene determinism test pattern

0.12.4.2 SVG-Level Verification

SVG output is compared byte-for-byte:

```
1 const svg1 = svgBackend.render(scene);
2 const svg2 = svgBackend.render(scene);
3 expect(svg1).toBe(svg2); // Exact string equality
```

Listing 13: SVG determinism test pattern

0.12.4.3 Raster-Level Verification

PNG output is compared pixel-by-pixel with zero tolerance (for resvg) or minimal tolerance (for Skia):

```
1 const png1 = pngBackend.render(scene);
2 const png2 = pngBackend.render(scene);
3 expect(pixelDiff(png1, png2)).toBeLessThan(tolerance);
```

Listing 14: Raster determinism test pattern

0.12.4.4 Cross-Platform Verification

CI runs on macOS, Linux, and Windows. Golden files committed on one platform must match on all platforms.

0.12.5 Determinism Across Backend Versions

Determinism is guaranteed *within* a backend version. When backend versions change (e.g., resvg 0.35 → 0.36), output may change. This is expected and managed by:

1. Recording backend version in output metadata
2. Regenerating golden files when backend versions are bumped
3. Semantic versioning: backend version bumps that change output are minor or major releases

0.12.6 The Exception: Cross-Backend Differences

Cross-backend pixel identity is *explicitly not promised*. The SVG backend and Skia backend, given the same Scene, produce visually different outputs:

- SVG may approximate or omit effects that Skia renders faithfully
- Anti-aliasing algorithms differ
- Font rendering differs (even with the same font metrics)

This is correct behavior. The determinism contract applies *within* a backend, not *across* backends. Cross-backend tests use per-backend golden images.

0.13 Animation: Additive and Backend-Conditional

Animation is modeled as an *additive, declarative, backend-conditional* layer on top of the static Scene IR. Backends that support animation (SVG, HTML) honor animation hints; backends that produce static output (PNG, PDF, PPTX) render the resting frame.

Implementation Status. Fully implemented with optional `animation` field in Scene IR. SVG backend emits SMIL `<animate>` and `<animateMotion>` elements; raster backends (PNG, Skia) render the byte-identical resting frame. Flow grammar edges support flowing dashes (`stroke-dashoffset` animation). All animation attributes deterministic and included in SVG hash verification.

0.13.1 Design Philosophy

Additive, not fundamental. Animation is not required for a diagram to be valid or useful. The vast majority of use cases (print, static slides, documentation) require no animation. Animation is an optional enhancement for web contexts.

Declarative, not frame-based. Animation hints describe *what* should animate, not *how* to render each frame. The implementation uses declarative mechanisms (SVG SMIL, CSS animations) that the browser or renderer interprets. There is no frame-by-frame generation, no video encoding, no GIF pipelines.

Backend-conditional. Animation hints are carried in the Scene IR. Backends that support animation render them; backends that don't, ignore them. This mirrors the existing pattern for effects (glow is Skia-only; SVG approximates or omits).

Determinism preserved. An animated SVG is still byte-identical markup. The animation is expressed as declarative attributes (`<animate>`, `<animateMotion>`, CSS `@keyframes`), not as different frames. Golden tests compare SVG text, including animation elements.

0.13.2 Animation Patterns from the Corpus

Analysis of the inspiration corpus reveals several animation patterns used in technical explainer infographics:

Flowing dashes / marching ants

Arrows or connectors with animated `stroke-dashoffset`, creating a sense of flow or data movement. Common in flow diagrams and architecture visualizations.

Draw-on

Path strokes that animate from zero length to full length, as if being drawn by hand. Used for emphasis when diagrams appear.

Dot along path

A small circle that travels along a connector path, representing data or signal flow.

Pulse / glow cycle

Elements that pulse (opacity or scale) to draw attention. Used sparingly for emphasis.

Sequential reveal

Elements that appear in sequence (fade-in with delay) rather than all at once. Used for step-by-step explanations.

0.13.3 Animation Hint Types

Animation hints are attached to Scene IR primitives via `animation_ref` fields:

0.13.3.1 FlowingDashes

Animates `stroke-dashoffset` to create flowing/marching effect on lines and paths.

```
FlowingDashes {
  target_id: string,           // Line or Path id
  dash_length: number,        // px
  gap_length: number,         // px
  speed_px_per_sec: number,   // flow speed
  direction: 'forward' | 'reverse'
}
```

SVG implementation.

```
1 <line id="flow-1" stroke-dasharray="10 5">
2   <animate attributeName="stroke-dashoffset"
3     from="0" to="15" dur="1s"
4     repeatCount="indefinite"/>
5 </line>
```

Listing 15: SVG flowing dashes animation

0.13.3.2 DrawOn

Animates a path from fully hidden (`stroke-dashoffset` = path length) to fully visible.

```
DrawOn {
  target_id: string,           // Path id
  duration_ms: number,
  easing: 'linear' | 'ease-in-out' | 'ease-out',
  delay_ms?: number
}
```

0.13.3.3 DotAlongPath

Animates a dot (circle) traveling along a path using <animateMotion>.

```
DotAlongPath {
  target_id: string,           // Path id to follow
  dot_radius: number,
  dot_fill: string,           // color
  duration_ms: number,
  repeat: number | 'infinite'
}
```

0.13.3.4 Pulse

Animates opacity, scale, or fill color in a repeating cycle.

```
Pulse {
  target_id: string,
  property: 'opacity' | 'scale' | 'fill',
  from: number | string,
  to: number | string,
  duration_ms: number,
  repeat: number | 'infinite'
}
```

0.13.3.5 FadeIn

Animates opacity from 0 to 1, optionally with delay for sequential reveal.

```
FadeIn {
  target_id: string,
  duration_ms: number,
  delay_ms?: number,
  easing: 'linear' | 'ease-in-out'
}
```

0.13.4 Theme-Level Animation Defaults

Themes may specify default animation behavior:

```
1 animation:
2   enabled: true                # master switch
3   default_duration_ms: 500
4   default_easing: ease-in-out
5   connectors:
6     style: flowing-dashes      # none | flowing-dashes | draw-on
7     speed_px_per_sec: 30
8   appear:
9     style: fade-in             # none | fade-in | draw-on
10    stagger_ms: 100            # delay between elements
```

Listing 16: Theme animation configuration

0.13.5 Backend Behavior

Backend	Animation Support	Behavior
SVG	Full	Emits SMIL <code><animate></code> / <code><animateMotion></code>
HTML	Full	Emits CSS <code>@keyframes</code> and <code>animation</code> properties
PNG (resvg)	None	Renders resting frame ($t=0$ or $t = \infty$)
PNG (Skia)	None	Renders resting frame
PDF	None	Renders resting frame
PPTX	Partial	Ignored (PPTX animation is a separate system)

Resting frame definition. When animation is ignored, the resting frame is:

- For `FlowingDashes`: solid stroke (no dash animation)
- For `DrawOn`: fully visible stroke
- For `DotAlongPath`: dot at start of path
- For `Pulse`: element at from value
- For `FadeIn`: fully opaque ($t = \infty$ state)

0.13.6 Scope Discipline: What's Out

The following animation capabilities are explicitly **out of scope** for v1:

Frame-by-frame generation

No rendering of individual animation frames. No GIF/APNG/WebP animated image output.

Video encoding

No MP4/WebM output. Video requires frame rendering, audio timing, and codec complexity.

Lottie/Bodmovin export

Lottie is a rich animation format that would require a separate export path. Deferred to future.

Interactive animation

No animation triggered by user interaction (hover, click). The diagrams are declarative, not interactive.

Complex sequencing

No timeline-based animation sequencing (keyframes at specific times). Only simple, repeating animations.

Rationale. These exclusions preserve the elegance of the small-declarative-spec $\rightarrow$ crisp-vector pipeline. Frame rendering, video encoding, and Lottie export would require heavy dependencies, break the text-based-determinism property, and add significant complexity. They may be valuable future extensions, but they are not v1.

Part IV

The Diagram Families

0.14 The Grammar Concept: Two-IR-Layer Model

This section defines the “grammar” abstraction that structures the diagram compiler. A grammar is a self-contained visual vocabulary with its own Domain IR and layout engine(s), all compiling down to the shared Scene IR.

0.14.1 What Is a Grammar?

A **grammar** in this architecture is:

A grammar = (Domain IR, Layout Engine(s), Theme Extensions) that compiles a declarative description of *what to communicate* into Scene IR primitives specifying *what to draw*.

Each grammar is:

- **Self-contained:** It defines its own IR schema, validation rules, and layout logic.
- **Kernel-dependent:** It consumes kernel services (Scene IR, backends, themes, icons) but does not modify them.
- **Peer to other grammars:** Timeline, flow, and graph grammars are siblings, not nested.

0.14.2 Grounding in the Grammar of Graphics

What the literature says. This architecture stands on the shoulders of three decades of visualization-grammar research.

Wilkinson’s *Grammar of Graphics* [64] established that any statistical graphic can be decomposed into a small algebra of orthogonal components: **data**, **aesthetic mappings**, **geometric marks**, **statistical transforms**, **scales**, **coordinates**, and **facets**. The key insight is generativity: a finite, composable set of primitives describes an infinite variety of charts. This is precisely the principle underlying the diagram compiler’s grammar abstraction — a finite set of mark types (box, arrow, label, group, connector) combines through composition to cover all target diagram classes.

Wickham’s layered grammar [63] operationalized Wilkinson into ggplot2, adding two engineering contributions of direct relevance: (1) **default inference** — the system fills in unspecified dimensions (axes, colors, font sizes) from data type and context, so the author specifies only deviations from sensible defaults; and (2) **spec/render separation** — a ggplot object is a spec; calling `ggsave()` triggers the rendering pipeline. Both principles are adopted verbatim: Domain IR fields have sensible defaults inferred by the layout engine, and the render pass is strictly separated from the compile pass.

Satyanarayan et al.’s Vega-Lite [48] pushed the grammar further: a concise, JSON-serializable spec compiles to Vega’s lower-level dataflow IR, which backends render to SVG or Canvas. Vega-Lite’s composition algebra (`layer`, `hconcat`, `vconcat`, `facet`) is the direct ancestor of the composition layer in Section 0.24.

The diagram extension. Where the GoG lineage addresses *statistical graphics*, this architecture addresses *diagram types* — swimlane timelines, flow pipelines, node-link graphs, step-card sequences, and comparison matrices. These diagrams have no *data/aesthetic-mapping* duality; they have a *semantic entity / visual layout* duality instead. A Timeline activity has *start*, *end*, and *track*; the compiler maps these to rectangle position, width, and vertical row — an analogous but distinct encoding to GoG’s *field* → *channel* mapping.

0.14.3 Agent-Authoring Reliability: Constrained DSL

What the literature says. A growing body of constrained-generation research establishes that *grammar-constrained decoding is the essential reliability lever for LLM-DSL pipelines*, and that small, well-constrained grammars lower the failure surface dramatically.

Willard & Louf [65] show that reformulating LLM generation as transitions in a finite-state machine derived from a formal grammar (or regex) eliminates an entire class of syntax failures with negligible per-token overhead. Wang et al. [62] demonstrate that a *minimal grammar fragment* for a specific diagram instance is more reliable than the full schema: “grammar constraints derived from domain-specific knowledge enable LLMs to generalize from few examples on complex DSL generation tasks.” Dong et al.’s XGrammar [9] achieves up to 100× speedup over naive per-step CFG execution by separating context-independent from context-dependent tokens at grammar-compilation time, making schema-constrained diagram generation practical in production LLM serving stacks.

Tian et al. [57] studied LLM failure modes for chart spec generation: LLMs produce syntactically valid but semantically wrong specs, miss required fields, and use ambiguous field names. Their mitigations map directly to the compiler’s architecture: (1) grammar constraints eliminate syntactic failures; (2) the validation layer (§0.14.4) catches semantic failures; (3) decomposed generation (entity identification, then IR emission) handles complex diagrams.

Narechania et al. [37] showed that targeting a well-specified JSON grammar (Vega-Lite’s schema) rather than a general-purpose API reduces NL-to-visualization complexity substantially. For small or on-device models, Ray [46] characterises the “constraint tax” — hard schema-decoding improves syntactic validity but can lower semantic accuracy for sub-3B models — and recommends a “reason free, constrain late” strategy.

What we recommend. The Domain IR JSON Schema (one per grammar) is published as a formal constraint and submitted to OpenAI Structured Outputs [41], XGrammar [9], or llama.cpp GBNF [20] for grammar-constrained generation. Keeping each Domain IR *small* (few required fields, flat nesting, minimal enum values) is not a simplification concession — it is a reliability design decision grounded in the literature above. The god-IR (Section 0.14.4) is rejected in part because a large, polymorphic schema is an agent-authoring failure vector.

0.14.4 The Two-IR-Layer Model

The architecture uses **two layers of intermediate representation**:

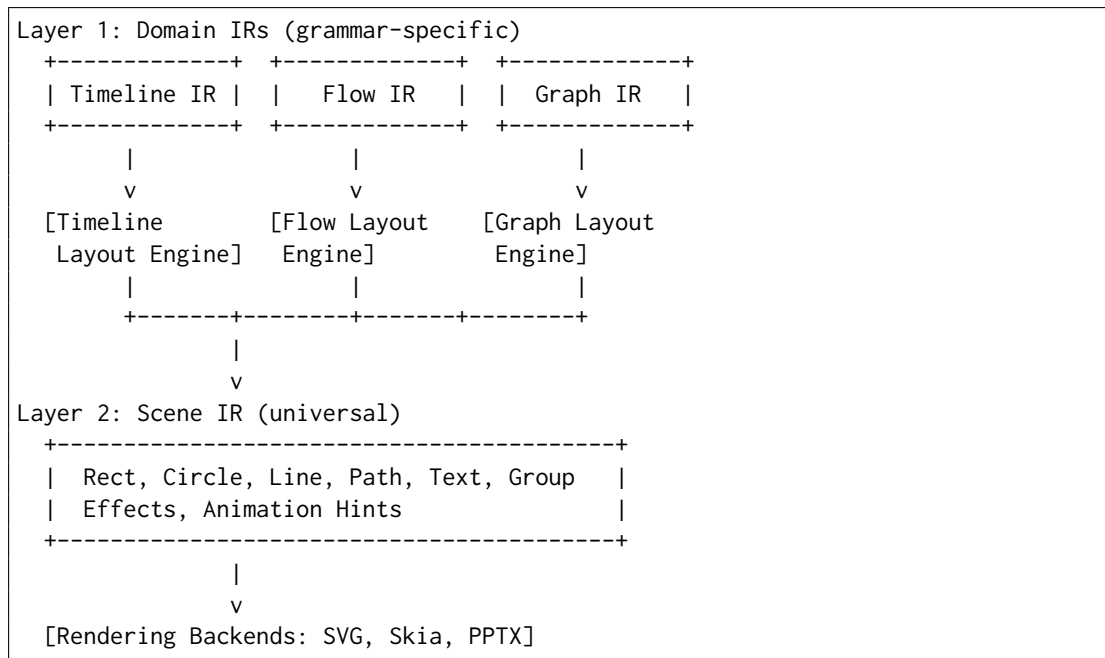


Figure 4: The two-IR-layer model: diverse Domain IRs compile to unified Scene IR

Domain IRs are diverse, small, and grammar-specific. The Timeline IR has tracks, activities, milestones. The Flow IR has nodes, edges, lanes. The Graph IR has vertices, edges, clusters. These entities are *semantically meaningful* within their grammar—an Activity knows it has a start date; a Node knows it has incoming edges.

Domain IRs are kept **small on purpose**:

- A small grammar is reliably emittable by an LLM
- A small schema fits in a context window
- A small grammar has clear boundaries and fewer edge cases

Scene IR is the universal assembly language. All Domain IRs compile *down* to the single shared Scene IR. The Scene IR knows nothing about timelines, flows, or graphs—it knows only primitives (Rect, Circle, Line, Path, Text, Group) and effects.

The Scene IR is the **one integration point**:

- Backends consume only Scene IR
- Themes style Scene IR primitives
- Golden tests can verify Scene IR determinism
- Animation hints attach to Scene IR primitives

0.14.5 Rejecting the God-IR Approach

An alternative architecture would use a single “god IR” that tries to express all diagram types in one schema:

```

1 # DON'T DO THIS
2 diagram:
3   type: timeline | flow | graph | comparison | ...
4   entities:
5     - kind: activity | node | cell | stat | ...
6     properties: { ... massive union of all possible fields ... }

```

Listing 17: Hypothetical god-IR (rejected)

This approach is explicitly rejected because:

Semantic muddiness

A single schema that means different things in different contexts is harder to validate, harder to understand, and harder to document.

Schema bloat

Every grammar's fields appear in the union, even when irrelevant. An activity has start/end; a graph node does not. The god-IR carries dead weight.

LLM hostility

A large, polymorphic schema with conditional validity rules is difficult for agents to emit correctly. Small, single-purpose schemas are easier.

Brittle evolution

Adding a new grammar requires modifying the central schema, risking regressions in all other grammars.

No clear ownership

Who owns the god-IR? Every grammar must negotiate changes. Separate Domain IRs have clear ownership.

The two-IR-layer model inverts the coupling: Domain IRs are independent; the Scene IR is the shared core. Adding a new grammar does not touch existing grammars' IRs.

0.14.6 Composition IR: Above Domain IRs

For multi-panel posters (Section 0.24), a Composition IR sits *above* the Domain IRs, not merged into them:

```

Composition IR
+-----+
| panels: [          |
|   { type: timeline, ir: {...} },
|   { type: flow,     ir: {...} },
|   { type: stat,     ir: {...} }
| ]                  |
| layout: grid(2,2) |
+-----+
      |
      v [Composition Layout Engine]
      |
(compiles each panel's Domain IR, then arranges in Scene IR)

```

The Composition IR *references* panels, each with its own grammar type and Domain IR. It does not flatten them into a single schema.

0.14.7 Grammar Interface Contract

Every grammar must implement:

```

1 interface Grammar<DomainIR> {
2   // Schema
3   readonly name: string;           // e.g., 'timeline', 'flow'
4   readonly schema: JsonSchema;     // JSON Schema for DomainIR
5
6   // Validation
7   validate(ir: unknown): ValidationResult;
8
9   // Layout
10  layout(ir: DomainIR, theme: Theme): Scene;
11
12  // Optional: grammar-specific theme extensions
13  readonly themeSchema?: JsonSchema;
14 }

```

Listing 18: Grammar interface contract

Schema. The JSON Schema for the Domain IR, used for validation and agent generation.

Validate. Checks the IR against schema and semantic rules. Returns diagnostics.

Layout. The core transformation: Domain IR + Theme $\rightarrow$ Scene IR. This is where all layout computation happens.

Theme extensions. Grammars may extend the theme schema with grammar-specific properties (e.g., `timeline.trackHeight`, `flow.nodeSpacing`).

0.14.8 Current and Planned Grammars

Grammar	Domain Entities	Status	Family
Timeline	tracks, activities, milestones, sections	Implemented	Timeline/project
Flow	nodes, edges, lanes, legends	Implemented	Node-link/graph
Sequence	participants, messages, activations, fragments	Implemented	UML/software
Tree	root, children (recursive), labels	Implemented	Tree/hierarchy
Class	classes, relationships, packages	Planned (Tier-1)	UML/software
State	states, transitions, composites	Planned (Tier-1)	UML/software
ER	entities, relationships, cardinalities	Planned (Tier-1)	UML/software
Chart	data, encoding, marks, scales	Planned (Tier-2)	Charts

The full 22-type coverage matrix and family taxonomy are detailed in Section 0.15.

0.14.9 The Compounding Payoff

The kernel/grammar separation creates a compounding payoff:

- **Animation** added to the kernel is inherited by all grammars
- **New backends** (e.g., Keynote) serve all grammars immediately
- **Theme improvements** (new fonts, colors, effects) apply everywhere
- **Determinism verification** at the Scene level covers all grammars
- **Icon registry** additions are available to all grammars

Each new grammar starts with a full feature set: determinism, theming, SVG/PNG/PDF/PPTX output, animation support, validation, and agent integration—all for free.

0.15 The Five Diagram Families

The 22 Mermaid diagram types (plus our extensions) are organized into five families. Each family shares a layout kernel, a visual vocabulary, and a set of Domain IR patterns. This taxonomy drives architecture decisions: new types within a family reuse the existing kernel and require primarily a new parser and minor IR extensions.

0.15.1 Family Overview

0.15.2 Family 1: Node-Link / Graph

The largest family. All members share the fundamental structure: **nodes** connected by **edges**, laid out by a directed-graph algorithm.

Shared kernel. The Sugiyama four-phase layered layout (Section 0.18): cycle removal, rank assignment, crossing minimization, coordinate assignment. Variants (LR, TB, RL, BT) are direction parameters.

Type-specific extensions:

flowchart

Core type. Subgraphs, diverse node shapes, edge labels. **Built.**

C4 Context/Container/Component/Dynamic

Bounded contexts as styled node clusters; stereotyped relationships. Reuses flow kernel with C4-specific node rendering.

architecture

Icon-heavy nodes in a layered or grouped topology.

block

Block diagrams with nested containers—layout variation of flowchart.



Figure 5: The five diagram families and their 22 member types, rendered as a mindmap using the `blueprint` theme. Each family shares a layout kernel; new types within a family require only a new parser and minor IR extensions.

Rendered by the diagram compiler this document describes (source: `design/figures/src/family-taxonomy.mmd`, built with `make figures`).

requirement

Requirement nodes + satisfy/derive/refine edges.

gitGraph

Commit DAG with branch/merge topology. Specialized rank = commit order.

sankey

Weighted directed edges rendered as variable-width flows.

0.15.3 Family 2: UML / Software (Tier-1 Priority)

The differentiating family. These are the diagrams software teams use daily—and where PlantUML’s dated rendering creates the biggest opportunity.

Sequence diagrams. Order-deterministic column layout. Participants, messages, activations, fragments (alt/opt/loop/par/critical/break). **Built** (Section 0.19).

Class diagrams. Classes with attributes/methods, relationships (inheritance, composition, aggregation, association, dependency), packages/namespaces. Domain IR: `classes[]`, `relationships[]`. Layout: hierarchical with inheritance edges directing rank.

State diagrams. States, transitions, composite states (nested), start/end pseudo-states, fork/join bars, history nodes. Domain IR: `states[]`, `transitions[]`. Layout: layered graph with composite-state nesting.

Entity-Relationship diagrams. Entities with attributes, relationships with cardinality (1, 0..1, 1..*, 0..*). Domain IR: `entities[]`, `relationships[]`. Layout: force-directed or layered with relationship-label placement.

0.15.4 Family 3: Charts (Data → Geometry)

A genuinely new grammar-of-graphics layer. Unlike the other families, charts map **quantitative data** to **geometric marks** through scales and coordinate systems. This is the Vega-Lite [48] / Grammar of Graphics [64] lineage adapted to our architecture.

See Section 0.21 for the full specification.

Types:**pie**

Proportional sectors from labeled values.

xychart

Bar and line charts on Cartesian axes.

quadrant

Four-quadrant scatter with labeled axes and positioned items.

radar

Multi-axis radial chart (spider/radar plot).

0.15.5 Family 4: Timeline / Project

Track-based temporal layouts. All members share the concept of a time axis with items positioned along it.

Shared kernel. Track-based layout with configurable time axis, row allocation, and activity/milestone placement (Section 0.16).

Types:**gantt**

Tasks with durations, sections, dependencies (visual arrows), today marker. **Built** (maps to Timeline IR).

timeline

Sectioned event list on a time axis. **Built** (maps to Timeline IR).

journey

User-journey map with tasks scored by satisfaction level. Extension: satisfaction score → color/position mapping.

kanban

Column-based board (backlog/doing/done). Extension: columns as vertical tracks, cards as items.

0.15.6 Family 5: Tree / Hierarchy

Recursive parent-child structures laid out with tidy-tree algorithms.

Shared kernel. Buchheim–Jünger–Leipert $O(n)$ tidy-tree layout (Section 0.20).

Types:**mindmap**

Indentation-defined hierarchy with styled nodes. **Built.**

treemap

Area-proportional nested rectangles from weighted hierarchy. Extension: squarified treemap algorithm for the layout kernel.

0.15.7 Coverage Roadmap Tiers

Tier 0 — Wire existing grammars to Mermaid syntax.

flowchart, sequence, gantt, timeline, mindmap. Work: write parsers for Mermaid syntax → existing Domain IRs. **5 types. Kernel: already built.**

Tier 1 — UML/software line.

class, state, erDiagram, C4 (4 variants). Work: new Domain IRs + layout extensions + Mermaid parsers. **7 types. High priority; key differentiator.**

Tier 2 — Charts.

pie, xychart, quadrant, radar. Work: new chart grammar-of-graphics kernel + parsers. **4 types. New architecture required.**

Tier 3 — Remaining types.

sankey, requirement, gitGraph, block, architecture, treemap, kanban, journey, packet. Work: mostly parser + IR variants on existing kernels. **9 types. Lower priority; incremental.**

0.16 Timeline Grammar: Domain IR

The Timeline Grammar is the first grammar implemented in the diagram compiler. This section specifies its Domain IR—the declarative representation of *what* a timeline communicates.

Note: This section re-homes and re-contextualizes the original Timeline IR specification as “Grammar #1” in the broader diagram compiler architecture. The IR definition is unchanged; only the framing is updated.

Implementation Status. Fully implemented in `packages/core/src/grammars/timeline/` with all four layout families (horizontal, vertical-spine, serpentine, roadmap). Theme-driven rendering with support for ByteByteGo style, dark variants, and animated arc-spine nodes. No deferrals.

0.16.1 Design Principles

The Timeline Domain IR follows the general grammar design principles (Section 0.14), with timeline-specific elaboration:

1. **Make illegal states unrepresentable.** The IR’s type system should prevent malformed timelines at the structural level, not merely validate them after the fact.
2. **Rendering-agnostic.** The IR describes meaning, not pixels. Visual decisions—colors, fonts, positioning—belong to the theme engine.
3. **Git-friendly.** Stable field ordering, line-oriented YAML syntax, deterministic serialization, minimal diff churn.
4. **Agent-generatable.** Clear required-vs-optional distinctions, explicit defaults, forgiving structure that remains validatable.

5. **Orthogonal core.** A small set of primitives; convenience features are optional layers, not core complexity.

0.16.2 Type Vocabulary

The IR uses the following type vocabulary consistently throughout this specification:

0.16.3 Document Structure

A Timeline IR document is a single YAML file with the following root structure:

```

1 version: "1.0"
2 metadata: { ... }
3 tracks: [ ... ]
4 groups: [ ... ]      # optional
5 activities: [ ... ]
6 milestones: [ ... ]  # optional
7 annotations: [ ... ] # optional
8 sections: [ ... ]    # optional
9 legend: { ... }      # optional

```

Listing 19: Root document structure

0.16.3.1 Root Fields

Invariant: The document must contain at least one track. Activities and milestones may both be empty, but a timeline with neither is semantically vacuous (valid but degenerate).

0.16.4 Metadata

The metadata object contains document-level information and temporal framing.

0.16.4.1 TimeRange

The `time_range` defines the temporal viewport of the timeline—the span that will be rendered. Activities or milestones outside this range are valid but may be clipped.

```

1 time_range:
2   start: 2026-Q1
3   end: 2026-Q4

```

0.16.4.2 AxisUnit

```

1 enum AxisUnit { day, week, month, quarter, half, year }

```

If omitted, the renderer infers axis unit from the time range span. The axis unit affects visual tick marks and labels, not the precision of individual dates.

0.16.4.3 Date Anchor and Fiscal Calendar

Date anchor (`today`). The symbolic date `now` and all relative date offsets (`+3m`, `-2w`, etc.) resolve against the *date anchor*, evaluated in this order:

1. `metadata.today` — if present, this value is used exclusively.
2. `metadata.created` — fallback when `today` is absent.
3. **Hard error** — if neither is present and any `now` or relative date appears in the document, the document is not deterministically evaluable and validation must reject it.

Renderers **must not** consult the system clock to resolve `now` or relative dates. The today-marker annotation (type: `today-marker`) also uses this anchor.

Determinism caveat: When `today` is omitted and `created` serves as the anchor, output is deterministic (the `created` date is fixed in the document). However, the document's visual "current position" does not advance with real time. Authors and agents **should** set `today` explicitly whenever `now` or relative dates appear, so that the document's meaning is unambiguous and byte-stable across environments and rendering runs.

Fiscal calendar (`fiscal_year_start`). `fiscal_year_start` specifies the calendar month (1–12) on which the fiscal year begins. The default is 1 (January), which makes fiscal and calendar years identical.

Fiscal quarter dates (`FYnn-Qk`) map to calendar dates as follows. `FYnn` denotes the fiscal year beginning on month `fiscal_year_start` of calendar year `20nn`. Fiscal quarter k ($k \in \{1, 2, 3, 4\}$) spans three consecutive calendar months starting at calendar month m_k , where:

$$m_k = ((\text{fiscal_year_start} - 1) + (k - 1) \times 3) \bmod 12 + 1$$

and the calendar year is `20nn` if $m_k \geq \text{fiscal_year_start}$, otherwise `20nn + 1`.

Examples with `fiscal_year_start: 1` (default): `FY26-Q1` = Jan–Mar 2026; `FY26-Q2` = Apr–Jun 2026 (identical to calendar quarters).

Examples with `fiscal_year_start: 4` (April fiscal year): `FY26-Q1` = Apr–Jun 2026; `FY26-Q2` = Jul–Sep 2026; `FY26-Q3` = Oct–Dec 2026; `FY26-Q4` = Jan–Mar 2027.

If any `FY...` date appears in the document and `fiscal_year_start` $\neq 1$, authors **should** set `fiscal_year_start` explicitly to ensure all renderers produce consistent x-coordinates for fiscal dates.

0.16.5 Date Model

Timelines present temporal information at varying granularities. A product roadmap may show quarters; a conference schedule shows hours. The IR must represent this heterogeneity without forcing false precision or losing meaningful vagueness.

0.16.5.1 Date Representations

A date in the IR is one of the following forms:

Design rationale: We deliberately support coarse granularity (quarters, halves, years) as first-class concepts rather than forcing conversion to day-level dates. This preserves authorial intent: “Q3 2026” means the entire quarter, not “2026-07-01”.

0.16.5.2 Uncertain and Open Dates

For dates that are genuinely unknown, tentative, or open-ended, the IR provides explicit encodings rather than using null-as-magic:

Invariant: `tbd`, `ongoing`, and `unknown` are valid only in positions where the schema allows uncertainty (e.g., end of a daterange, not `start` of the document’s `time_range`).

0.16.5.3 DateRange

A daterange represents a temporal interval:

```

1 # Explicit start and end
2 start: 2026-04-01
3 end: 2026-06-30
4
5 # Open-ended (no planned end)
6 start: 2026-04-01
7 end: ongoing
8
9 # Shorthand for single-granularity span
10 span: 2026-Q2 # equivalent to start: 2026-04-01, end: 2026-06-30

```

Invariant: When both `start` and `end` are concrete dates, `end` $\geq$ `start`. The `span` shorthand expands to the natural bounds of its granularity (e.g., `span: 2026-Q2` expands to Q2’s first and last days).

Invariant (exclusivity): `span` is *mutually exclusive* with `start` and `end`. A daterange must use exactly one of:

- `span` alone (shorthand form), or
- `start` alone or `start` + `end` (explicit form).

Co-presence of `span` with `start` or `end` on the same entity is a well-formedness error (`SPAN_START_CONFLICT`).

Open/ongoing semantics: An Activity or daterange with `start` specified but `end` omitted is an *open/ongoing interval*. This is semantically identical to writing `end: ongoing` and is an explicit, valid state (not an authoring error). Renderers must extend the bar to the canvas right edge and apply an open-end indicator (e.g., right-pointing chevron).

0.16.6 Tracks (Swimlanes)

Tracks are horizontal lanes that organize activities visually. Every activity belongs to exactly one track. Tracks appear in document order (first track at top).

```

1 tracks:
2   - id: platform
3     label: Platform Team
4     description: Core infrastructure work
5
6   - id: mobile
7     label: Mobile Apps
8     color: blue           # optional hint

```

Listing 20: Track examples

Invariant: Track id values must be unique within the document.

0.16.7 Groups

Groups provide logical clustering of activities, tracks, or other groups. They enable hierarchical organization (e.g., “Engineering” containing “Platform” and “Mobile” tracks) without conflating logical grouping with visual swimlanes.

```

1 groups:
2   - id: engineering
3     label: Engineering
4     children:
5       - platform      # ref to track
6       - mobile        # ref to track
7
8   - id: releases
9     label: Release Milestones
10    members:
11      - alpha-release  # ref to milestone
12      - beta-release

```

Listing 21: Group example

Invariant: Group hierarchies must be acyclic. A group cannot be its own ancestor.

0.16.8 Activities

Activities are time-spanning items—phases, projects, tasks, or efforts. They are the primary content of most timelines.

```

1 activities:
2   - id: api-redesign
3     label: API Redesign
4     track: platform
5     start: 2026-Q1
6     end: 2026-Q2
7     status: in-progress

```

```

8   progress: 0.4
9
10  - id: mobile-launch
11    label: Mobile App Launch
12    track: mobile
13    span: 2026-Q3
14    status: planned
15    category: launch

```

Listing 22: Activity examples

0.16.8.1 Status Enum

The `status` field communicates *visual* state for presentation purposes. This is explicitly **not** a project-management workflow state; it describes what the audience should understand about the item’s current condition.

```

1  enum Status {
2    planned,      # Scheduled but not yet started
3    in-progress,  # Currently active
4    done,         # Completed
5    at-risk,      # May miss target (visual warning)
6    blocked,      # Cannot proceed (visual alert)
7    cancelled,    # No longer planned
8    tentative     # Not yet confirmed
9  }

```

Design rationale: We include `at-risk` and `blocked` because executive timelines frequently need to communicate risk visually. We omit workflow states like “in review” or “approved” which belong to project-management tools, not visual communication artifacts.

0.16.8.2 Progress

The `progress` field is a number in $[0, 1]$ representing completion fraction. It has no inherent visual meaning—the theme decides whether to render it as a progress bar, percentage label, or ignore it entirely.

Invariant: If `progress` is provided, it must be in $[0, 1]$. `progress: 1.0` does not imply `status: done`; these are orthogonal.

0.16.9 Milestones

Milestones are point-in-time markers—releases, deadlines, decision points, or events. Unlike activities, they have no duration.

```

1  milestones:
2    - id: alpha-release
3      label: Alpha Release
4      date: 2026-03-15
5      track: platform
6      status: done

```



```

7   icon: flag
8   color: cyan
9
10  - id: board-review
11    label: Board Review
12    date: 2026-Q2          # first day of Q2
13    category: governance

```

Listing 23: Milestone examples

Note: When `track` is omitted, the milestone is “global” and may be rendered spanning all tracks (e.g., a vertical line with label at top).

0.16.10 Annotations

Annotations add contextual information—callouts, notes, highlights, or today-markers—without being first-class timeline entities.

```

1  annotations:
2    - type: today-marker
3      date: now
4
5    - type: callout
6      target: api-redesign
7      text: "Critical path item"
8      position: above
9
10   - type: bracket
11     targets: [alpha-release, beta-release]
12     label: "Testing Phase"
13
14   - type: period
15     start: 2026-06-01
16     end: 2026-06-15
17     label: "Code Freeze"
18     style: highlight

```

Listing 24: Annotation examples

0.16.10.1 Annotation Types

```

1  enum AnnotationType {
2    today-marker, # Vertical line at current date
3    callout,      # Note attached to an entity
4    bracket,      # Visual bracket spanning entities
5    period,       # Highlighted time period
6    note,         # Freestanding note
7    connector     # Line connecting two entities
8  }

```

0.16.11 Sections

Sections divide the timeline into logical or visual chapters. They enable multi-page timelines or distinct phases that should be visually separated.

```

1 sections:
2   - id: current-state
3     label: "Current State (Q1-Q2)"
4     time_range:
5       start: 2026-Q1
6       end: 2026-Q2
7
8   - id: future-state
9     label: "Future Roadmap (Q3-Q4)"
10    time_range:
11      start: 2026-Q3
12      end: 2026-Q4

```

Listing 25: Section example

0.16.12 Legend

The legend documents the meaning of statuses, categories, and visual conventions used in the timeline. If omitted, the renderer may auto-generate a legend from used values.

```

1 legend:
2   show: true
3   position: bottom-right
4   entries:
5     - key: in-progress
6       label: In Progress
7       description: Currently being worked on
8     - key: at-risk
9       label: At Risk
10      description: May miss planned date
11     - key: launch
12       label: Launch Event
13       category: true

```

Listing 26: Legend example

0.16.13 ID and Reference System

0.16.13.1 Identifier Format

All id fields must be:

- Non-empty strings
- Composed of lowercase letters, digits, and hyphens
- Starting with a letter
- Unique within their entity type AND globally within the document

Regex: `^[a-z][a-z0-9-]*$`

Rationale for global uniqueness: References may appear in contexts where the entity type is ambiguous (e.g., annotation targets). Global uniqueness eliminates the need for type prefixes in references.

```

1 # Valid IDs
2 id: api-v2
3 id: q3-release
4 id: mobile-team-ios
5
6 # Invalid IDs
7 id: API-v2           # uppercase
8 id: 2026-release    # starts with digit
9 id: api_v2          # underscore

```

Listing 27: ID examples

0.16.13.2 References

A reference is a string containing the id of another entity. The IR uses bare strings for references (not structured objects) for YAML terseness:

```

1 activities:
2   - id: feature-a
3     track: platform      # ref<Track>
4     group: engineering   # ref<Group>
5
6 annotations:
7   - type: callout
8     target: feature-a    # ref<Activity>

```

Invariant: Every reference must resolve to exactly one entity in the document. A validator must reject documents with dangling references.

0.16.13.3 Reference Namespacing

Because IDs are globally unique, references are unambiguous without type prefixes. The schema context determines what types are valid:

- `activity.track`: must reference a `Track`
- `annotation.target`: may reference `Activity` or `Milestone`
- `group.children`: may reference `Track` or `Group`

A validator checks that the referenced entity has the correct type for its context.

0.16.14 Well-Formedness Rules

A Timeline IR document is **well-formed** if and only if all the following invariants hold:

0.16.14.1 Structural Invariants

1. **Version present:** version field exists and matches a known specification version.
2. **Required fields:** All fields marked “Req: Yes” are present and non-null.
3. **At least one track:** The tracks list is non-empty.
4. **Unique IDs:** All id values are globally unique within the document.
5. **Valid ID format:** All id values match `^[a-z][a-z0-9-]*$`.

0.16.14.2 Referential Invariants

6. **References resolve:** Every reference points to an existing entity.
7. **Reference types match:** References point to entities of the expected type (e.g., `activity.track` references a `Track`, not a `Group`).
8. **No circular groups:** The group parent-child graph is acyclic.

0.16.14.3 Temporal Invariants

9. **Valid time range:** If `metadata.time_range` has both start and end, then `end >= start`.
10. **Activity duration non-negative:** For activities with concrete start and end, `end >= start`.
11. **Date format valid:** All date values conform to the date model (§0.16.5).
12. **Activity date-source exclusive:** Every activity must satisfy exactly one of: (a) `span` is present and neither `start` nor `end` is present; or (b) `start` is present and `span` is absent (`end` optional). Co-presence of `span` with `start` or `end` is a hard well-formedness error: `SPAN_START_CONFLICT`.
13. **Date anchor present when required:** If any date field in the document contains now or a relative offset (e.g., `+3m`, `-2w`), then at least one of `metadata.today` or `metadata.created` must be present. Absence of both is a hard error: `DATE_ANCHOR_MISSING`.
14. **Track index values unique:** When `track.index` is present, all supplied values must be unique non-negative integers within the document.

0.16.14.4 Value Invariants

15. **Progress in range:** If `progress` is present, $0 \leq \text{progress} \leq 1$.
16. **Status valid:** All status values are members of the `Status` enum.
17. **Axis unit valid:** If present, `axis_unit` is a member of `AxisUnit` enum.

0.16.14.5 Soft Warnings (Valid but Suspicious)

The following are not errors but may indicate authorial mistakes:

- Activity or milestone outside `metadata.time_range` (will be clipped)
- `progress: 1.0` with `status: in-progress` (likely stale)
- Empty activities and milestones lists (vacuous timeline)

- Unused tracks (no activities reference them)

0.16.15 Complete Examples

The following examples demonstrate realistic Timeline IR documents. Each is a complete, valid document showcasing different timeline use cases.

0.16.15.1 Example 1: Product Roadmap

A quarterly product roadmap with multiple teams, showing how the IR represents a typical software planning artifact.

```

1 version: "1.0"
2
3 metadata:
4   title: "Product Roadmap 2026"
5   subtitle: "Engineering & Design"
6   author: "Product Team"
7   created: 2026-06-09
8   time_range:
9     start: 2026-Q1
10    end: 2026-Q4
11   axis_unit: quarter
12   theme: corporate-blue
13
14 tracks:
15   - id: platform
16     label: Platform
17     description: Core infrastructure and APIs
18
19   - id: mobile
20     label: Mobile Apps
21     description: iOS and Android applications
22
23   - id: design
24     label: Design System
25     description: Component library and guidelines
26
27 groups:
28   - id: foundation
29     label: Foundation Work
30     members: [api-v2, component-lib]
31
32 activities:
33   - id: api-v2
34     label: API v2 Migration
35     track: platform
36     start: 2026-Q1
37     end: 2026-Q2
38     status: in-progress
39     progress: 0.6
40     category: infrastructure
41
42   - id: mobile-redesign
43     label: Mobile App Redesign

```

```

44     track: mobile
45     start: 2026-Q2
46     end: 2026-Q3
47     status: planned
48     description: Complete visual refresh
49
50   - id: component-lib
51     label: Component Library
52     track: design
53     start: 2026-Q1
54     end: 2026-Q4
55     status: in-progress
56     progress: 0.3
57
58   - id: analytics
59     label: Analytics Integration
60     track: platform
61     span: 2026-Q3
62     status: planned
63
64 milestones:
65   - id: api-ga
66     label: API v2 GA
67     date: 2026-06-30
68     track: platform
69     status: planned
70     icon: flag
71
72   - id: app-launch
73     label: App Store Launch
74     date: 2026-Q4
75     track: mobile
76     status: planned
77     category: launch
78
79 annotations:
80   - type: today-marker
81     date: now
82
83 legend:
84   show: true
85   entries:
86     - key: infrastructure
87       label: Infrastructure
88       category: true
89     - key: launch
90       label: Launch Event
91       category: true

```

Listing 28: Product roadmap example

0.16.15.2 Example 2: Executive Program Timeline

A high-level transformation program timeline suitable for board presentations, demonstrating status communication and risk visibility.

```

1 version: "1.0"

```

```
2
3 metadata:
4   title: "Digital Transformation Program"
5   subtitle: "FY26 Executive View"
6   author: "PMO"
7   created: 2026-06-09
8   time_range:
9     start: 2026-01-01
10    end: 2026-12-31
11   axis_unit: month
12   theme: executive
13
14 tracks:
15   - id: strategy
16     label: Strategy & Governance
17
18   - id: technology
19     label: Technology Modernization
20
21   - id: people
22     label: People & Change
23
24 activities:
25   - id: strategy-define
26     label: Strategy Definition
27     track: strategy
28     start: 2026-01-01
29     end: 2026-03-31
30     status: done
31     progress: 1.0
32
33   - id: vendor-selection
34     label: Vendor Selection
35     track: technology
36     start: 2026-02-01
37     end: 2026-04-30
38     status: done
39
40   - id: platform-build
41     label: Platform Build
42     track: technology
43     start: 2026-04-01
44     end: 2026-09-30
45     status: in-progress
46     progress: 0.35
47
48   - id: data-migration
49     label: Data Migration
50     track: technology
51     start: 2026-07-01
52     end: 2026-11-30
53     status: at-risk
54     description: Dependency on legacy system documentation
55
56   - id: training
57     label: Training Program
58     track: people
59     start: 2026-06-01
```

```

60     end: 2026-12-31
61     status: planned
62
63     - id: change-mgmt
64       label: Change Management
65       track: people
66       start: 2026-03-01
67       end: ongoing
68       status: in-progress
69
70 milestones:
71   - id: board-approval
72     label: Board Approval
73     date: 2026-03-15
74     status: done
75     icon: star
76
77   - id: go-live
78     label: Go Live
79     date: 2026-10-01
80     track: technology
81     status: planned
82     icon: flag
83
84   - id: program-close
85     label: Program Close
86     date: 2026-12-15
87     status: planned
88
89 annotations:
90   - type: today-marker
91     date: now
92
93   - type: callout
94     target: data-migration
95     text: "Risk: Legacy documentation gaps"
96     position: above
97
98   - type: bracket
99     targets: [platform-build, data-migration]
100    label: "Critical Path"
101
102 sections:
103   - id: h1
104     label: "H1 2026: Foundation"
105     time_range:
106       start: 2026-01-01
107       end: 2026-06-30
108
109   - id: h2
110     label: "H2 2026: Execution"
111     time_range:
112       start: 2026-07-01
113       end: 2026-12-31

```

Listing 29: Executive program timeline

0.16.15.3 Example 3: Architecture Evolution Timeline

A technical architecture timeline showing system evolution over multiple years, demonstrating coarse granularity and technical categorization.

```

1 version: "1.0"
2
3 metadata:
4   title: "Platform Architecture Evolution"
5   subtitle: "2026-2028 Technical Roadmap"
6   author: "Architecture Team"
7   created: 2026-06-09
8   time_range:
9     start: 2026
10    end: 2028
11   axis_unit: half
12   theme: technical
13
14 tracks:
15   - id: frontend
16     label: Frontend
17
18   - id: backend
19     label: Backend Services
20
21   - id: data
22     label: Data Platform
23
24   - id: infra
25     label: Infrastructure
26
27 activities:
28   - id: react-migration
29     label: React 19 Migration
30     track: frontend
31     start: 2026-H1
32     end: 2026-H2
33     status: in-progress
34     category: modernization
35
36   - id: micro-frontend
37     label: Micro-Frontend Architecture
38     track: frontend
39     start: 2027-H1
40     end: 2027-H2
41     status: planned
42     category: architecture
43
44   - id: api-gateway
45     label: API Gateway
46     track: backend
47     span: 2026-H1
48     status: done
49     category: infrastructure
50
51   - id: service-mesh
52     label: Service Mesh Adoption
53     track: backend

```

```
54     start: 2026-H2
55     end: 2027-H1
56     status: planned
57     category: infrastructure
58
59 - id: event-driven
60   label: Event-Driven Architecture
61   track: backend
62   start: 2027
63   end: 2028
64   status: tentative
65   category: architecture
66
67 - id: lakehouse
68   label: Data Lakehouse
69   track: data
70   start: 2026-H1
71   end: 2027-H1
72   status: in-progress
73   progress: 0.4
74   category: data
75
76 - id: ml-platform
77   label: ML Platform
78   track: data
79   start: 2027
80   end: ongoing
81   status: tentative
82   category: ai
83
84 - id: k8s-migration
85   label: Kubernetes Migration
86   track: infra
87   span: 2026
88   status: in-progress
89   progress: 0.7
90   category: infrastructure
91
92 - id: multi-cloud
93   label: Multi-Cloud Strategy
94   track: infra
95   start: 2027
96   end: 2028
97   status: planned
98   category: infrastructure
99
100 milestones:
101 - id: legacy-sunset
102   label: Legacy System Sunset
103   date: 2027-H1
104   track: backend
105   status: planned
106   icon: flag
107
108 - id: cloud-native
109   label: Cloud Native Milestone
110   date: 2026
111   track: infra
```

```

112     status: planned
113
114   annotations:
115     - type: period
116       start: 2026-H2
117       end: 2027-H1
118       label: "Transition Period"
119       style: highlight
120
121   legend:
122     entries:
123       - key: modernization
124         label: Modernization
125         category: true
126       - key: architecture
127         label: Architecture Change
128         category: true
129       - key: infrastructure
130         label: Infrastructure
131         category: true
132       - key: data
133         label: Data Platform
134         category: true
135       - key: ai
136         label: AI/ML
137         category: true

```

Listing 30: Architecture evolution timeline

0.16.16 Extensibility

The IR is designed to be extended without breaking existing consumers.

0.16.16.1 Metadata Extension

Every entity type includes a metadata field (type `map<string,any>`) for application-specific data:

```

1   activities:
2     - id: feature-x
3       label: Feature X
4       track: platform
5       span: 2026-Q2
6       metadata:
7         jira_key: PROJ-1234
8         owner: alice@example.com
9         priority: high

```

Renderers should ignore unrecognized metadata keys. Source integrations may use metadata to preserve round-trip fidelity with external systems.

0.16.16.2 Custom Categories

The `category` field accepts arbitrary strings. The theme engine maps categories to visual styles. New categories can be introduced without IR changes:

```

1 activities:
2   - id: security-audit
3     category: security    # custom category
4     # ...
5
6 legend:
7   entries:
8     - key: security
9       label: Security Work
10      category: true

```

0.16.16.3 Version Compatibility

The `version` field enables forward compatibility:

- Consumers should accept documents with higher minor versions (1.1, 1.2, etc.) and ignore unknown fields.
- Major version changes (2.0) may introduce breaking changes; consumers should reject documents with unsupported major versions.

0.16.17 Serialization and Syntax

0.16.17.1 Canonical Format

YAML 1.2 is the canonical surface syntax for Timeline IR documents. Design decisions prioritizing YAML:

- **Minimal punctuation:** No braces for simple mappings, no quotes for most strings.
- **Line-oriented:** Each field on its own line for clean diffs.
- **Stable ordering:** Fields should appear in specification order for consistency; validators may enforce this.
- **Comments preserved:** YAML allows comments; tools should preserve them.

0.16.17.2 Alternative Formats

Tooling may support additional formats with lossless conversion:

- **JSON:** Direct mapping; verbose but universally parseable.
- **TOML:** Alternative for users who prefer it.

The IR is defined abstractly; the YAML syntax is one serialization, not the definition.

0.16.17.3 Git Friendliness

For minimal diff churn:

1. Lists use block style (one item per line), not flow style.
2. Fields appear in consistent order.
3. Auto-generated fields (like `updated` timestamps) are optional and may be omitted in version-controlled documents.
4. IDs are stable slugs, not generated UUIDs.

```

1 # Good: block style, stable order
2 activities:
3   - id: feature-a
4     label: Feature A
5     track: platform
6     span: 2026-Q2
7
8 # Avoid: flow style, hard to diff
9 activities: [{id: feature-a, label: Feature A, track: platform,
    span: 2026-Q2}]

```

Listing 31: Git-friendly style

0.16.18 Validation

A conforming validator must check all invariants in §0.16.14. Validation is a function:

$$\text{validate} : \text{Document} \rightarrow \text{Valid} \mid \text{Errors}$$

0.16.18.1 Error Reporting

Validation errors should include:

- Path to the offending field (e.g., `activities[2].track`)
- The invariant violated
- The actual value
- Suggested fix (when determinable)

0.16.18.2 Strict vs. Lenient Mode

- **Strict mode:** All invariants enforced; required for rendering.
- **Lenient mode:** Warnings instead of errors for soft issues; useful for work-in-progress documents.

0.16.19 Relationship to Other Components

The IR is the stable contract between pipeline stages:

Ingestion (upstream)

Source adapters (ADO, GitHub, Markdown, etc.) produce IR documents. The IR defines the target shape; adapters handle mapping.

Theme Engine (downstream)

Themes consume IR and produce styled visual specifications. Themes interpret **status**, **category**, and **hint** fields (**color**, **icon**) but cannot require additional IR fields.

Renderer (downstream)

The renderer consumes themed IR (or IR + theme) and produces output (SVG, PNG, PDF, PPTX). The IR remains rendering-agnostic; pixel-level decisions are not IR concerns.

Agent Integration

AI agents generate IR directly. The IR's explicit typing, clear required/optional fields, and forgiving structure enable reliable generation. Agents need not understand rendering.

Contract stability: Downstream components may not require IR fields beyond this specification. Extension fields (**metadata**) are pass-through.

0.16.20 Build vs. Adopt: A Survey of Existing Representations

Before committing to a fully greenfield IR, this section surveys existing formats, grammars, and standards as potential adoption or extension candidates. Every candidate is assessed against the five criteria derived from the design principles (§0.16.3):

1. **Visual-communication focus:** roadmap/milestone/swimlane semantics, *not* task-scheduling (no dependencies, resource levelling, or critical path);
2. **Git-friendly:** line-oriented text, stable field order, minimal diff churn;
3. **LLM-generatable:** clear required/optional schema, simple references, forgiving structure;
4. **Render/theme separation:** clean boundary between *what* is communicated and *how* it looks; and
5. **Open licence with community or specification body.**

0.16.20.1 Timeline Text Languages

Markwhen. Markwhen [29] (MIT licence) is the closest existing tool to our surface-syntax goals. Events are written as **date:** **label** or **start/end:** **label** lines; **group:** blocks provide swimlane-like structure; **#tag** tokens allow lightweight categorisation. The format is text-native, line-oriented, and git-diffable.

Critical gaps exist, however. There is no first-class status concept (no equivalent of **status:** **at-risk**); granularity is limited to ISO dates and informal text strings (no

2026-Q3 or fiscal-quarter shorthand); render/theme separation is absent—the view container interprets colour from tags, not a separate theme layer; static SVG, PDF, and PPTX output are unsupported; and Markwhen does not publish a versioned, machine-verifiable schema for its internal JSON parse tree, which makes reliable LLM generation fragile. The editor (markwhen.com) is proprietary; only the parser and view container are open source.

Mermaid timeline and Gantt. The Mermaid `timeline` type [31] groups events into sections on a free-text time axis. It has no swimlane concept, no status enum, no PPTX output, and no render/theme separation (styles are hard-coded in the renderer). The Mermaid `gantt` type [32] adds dependency keywords (`crit`, `done`, `active`)—precisely the scheduling semantics this specification rejects. Neither type exposes a stable IR that downstream tooling can consume.

PlantUML Gantt. PlantUML’s `@startgantt` [43] is experimental and scheduling-oriented (dependency declarations, work-remaining percentages). Its verbose, UML-rooted syntax is not LLM-friendly for presentation timelines.

D2 and Structurizr. D2 [55] is a general-purpose diagram DSL with no native timeline type. Structurizr [51] is domain-specific to software architecture (C4 model). Neither is a relevant adoption candidate.

0.16.20.2 Visualization Grammars

Vega-Lite. Vega-Lite [48] is the most significant analogy in this category. It demonstrates that a high-level declarative JSON grammar can cleanly separate *what to visualize* (data, encodings, marks) from *how to render it* (scales, axes, guides, legends), with actual rendering delegated to the Vega runtime. This is precisely the WHAT/HOW philosophy that this IR embodies (see §0.16.19).

Vega-Lite is, however, a statistical-graphics grammar, not a roadmap grammar. Encoding a swimlane timeline in Vega-Lite requires manual mark composition: a `rect` mark with `y: track`, `x: start`, `x2: end`, plus separate layers for milestones and annotations. There is no first-class concept of `tracks`, `milestones`, `status`, `span`, or `annotations`. The resulting spec is verbose (100+ JSON lines for a five-activity roadmap), fragile for LLM generation, and not human-editable in the YAML sense intended here. Vega-Lite is the strongest *architectural analogy*—its declarative WHAT/HOW separation is the direct design precedent for this IR—but cannot be adopted wholesale.

The Grammar of Graphics and ggplot2. Wilkinson’s grammar [64] and Wickham’s layered refinement [63] establish the theoretical foundation for declarative visual grammars: decompose a graphic into data, aesthetics, geometry, statistics, and scales. The key insight—separating semantic content from visual treatment—directly motivates this IR’s separation of tracks/activities/status from theme/colours/layout. These are *conceptual donors*, not adoptable formats.

Observable Plot and Apache ECharts. Observable Plot (ISC licence [39]) is a concise JavaScript charting API. It has no native swimlane roadmap type and requires imperative programming, not declarative text authoring. Apache ECharts has a `timeline` component, but it is an animated-playback control that cycles through multiple chart states over time—not a swimlane roadmap grammar. Neither is suitable for adoption.

0.16.20.3 Timeline Data Models

Knight Lab TimelineJS. TimelineJS [28] accepts a flat JSON array of events with `start_date`, `end_date`, `text.headline`, and `text.text`. It is storytelling-oriented (narrative slides, embedded media) with no swimlanes, no status enum, no PPTX output, and no render/theme separation.

vis-timeline. vis-timeline [58] is a browser-only interactive widget requiring JavaScript item objects (`{id, content, start, end, group}`). There is no declarative text format, no static SVG/PDF/PPTX export, and no theme-layer concept. Not adoptable.

react-chrono and Timeline Storyteller. react-chrono [45] (MIT) accepts a JavaScript items array with `title`, `cardTitle`, `cardDetailedText`; date fields are plain strings with no semantic granularity and no swimlanes. Microsoft Research’s Timeline Storyteller [36] (open-sourced 2019) accepts a simple `[{start, end, label}]` JSON array—no swimlanes, no status, no maintained community. Neither is adoptable.

0.16.20.4 Temporal and Calendar Standards

iCalendar (RFC 5545). RFC 5545 [8] defines `VEVENT` with properties `DTSTART`, `DTEND`, `SUMMARY`, `DESCRIPTION`, `CATEGORIES`, `STATUS` (`TENTATIVE`, `CONFIRMED`, `CANCELLED`), and `URL`. This vocabulary is the outstanding *donor*: Mark’s `start/end/label/description/category/status/url` all map directly to iCalendar terms, establishing prior-art legitimacy for these names (see Table 25).

The ICS wire format is wholly unsuitable: property-folded lines (`DTSTART:20260609T140000Z`), no YAML/JSON structure, and no swimlane, visual-status, or render-pipeline concept. The `STATUS` enum covers scheduling state (`CONFIRMED`) but not visual-communication urgency (`at-risk`, `blocked`). **Not adoptable; strongest single vocabulary donor.**

Schema.org/Event. The schema.org Event type [50] defines `name`, `startDate`, `endDate`, `description`, `url`, and `eventStatus`. These are the canonical web vocabulary for events and corroborate Mark’s field naming. Schema.org is a semantic-markup standard (JSON-LD/Microdata), not a visual representation format. **Not adoptable; useful vocabulary donor.**

W3C OWL-Time. OWL-Time [60] (W3C Recommendation, 2022) formalises temporal entities as `Instant` and `Interval` with object properties `hasBeginning`, `hasEnd`, and `hasTemporalDuration`. Crucially, omitting `hasEnd` explicitly represents an open/ongoing interval—the precise semantics of the IR’s `end`: `ongoing`. OWL-Time implements Allen’s thirteen interval relations [2] and is the authoritative formal reference for the IR’s open-ended activity semantics.

Allen’s Interval Algebra. Allen’s 1983 paper [2] establishes 13 binary relations between time intervals: *before*, *meets*, *overlaps*, *starts*, *during*, *finishes*, *equals*, and their converses. The algebra provides formal vocabulary for the uncertain/open date model: an activity with end: ongoing participates in the *started-by* relation with the document’s `time_range` without requiring a concrete end date. Not a format; the formal grounding for temporal semantics.

ISO 8601. ISO 8601 [24] is already the foundation of the date model (§0.16.5). The IR’s 2026-06-09, 2026-06, and 2026 forms are ISO 8601; the 2026-Q2 and 2026-H1 extensions follow the same YYYY-unit pattern as a natural extension. No further alignment is needed.

0.16.20.5 Scheduling and PM Formats

TaskJuggler [54] (GPL v2), MS Project XML [34], and Primavera P6 are scheduling grammars built around *computed* dates: dependency resolution, resource levelling, and critical-path analysis. All are explicitly out of scope per the Timeline Grammar decision. MS Project XML additionally fails the git-friendly criterion: the format is non-deterministic between saves (GUIDs and timestamps change on every write). No vocabulary from these formats merits borrowing beyond what iCalendar and schema.org already provide.

0.16.20.6 Analogous Document IRs

The Pandoc AST [26] is the strongest structural analogy: a typed, versioned, language-agnostic JSON document IR with a `pandoc-api-version` key, a `meta` block, and a sequence of typed block elements. This pattern—`version` + `metadata` + typed entity lists—maps directly onto the root structure of §0.16.3. The Dragon Book [1] provides theoretical backing for a typed, pipeline-stable IR as the contract between compilation stages. Both are *structural analogies*, not adoptable formats.

0.16.20.7 Comparison Table

Table 24 scores the seven strongest candidates against the five criteria. **Y** = good fit; **P** = partial; **N** = poor fit or not applicable.

No candidate scores **Y** on all five dimensions. Markwhen—the closest—fails on render/theme separation, lacks a stable machine-verifiable schema, and has no PPTX output path.

0.16.20.8 Recommendation: Build with Borrowed Vocabulary

Verdict: Build a bespoke IR. No existing format is adoptable wholesale and no viable extension profile exists.

Two orthogonal gaps eliminate every candidate:

1. **Semantic gap.** All formats with real communities (Mermaid, iCalendar, Vega-Lite) are scheduling-oriented, statistical-graphics-oriented, or calendar-oriented. None is a visual-communication roadmap grammar. Defining a “profile” of iCalendar or a “mark recipe” in Vega-Lite would require adding swimlanes, visual status, coarse-grain dates, PPTX output, and a theme engine as first-class concepts—at which point we have built a new layer regardless.
2. **Pipeline gap.** The requirement of a static, multi-backend render pipeline (SVG / PDF / PPTX) with a separate theme engine has no counterpart in any existing open-source tool. Mermaid, Markwhen, and vis-timeline all bake rendering into the format. Vega-Lite delegates to a JavaScript runtime, making it unsuitable for PPTX or print-PDF output without adaptation that would negate any adoption benefit.

A BUILD decision is therefore justified. It must not be a wheel-reinvention: the following **vocabulary and semantic donors** are explicitly leveraged so that the IR speaks standard language wherever standards exist.

ISO 8601 [24]

Date string syntax (2026-06-09, 2026-Q2). Already used in the date model (§0.16.5); no change needed.

iCalendar RFC 5545 [8]

Field naming: DTSTART → start, DTEND → end, SUMMARY → label, DESCRIPTION → description, CATEGORIES → tags/category, URL → url. Status vocabulary: TENTATIVE appears verbatim in the Status enum; CANCELLED maps to cancelled.

Schema.org/Event [50]

Confirms name/startDate/endDate/ description/url as the canonical web vocabulary for events. Mark’s label/start/end are direct equivalents (shorter names preferred for human authoring).

W3C OWL-Time [60] and Allen’s Algebra [2]

Open-interval semantics: omitting hasEnd denotes an ongoing interval, validating end: ongoing. The Instant/Interval duality formalises the Milestone (instant) vs. Activity (interval) distinction.

Vega-Lite [48]

Architectural precedent: strict WHAT/HOW separation between a declarative specification (this IR) and the rendering runtime (Theme Engine + Renderer). The pattern of a versioned, typed YAML/JSON document as the sole contract between source and output is adopted from Vega-Lite’s design.

Markwhen [29] and Mermaid [31]

Surface-syntax precedents: readable event lines, section/group structure, tag-based categorisation. These inform future higher-level authoring syntaxes that compile down to this IR.

Pandoc AST [26]

Structural analogy: versioned, typed, language-agnostic document IR with version + meta + typed entity lists. Validates the root structure (§0.16.3) and the principle of a stable, machine-verifiable schema.

0.16.20.9 Alignment of Mark’s IR with Prior Art

Table 25 maps Mark’s field names to their closest standard equivalents. Fields marked *Original* have no adequate standard analog and are justified by the visual-communication use case.

Flags for Mark and Leslie (no schema changes required).

- **No schema changes are recommended.** Mark’s field choices are well-aligned with iCalendar RFC 5545, schema.org/Event, and OWL-Time. The original contributions (*at-risk*, *blocked*, *span*, *progress*, *track*) are justified by the visual-communication use case and have no adequate standard equivalent.
- **Document vocabulary donors in the spec preamble.** This section serves that purpose. Future contributors should not propose renames (e.g., *name* for *label*, *startDate* for *start*) without consulting this survey, since shorter names are deliberate and standards-corroborated.
- **Optional surface alias type: event:** schema.org uses *Event* and OWL-Time uses *Instant* for point-in-time items. Exposing type: *event* as a surface-syntax alias for milestones could ease LLM generation without changing the IR schema. Flagged for future surface-language design.

0.17 Timeline Layout Engine

This section specifies the layout engine for the Timeline Grammar—the deterministic transformation from Timeline Domain IR to Scene IR. The layout engine is grammar-specific; the rendering backends (Section 0.11) are shared across all grammars.

0.17.1 Layout Pipeline Overview

The Timeline layout engine defines a *deterministic mapping* from a Timeline IR document and a theme to Scene IR geometry: positions, dimensions, label placements, and visual treatments. “Deterministic” is not aspirational; it is a hard contract. Two conforming layout engines, given identical IR and theme, must produce geometrically identical Scene IR [48].

The engine is organized as an ordered six-phase pipeline. Each phase consumes only outputs of preceding phases and produces geometry for later phases. The pipeline is *pure*: no phase reads from the IR after its designated turn, and no phase performs I/O, reads a system clock, or invokes a random number generator.

0.17.2 Determinism Contract

Two conforming layout engines R_1 and R_2 , given the same IR document D and theme θ , must produce identical coordinate values, label positions, and visual-treatment assignments. This contract requires:

1. **Pure function.** Output is a pure function of (D, θ) . No external state—timestamps, random values, environment variables, system locale, or screen resolution—influences any computed value.
2. **Stable sort keys.** Every ordered collection is sorted by an explicit, unambiguous key sequence. Collections without semantic ordering sort by `id` alphabetically:
 - Tracks: by `index` ascending (integer, unique by well-formedness invariant).
 - Activities within a track: by `(start_ordinal, id)` ascending.
 - Milestones: by `(date_ordinal, id)` ascending.
 - Annotations, sections, groups: by `id` ascending.

The ID values break all ties; global uniqueness is guaranteed by the IR invariant.

3. **Fixed rounding.** All intermediate floating-point values are rounded using *round-half-up* ($\lfloor v + 0.5 \rfloor$). Scene geometry values are expressed to two decimal places using the same convention. No other rounding rule is used anywhere in the pipeline.
4. **Deterministic symbolic date resolution.** The symbolic date now and relative dates (`+3m`, `-2w`) resolve to a fixed anchor. The anchor is: `metadata.today` if present; else `metadata.created`; else a validation error (see Gap 1 in §0.17.8). The system clock is never consulted.
5. **Embedded font metrics.** Label-width and line-height computations use metrics from font files bundled with the theme. System fonts are not consulted for any layout-critical measurement. This guarantees that label placement is identical on macOS, Linux, and Windows.
6. **Specification-version governance.** The renderer reads `version` from the IR document and verifies it against a supported specification version before beginning layout. A version mismatch is a hard error, not a warning.
7. **Layered determinism.** The determinism contract applies at three levels. (i) *Scene geometry* is always byte-deterministic: the layout pipeline output is bitwise-reproducible across runs, platforms, and renderer implementations. (ii) *Per-backend output* is deterministic given a pinned backend version and fixed effect seeds; the backend version is an explicit parameter of the contract. (iii) *Cross-backend pixel identity* is explicitly *not* promised: the SVG backend and the Raster backend are permitted and expected to produce visually different outputs from the same Scene. This is correct behaviour at different fidelity tiers, not a defect. See Section 0.10.4 for the full layered contract.

Consequence. If R_1 and R_2 both satisfy this contract and their outputs diverge, the divergent renderer has a defect. The specification is the arbiter.

0.17.3 Coordinate System

The renderer works in a *logical-pixel* coordinate system. The top-left corner of the canvas is the origin $(0, 0)$; x increases rightward, y downward. All layout computations use integer arithmetic in logical pixels; the Scene/Render IR records decimal-valued coordinates as its output (see Section 0.17.6).

The canvas *width* is fixed by the theme. The canvas *height* is always computed from track count and sub-lane expansion; it must never be truncated.

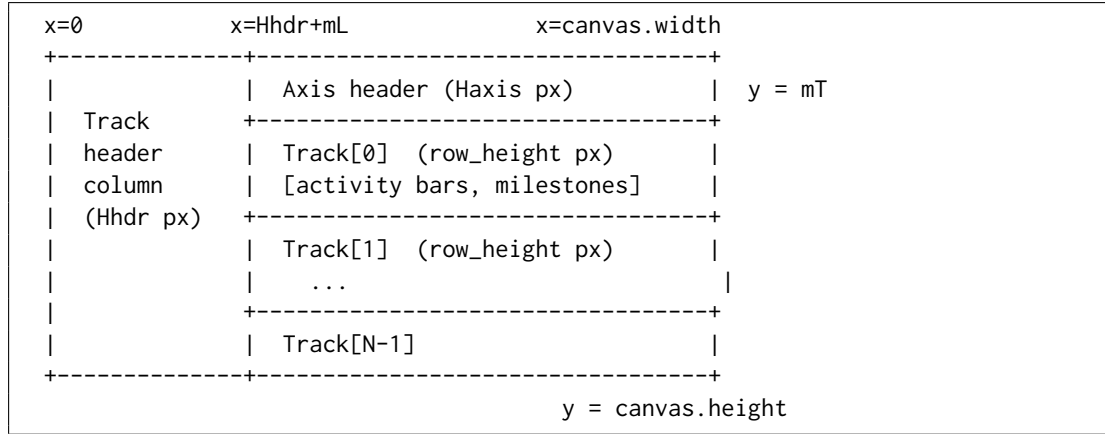


Figure 6: Canvas coordinate system and layout zones

0.17.4 Timeline Axis

0.17.4.1 Axis Unit and Inference

`metadata.axis_unit` controls tick granularity. If absent, the renderer infers a suitable unit from the `time_range` span, applying the first matching threshold:

These thresholds yield 4–26 visible ticks at a 1200-pixel canvas width—the range that avoids crowding while remaining informative for the executive-roadmap use cases described in Section 0.3.

0.17.4.2 Date-to-Coordinate Function

All horizontal positions derive from a single deterministic function $x(T)$. Let T_{ord} denote the *day ordinal* of date T : an integer count of days since 2000-01-01 (epoch day 0). Coarser-precision dates are resolved to their period-start day before ordinal conversion (see §0.17.4.3).

$$x(T) = H_{\text{hdr}} + m_L + \left\lfloor \frac{(T_{\text{ord}} - T_{s,\text{ord}}) \cdot W_{\text{draw}}}{T_{e,\text{ord}} - T_{s,\text{ord}}} + 0.5 \right\rfloor$$

where $T_{s,\text{ord}}$ and $T_{e,\text{ord}}$ are the day ordinals of the axis domain start and end after coercion. The numerator product $(T_{\text{ord}} - T_{s,\text{ord}}) \cdot W_{\text{draw}}$ is computed before division to prevent precision loss. For a 1200-pixel canvas spanning ten years (3650 days), the intermediate product is at most $1200 \times 3650 = 4\,380\,000$, well within 32-bit integer range.

Boundary invariants. $x(T_s) = H_{\text{hdr}} + m_L$ exactly; $x(T_e) = H_{\text{hdr}} + m_L + W_{\text{draw}}$ exactly. Any date at or before T_s maps to the left canvas boundary; any date at or after T_e maps to the right canvas boundary, before clipping.

0.17.4.3 Date Coercion for Bar Edges

Dates coarser than `day` must be coerced to a specific calendar day before ordinal conversion. The rule depends on whether the date is a *left edge* (start of a bar) or a *right edge* (end of a bar):

Fiscal dates require a fiscal-year start month that is currently absent from the IR contract (see Gap 2 in §0.17.8).

The same left-edge/right-edge coercion applies to `time_range.start` (left) and `time_range.end` (right) when establishing the axis domain.

0.17.4.4 Tick and Gridline Placement

1. Enumerate all `axis_unit` period-start dates within $[T_s, T_e]$ inclusive.
2. For each period boundary T_k : compute $x_k = x(T_k)$ per §0.17.4.2.
3. Place a tick mark of height `theme.axis.tick_height` px at x_k , at the bottom of the axis header band.
4. Place a vertical gridline at x_k spanning from the top of `Track[0]` to the bottom of `Track[N - 1]`, styled per `theme.axis.gridline_*` properties.
5. Place a tick label centered at x_k , `theme.axis.tick_label_offset` px below the tick. Format using `metadata.locale` and the label rules in Table 29. If a label would overlap its neighbor (right edge > neighbor's left edge), suppress the label for this tick only.

Week ticks. For `axis_unit = week`, week boundaries are ISO 8601 Monday starts regardless of locale. The locale affects label string formatting only (language, order), never tick positions.

Section bands. If `sections[]` is non-empty, the theme may render alternating column shading between section boundaries. Band edges coincide with section start/end dates coerced per Table 28. Section bands are painted at the lowest z-order, below gridlines and all content elements.

0.17.5 The Six-Phase Layout Algorithm

The renderer executes geometry computation in exactly six phases in the order given below. Each phase's output is immutable input to subsequent phases. No phase may invoke logic from a later phase.

1	Phase 1		AXIS COMPUTATION
2			Input: IR metadata (time_range, axis_unit, today-anchor)
3			Output: axis domain [Ts, Te], axis_unit, tick_positions[]
4			
5	Phase 2		TRACK PLACEMENT
6			Input: tracks[] sorted by index
7			Output: track.y_top[], track.height[] (provisional)
8			

```

9 Phase 3 | ACTIVITY GEOMETRY
10       |   Input:  activities[], Phase-1 axis, Phase-2 track
           |   positions
11       |   Output: activity.{x_left, x_right, y, lane, width}
12       |           track.height[] (final, expanded for sub-lanes)
13
14 Phase 4 | MILESTONE GEOMETRY
15       |   Input:  milestones[], Phase-1 axis, Phase-3 final track
           |   heights
16       |   Output: milestone.{x_center, y_center, stack_index}
17
18 Phase 5 | SECTIONS AND ANNOTATIONS
19       |   Input:  sections[], annotations[], all Phase 1-4 geometry
20       |   Output: section.{x_left, x_right}, annotation.{x, y}
21
22 Phase 6 | LABEL COLLISION RESOLUTION
23       |   Input:  all geometry from Phases 3-5
24       |   Output: final label positions for milestones

```

Listing 32: Layout pipeline overview

0.17.5.1 Phase 1: Axis Computation

```

1 1. Resolve today-anchor:
2   if metadata.today is set : anchor = metadata.today
3   elif metadata.created is set : anchor = metadata.created
4   else : ERROR("Cannot resolve symbolic/relative dates: no
           today/created")
5
6 2. Resolve time_range.start:
7   a. Expand 'now' -> anchor; expand '+3m' -> anchor + 3 months;
           etc.
8   b. Coerce to period start (left-edge rule, Table 5.3). -> T_s
9
10 3. Resolve time_range.end (same expansion; coerce to period end ->
    T_e).
11   ASSERT T_e >= T_s; else ERROR.
12
13 4. Infer axis_unit if absent:
14   span_days = T_e_ord - T_s_ord
15   Apply Table 5.2 thresholds in order; assign axis_unit.
16
17 5. Compute W_draw = canvas.width - mL - mR -
    theme.track.header_width.
18
19 6. Enumerate tick positions:
20   T_k = all period-start dates of axis_unit in [T_s, T_e]
           inclusive.
21   For each T_k: x_k = x(T_k) per Section 5.3.2.
22   Compute tick label strings using metadata.locale.

```

Listing 33: Phase 1: Axis computation

0.17.5.2 Phase 2: Track Placement

```

1 1. Sort tracks by index ascending.
2   ASSERT all index values are unique non-negative integers.
3
4 2. y_cursor = mT + Haxis
5
6 3. For i in 0 .. N_tracks-1:
7     track[i].y_top = y_cursor
8     track[i].height = theme.track.row_height      -- provisional
9     y_cursor      += theme.track.row_height + theme.track.row_gap
10
11 4. Record group bracket extents after Phase 3 finalises track
    heights.

```

Listing 34: Phase 2: Track placement

Track heights set here are provisional. Phase 3 may expand them when overlapping activities require sub-lanes. Downstream phases must use the Phase-3-finalised heights.

0.17.5.3 Phase 3: Activity Geometry

Sub-lane assignment.

```

1 For each track T (in index order):
2   1. A = activities where A.track == T.id.
3   2. Resolve each A.start_ord = ordinal(coerce_left(A.start)).
4     Resolve each A.end_x per special-end-date rules below.
5   3. Sort A by (start_ord, id) ascending (stable sort).
6   4. Greedy sub-lane assignment:
7     lane_end_x = [-infinity]
8     For each A_i in sort order:
9       k = smallest index where lane_end_x[k] <= x(A_i.start_ord)
10      if no such k exists:
11        if len(lane_end_x) < theme.track.max_sub_lanes:
12          append -infinity to lane_end_x; k = new last index
13        else:
14          k = theme.track.max_sub_lanes - 1  -- cap; emit WARNING
15          A_i.lane = k
16          lane_end_x[k] = A_i.end_x
17 5. n_lanes = max(A.lane) + 1
18 6. if n_lanes > 1:
19     track[T].height = theme.track.row_height
20                     + (n_lanes - 1) * theme.track.sub_lane_height
21 7. For each A_i:
22     A_i.y = track[T].y_top
23           + theme.activity.bar_top_margin
24           + A_i.lane * theme.track.sub_lane_height
25     A_i.x_left = x(A_i.start_ord)
26     A_i.x_right = A_i.end_x
27     logical_w = A_i.x_right - A_i.x_left
28     if logical_w < theme.activity.min_width:
29       mid = (A_i.x_left + A_i.x_right) / 2  --
30         round-half-up
31       A_i.x_left = mid - theme.activity.min_width / 2
32       A_i.x_right = A_i.x_left + theme.activity.min_width
33     A_i.width = A_i.x_right - A_i.x_left

```

Listing 35: Phase 3: Activity geometry and greedy sub-lane assignment

Special end-date resolution.

- **end absent** (omitted): treated identically to **ongoing**.
- **ongoing**: $\text{end_x} = H_{\text{hdr}} + m_L + W_{\text{draw}}$ (right canvas boundary); append right-chevron ($\rightarrow$) decoration.
- **tbd**: $\text{end_x} = \text{x_left} + \text{theme.activity.tbd_extension_px}$ (default: $2 \times \overline{\Delta x_{\text{tick}}}$, the mean tick spacing in pixels); render rightward extension with dashed border at 50% opacity; place “TBD” label inside extension zone.
- **unknown**: same geometry as **tbd**; use unknown visual treatment from theme status map.
- **~date**: coerce date normally to ordinal; $\text{end_x} = x(\text{date_ord})$; flag right edge as approximate for gradient-fade rendering.

Label placement (deterministic three-way rule). The rule is applied exactly once per activity; no iteration:

1. **Inside**: if $A.\text{width} \geq \text{theme.activity.label_inside_min_width}$, render label horizontally centered in bar, vertically centered on bar height, in $\text{theme.activity.label_color_inside}$.
2. **Right**: render label starting at $A.\text{x_right} + 4$ px in $\text{theme.activity.label_color_outside}$. If label right edge $> H_{\text{hdr}} + m_L + W_{\text{draw}} - 4$, proceed to step 3.
3. **Left**: render label ending at $A.\text{x_left} - 4$ px. If label left edge $< H_{\text{hdr}} + m_L$, truncate label to $\text{theme.activity.label_truncate_chars}$ characters + “...” (U+2026) and re-attempt step 2.

Progress indicator. If activity.progress is set: render a filled strip of height $\text{theme.activity.progress_bar_height}$ px at the bottom of the bar, width = $\lfloor A.\text{width} \cdot A.\text{progress} + 0.5 \rfloor$ px, inset inside the bar border.

0.17.5.4 Phase 4: Milestone Geometry

Placement and stacking.

```

1 For each milestone M (sorted by date_ord, id):
2   1. x_center = x(M.date_ordinal).
3   2. Determine base_y:
4       if M.track is set:
5         base_y = track[M.track].y_top + track[M.track].height / 2
6       else (cross-track):
7         base_y = mT + H_axis + H_draw / 2
8
9   3. Stacking within (effective_track_id, x_center) group:
10      Sort group members by id ascending.
11      Assign stack_index k = 0, 1, 2, ...
12      M.y_center = base_y + k * theme.milestone.stack_offset_y
13
14   4. Diamond vertices (integer arithmetic):
15      d = theme.milestone.diamond_size
16      top    = (x_center,      M.y_center - d)
17      right  = (x_center + d, M.y_center    )
18      bottom = (x_center,      M.y_center + d)
19      left   = (x_center - d, M.y_center    )

```

Listing 36: Phase 4: Milestone placement and stacking

The diamond is a four-vertex polygon with vertices computed directly. No `transform="rotate(...)"` attribute is used; rotation-transform-dependent rendering differences across SVG viewers are avoided.

Cross-track milestones. When `track` is null, the milestone is drawn at the vertical centre of the full rendered area. Its diamond size is `theme.milestone.diamond_size` (not scaled); visual emphasis comes from the spanning vertical extent of the label, which may be increased by a theme multiplier `theme.milestone.cross_track_label_scale` (default: 1.2).

0.17.5.5 Phase 5: Sections and Annotations

Section bands. For each section S (sorted by start date, then id):

- $x_{\text{left}}(S) = x(\text{coerce_left}(S.\text{start}))$.
- $x_{\text{right}}(S) = x(\text{coerce_right}(S.\text{end}))$.
- Band spans vertically from $m_T + H_{\text{axis}}$ to $m_T + H_{\text{axis}} + H_{\text{draw}}$.
- Z-order: sections paint first (below gridlines, activities, milestones, labels).

Overlapping sections are valid; a later section in sort order paints over an earlier one.

Annotation placement.

1. Sort annotations by id ascending.
2. For each annotation: compute anchor as centre of target entity's bounding box.
3. Place annotation box in the first quadrant that keeps it entirely within the canvas. Preference order: top-right, top-left, bottom-right, bottom-left. If no quadrant fits, use top-right and clip to canvas boundary.
4. Draw connector (line/elbow) from annotation box edge to anchor, styled per `theme.annotation.connector`.

0.17.5.6 Phase 6: Label Collision Resolution

Activity labels are placed definitively in Phase 3 (no iteration). Milestone labels require a collision-resolution pass because multiple milestones at similar x positions can produce overlapping labels.

```

1 1. Collect all milestone label bounding boxes (x_label, y_label,
   width, height, id).
2 2. Sort by (x_label, y_label, id) ascending (stable).
3 3. Scan pass (bounded by N iterations, N = label count):
4   For each consecutive pair (L_i, L_{i+1}):
5       if L_i.right_edge > L_{i+1}.left_edge
6           AND |L_i.y_label - L_{i+1}.y_label| < label_height:
7           L_{i+1}.y_label += theme.milestone.label_stack_offset

```

```

8 4. Repeat step 3 until no overlapping pairs remain OR N iterations
   reached.
9   If overlaps persist after N iterations: truncate labels to
10  theme.milestone.label_max_width px, accepting visual overlap.
11 5. Clamp all final y_label values to [mT, mT + Haxis + H_draw + mB].

```

Listing 37: Phase 6: Milestone label collision resolution

Because the input sort and every shift decision are deterministic, two renderers executing this algorithm on identical inputs produce identical label placements.

0.17.6 Scene / Render IR — Output of the Pipeline

The six-phase layout pipeline does not target any output format directly. Its output is the **Scene / Render IR**: a byte-deterministic, backend-agnostic record of every drawing primitive, resolved coordinate, visual treatment, and effect request. The Scene is described fully in Section 0.10; its role here is to name what the pipeline produces and to record what each phase contributes.

Every value computed in Phases 1–6 becomes a field of a Scene drawing primitive or a top-level Scene property. The Scene is a pure data record: it carries no rendering instructions specific to SVG, rasterisation, or PowerPoint. Rendering backends (SVG, Raster, PPTX, HTML) consume the Scene and emit their respective formats; they are described in Section ??.

What the Scene preserves from the pipeline.

- **Phase 1** outputs: axis domain $[T_s, T_e]$, axis unit, tick positions x_k , and tick label strings.
- **Phases 2–3** outputs: all activity bar coordinates $(x_{\text{left}}, x_{\text{right}}, y, \text{width})$, lane assignments, and final track heights.
- **Phase 4** outputs: all milestone vertex coordinates and stack indices.
- **Phase 5** outputs: section band bounds $(x_{\text{left}}, x_{\text{right}})$, annotation positions, and connector endpoints.
- **Phase 6** outputs: final label positions after collision resolution.
- **Theme-resolved visual treatments**: fill colours, stroke styles, opacity, corner radii, pattern references, and effect requests with their fallback policies (see Section ??).

The Scene does not carry the IR document or theme; it is the fully resolved, self-contained rendering specification. A backend needs only the Scene and its own capability profile to produce output.

0.17.7 Edge Cases

Every case below is normative. A renderer that produces different behaviour for any case has a defect. The summary table gives the complete catalogue; extended descriptions follow for the most structurally complex cases.

Table 30: Edge-case definitions (normative)

Case	Condition	Rendering Rule
Zero-duration activity	<code>start == end</code> or <code>span</code> that resolves to a single point	Render bar of <code>min_width</code> px centred at $x(\text{start})$. Never render 0-width. Label placed right (outside).
Overlapping activities (same track)	Two activities in same track whose date ranges overlap	Greedy sub-lane assignment (Phase 3). Track height expands. Cap at <code>max_sub_lanes</code> .
Open interval (ongoing)	<code>end: ongoing</code> or <code>end</code> absent	Bar extends to right canvas boundary. Right-chevron decoration. No overflow beyond canvas edge.
TBD end date	<code>end: tbd</code>	Bar extends <code>tbd_extension_px</code> beyond <code>x_left</code> . Dashed, 50% opacity. “TBD” label inside extension.
Both dates TBD	<code>start: tbd</code> or equivalent	Full-track-width hatched block at 40% opacity. Activity label + “(TBD)”.
Unknown start	<code>start: unknown</code>	Left edge placed at $x(T_s)$. Left-fade gradient over <code>approx_fade_px</code> .
Unknown end	<code>end: unknown</code>	Same geometry as <code>tbd</code> ; rendered with <code>unknown</code> visual treatment from theme.
Approximate start	<code>start: ~date</code>	Geometry at nominal date; left edge rendered with gradient fade of <code>approx_fade_px</code> .
Approximate end	<code>end: ~date</code>	Geometry at nominal date; right edge rendered with gradient fade.
Activity fully before range	<code>end < T_s</code>	Not rendered. Renderer warning. Activity still appears in legend.
Activity fully after range	<code>start > T_e</code>	Not rendered. Renderer warning. Activity still appears in legend.
Activity partially before range	<code>start < T_s, end ≥ T_s</code>	Bar clipped to $[x(T_s), x(\text{end})]$. Left-clip indicator on clipped edge.
Activity partially after range	<code>start ≤ T_e, end > T_e</code>	Bar clipped to $[x(\text{start}), x(T_e)]$. Right-clip indicator on clipped edge.
Very short span	$x(\text{end}) - x(\text{start}) < \text{min_width}$ after rounding	Render at <code>min_width</code> ; centre on logical midpoint (round-half-up). Label right (outside).
Very long span	Bar occupies $> 80\%$ of W_{draw}	Render normally. Prefer inside label placement if label fits within bar.
Simultaneous milestones (same track)	Two milestones with equal <code>date</code> on the same <code>track</code>	Stack vertically by <code>stack_offset_y</code> , sorted by <code>id</code> ascending (Phase 4).

continued ...

Table 30 continued

Case	Condition	Rendering Rule
Simultaneous milestones (cross-track)	Two milestones with <code>track: null</code> and equal <code>date</code>	Stack at full-timeline vertical centre, sorted by <code>id</code> ascending.
Milestone outside range	Milestone date $< T_s$ or $> T_e$	Not rendered. Renderer warning. Appears in legend.
Empty track	Track has no activities and no milestones	Row rendered at <code>row_height</code> . Label visible. Body empty. Never collapsed.
Sub-lane cap exceeded	Overlapping activities require $> \text{max_sub_lanes}$ lanes	Excess activities placed in the last lane (visible overlap). Renderer warning.

Overlapping activities: sub-lane assignment in detail. The greedy algorithm assigns each activity (in stable (`start_ord`, `id`) order) to the lowest-indexed lane where it does not overlap any previously placed activity. Overlap is defined as $x_{\text{left}}(A_j) < x_{\text{right}}(A_i)$ for activities A_i , A_j in the same lane. Because sort order is deterministic and each assignment is a pure function of the current `lane_end_x` state, all renderers produce identical lane assignments for identical inputs.

Simultaneous milestones in detail. When milestones share an (`track`, x_{center}) pair, they are stacked downward from the nominal track centre. The first milestone in `id`-ascending order is at the base position (stack index 0). Subsequent milestones are offset by $k \times \text{theme.milestone.stack_offset_y}$. Stacking is permitted to overflow into the inter-track gap; milestones must *not* be silently suppressed if they cannot fit within track height.

Clip indicators. An activity bar clipped at the axis boundary receives a *clip indicator*: a small “//” mark (two angled lines, 4×8 px) drawn in the activity’s fill colour at full opacity, positioned at the clipped edge centre. The indicator signals to the reader that the activity continues beyond the visible axis range.

Minimum-width semantics. `theme.activity.min_width` (default: 4 px) prevents activities from being rendered invisibly narrow at coarse zoom levels. A zero-duration activity and a very-short-span activity both produce a bar of exactly `min_width` px. The two cases are *visually indistinguishable*; distinguishing them would require a separate icon or tooltip, which is a theme-level option outside the core rendering contract.

0.17.8 Known IR Gaps

The following gaps in Mark’s IR contract (§??) affect rendering determinism. They are flagged here for Mark and Leslie to resolve; this spec does *not* unilaterally extend the IR. Renderers encountering these gaps should apply the documented fallback and emit a validation warning.

Gap 1 — `metadata.today` field missing.

Leslie’s decision record mandates an explicit `today` field in the IR for deterministic resolution of the now symbolic date and the today-marker feature. Mark’s current `metadata` object does not include this field. **Proposed resolution:** add `today` (type `date`, optional) to `metadata`. Renderer fallback chain: `metadata.today` → `metadata.created` → validation error.

Gap 2 — `metadata.fiscal_year_start` missing.

The FY26-Q2 fiscal-date format requires knowing the fiscal-year start month. Two renderers assuming different organisations’ fiscal calendars (US federal: October; UK: April; common corporate: January) will produce different x -coordinates for identical fiscal dates. **Proposed resolution:** add `fiscal_year_start` (type `int`, range 1–12, default 1 = January) to `metadata`.

Gap 3 — Relative date anchor undefined.

Relative dates (+3m, -2w) are described in Mark’s contract as resolving from “context”, which is insufficient. A renderer must know the anchor date to compute an ordinal. **Proposed resolution:** define that relative dates use the same anchor as Gap 1 (`today` → `created` → error).

Gap 4 — Omitted end semantics ambiguous.

Mark’s Activity contract marks `end` as optional but does not state whether an absent `end` is equivalent to `ongoing` or a validation error. This rendering model treats omitted `end` as `ongoing` (see the open-interval row in Table 30). The IR spec should make this explicit.

0.18 Flow Grammar: Domain IR

The Flow Grammar is the second grammar implemented in the diagram compiler and the **template grammar** for all future grammar additions. Where the Timeline Grammar (Section 0.16) proved the kernel’s Scene IR contract with a time-axis-centric vocabulary, the Flow Grammar proves that the kernel is genuinely grammar-agnostic: it supports node-link topologies, pipeline sequences, and process flows—fundamentally different visual semantics—without kernel modifications.

Implementation Status. Fully implemented in `packages/core/src/grammars/flow/` with Sugiyama four-phase layered layout (cycle detection, longest-path rank assignment, barycenter-based crossing minimization, Brandes–Köpf coordinate assignment). FlowTheme with light and dark variants. **Deferred:** general non-layered layouts (force-directed, stress-majorization).

0.18.1 Scope & Intent

0.18.1.1 What a Flow Diagram IS

A **flow diagram** in this grammar is a *directed node-link diagram* representing:

- **Pipelines/sequences:** ordered chains of processing steps (e.g., CI/CD, RAG retrieval, ETL).

- **Process flows:** multi-path directed graphs with branching and joining (e.g., decision trees, agent loops, state machines).
- **Architecture topologies:** layered or grouped node-link structures (e.g., microservice communication, data flow).

These correspond to the *node-link/graph* family’s *topology* and *pipeline/sequence* kinds (Section 0.15.2). The animated-arrow variant (flowing dashes along connectors) is a first-class pattern in this grammar.

0.18.1.2 What a Flow Diagram is NOT

Not a comparison/matrix.

Tabular grid layouts with cells (Section ??) belong to the Comparison grammar. Flows have directed edges; comparisons do not.

Not a timeline.

The timeline grammar’s entities have temporal semantics (**start**, **end**, **track**). Flow nodes have no time axis.

Not a general graph.

The Graph grammar (Section ??) covers undirected networks, trees with Reingold-Tilford layout, and hub-and-spoke topologies. The Flow grammar is restricted to *directed* structures with a declared layout intent—it does not attempt general force-directed placement.

0.18.1.3 Modeling Boundary

The Flow grammar covers diagrams where:

1. Entities are discrete **nodes** (processing steps, decisions, data stores).
2. Relationships are **directed edges** (data flow, control flow, sequence).
3. Layout admits a natural **directional reading order** (left-to-right, top-to-bottom).

Diagrams without directed edges, or without a dominant reading direction, are out of scope for this grammar.

0.18.2 Flow Domain IR

The Flow Domain IR is the small, grammar-specific representation that compiles *down* to the shared Scene IR (Section 0.10). It is **not** a “god-IR”—it contains only the constructs necessary to describe directed node-link diagrams with optional grouping.

0.18.2.1 Document Structure

A Flow IR document is a single YAML/JSON object with the following root structure:

```
1 version: "1.0"
2 metadata:
```

```

3  title: string          # required
4  subtitle: string       # optional; default null
5  theme: string          # optional; default "default-flow"
6  flow:
7    direction: left-to-right | top-to-bottom # required
8    nodes: [...]         # required; list<FlowNode>; may be empty
9    edges: [...]         # optional; list<FlowEdge>; default []
10   groups: [...]        # optional; list<FlowGroup>; default []

```

Listing 38: Flow IR root document structure

0.18.2.2 Root Fields

0.18.2.3 FlowNode

A node is a discrete entity in the flow: a processing step, decision point, data store, or any labeled box.

Identity. Node `id` is the primary key within a document. It must be unique across all nodes. References in edges and groups use this `id`. Two nodes with the same `id` are a validation error.

Ordering. Nodes are ordered by their position in the `nodes` list. This order is the **canonical order** used by the layout engine for tie-breaking (determinism) and by serialization for stable output.

0.18.2.4 FlowEdge

An edge represents a directed relationship between two nodes.

Identity. Edges are identified by position in the `edges` list (index). Unlike nodes, edges do not carry an explicit `id`—their identity is their list position. This list position is the canonical order for deterministic processing.

Directedness.

directed

Arrow from source to target (default).

bidirectional

Arrows on both ends.

undirected

No arrowheads (connector line only).

Port resolution. When `source_port` or `target_port` is `auto`, the layout engine assigns the port based on the relative positions of source and target nodes. For left-to-right flows, `auto` resolves to `right` on source and `left` on target for forward edges.

0.18.2.5 FlowGroup

Groups are optional visual containers (lanes, clusters, swim-lanes) that partition nodes.

Group membership resolution. A node belongs to a group if *either* it appears in the group’s nodes list *or* it has a `group` field referencing the group’s id. Both mechanisms are equivalent; using both for the same node–group pair is valid (idempotent). A node may belong to at most one group; dual membership is a validation error.

Ungrouped nodes. Nodes not assigned to any group are rendered in the main flow area, outside any group container. They participate in layout normally.

0.18.2.6 Layout Intent

The `flow.direction` field declares the dominant layout direction:

left-to-right

Nodes are ranked left-to-right; edges flow rightward. This is the default for pipeline/sequence diagrams.

top-to-bottom

Nodes are ranked top-to-bottom; edges flow downward. This suits vertical process flows.

The declared direction is a **hard constraint** on the layout engine: it determines layer-assignment orientation in the Sugiyama algorithm and reading order in the linear-sequence layout.

0.18.3 Layout Contract

The Flow grammar’s layout engine converts the Domain IR into Scene IR coordinates. The choice of algorithm depends on graph topology, but all algorithms must satisfy the **determinism contract**: identical IR in $\rightarrow$ byte-identical Scene out.

0.18.3.1 Layout Family Selection

Topology	Algorithm	When Applied
Linear chain	Linear sequence layout	All nodes have in-degree ≤ 1 and out-degree ≤ 1 (a simple path or set of disjoint paths).
DAG	Sugiyama layered [52]	Directed acyclic graph with branching/joining.
Cyclic	Sugiyama with cycle removal	Cycles detected; feedback arcs reversed before layering.

Topology detection is automatic. The layout engine inspects the graph structure and selects the appropriate algorithm. The author does not choose the algorithm—only the direction. This is a key difference from the Graph grammar (Section ??), which requires explicit layout selection.

0.18.3.2 Sugiyama Layered Layout (DAGs and Cyclic Flows)

The primary layout algorithm for non-trivial flows follows the four-phase Sugiyama framework [52]:

1. **Cycle removal.** Detect cycles via DFS. Reverse a minimal feedback arc set to produce a DAG. Reversed edges are rendered with a visual back-edge indicator (curved path routed behind the main flow).
2. **Layer assignment.** Assign each node to a layer (rank) using network simplex [17], minimizing total edge span. The direction field determines whether layers are columns (left-to-right) or rows (top-to-bottom).
3. **Crossing minimization.** Barycenter heuristic iterated over adjacent layer pairs (24 sweeps, per standard practice [52]). Ties broken by canonical node list order (determinism).
4. **Coordinate assignment.** Brandes & Köpf [5] linear-time algorithm: four candidate alignments, median selected. Guarantees at most two bends per long edge.

Edge routing. Edges are routed orthogonally where the Brandes–Köpf coordinate assignment produces aligned segments. For non-orthogonal cases (long spans, back-edges), edges use smooth cubic Bézier curves. The choice is deterministic: given fixed node positions, edge routing is a pure function.

Orthogonal routing follows the principles of Tamassia’s topology–shape–metrics framework [53]: edges consist of horizontal and vertical segments only, with bends minimized by the coordinate-assignment phase. This produces clean, architecture-diagram-style connectors.

0.18.3.3 Linear Sequence Layout (Pipelines)

When the flow graph is a simple chain (every node has at most one predecessor and one successor), the layout engine uses a trivial linear placement:

- Nodes placed at equal intervals along the primary axis (determined by direction).
- Edges are straight horizontal (LTR) or vertical (TTB) connectors.
- Groups (if present) span the extent of their member nodes.

This produces clean, minimal pipeline diagrams without the overhead of Sugiyama layer assignment.

0.18.3.4 Determinism Mandate

All flow layouts are fully deterministic. Specifically:

1. No randomized initialization. No force-directed simulation. No RNG.
2. Tie-breaking uses the canonical node order (list position in the IR).
3. Crossing minimization uses a fixed sweep count (24), not convergence-based termination.
4. The same IR document, processed by the same engine version, produces byte-identical Scene IR on every execution.

If a force-like aesthetic is ever required for a flow diagram (not anticipated for v1), the implementation **must** use stress majorization [18] with a deterministic initial layout (nodes on a circle in canonical order) and a fixed iteration count. Raw Fruchterman-Reingold [16] with random initialization is prohibited.

Stress majorization rationale. Stress majorization (SMACOF) is a pure function of the graph-distance matrix given a fixed initial configuration. It converges monotonically (no oscillation), producing identical results across runs [18]. This is the only acceptable “force-like” family for a deterministic compiler.

0.18.4 Lowering to the Scene IR

The flow layout engine emits Scene IR primitives (Section 0.10). This section specifies the mapping from flow constructs to Scene primitives. **No new Scene IR primitives are required**—the existing kernel is sufficient.

0.18.4.1 Node Lowering

Flow Shape	Scene IR Primitive(s)
rect	Rect{corner_radius: 0}
rounded-rect	Rect{corner_radius: theme.flow.nodeRadius}
circle	Circle
diamond	Path (four-point rhombus via M/L/L/L/Z commands)
stadium	Rect{corner_radius: height/2} (pill shape)

Each node emits a `Group` containing:

1. The shape primitive (background/border).
2. A `Text` primitive for the label (centered within the shape).
3. Optionally, an `Image` primitive for the icon (positioned above or left of the label, per theme).

Status → **color**. The node’s `status` field is resolved by the theme to `fill` and `stroke` colors on the shape primitive. The Domain IR does *not* carry color values—color is always theme-resolved.

0.18.4.2 Edge Lowering

Each edge emits:

1. A `Path` primitive for the connector line (routed by the layout engine). Stroke style (solid, dashed, dotted) maps directly to the Scene Path's `stroke_style` field.
2. An arrowhead: a small filled `Path` (triangle) at the target end for `directed` edges; at both ends for `bidirectional`; omitted for `undirected`.
3. Optionally, a `Text` primitive for the edge label, positioned at the midpoint of the path (offset perpendicular to the edge direction to avoid overlap).

Animated edges. When `animated: true`, the edge's `Path` primitive receives a `FlowingDashes` animation hint (Section 0.10.3):

```
FlowingDashes {
  target_id: <edge_path_id>,
  dash_length: theme.flow.animDashLength,    // default 8px
  gap_length:  theme.flow.animGapLength,     // default 4px
  speed_px_per_sec: theme.flow.animSpeed,    // default 40
  direction: 'forward'
}
```

SVG/HTML backends render this as `stroke-dashoffset` animation. Raster backends (PNG/Skia) render the resting frame: a static dashed stroke.

0.18.4.3 Group Lowering

Groups emit:

1. A background `Rect` spanning the bounding box of all member nodes (with padding from theme).
2. A `Text` primitive for the group label (positioned at the top of the rect).
3. All member nodes rendered as children of a `Group` primitive (for logical containment, not visual nesting—nodes are positioned absolutely).

No new primitives needed. The flow grammar maps entirely to existing Scene IR primitives: `Rect`, `Circle`, `Path`, `Text`, `Image`, and `Group`. No kernel changes are required.

Open question for Barbara/Mark: If future flow diagrams require *nested* groups (groups within groups), should the Scene IR `Group` primitive support recursive clip regions, or should nesting be flattened at lowering time? Current spec assumes single-level groups only. Nested groups are deferred to v2.

0.18.5 Edge Cases & Total Semantics

Every possible input must have a defined meaning or produce a clear validation error. This section pins down the edge cases.

0.18.5.1 Cycles

Cycles are valid. A flow may contain cycles (e.g., retry loops, feedback arcs in agent pipelines). The layout engine handles cycles by:

1. Detecting strongly-connected components via Tarjan’s algorithm.
2. Selecting a minimal feedback arc set (edges to reverse for layering). Selection is deterministic: among equally-scored candidates, the edge appearing *later* in the canonical edge list is chosen for reversal.
3. Reversed edges are rendered as *back-edges*: routed with a longer path (curved or stepped) that visually indicates “going back” in the flow direction.

The resulting diagram remains readable: forward edges flow in the declared direction; back-edges are visually distinct.

0.18.5.2 Self-Loops

Self-loops are valid. An edge where `source == target` is rendered as a loopback connector: a curved path that exits the node from one port and re-enters at an adjacent port (e.g., exits *right*, loops around, enters *top*). The specific port pair is determined by the flow direction and node position.

0.18.5.3 Multi-Edges

Multiple edges between the same pair of nodes are valid. Each edge is rendered as a separate path, offset perpendicular to the primary path direction to avoid visual overlap. Offset distance is `theme.flow.multiEdgeGap` (default: 6px). Edges are offset in canonical order (list position).

0.18.5.4 Disconnected / Orphan Nodes

Nodes with no edges are valid. Orphan nodes (zero in-degree and zero out-degree) are placed in a designated “orphan region”—below the main flow (for LTR) or to the right (for TTB)—sorted by canonical order. They participate in group membership normally.

Disconnected components (multiple disjoint subgraphs) are each laid out independently using the appropriate algorithm, then arranged sequentially along the secondary axis (vertical for LTR, horizontal for TTB).

0.18.5.5 Missing Target/Source ID

Validation error. If an edge references a node id that does not exist in the nodes list, the document fails validation with a diagnostic: “Edge at index N references unknown node id ‘X’ in field ‘source|target’”. The layout engine does not process invalid documents.

0.18.5.6 Empty Flow

Valid but degenerate. A flow with zero nodes and zero edges produces a Scene with only the canvas and metadata (no drawing primitives). This is analogous to the timeline grammar's empty-activities case. A lint warning is emitted: "Flow contains no nodes; output will be empty."

0.18.5.7 Duplicate Node IDs

Validation error. Two nodes with the same id fail validation: "Duplicate node id 'X' at indices M and N."

0.18.5.8 Edge Referencing Same Source and Target (Self-Loop)

Covered above (§self-loops). This is valid, not an error.

0.18.6 Worked Example: RAG Pipeline

This example is drawn from a RAG (Retrieval-Augmented Generation) pipeline: a simplified architecture flow.

0.18.6.1 Flow Domain IR

```

1 version: "1.0"
2 metadata:
3   title: "RAG Retrieval Pipeline"
4   theme: dark-flow
5 flow:
6   direction: left-to-right
7   nodes:
8     - id: query
9       label: "User Query"
10      shape: stadium
11      icon: message-circle
12      status: active
13     - id: embed
14       label: "Embed Query"
15       shape: rounded-rect
16       icon: cpu
17     - id: retrieve
18       label: "Vector Search"
19       shape: rounded-rect
20       icon: database
21     - id: rerank
22       label: "Re-rank"
23       shape: rounded-rect
24       icon: filter
25     - id: generate
26       label: "Generate Answer"
27       shape: rounded-rect
28       icon: sparkles

```

```

29     status: success
30 edges:
31   - source: query
32     target: embed
33     style: solid
34   - source: embed
35     target: retrieve
36     style: solid
37     animated: true
38   - source: retrieve
39     target: rerank
40     style: dashed
41     label: "top-k"
42   - source: rerank
43     target: generate
44     style: solid
45     animated: true
46   - source: generate
47     target: query
48     style: dotted
49     label: "follow-up"
50     directedness: directed
51 groups:
52   - id: retrieval
53     label: "Retrieval Stage"
54     nodes: [embed, retrieve, rerank]
55     style: lane

```

Listing 39: RAG pipeline — Flow Domain IR

0.18.6.2 Lowering to Scene IR (Prose Description)

The layout engine processes this document as follows:

1. **Topology detection.** The edge `generate` $\rightarrow$ `query` creates a cycle. The engine identifies this as a feedback arc and reverses it for layering purposes.
2. **Layer assignment.** Network simplex assigns five layers (left-to-right): L0=query, L1=embed, L2=retrieve, L3=rerank, L4=generate. The reversed back-edge `generate` $\rightarrow$ `query` spans 4 layers.
3. **Crossing minimization.** With a single node per layer, no crossings exist. The barycenter pass completes trivially.
4. **Coordinate assignment.** Nodes are placed at equal horizontal intervals. The “Retrieval Stage” group’s bounding box spans L1–L3.
5. **Scene IR emission.** The resulting Scene contains:
 - 5 Group primitives (one per node), each containing a Rect (stadium/rounded-rect) + Text (label) + Image (icon).
 - 5 Path primitives (edges). The `embed` $\rightarrow$ `retrieve` and `rerank` $\rightarrow$ `generate` paths carry `FlowingDashes` animation hints. The `generate` $\rightarrow$ `query` back-edge is routed as a curved path below the main flow.
 - 5 small Path primitives (arrowheads on directed edges).
 - 2 Text primitives for edge labels (“top-k” and “follow-up”).

- 1 Rect (group background for “Retrieval Stage”) + 1 Text (group label).
- Canvas: dimensions computed from content bounding box + padding.
- Meta: scene_hash computed from the serialized element list.

Determinism verification. Running this IR through the layout engine twice produces an identical scene_hash. The cycle-handling, layer assignment, and coordinate assignment are all pure functions of the input and canonical ordering.

0.18.7 Determinism & Opt-In Discipline

The flow grammar respects the project’s sacred determinism contract (Section 0.12):

Byte-identical output.

The same Flow IR + same engine version + same theme = identical Scene IR (and therefore identical SVG). No exceptions.

No-change defaults.

All optional fields (shape, status, style, animated, directedness, ports, groups) have explicit defaults. Omitting optional fields produces the same output as specifying the default value explicitly.

New tokens are opt-in.

Future additions to the Flow IR (e.g., new shape values, new edge styles) must have no-change defaults: existing documents that do not use new features must produce byte-identical output after an engine upgrade.

Canonical ordering.

List position in nodes and edges is the tie-breaking order for all layout decisions. Re-ordering nodes in the IR may change layout (this is intentional—order is semantic).

Animation does not break determinism.

The animated flag adds a declarative FlowingDashes hint to the Scene. The SVG markup is byte-identical regardless of whether the animation is “playing” — animation is CSS/S-MIL, not frame-by-frame.

0.18.8 Open Questions (Deferred to Mark/Barbara)

The following design points are flagged for specialist review before the schema is finalized:

- Mark (IR schema detail):**
- Exact JSON Schema for the Flow Domain IR (enum values, string patterns, min/max constraints).
 - Whether FlowEdge should carry an explicit id field for referencing in composition or annotation contexts.
 - Port model: is the 5-value enum (auto, top, right, bottom, left) sufficient, or should named custom ports be supported (useful for nodes with multiple outgoing paths)?
 - Validation rule set: exhaustive list of semantic validation rules beyond schema conformance.

- Barbara (rendering semantics):**
- Self-loop routing: exact curve parameters (radius, port pair selection heuristic).
 - Back-edge rendering: curve style (cubic Bézier, stepped, or arc) and visual offset distance.
 - Multi-edge offset geometry: perpendicular offset vs. curvature-based separation.
 - Group visual style details: border radius, padding, header positioning rules.
 - Edge label collision avoidance with nodes/other labels: post-layout adjustment rules.

Kernel change flag. No kernel (Scene IR) changes are required for the flow grammar as specified. If nested groups or custom port models later prove necessary, those would require kernel-level changes needing Barbara and Mark sign-off before adoption.

0.19 Sequence Grammar: Domain IR

The Sequence Grammar is the third grammar implemented in the diagram compiler and the **first de-risked new grammar**: its layout is deterministic-by-construction—no graph auto-layout algorithm is required. Where the Flow Grammar (Section 0.18) requires Sugiyama layered layout [52] with cycle removal, crossing minimization, and coordinate assignment (Section 0.28), the Sequence Grammar’s placement is fully determined by two ordered lists: participants declared left-to-right, and messages declared top-to-bottom. This makes it the lowest-risk grammar to add after Timeline and Flow.

Implementation Status. Fully implemented in `packages/core/src/grammars/sequence/` with participant/message/activation/fragment IR compilation. Supports all fragment kinds (alt with multi-guard, loop, opt, par), self-messages with curved paths, and stereotype rendering (actor, boundary, control, entity, database). SequenceTheme with light and ByteByteGo variants.

0.19.1 Scope & Intent

0.19.1.1 What a Sequence Diagram IS

A **sequence diagram** in this grammar is a *time-ordered interaction diagram* representing:

- **Participants:** Named entities (actors, objects, services, boundaries) declared in a fixed horizontal order.
- **Messages:** Ordered communications between participants—synchronous calls, asynchronous signals, and return/reply messages—flowing downward along vertical lifelines.
- **Lifelines:** Vertical dashed lines descending from each participant, representing the entity’s existence over time.
- **Activations** (optional): Thin rectangles on lifelines indicating processing spans.

- **Fragments** (optional): Labeled rectangles spanning a range of messages, representing control-flow semantics (loop, alt, opt, par).

The visual lineage is UML 2.x Sequence Diagrams [38] and ITU-T Message Sequence Charts [25]. The grammar captures the essential structural semantics—participant ordering plus message ordering—without the full UML behavioral metamodel.

0.19.1.2 What a Sequence Diagram is NOT

Not a free node-link flow.

Flows (Section 0.18) have nodes at arbitrary graph positions connected by routed edges. Sequence diagrams have a rigid two-axis structure: participants span x, messages span y. There is no node-link topology to lay out.

Not a timeline.

The timeline grammar’s entities have temporal semantics (**start**, **end**, **track**) on a calibrated time axis. Sequence diagrams use *ordinal* vertical position (message 1, message 2, ...)—not absolute time.

Not a general graph.

No force-directed, Sugiyama, or tree-layout algorithm is involved. Placement is tabular.

0.19.1.3 Modeling Boundary

The Sequence Grammar covers diagrams where:

1. Entities are **participants** arranged in a fixed horizontal order.
2. Interactions are **messages** between participants, with a total order defining vertical position.
3. Time is **ordinal** (row index), not metric.

Diagrams requiring metric time (Gantt-like durations) belong to the Timeline grammar. Diagrams requiring arbitrary node placement belong to Flow or Graph grammars.

0.19.1.4 Why De-Risked

Deterministic layout is trivial. Given n participants and m messages:

- Participant i is placed at x_i determined solely by declared order and measured label widths (no optimization).
- Message j is placed at $y_j = \text{headerHeight} + j \times \text{rowHeight}$ (no crossing minimization, no coordinate assignment).
- No iterative algorithm, no RNG, no convergence criterion.

Contrast with the Flow grammar’s reliance on four-phase Sugiyama layout (Section 0.18.3) where even tie-breaking heuristics must be carefully pinned for determinism. Here, placement is a pure arithmetic function of declaration order—the “hard problem” of §0.28 simply does not apply.

0.19.2 Sequence Domain IR

The Sequence Domain IR is the small, grammar-specific representation that compiles *down* to the shared Scene IR (Section 0.10). It is **not** a “god-IR”—it contains only the constructs necessary to describe participant-message interaction diagrams.

0.19.2.1 Document Structure

A Sequence IR document is a single YAML/JSON object with the following root structure:

```

1 version: "1.0"
2 metadata:
3   title: string           # required
4   subtitle: string        # optional; default null
5   theme: string           # optional; default "default-sequence"
6 sequence:
7   participants: [...]     # required; list<Participant>; min 1
8   messages: [...]        # required; list<Message>; may be empty
9   activations: [...]     # optional; list<Activation>; default []
10  fragments: [...]       # optional; list<Fragment>; default []

```

Listing 40: Sequence IR root document structure

0.19.2.2 Root Fields

0.19.2.3 Participant

A participant is a named entity with a vertical lifeline.

Identity. Participant id is the primary key. It must be unique across all participants. Messages reference participants by this id. Two participants with the same id are a validation error.

Declared order. Participants are placed left-to-right in the order they appear in the participants list. This order is the **canonical horizontal order**—the sole determinant of x-position. Reordering participants in the IR changes the diagram (this is intentional; order is semantic).

Kind semantics. The kind field controls the visual stereotype of the participant header:

actor

Stick-figure icon above the label (UML actor).

object

Rectangular box with label (default, most common).

boundary

Box with a vertical bar on the left edge.

control

Box with a circular arrow icon.

entity

Box with a horizontal underline.

database

Cylinder (DB drum) icon above the label.

The kind does NOT affect layout—only the header’s visual representation. All participants occupy the same column width regardless of kind.

0.19.2.4 Message

A message is a directed communication between two participants (or a self-message within one participant).

Identity. Messages are identified by their **order** field—the explicit sequence index. The **order** is the sole determinant of vertical position. Messages with lower **order** values appear higher in the diagram.

Total order contract. The **order** field imposes a **total order** on messages. This is the deterministic backbone of the layout:

- Messages are sorted by **order** (ascending) before layout.
- Message at rank k (0-indexed after sorting) is placed at $y_k = \text{headerHeight} + k \times \text{rowHeight}$.
- If two messages share the same **order** value, the tie is broken by list position in the messages array (stable sort). See edge cases (§0.19.5).

Self-message. When **from** == **to**, the message is a self-message: a loopback arrow on a single lifeline. It occupies one message row like any other message, but is rendered as a small rightward loop returning to the same lifeline (see §0.19.4).

Message kind semantics.**sync**

Synchronous call. Rendered as a solid line with a **filled** triangular arrowhead at the target.

async

Asynchronous signal. Rendered as a solid line with an **open** (line-only) arrowhead at the target.

reply

Return/response message. Rendered as a **dashed** line with an open arrowhead at the target.

0.19.2.5 Activation

An activation represents a processing span on a participant’s lifeline—the period during which the participant is actively handling a request.

Semantics. An activation bar is drawn as a thin filled rectangle on the participant’s lifeline, spanning from the y-position of the message with `order == from_order` to the y-position of the message with `order == to_order`. The bar is centered on the lifeline.

Validation. `from_order` must be $\leq$ `to_order`. Both values must reference `order` values that exist in the `messages` list (not necessarily on messages involving this participant—activations can span any message range). A zero-height activation (`from_order == to_order`) is valid: it renders as a minimal-height bar at that message row.

0.19.2.6 Fragment

A fragment represents a combined fragment (UML 2.x interaction fragment)—a labeled region spanning a range of messages that carries control-flow semantics.

Semantics. A fragment is rendered as a labeled rounded rectangle spanning:

- Vertically: from the y-position of `from_order` to the y-position of `to_order` (with padding).
- Horizontally: from the leftmost to the rightmost participant in `participants` (or all participants if omitted), with padding.

The `kind` keyword is rendered in a small tab in the upper-left corner of the rectangle (e.g., “loop”, “alt”).

Nesting. Fragments may nest: a fragment with `from_order=2`, `to_order=5` may contain another with `from_order=3`, `to_order=4`. Nested fragments are rendered inside their parent with visual inset. Overlapping but non-nested fragments (partial overlap) are a validation error.

0.19.3 Layout Contract (Deterministic-by-Construction)

The Sequence Grammar’s layout is **fully determined by two ordered lists**: participants (horizontal) and messages (vertical). No optimization, heuristic, or iterative algorithm is involved.

0.19.3.1 Participant Placement (x-axis)

Given n participants $p_1, p_2, \dots, p_n$ in declared order:

1. Compute the header width for each participant: $w_i = \text{measureText}(p_i.\text{label}) + 2 \times \text{headerPadX}$.
2. Apply a minimum column width: $\text{colW}_i = \max(w_i, \text{minColWidth})$.
3. Place participant centers at cumulative positions:

$$\begin{aligned} x_1 &= \text{marginLeft} + \text{colW}_1/2 \\ x_i &= x_{i-1} + \text{colW}_{i-1}/2 + \text{colGap} + \text{colW}_i/2 \quad (i > 1) \end{aligned}$$

Determinism. Placement is a pure function of the participant list and theme geometry tokens (`headerPadX`, `minColWidth`, `colGap`, `marginLeft`). Same input $\rightarrow$ identical pixel positions. No force-directed layout, no crossing minimization, no Sugiyama—contrast with the Flow Grammar’s four-phase Sugiyama pipeline (Section 0.18.3).

0.19.3.2 Lifelines (y-axis skeleton)

Each participant p_i emits a vertical dashed line from the bottom of its header box to the bottom of the diagram:

$$\text{lifeline}_i : (x_i, y_{\text{headerBottom}}) \longrightarrow (x_i, y_{\text{diagramBottom}})$$

The lifeline is purely decorative; it does not participate in layout computation.

0.19.3.3 Message Placement (y-axis)

Given m messages sorted by `order` (ties broken by list position), message at rank k (0-indexed) is placed at:

$$y_k = y_{\text{headerBottom}} + \text{firstMsgGap} + k \times \text{rowHeight}$$

Each message is rendered as a horizontal (or angled) arrow from x_{from} to x_{to} at height y_k , with its label text positioned above the arrow.

Self-messages. A self-message (from = to) is rendered as a small rightward loop: an arrow that exits the lifeline to the right, descends by a fixed `selfMsgHeight`, and returns to the same lifeline. It still occupies exactly one message row at y_k .

0.19.3.4 Activation Bars

An activation bar for participant p_i spanning orders $[a, b]$ is placed as a thin filled rectangle:

$$\text{rect} : (x_i - \text{barHalfW}, y_{\text{rank}(a)}) \longrightarrow (x_i + \text{barHalfW}, y_{\text{rank}(b)})$$

where $\text{rank}(a)$ is the y-position of the message with that order value.

0.19.3.5 Fragment Rectangles

A fragment spanning orders $[a, b]$ over participants $[p_l, p_r]$ is placed as a rounded rectangle:

$$\begin{aligned} x_{\text{left}} &= x_l - \text{colW}_l/2 - \text{fragPadX} \\ x_{\text{right}} &= x_r + \text{colW}_r/2 + \text{fragPadX} \\ y_{\text{top}} &= y_{\text{rank}(a)} - \text{fragPadY} \\ y_{\text{bot}} &= y_{\text{rank}(b)} + \text{fragPadY} \end{aligned}$$

The operator keyword `tab` is positioned at $(x_{\text{left}} + \text{tabInset}, y_{\text{top}} + \text{tabInset})$.

0.19.3.6 Contrast with Flow Grammar Layout

Aspect	Flow Grammar	Sequence Grammar
Layout family	Sugiyama 4-phase (layer assignment, crossing min., coordinate assignment)	Arithmetic placement (declared order $\rightarrow$ position)
Determinism mechanism	Fixed sweep count, canonical tie-breaking	Trivially deterministic (no optimization)
Risk	Cycle removal heuristics, convergence control	None—placement is a closed-form function
New algorithms needed	Yes (Sugiyama, network simplex)	No

0.19.4 Lowering to the Scene IR

The sequence layout engine emits Scene IR primitives (Section 0.10). **No new Scene IR primitives are required**—the existing kernel is sufficient.

0.19.4.1 Participant Headers

Each participant header emits:

1. A `Rect` primitive for the header box (or a `Path` for non-rectangular kinds like database cylinder).
2. A `Text` primitive for the participant label, centered within the box.
3. For kind: `actor`: an additional small `Path` (stick-figure) above the label.

0.19.4.2 Lifelines

Each lifeline emits a single `Line` (or `Path`) primitive:

```
Line {
  x1: x_i, y1: headerBottom,
```

```

x2: x_i, y2: diagramBottom,
stroke_style: dashed,
stroke_width: theme.sequence.lifelineWidth // default 1px
}

```

0.19.4.3 Messages

Each message emits:

1. A `Line` or `Path` primitive for the connector:

sync

Solid stroke.

async

Solid stroke.

reply

Dashed stroke.

2. An arrowhead `Path`:

sync

Filled triangular arrowhead (solid, closed polygon).

async

Open arrowhead (two lines forming a “V”).

reply

Open arrowhead (two lines forming a “V”), on a dashed connector.

3. A `Text` primitive for the message label, positioned above the connector midpoint.

Self-message rendering. A self-message emits a `Path` with three segments: rightward horizontal, downward vertical (`height = selfMsgHeight`), leftward horizontal returning to the lifeline. The arrowhead is placed at the return point.

Open question for Barbara: The exact curve geometry for self-messages (rounded corners vs. sharp right angles vs. a smooth arc) is a rendering-detail decision. The spec defines the logical shape (exit right, descend, return left); Barbara owns the curve parameterization.

0.19.4.4 Activation Bars

Each activation emits a single `Rect` primitive:

```

Rect {
  x: x_i - barHalfW, y: y_rank(from_order),
  width: 2 * barHalfW,
  height: y_rank(to_order) - y_rank(from_order),
  fill: theme.sequence.activationFill,
  stroke: theme.sequence.activationStroke
}

```


0.19.4.5 Fragments

Each fragment emits:

1. A `Rect` primitive (rounded corners, no fill or light fill per theme) for the frame.
2. A small `Rect + Text` for the operator keyword tab in the upper-left corner.
3. A `Text` primitive for the guard label, positioned after the keyword tab.

No new kernel primitives needed. The entire sequence grammar maps to existing Scene IR primitives: `Rect`, `Line/Path`, `Text`, and `Group`. No kernel changes are required. If future extensions (e.g., destruction markers, creation messages with offset lifeline starts) prove necessary, those would require kernel review—**flagged for Barbara/Mark sign-off** rather than assumed.

Animation. Animation semantics are out of scope for this grammar specification. Per the project’s additive animation model (Section 0.10.3), future animation hints (e.g., sequential message fade-in) can be layered onto the Scene IR without modifying the domain IR.

0.19.5 Edge Cases & Total Semantics

Every possible input must have a defined meaning or produce a clear validation error.

0.19.5.1 Self-Message

Valid. A message where `from == to` is rendered as a loopback on the participant’s lifeline (see §0.19.4). It occupies one row like any other message.

0.19.5.2 Message Referencing an Undeclared Participant

Validation error. If a message’s `from` or `to` references a participant id not in the participants list, the document fails validation: "Message at order N references unknown participant id 'X' in field 'from|to'."

0.19.5.3 Duplicate Order Indices

Valid (tie-broken). If two or more messages share the same order value, they are placed at consecutive y-positions in the order they appear in the `messages` array (stable sort). A lint warning is emitted: "Messages at indices M and N share order value K; list position used as tie-breaker."

This ensures total determinism even with duplicate orders—but authors are encouraged to use unique order values for clarity.

0.19.5.4 Participant with No Messages

Valid. A participant that is never referenced by any message still appears in the diagram with its header box and lifeline. It occupies its declared column position. A lint warning is emitted: "Participant 'X' has no messages; lifeline will be empty."

0.19.5.5 Empty Diagram

Partially valid. A sequence with at least one participant but zero messages produces a diagram showing only participant headers and lifelines (no message arrows). A sequence with zero participants is a validation error: "Sequence must declare at least one participant."

0.19.5.6 Nested Fragments

Valid. Fragment *A* may fully contain fragment *B* ($A.\text{from_order} \leq B.\text{from_order}$ and $B.\text{to_order} \leq A.\text{to_order}$). Nested fragments are rendered with visual inset (each nesting level offsets inward by `fragNestInset` pixels). Rendering order: outermost fragments first (painter's algorithm), innermost on top.

Partial overlap is invalid. If fragment *A* and fragment *B* overlap but neither fully contains the other ($A.\text{from_order} < B.\text{from_order} < A.\text{to_order} < B.\text{to_order}$), validation fails: "Fragments at indices M and N partially overlap; fragments must nest or be disjoint."

Open question for Barbara: Maximum nesting depth for readable rendering. The spec imposes no hard limit; Barbara may recommend a soft limit (e.g., 3–4 levels) with a lint warning.

0.19.5.7 Async Message with No Corresponding Reply

Valid. An async message with no subsequent reply from the target back to the source is perfectly valid—it represents a fire-and-forget signal. No implicit reply is generated.

0.19.5.8 Activation Referencing Non-Existent Order

Validation error. If `from_order` or `to_order` does not match any message's order value, validation fails: "Activation for participant 'X' references non-existent order value N."

0.19.6 Worked Example: Token-Based REST API Authentication

This example models a simple token-based REST API authentication flow: a client requests a token, the server returns it, then the client uses the token in a subsequent authenticated request.

0.19.6.1 Sequence Domain IR

```

1 version: "1.0"
2 metadata:
3   title: "Token-Based REST API Authentication"
4   theme: default-sequence
5 sequence:
6   participants:
7     - id: client
8       label: "Client"
9       kind: actor
10    - id: server
11      label: "Auth Server"
12      kind: object
13  messages:
14    - from: client
15      to: server
16      label: "POST /auth/token (credentials)"
17      order: 1
18      kind: sync
19    - from: server
20      to: client
21      label: "200 OK { token: 'jwt...' }"
22      order: 2
23      kind: reply
24    - from: client
25      to: server
26      label: "GET /api/data (Authorization: Bearer jwt...)"
27      order: 3
28      kind: sync
29    - from: server
30      to: client
31      label: "200 OK { data: [...] }"
32      order: 4
33      kind: reply
34  activations:
35    - participant: server
36      from_order: 1
37      to_order: 2
38    - participant: server
39      from_order: 3
40      to_order: 4

```

Listing 41: REST API auth — Sequence Domain IR

0.19.6.2 Lowering to Scene IR (Detailed)

The layout engine processes this document as follows:

1. **Participant placement.** Two participants in declared order: Client at x_1 , Auth Server at x_2 . With `colGap=80px` and measured label widths, suppose $x_1 = 120\text{px}$, $x_2 = 320\text{px}$.
2. **Message placement.** Four messages sorted by order (1, 2, 3, 4). With `rowHeight=50px` and `firstMsgGap=30px`:
 - Message 1 at $y = 30 + 0 \times 50 = 30\text{px}$ below header.

- Message 2 at $y = 30 + 1 \times 50 = 80\text{px}$ below header.
- Message 3 at $y = 30 + 2 \times 50 = 130\text{px}$ below header.
- Message 4 at $y = 30 + 3 \times 50 = 180\text{px}$ below header.

3. **Scene IR emission.** The resulting Scene contains:

- **2 header Groups:** each a Rect + Text. Client additionally has a stick-figure Path (kind=actor).
- **2 lifeline Lines:** dashed vertical lines from header bottom to diagram bottom, at x_1 and x_2 .
- **4 message Lines:** horizontal arrows from x_1 to x_2 (messages 1, 3) and from x_2 to x_1 (messages 2, 4). Messages 2 and 4 have `stroke_style: dashed` (reply kind).
- **4 arrowhead Paths:** filled triangles for messages 1, 3 (sync); open V-shapes for messages 2, 4 (reply).
- **4 label Texts:** positioned above each message arrow at the midpoint.
- **2 activation Rects:** thin filled rectangles on the server lifeline (x_2), spanning $[y_1, y_2]$ and $[y_3, y_4]$.
- **Canvas:** width computed from rightmost participant + margin; height from last message row + footer margin.
- **Meta:** `scene_hash` computed from serialized element list.

Primitive count. Total Scene primitives: 2 Groups (headers) + 2 Lines (lifelines) + 4 Lines (messages) + 4 Paths (arrowheads) + 4 Texts (labels) + 2 Rects (activations) + canvas + meta = 18 drawing primitives. All are existing kernel types—no extensions required.

Determinism verification. Running this IR through the layout engine twice produces an identical `scene_hash`. Participant placement is a function of declaration order and label widths; message placement is a function of order values. No randomness, no iteration, no heuristic.

0.19.7 Determinism & Opt-In Discipline

The Sequence Grammar respects the project’s sacred determinism contract (Section 0.12):

Byte-identical output.

The same Sequence IR + same engine version + same theme = identical Scene IR (and therefore identical SVG). No exceptions.

No-change defaults.

All optional fields (`kind`, `description`, `activations`, `fragments`) have explicit defaults. Omitting optional fields produces the same output as specifying the default value explicitly.

New tokens are opt-in.

Future additions to the Sequence IR (e.g., new participant kinds, destruction markers, creation messages) must have no-change defaults: existing documents that do not use new features must produce byte-identical output after an engine upgrade.

Canonical ordering.

Declaration order in `participants` determines x-position; `order` field in messages determines y-position; list position is the final tie-breaker. These are the *only* inputs to layout.

No iterative algorithms.

Unlike the Flow Grammar’s Sugiyama layout, the Sequence Grammar uses no iterative or heuristic algorithms. Placement is a closed-form arithmetic function. This is the strongest possible determinism guarantee.

0.19.8 Open Questions (Deferred to Mark/Barbara)

The following design points are flagged for specialist review before the schema is finalized:

Mark (IR schema detail): • Exact JSON Schema for the Sequence Domain IR (enum values, string patterns, min/max constraints on `order`).

- Whether `Message` should carry an optional explicit `id` field for cross-referencing in composition or annotation contexts.
- Validation rule completeness: the exhaustive set of semantic checks beyond schema conformance (e.g., fragment overlap detection algorithm, activation range validation).
- Whether `order` should be required or whether implicit ordering (list position as `order`) is acceptable as an alternative mode.

Barbara (rendering semantics): • Self-message curve geometry: rounded corners, smooth arc, or sharp right-angle bends. Radius parameters.

- Fragment nesting depth: recommended maximum for readability; visual inset scaling per level.
- Participant header rendering for non-object kinds: exact icon geometries for actor (stick-figure), boundary, control, entity, database stereotypes.
- Arrowhead sizing: scale relative to stroke width or fixed pixel size.
- Activation bar width: fixed or proportional to lifeline stroke width.

Kernel change flag. No kernel (Scene IR) changes are required for the Sequence Grammar as specified. If future extensions (destruction markers with “X” terminators, participant creation mid-diagram with offset lifeline starts, or “found/lost” messages from/to diagram boundaries) prove necessary, those would require kernel-level review needing Barbara and Mark sign-off before adoption.

0.20 Tree Grammar: Domain IR

The Tree Grammar is the fourth grammar in the diagram compiler and the **second de-risked grammar**: its layout is the Buchheim–Jünger–Leipert tidy-tree algorithm [6]—deterministic, $O(n)$, and fully solved. Where the Flow Grammar (Section 0.18) requires four-phase Sugiyama layered layout with cycle removal and crossing minimization (Section 0.28), and the Sequence Grammar (Section 0.19) sidesteps layout entirely via arithmetic placement, the Tree Grammar occupies a middle ground: it *does* require a real

layout algorithm, but one that is provably deterministic and linear-time for its constrained input class (rooted trees).

Implementation Status. Fully implemented in `packages/core/src/grammars/tree/` with the Buchheim–Jünger–Leipert $O(n)$ tidy-tree layout algorithm. Supports recursive node nesting, custom node shapes, and full tree traversal. TreeTheme with light and dark variants.

0.20.1 Scope & Intent

0.20.1.1 What a Tree Diagram IS

A **tree diagram** in this grammar is a *node-link rendering of a rooted tree*: a connected, acyclic graph with exactly one root node and where every non-root node has exactly one parent. Trees represent containment, decomposition, and hierarchical relationships:

- **Document structure:** document → chapters → sections → subsections.
- **Organizational hierarchies:** CEO → VPs → directors → teams.
- **Decision trees:** root question → branches at each decision point.
- **File systems:** root directory → subdirectories → files.
- **Taxonomies and classification:** kingdom → phylum → class → order.

The visual form is a *tidy tree*: nodes placed at discrete depth levels with sibling groups spread horizontally, connected by parent-to-child edges. This is the canonical “org chart” or “tree view” layout.

0.20.1.2 What a Tree Diagram is NOT

Not a general directed graph.

Flows (Section 0.18) support cycles, multi-parent nodes, and arbitrary DAG topologies. Trees require exactly one parent per non-root node—no shared children, no cycles.

Not a sequence diagram.

Sequence diagrams (Section 0.19) have a rigid participant×message grid structure. Trees have variable branching and depth.

Not a timeline.

Timelines have a calibrated time axis. Tree depth levels are ordinal (level 0, 1, 2...), not temporal.

Not a mind map.

Mind maps are typically radial, free-form, and allow cross-links. This grammar produces strictly hierarchical tidy-tree layouts with no cross-edges.

0.20.1.3 Modeling Boundary

The Tree Grammar covers diagrams where:

1. Entities are **nodes** in a containment/decomposition hierarchy.
2. Relationships are strictly **parent**→**child** (one parent per node, one root).
3. Layout admits a natural **level-based reading** (root at top, children below).

Diagrams with shared children (a node having two parents) belong to the Flow grammar (DAG). Diagrams with no clear hierarchy belong to the Graph grammar’s force-directed or hub-spoke layouts.

0.20.1.4 Why De-Risked

Tidy-tree layout is deterministic and $O(n)$. The Buchheim–Jünger–Leipert algorithm [6] (correcting Walker [61], itself extending Reingold–Tilford [47]) computes compact, non-overlapping node positions in a single linear-time pass:

- No NP-hard subproblems (contrast: crossing minimization in Sugiyama is NP-complete [19]).
- No iterative convergence (contrast: force-directed methods).
- No randomized initialization.
- Fully deterministic given a fixed sibling order.
- Proven $O(n)$ time and space.

This makes Tree the lowest-risk graph-layout grammar after Sequence (which requires no layout algorithm at all). The “hard problem” of Section 0.28 is bypassed by the structural constraint: trees are a strict, easier subclass of general graphs.

0.20.2 Tree Domain IR

The Tree Domain IR is the small, grammar-specific representation that compiles *down* to the shared Scene IR (Section 0.10). It is **not** a “god-IR”—it contains only the constructs necessary to describe rooted-tree hierarchies.

0.20.2.1 Document Structure

A Tree IR document is a single YAML/JSON object with the following root structure:

```

1 version: "1.0"
2 metadata:
3   title: string           # required
4   subtitle: string        # optional; default null
5   theme: string           # optional; default "default-tree"
6 tree:
7   root: TreeNode          # required; the single root node

```

Listing 42: Tree IR root document structure

0.20.2.2 Root Fields

0.20.2.3 TreeNode

A `TreeNode` is the sole structural construct. Trees are recursive: each node may contain child nodes.

Canonical representation: children-list. The **canonical** tree representation uses an embedded children list on each node. This is chosen over a flat-list-with-parent-ref representation for three reasons:

1. **Natural authoring:** trees are written top-down; nesting children inside parents mirrors the mental model.
2. **Structural guarantee:** a well-formed nested document is automatically a valid tree (no cycles, no orphans, exactly one root). Validation is trivial—only duplicate id checks are needed.
3. **Sibling order is explicit:** the list order of children defines left-to-right placement. No separate ordering field is required.

Alternative: flat list with parent refs. An alternative flat representation (a list of nodes where each non-root carries a `parent` ref) is a valid *input convenience*. If supported, the validator **normalizes** it to the canonical children-list form before layout:

1. Verify exactly one node has no `parent` (the root).
2. Verify all `parent` refs resolve to existing node ids.
3. Verify no cycles (topological sort succeeds).
4. Group nodes by parent; sibling order = list order among nodes sharing the same parent.

Open question for Mark: Whether to support the flat parent-ref form as an alternative input mode, or require the canonical children-list form exclusively. The spec defines the canonical form; the schema may optionally accept both.

Identity. Node id is the primary key across the entire document. It must be globally unique (across all levels of nesting). Two nodes with the same `id`—regardless of depth—are a validation error.

Sibling order. Children are placed left-to-right in the order they appear in the children list. This order is the **canonical horizontal order** and the sole determinant of sibling arrangement. Reordering children in the IR changes the diagram (this is intentional; order is semantic).

Kind, icon, collapsed: semantic hints only. Per the grammar=semantics / theme=style principle (Section 0.14):

- **kind** is a semantic tag (“chapter”, “person”, “folder”) that a `TreeTheme` *may* use to select visual treatment (e.g., different fill colors or border styles per kind). The grammar assigns no visual meaning to kind values—that is a theme concern.
- **icon** references the shared icon registry. Placement and sizing are theme-driven.
- **collapsed** is a rendering hint: when **true**, the subtree rooted at this node is visually elided (e.g., replaced by an expander indicator). The node itself is still rendered; its children are hidden. This is a *hint*—static backends render it as a visual marker; interactive backends may allow expansion.

None of these fields affect layout geometry of *other* nodes. A collapsed subtree occupies zero layout space for its hidden children (the collapsed node itself retains its measured size).

0.20.3 Layout Contract (Deterministic Tidy-Tree)

The Tree Grammar’s layout engine converts the Domain IR into Scene IR coordinates using the layered tidy-tree algorithm family. The layout is **fully deterministic**: identical IR in $\rightarrow$ byte-identical Scene out.

0.20.3.1 Algorithm: Buchheim–Jünger–Leipert (2002)

The layout follows the Reingold–Tilford [47] / Walker [61] / Buchheim–Jünger–Leipert [6] lineage:

Depth assignment.

Each node is assigned to a **depth level** d : the root at $d = 0$, its children at $d = 1$, and so on. Depth determines the y-coordinate: $y = d \times \text{levelHeight}$.

Horizontal placement.

Siblings are spread horizontally such that:

- No two node bounding boxes overlap (minimum horizontal separation = `siblingGap`).
- Subtrees of adjacent siblings do not overlap (the algorithm walks contours of left and right subtrees, advancing a “thread” pointer to detect conflicts).
- A parent is centered above its children (the midpoint of the leftmost and rightmost child x-positions).

Complexity.

$O(n)$ time and $O(n)$ space, where n is the number of nodes [6]. The thread-pointer technique avoids Walker’s [61] quadratic worst case.

Determinism.

The algorithm is a pure function of the tree structure and sibling order. No randomness, no iteration count, no convergence criterion. Same input $\rightarrow$ same output, always.

0.20.3.2 Orientation

The primary (default) orientation is **top-down**: root at the top of the canvas, children below. The depth axis maps to y (increasing downward); the sibling axis maps to x.

Left-to-right orientation (root at left, children to the right) is supported as a theme/layout option—it is the same algorithm with axes transposed. Additional orientations (bottom-up, right-to-left) follow the same transposition pattern.

Note: Orientation is a layout/theme option, not a grammar-level semantic field. The IR describes structure; orientation is presentation. A `TreeTheme` declares which orientation to use.

0.20.3.3 Node Sizing

Each node’s bounding box is computed as:

$$w_i = \text{measureText}(\text{node}_i.\text{label}) + 2 \times \text{nodePadX} + \text{iconWidth}_i$$

$$h_i = \text{lineHeight} + 2 \times \text{nodePadY}$$

All sizing parameters (`nodePadX`, `nodePadY`, `iconWidth`, `lineHeight`) are **theme tokens**. The grammar does not prescribe pixel values—only the formula. This ensures that node size is a pure function of label content + theme geometry.

0.20.3.4 Spacing Parameters (Theme Tokens)

0.20.3.5 Contrast with Flow Grammar’s Sugiyama

Aspect	Flow Grammar (Sugiyama)	Tree Grammar (Tidy-Tree)
Input class	General directed graphs (DAGs, cyclic with FAS removal)	Rooted trees only (one parent per node, no cycles)
Phases	4 (cycle removal, layer assignment, crossing min., coordinate assignment)	1 (recursive bottom-up + top-down adjustment)
Crossing minimization	NP-complete; barycenter heuristic, 24 sweeps	Not applicable—trees have no edge crossings
Complexity	$O(V ^2)$ for crossing minimization	$O(n)$
Determinism mechanism	Fixed sweep count + canonical tie-breaking	Inherently deterministic (pure recursion over fixed structure)

Trees are a strict subclass of DAGs. The tidy-tree algorithm exploits this constraint to eliminate all hard subproblems that Sugiyama must address.

0.20.4 Lowering to the Scene IR

The tree layout engine emits Scene IR primitives (Section 0.10). **No new Scene IR primitives are required**—the existing kernel is sufficient.

0.20.4.1 Node Lowering

Each `TreeNode` emits a `Group` containing:

1. A `Rect` primitive for the node box. Shape is theme-driven: default is rounded rectangle (`corner_radius: theme.tree.nodeRadius`). The theme may select different shapes per kind (e.g., circles for leaf nodes, diamonds for decision nodes).
2. A `Text` primitive for the node label, centered within the box.
3. Optionally, an `Image` primitive for the icon (positioned left of label or above, per theme).

Collapsed nodes. When `collapsed: true`, the node is rendered normally but:

- Its children are *not* emitted to the Scene IR (zero primitives for the subtree).
- An expander indicator is added: a small `Path` or `Text` glyph (e.g., “+” or “...”) positioned below or beside the node, per theme.

0.20.4.2 Edge Lowering

Each parent→child relationship emits a connector:

Orthogonal elbow (default).

A `Path` with three segments: vertical from parent bottom-center down to the midpoint between levels, horizontal to the child’s x-position, then vertical down to the child’s top-center. This produces the classic “org chart” connector style.

Straight line.

A single `Line` from parent bottom-center to child top-center. Simpler but less readable for wide trees.

Curved.

A cubic Bézier `Path` from parent to child. Produces smooth, flowing connectors.

The choice among these styles is a **theme option** (`theme.tree.edgeStyle: elbow | straight | curved`). The grammar does not prescribe edge rendering—only that a visual connector exists for each parent-child pair.

Edge directionality. Edges are rendered **without arrowheads** by default (the top-down reading direction is implicit). Arrowheads are a theme option for cases where directionality needs emphasis.

0.20.4.3 Scene IR Mapping Summary

Tree Construct	Scene IR Primitive(s)
Node box	Rect (rounded, theme-styled)
Node label	Text (centered in box)
Node icon	Image (optional, positioned per theme)
Parent→child edge	Path (elbow/straight/curved, per theme)
Collapsed indicator	Path or Text (expander glyph)
Canvas	Computed from tree bounding box + margins

No new kernel primitives needed. The entire Tree Grammar maps to existing Scene IR primitives: Rect, Text, Path, Line, Image, and Group. No kernel changes are required. If future extensions (e.g., cross-links between tree nodes for showing relationships outside the hierarchy) prove necessary, those would require kernel-level review—**flagged for Barbara/Mark sign-off** rather than assumed.

Styling deferred to TreeTheme. Consistent with the SequenceTheme and FlowTheme precedent, all visual styling (colors, font sizes, border widths, edge styles, node shapes per kind, spacing) is controlled by a TreeTheme type. The grammar specifies structure; the theme specifies presentation.

0.20.5 Edge Cases & Total Semantics

Every possible input must have a defined meaning or produce a clear validation error.

0.20.5.1 Multiple Roots (Forest)

Rejected. The Tree Grammar requires exactly one root. A document with multiple roots (e.g., a flat list with two nodes having no parent) is a validation error: "Tree must have exactly one root node; found N root candidates."

Rationale: A forest (multiple disjoint trees) is a fundamentally different layout problem—it requires deciding how to arrange separate trees relative to each other. This is a composition concern. Authors who need a forest should use the Composition layer (Section 0.24) with multiple Tree panels.

0.20.5.2 Cycles

Rejected. A cycle in the tree structure (node *A* is a descendant of node *B* which is a descendant of *A*) is a validation error: "Cycle detected involving node 'X'; tree structures must be acyclic."

In the canonical children-list representation, cycles are structurally impossible (a node cannot appear as its own ancestor in a nested JSON/YAML document). Cycles can only arise in the flat parent-ref alternative input form, where validation must perform a topological sort to detect them.

0.20.5.3 Orphan Nodes (Unresolved Parent)

Rejected. In the flat parent-ref input form, if a node references a parent id that does not exist, validation fails: "Node 'X' references non-existent parent 'Y'."

In the canonical children-list form, orphans are structurally impossible—every node is either the root or a child of another node.

0.20.5.4 Single-Node Tree

Valid. A tree with only a root node (no children) is valid. It produces a single node box centered on the canvas. This is the minimal valid tree.

0.20.5.5 Very Wide Trees (Many Siblings)

Valid. A node with many children (e.g., 50+ siblings) produces a wide diagram. The layout engine does not impose a hard limit on sibling count. The tidy-tree algorithm handles this correctly—it simply places all siblings in a row. A lint warning may be emitted for extreme cases: "Node 'X' has N children; consider restructuring for readability." (Threshold is theme-configurable, e.g., > 20.)

0.20.5.6 Very Deep Trees

Valid. A tree with many levels (e.g., 20+ depth) produces a tall diagram. No hard limit is imposed. The $O(n)$ algorithm handles deep trees efficiently. As with wide trees, a lint warning may be emitted for extreme depth.

0.20.5.7 Duplicate IDs

Validation error. Two nodes anywhere in the tree with the same id fail validation: "Duplicate node id 'X' at paths 'P1' and 'P2'." (Paths indicate the nesting location for debugging.)

0.20.5.8 Empty Children List

Valid. A node with children: [] is a leaf node. This is the common case for terminal nodes and is not degenerate.

0.20.6 Worked Example: Document Structure Hierarchy

This example models a document hierarchy: a root "Document" node with three chapters, each containing a few sections. This is drawn from the corpus's document-structure diagram kind.

0.20.6.1 Tree Domain IR

```

1 version: "1.0"
2 metadata:
3   title: "Document Structure"
4   theme: default-tree
5 tree:
6   root:
7     id: doc
8     label: "Document"
9     kind: root
10    children:
11      - id: ch1
12        label: "Chapter 1: Introduction"
13        kind: chapter
14        children:
15          - id: s1-1
16            label: "1.1 Background"
17            kind: section
18          - id: s1-2
19            label: "1.2 Motivation"
20            kind: section
21      - id: ch2
22        label: "Chapter 2: Methods"
23        kind: chapter
24        children:
25          - id: s2-1
26            label: "2.1 Data Collection"
27            kind: section
28          - id: s2-2
29            label: "2.2 Analysis"
30            kind: section
31          - id: s2-3
32            label: "2.3 Validation"
33            kind: section
34      - id: ch3
35        label: "Chapter 3: Results"
36        kind: chapter
37        children:
38          - id: s3-1
39            label: "3.1 Findings"
40            kind: section

```

Listing 43: Document hierarchy — Tree Domain IR

0.20.6.2 Layout Computation

The layout engine processes this document as follows:

1. **Depth assignment.** Three levels: doc at $d = 0$; ch1, ch2, ch3 at $d = 1$; all sections at $d = 2$.
2. **Node sizing.** Each node's width is measured from its label text + padding. Suppose (with default theme): doc= 180px, chapters $\approx$ 220px, sections $\approx$ 200px wide; all= 36px tall.
3. **Tidy-tree placement (Buchheim et al.).**

- Leaf nodes (sections) are placed with `siblingGap= 20px` between them.
 - Each chapter is centered above its children.
 - Subtree gaps ensure `ch1`'s rightmost section and `ch2`'s leftmost section maintain at least `subtreeGap= 40px` separation.
 - The root `doc` is centered above the three chapter nodes.
4. **Vertical positions.** With `levelHeight= 100px`: $y_{\text{doc}} = 50$, $y_{\text{chapters}} = 150$, $y_{\text{sections}} = 250$.

0.20.6.3 Lowering to Scene IR

The resulting Scene contains:

- **10 node Groups:** each containing a `Rect` (rounded, theme-styled) + `Text` (label). The root node may have a distinct fill color (theme resolves `kind: root` to a highlight fill).
- **9 edge Paths:** one for each parent→child relationship. With the default `elbow` edge style, each path has three segments (vertical-horizontal-vertical).
- **Canvas:** width computed from the leftmost section's left edge to the rightmost section's right edge, plus margins. Height computed from top margin to depth-2 bottom edge, plus footer margin.
- **Meta:** `scene_hash` computed from the serialized element list.

Primitive count. Total Scene primitives: 10 Groups (containing 10 Rects + 10 Texts) + 9 Paths (edges) + canvas + meta = 29 drawing primitives. All are existing kernel types—no extensions required.

Determinism verification. Running this IR through the Buchheim layout engine twice produces an identical `scene_hash`. Node positions are a pure function of the tree structure, sibling order, and theme geometry tokens. No randomness, no iteration, no heuristic.

0.20.7 Determinism & Theme/Semantics Split

The Tree Grammar respects the project's sacred determinism contract (Section 0.12) and the grammar=semantics / theme=style principle:

Byte-identical output.

The same Tree IR + same engine version + same theme = identical Scene IR (and therefore identical SVG). No exceptions.

No-change defaults.

All optional fields (`kind`, `icon`, `collapsed`, `description`, `children`) have explicit defaults. Omitting optional fields produces the same output as specifying the default value explicitly.

New tokens are opt-in.

Future additions to the Tree IR (e.g., new semantic hint fields, annotation support) must have no-change defaults: existing documents that do not use new features must produce byte-identical output after an engine upgrade.

Canonical ordering.

Sibling order in children lists is the sole determinant of horizontal arrangement. Re-ordering children changes the diagram (intentional—order is semantic).

Deterministic algorithm.

Buchheim–Jünger–Leipert is a pure function over a tree structure. No iteration count, no sweep count, no RNG. This provides the strongest possible determinism guarantee short of trivial arithmetic (Sequence Grammar).

Grammar is semantic-only.

The Tree Domain IR carries *structure* (parent-child hierarchy, sibling order) and *semantic hints* (kind, icon, collapsed). It does **not** carry:

- Colors, fills, or strokes (theme tokens).
- Font sizes or families (theme tokens).
- Node shapes (theme resolves kind to a shape).
- Edge styles (theme option: elbow/straight/curved).
- Spacing values (theme geometry tokens).
- Orientation (theme layout option).

All visual presentation is controlled by a `TreeTheme` type, following the `SequenceTheme` and `FlowTheme` precedent.

0.20.8 Open Questions (Deferred to Mark/Barbara)

The following design points are flagged for specialist review before the schema is finalized:

Mark (IR schema detail):

- **Canonical form:** Should the schema accept *only* the children-list form, or also support a flat parent-ref alternative input? If both, specify the normalization algorithm and which takes precedence on conflict.

- **Forest handling:** Confirm rejection of multiple roots. If future requirements demand forest support, specify it as a separate grammar or as a composition-layer concern.
- **Validation invariants:** Exhaustive list of semantic checks beyond schema conformance (duplicate id detection across nested levels, maximum depth/breadth lint thresholds).
- **Kind enum:** Should kind be a free string or a closed enum? Free string is more extensible (themes can map arbitrary kinds); closed enum enables schema-level validation.
- **Node id format:** Confirm kebab-case requirement. Nested ids may benefit from path-based namespacing (e.g., `ch1/s1-1`) vs. global flat namespace.

Barbara (rendering semantics):

- **Edge routing style:** Default elbow geometry—exact midpoint calculation, corner radius for rounded elbows, minimum segment length before defaulting to straight.

- **Collapsed-node handling:** Visual indicator design (“+” glyph, ellipsis, count badge showing hidden children). Interactive vs. static rendering semantics.
- **TreeTheme token surface:** Complete list of theme tokens (colors, typography, geometry, edge style) needed for the initial default theme and at least one showcase theme.
- **Kind→shape mapping:** Default visual mappings for common kind values (e.g., person→circle, folder→rounded-rect with icon). How many built-in kind mappings should the default theme provide?
- **Label overflow:** Behavior when label text exceeds available node width—truncate with ellipsis, wrap to multiple lines, or auto-expand node width (current spec assumes auto-expand via `measureText`).

Kernel change flag. No kernel (Scene IR) changes are required for the Tree Grammar as specified. If future extensions (cross-links between tree nodes, animated expand/-collapse transitions, or multi-root forest layout) prove necessary, those would require kernel-level review needing Barbara and Mark sign-off before adoption.

0.21 The Chart Family: A Grammar-of-Graphics Layer

The chart family is the one genuinely new architectural component required by the Mermaid-superset strategy. Unlike node-link, timeline, and tree families (which map *structural* entities to *positioned shapes*), charts map **quantitative data** to **geometric marks** through scales and coordinate systems.

This section specifies the chart grammar drawing on the Grammar of Graphics [64] and Vega-Lite [48] traditions, adapted to our two-IR-layer architecture.

0.21.1 Design Principles for Charts

1. **Data → geometry, not data → pixels.** The Domain IR specifies data and encoding; the layout engine computes geometry; the theme styles it.
2. **Mermaid-compatible syntax.** Each chart type’s Mermaid syntax parses faithfully to our Chart IR.
3. **Deterministic layout.** All scale computations, tick placement, and mark positioning are closed-form (no iteration, no randomness).
4. **Theme-driven aesthetics.** Colors, fonts, gridlines, and labels are theme-controlled. The IR carries data and encoding, never style.

0.21.2 Chart Domain IR

```

1 interface ChartDocument {
2   version: string;
3   chartType: 'pie' | 'xy' | 'quadrant' | 'radar';
4   title?: string;
5   data: ChartData;

```

```

6   encoding: ChartEncoding;
7   config?: ChartConfig;
8 }
9
10 interface ChartData {
11   values: Array<Record<string, number | string>>;
12 }
13
14 interface ChartEncoding {
15   // Type-specific encoding channels
16   x?: FieldEncoding;      // xy charts
17   y?: FieldEncoding;      // xy charts
18   color?: FieldEncoding;  // categorical color
19   size?: FieldEncoding;   // mark size (radar axis values)
20   theta?: FieldEncoding;  // pie angle
21   label?: FieldEncoding;  // text labels
22 }
23
24 interface FieldEncoding {
25   field: string;          // key into data values
26   type: 'quantitative' | 'nominal' | 'ordinal';
27   scale?: ScaleConfig;
28 }

```

Listing 44: Chart Domain IR shape (TypeScript)

0.21.3 Chart Types

0.21.3.1 Pie Chart

Maps categorical values to angular sectors. Mermaid syntax:

```

pie title Pet Adoption
  "Dogs" : 386
  "Cats" : 85
  "Rats" : 15

```

Domain IR encoding: `theta` channel maps value to sector angle; `color` channel maps label to fill color (theme palette).

0.21.3.2 XY Chart (Bar/Line)

Cartesian charts with categorical or quantitative axes. Mermaid syntax:

```

xychart-beta
  title "Monthly Revenue"
  x-axis [Jan, Feb, Mar, Apr, May]
  y-axis "Revenue (USD)" 0 --> 5000
  bar [1200, 2400, 3100, 2800, 4200]
  line [1000, 2200, 2900, 2600, 4000]

```

Domain IR: `x` encodes categories or values; `y` encodes quantitative measure; mark type (bar/line) stored in encoding config.

0.21.3.3 Quadrant Chart

Four-quadrant scatter plot with labeled axes and positioned items. Mermaid syntax:

```
quadrantChart
  title Reach and engagement
  x-axis Low Reach --> High Reach
  y-axis Low Engagement --> High Engagement
  quadrant-1 We should expand
  quadrant-2 Need to promote
  Campaign A: [0.3, 0.6]
  Campaign B: [0.45, 0.23]
```

Domain IR: x and y encode position on [0,1] scales; quadrant labels are metadata.

0.21.3.4 Radar Chart

Multi-axis radial chart. Mermaid syntax (proposed):

```
radar
  title Skills Assessment
  axes: [Speed, Reliability, Comfort, Safety, Efficiency]
  "Model A": [4, 3, 2, 5, 4]
  "Model B": [2, 5, 4, 3, 3]
```

Domain IR: axes define radial dimensions; each series maps values to axis positions.

0.21.4 Layout Engine

The chart layout engine is shared across all four chart types:

1. **Scale computation.** From data domain (min/max or categories) to pixel range, using linear, band, or radial scales. All scales are closed-form.
2. **Axis/grid generation.** Tick positions, labels, and gridlines computed deterministically from scale + theme config.
3. **Mark placement.** Data points mapped through scales to Scene IR primitives: Rect (bars), Path (lines, pie sectors, radar polygons), Circle (scatter points).
4. **Label placement.** Deterministic label positioning with collision avoidance (priority-based, not iterative).

0.21.5 Relation to Vega-Lite

The chart grammar draws on Vega-Lite's decomposition (data, transform, mark, encoding, scale, axis, legend) but is deliberately simpler:

- No data transforms (filter, aggregate, window)—data arrives pre-aggregated.

- No interaction or selection.
- No composition operators (layer/concat/facet)—composition is handled by the poster layer (Part VI).
- Subset of mark types: rect, line, arc, point, text.

This simplification keeps the chart Domain IR small enough for reliable agent generation while covering the four Mermaid chart types completely.

0.21.6 Implementation Status

Not yet built. The chart family is the Tier-2 deliverable. It requires:

- New Chart Domain IR and JSON Schema
- Scale computation library (linear, band, radial)
- Chart layout engine compiling to Scene IR
- Mermaid parsers for pie, xychart, quadrant, radar syntax
- Chart-specific theme extensions (axis styling, gridlines, palettes)

The existing Scene IR and backends (SVG, PNG) require no changes—charts compile to the same Rect/Path/Text/Circle primitives used by all other families.

Table 8: Five diagram families with constituent types and kernel

Family	Diagram Types	Layout Kernel	Status
Node-link / Graph	flowchart, C4 (4 variants), architecture, block, requirement, git-Graph, sankey	Sugiyama (layered)	Flow built; rest planned
UML / Software	sequence class, state, er-Diagram	Grammar-specific	Sequence built; rest Tier-1
Charts (data→geometry)	pie, xy-chart, quadrant, radar	Grammar-of-graphics	New kernel (Tier-2)
Timeline / Project	gantt, time-line, journey, kan-ban	Track-based	Timeline built; rest planned
Tree / Hierarchy	mindmap, treemap	Buchheim–Jünger–Leipert	Tree built; treemap planned

Type	Description
string	UTF-8 text, non-empty unless explicitly noted
string?	Optional string (may be omitted or null)
int	Integer
number	Decimal number
number[0..1]	Decimal in closed interval [0, 1]
bool	Boolean (<code>true/false</code>)
date	A temporal point (see §0.16.5)
daterange	A temporal interval with start and optional end
id	Document-unique identifier (kebab-case slug, e.g., <code>alpha-release</code>)
ref<T>	Reference to an entity of type T by its id
list<T>	Ordered list of elements of type T
optional<T>	Value of type T that may be omitted
enum{...}	One of the listed literal values
map<K,V>	Key-value mapping

Table 9: IR type vocabulary

Field	Type	Req	Default	Description
version	string	Yes	—	Specification version (semver, e.g., <code>"1.0"</code>)
metadata	Metadata	Yes	—	Document-level metadata
tracks	list<Track>	Yes	—	Swimlanes/rows; at least one required
groups	list<Group>	No	[]	Logical groupings of activities
activities	list<Activity>	Yes	—	Time-spanning items; may be empty
milestones	list<Milestone>	No	[]	Point-in-time markers
annotations	list<Annotation>	No	[]	Callouts, notes, highlights
sections	list<Section>	No	[]	Visual/logical document sections
legend	Legend	No	auto	Legend mapping; auto-generated if omitted

Table 10: Root document fields

Field	Type	Req	Default	Description
title	string	Yes	—	Primary document title
subtitle	string?	No	null	Secondary title or tagline
author	string?	No	null	Author or team name
created	date	No	now	Document creation date
updated	date	No	now	Last modification date
time_range	TimeRange	Yes	—	Temporal bounds of the timeline
axis_unit	AxisUnit	No	inferred	Granularity of the time axis
theme	string?	No	null	Theme identifier (resolved by renderer)
locale	string?	No	"en"	BCP-47 locale for date formatting
today	date?	No	eval time	Reference date for now, relative dates, and today-marker; see §0.16.4.3
fiscal_year_start	int [1..12]	No	1	Calendar month on which the fiscal year begins (1 = January); see §0.16.4.3
description	string?	No	null	Extended description or notes

Table 11: Metadata fields

Field	Type	Req	Default	Description
start	date	Yes	—	Inclusive start of the viewport
end	date	No	open	Inclusive end; omit for open-ended

Table 12: TimeRange fields

Form	Example	Semantics
ISO date	2026-06-09	Specific calendar day
ISO datetime	2026-06-09T14:00	Specific moment (minute precision)
Year-month	2026-06	Entire month (June 2026)
Year only	2026	Entire year
Quarter	2026-Q2	Calendar quarter (Apr–Jun)
Half	2026-H1	Calendar half (Jan–Jun)
Fiscal quarter	FY26-Q2	Fiscal quarter (calendar mapping per <code>fiscal_year_start</code> ; see §0.16.4.3)
Relative	+3m	3 months from the date anchor (<code>metadata.today</code> → <code>metadata.created</code> ; see §0.16.4.3)
Symbolic	now	Resolves to the date anchor (<code>metadata.today</code> → <code>metadata.created</code> ; see §0.16.4.3)

Table 13: Date representation forms

Value	Semantics
tbd	Date is not yet determined; renders as “TBD”
ongoing	No planned end; activity continues indefinitely
unknown	Date exists but is not known to the author
~2026-Q3	Approximate/tentative (prefix tilde)

Table 14: Uncertain and open date markers

Field	Type	Req	Default	Description
start	date	Yes*	—	Interval start (inclusive)
end	date ongoing tbd	No	ongoing	Interval end (inclusive); omitting is equivalent to end: ongoing
span	date	Yes*	—	Single-unit shorthand; mutually exclusive with start/end

Table 15: DateRange fields (*exactly one of `span` or `start` (+ optional `end`) is required; co-presence of `span` with `start` or `end` is a well-formedness error)

Field	Type	Req	Default	Description
id	id	Yes	—	Unique track identifier
label	string	Yes	—	Display label
description	string?	No	null	Extended description
color	string?	No	theme	Color hint (theme may override)
group	ref<Group>?	No	null	Parent group (for nested tracks)
index	int?	No	position	Explicit sort index; unique non-negative integer; tracks render top-to-bottom in ascending index order
collapsed	bool	No	false	Initial collapsed state (if renderer supports)

Table 16: Track fields

Field	Type	Req	Default	Description
id	id	Yes	—	Unique group identifier
label	string	Yes	—	Display label
description	string?	No	null	Extended description
parent	ref<Group>?	No	null	Parent group (for nesting)
children	list<ref<Track Group>>	No	[]	Ordered child tracks/groups
members	list<ref<Activity Milestone>>	No	[]	Member entities
color	string?	No	theme	Color hint

Table 17: Group fields

Field	Type	Req	Default	Description
id	id	Yes	—	Unique activity identifier
label	string	Yes	—	Display label (shown on bar)
track	ref<Track>	Yes	—	Containing track
start	date	Yes*	—	Start date
end	date ongoing tbd	No	ongoing	End date; omitting is equivalent to end: ongoing (open interval)
span	date	Yes*	—	Single-unit shorthand; mutually exclusive with start/end
status	Status	No	planned	Visual communication status
progress	number[0..1]?	No	null	Completion fraction
category	string?	No	null	Semantic category (for styling)
description	string?	No	null	Extended description
group	ref<Group>?	No	null	Logical group membership
url	string?	No	null	External link
tags	list<string>	No	[]	Freeform tags
metadata	map<string,any>	No	{}	Extension metadata

Table 18: Activity fields (*exactly one of span or start (+ optional end) is required; co-presence of span with start or end is a well-formedness error)

Field	Type	Req	Default	Description
id	id	Yes	—	Unique milestone identifier
label	string	Yes	—	Display label
date	date	Yes	—	Point in time
track	ref<Track>?	No	global	Associated track (or global)
status	Status	No	planned	Visual status
category	string?	No	null	Semantic category
icon	string?	No	theme	Icon hint (diamond, flag, star, etc.)
color	string?	No	theme	Color hint (theme may override)
description	string?	No	null	Extended description
group	ref<Group>?	No	null	Logical group membership
url	string?	No	null	External link
tags	list<string>	No	[]	Freeform tags
metadata	map<string,any>	No	{}	Application data; renderers ignore unknown keys

Table 19: Milestone fields

Field	Type	Req	Default	Description
type	AnnotationType	Yes	—	Annotation kind
id	id?	No	auto	Optional identifier
target	ref<Activity Milestone>?	Cond	—	Single target entity
targets	list<ref<...>?	Cond	—	Multiple targets
date	date?	Cond	—	Point annotation
start/end	date?	Cond	—	Range annotation
text/label	string?	No	null	Display text
position	enum{above,below,left,right}?	No	auto	Hint only
style	string?	No	theme	Style hint

Table 20: Annotation fields (conditional fields depend on type)

Field	Type	Req	Default	Description
id	id	Yes	—	Unique section identifier
label	string	Yes	—	Section title
time_range	TimeRange?	No	inherit	Override document time range
tracks	list<ref<Track>?	No	all	Subset of tracks to show
description	string?	No	null	Section description
page_break	bool	No	false	Hint for paginated output

Table 21: Section fields

Field	Type	Req	Default	Description
show	bool	No	true	Whether to render legend
position	string?	No	theme	Position hint
title	string?	No	"Legend"	Legend title
entries	list<LegendEntry>	No	auto	Legend entries

Table 22: Legend fields

Field	Type	Req	Default	Description
key	string	Yes	—	Status or category key
label	string	Yes	—	Human-readable label
description	string?	No	null	Extended description
category	bool	No	false	True if this is a category (not status)

Table 23: LegendEntry fields

Candidate	Vis.	Git	LLM	Sep.	Licence	Tooling
Markwhen [29]	Y	Y	P	N	MIT	P
Mermaid	P	Y	P	N	MIT	Y
time-line [31]						
iCalendar	N	N	N	N	IETF	Y
VEVENT [8]						
TimelineJS	Y	P	Y	N	GPL	P
JSON [28]						
vis-timeline [58]	Y	N	N	N	MIT	Y
Vega-Lite [48]	P	P	P	Y	BSD-3	Y
TaskJuggler [54]	N	Y	N	N	GPLv2	P

Table 24: Build-vs-Adopt scoring. Vis. = visual-communication focus (not scheduling); Git = git-friendly text format; LLM = LLM-generatable schema; Sep. = render/theme separation; Tooling = active community. Y = good; P = partial; N = poor.

IR field	iCal schema.org	/	OWL- Time	Assessment
start, end	DTSTART/DTEND; startDate/endDate		hasBeginning/ hasEnd	Aligned. Standard names; shorter preferred for authoring.
label	SUMMARY; name	—		Aligned. Shorter form preferred for YAML.
description	DESCRIPTION	—		Aligned.
url	URL; url	—		Aligned.
tags	CATEGORIES (multi)	—		Aligned. Correct plural form.
category	CATEGORIES (single)	—		Aligned.
status:	STATUS:TENTATIVE	—		Aligned. Verbatim iCalendar value.
tentative				
status:	STATUS:CANCELLED	—		Aligned. Lower-case per IR convention.
cancelled				
status:	—	—		Original. No standard analog; justified by executive-presentation idiom.
at-risk				
status:	—	—		Original. No standard analog; justified by visual-communication need.
blocked				
end: ongoing	—		omit hasEnd	Semantically aligned. Explicit encoding is clearer than omission.
span	—	—		Original. Convenient shorthand; no standard equivalent.
track	CALENDAR (weak)	—		Original. Markwhen uses <code>group</code> ; <code>track</code> is clearer.
progress	—	—		Original. No standard analog; no change needed.
Activity (type)	VEVENT (duration > 0)		Interval	Aligned. Correct modelling of time-spanning entities.
Milestone (type)	VEVENT (zero dur.)		Instant	Aligned. Correct modelling of point-in-time markers.

Table 25: Alignment of Mark’s IR field names and types with prior-art standards.

Symbol	Definition
W	<code>theme.canvas.width</code> — total canvas width in logical pixels
m_L, m_R, m_T, m_B	Left, right, top, bottom margins
H_{hdr}	<code>theme.track.header_width</code> — left column for track labels
H_{axis}	<code>theme.axis.height</code> — horizontal time-axis strip
W_{draw}	$W - m_L - m_R - H_{\text{hdr}}$ — drawable timeline width
H_{draw}	$\sum_i h_i + \sum_{i>0} g$ — total height of all track rows plus gaps
H	Computed canvas height: $m_T + H_{\text{axis}} + H_{\text{draw}} + m_B$

Table 26: Coordinate system symbols

time_range span (days, inclusive)	Inferred axis_unit
≤ 14	day
≤ 91	week
≤ 548	month
≤ 1461	quarter
≤ 2922	half
> 2922	year

Table 27: Axis unit inference thresholds (first match wins)

Date form	As left edge (start)	As right edge (end)
2026-06-09 (day)	2026-06-09	2026-06-09
2026-06 (month)	2026-06-01	2026-06-30
2026 (year)	2026-01-01	2026-12-31
2026-Q2 (quarter)	2026-04-01	2026-06-30
2026-H1 (half)	2026-01-01	2026-06-30
FY26-Q2 (fiscal)	First day of fiscal Q2	Last day of fiscal Q2

Table 28: Date coercion to day boundaries by edge role

axis_unit	Primary label content	Secondary label (every N ticks)
day	Day-of-month number	Month name (every 7)
week	Week-start day	Month name (every 4)
month	Month abbreviation	Year number (every 12)
quarter	Q1 / Q2 / Q3 / Q4	Year number (every 4)
half	H1 / H2	Year number (every 2)
year	Year number	None

Table 29: Tick label content rules per axis unit

Field	Type	Req	Default	Description
version	string	Yes	—	Spec version (semver, e.g., "1.0")
metadata	FlowMetadata	Yes	—	Document-level metadata
flow	FlowDefinition	Yes	—	The flow graph definition

Table 31: Flow IR root fields

Field	Type	Req	Default	Description
id	id	Yes	—	Document-unique ID (kebab-case)
label	string	Yes	—	Display text
shape	enum{rect, rounded-rect, circle, diamond, stadium}	No	rounded-rect	Visual shape
icon	string?	No	null	Icon registry key
status	enum{default, active, success, warning, error, muted}	No	default	Semantic state
description	string?	No	null	Tooltip/secondary text
group	ref<FlowGroup>?	No	null	Group membership

Table 32: FlowNode fields

Field	Type	Req	Default	Description
source	ref<FlowNode>	Yes	—	Source node id
target	ref<FlowNode>	Yes	—	Target node id
source_port	enum{auto, top, right, bottom, left}	No	auto	Anchor point on source node
target_port	enum{auto, top, right, bottom, left}	No	auto	Anchor point on target node
label	string?	No	null	Edge annotation text
style	enum{solid, dashed, dotted}	No	solid	Stroke style
animated	bool	No	false	Enable flowing-dash animation
directedness	enum{directed, bidirectional, undirected}	No	directed	Arrow rendering

Table 33: FlowEdge fields

Field	Type	Req	Default	Description
id	id	Yes	—	Document-unique identifier
label	string	Yes	—	Group heading text
nodes	list<ref<FlowNode>>	No	[]	Node ids in this group (augments per-node group field)
style	enum{lane, cluster, outline}	No	lane	Visual rendering style

Table 34: FlowGroup fields

Field	Type	Req	Default	Description
version	string	Yes	—	Spec version (semver, e.g., "1.0")
metadata	SeqMetadata	Yes	—	Document-level metadata
sequence	SeqDefinition	Yes	—	The sequence definition

Table 35: Sequence IR root fields

Field	Type	Req	Default	Description
id	id	Yes	—	Document-unique identifier (kebab-case)
label	string	Yes	—	Display text box
kind	enum{actor, object, boundary, control, entity, database}	No	object	Participant effects header icon
description	string?	No	null	Tooltip/secondary text

Table 36: Participant fields

Field	Type	Req	Default	Description
from	ref<Participant>	Yes	—	Source participant id
to	ref<Participant>	Yes	—	Target participant id
label	string	Yes	—	Message annotation text
order	integer	Yes	—	Explicit sequence index (≥ 0); determines y-position
kind	enum{sync, async, reply}	No	sync	Message kind (controls arrowhead style)

Table 37: Message fields

Field	Type	Req	Default	Description
participant	ref<Participant>	Yes	—	Lifeline on which the activation appears
from_order	integer	Yes	—	Message order at which activation begins
to_order	integer	Yes	—	Message order at which activation ends

Table 38: Activation fields

Field	Type	Req	Default	Description
kind	enum{loop, alt, opt, par, critical, break}	Yes	—	Fragment operator
label	string	Yes	—	Guard condition or description text
from_order	integer	Yes	—	First message order in the span
to_order	integer	Yes	—	Last message order in the span
participants	list<ref<Participant>?	No	all	Subset of participants the fragment spans horizontally

Table 39: Fragment fields

Field	Type	Req	Default	Description
version	string	Yes	—	Spec version (semver, e.g., "1.0")
metadata	TreeMetadata	Yes	—	Document-level metadata
tree	TreeDefinition	Yes	—	The tree definition (contains the root)

Table 40: Tree IR root fields

Field	Type	Req	Default	Description
id	id	Yes	—	Document-unique identifier (kebab-case)
label	string	Yes	—	Display text for the node
children	list<TreeNode>	No	[]	Ordered child nodes
kind	string?	No	null	Semantic kind hint (e.g., "chapter", "person", "decision")
icon	string?	No	null	Icon registry key
collapsed	bool	No	false	Hint: render subtree as collapsed (elided)
description	string?	No	null	Tooltip or secondary text

Table 41: TreeNode fields

Token	Meaning
levelHeight	Vertical distance between depth levels (center-to-center)
siblingGap	Minimum horizontal gap between adjacent sibling bounding boxes
subtreeGap	Minimum horizontal gap between adjacent subtrees (may differ from siblingGap)
nodePadX, nodePadY	Internal padding within a node box
marginTop, marginLeft	Canvas margins

Table 42: Tree layout theme tokens (geometry)

Part V

Aesthetics & Theming

0.22 Theme Architecture

A theme is *one coherent design system*, defined once at a general, component-agnostic level. It is *not* an attribute owned separately by each diagram type; it is an authority that every component conforms to. Each component—flow, sequence, timeline, tree, class diagram, ER diagram, and so on—provides a *specific implementation* of that general system: it knows how to realise the theme’s tokens in its own geometric terms. The theme knows nothing about flowcharts or Gantt bars; the component knows nothing about brand palettes or spacing scales. The theme sets the contract; the component fulfils it.

Confirmed design principle (binding): A theme must be a coherent system applicable to many (ideally all) components. You cannot conceive of a theme as an attribute of each component as a separate system. A theme is defined once in general terms; each component provides its own specific implementation of that general definition.

Themes also drive **geometry, layout, and routing**—not just colour. A theme defines widths, fonts, weights, spacing rhythms, density levels, and shape language. The layout engine consults the theme to make rendering, layout, and routing decisions. The old rule that “a theme may not alter geometry” is explicitly retired: it contradicts both this model and the existing code, where the timeline’s `spineSpacing` and `nodeWrap` tokens already drive layout behaviour.

Implementation status. As of 2026-06-14 the compiler ships 16 grammar-specific `theme.ts` files (one per component: `FlowTheme`, `SequenceTheme`, `ClassTheme`, `TreeTheme`, etc.) plus the timeline’s `ResolvedTheme` (which covers 14 named timeline themes). These are separate, incompatible vocabularies—the fragmentation this section resolves. The migration path is to generalise `ResolvedTheme` upward into a shared general contract, then adapt each per-component file to consume that contract.

0.22.1 Current Reality: Per-Component Fragmentation

Today every grammar family carries an independent styling vocabulary:

- **Timeline:** `packages/core/src/themes/types.ts` — `ResolvedTheme` with `canvas`, `typography`, `axis`, `track`, `milestone`, `serpentine`, `roadmap`, and layout-affecting flags such as `spineSpacing` and `nodeWrap`. Fourteen named themes: `consulting`, `executive`, `product`, `release`, `minimal`, `showcase`, `bytebytego`, `ai-timeline`, and others.
- **Flow:** `FlowTheme`—orientation, edge routing style, node geometry, font sizes.
- **Sequence:** `SequenceTheme`—participant render mode (box vs. card), row heights, activation bar widths, fragment styling.
- **Sixteen further grammars** (class, ER, state, tree, C4, chart, `gitGraph`, journey, sankey, kanban, requirement, block, packet, architecture) each with their own isolated token structs.

No named theme (“consulting”, “executive”) spans more than one grammar today. There is no shared vocabulary for palette roles, type scale, or spacing rhythm across components. This is the problem the new model resolves.

0.22.2 The Themes $\times$ Components Matrix

The new model is a two-dimensional system:

Theme (the row)

A general design language. It defines palette by role, type scale, spacing rhythm, density, and shape language—nothing component-specific. Adding a new theme adds a new row.

Component (the column)

A diagram grammar. It knows how to realise the general theme in its own geometric terms. Adding a new component adds a new column.

Intersection ($T \times C$)

How theme T renders component C . This is computed by C ’s implementation consuming T ’s general tokens; it is not stored as a separate artefact.

Theme $\downarrow$ / Component $\rightarrow$	Timeline Flow		SequenceTree		Class	...
Consulting	✓	✓	✓	✓	✓	...
Executive	✓	✓	✓	✓	✓	...
ByteByteGo	✓	✓	✓	✓	✓	...
Minimal	✓	✓	✓	✓	✓	...
<i>New theme</i>	✓	✓	✓	✓	✓	...

Table 43: Themes $\times$ Components matrix. A theme is the row authority; a component is the column implementer. A ✓ means the component’s implementation consumes that theme’s tokens. Add a theme once—every component renders it. Add a component once (implement the contract)—it inherits every existing theme for free.

This is the inverse of the fragmented model. In the fragmented model each component owns its theme, so adding a new theme requires editing every component; adding a new component requires writing a new, isolated theme. In the matrix model each direction is $O(1)$ in authoring work: a new theme is one new general-contract file; a new component is one new implementation that automatically supports all existing themes.

0.22.3 The General Theme Contract

The general theme contract is the primary design artefact of the theming system. It is an abstract, component-agnostic token vocabulary expressive enough to drive any component’s geometry, routing, and look—with zero component-specific fields. Every component’s implementation consumes this vocabulary; nothing outside it may be required for a correct rendering [59].

General-contract rule (binding): The general theme vocabulary **MUST** be rich enough that a component’s layout engine can ask the theme for spacing, density, routing style, and palette roles and receive a deterministic

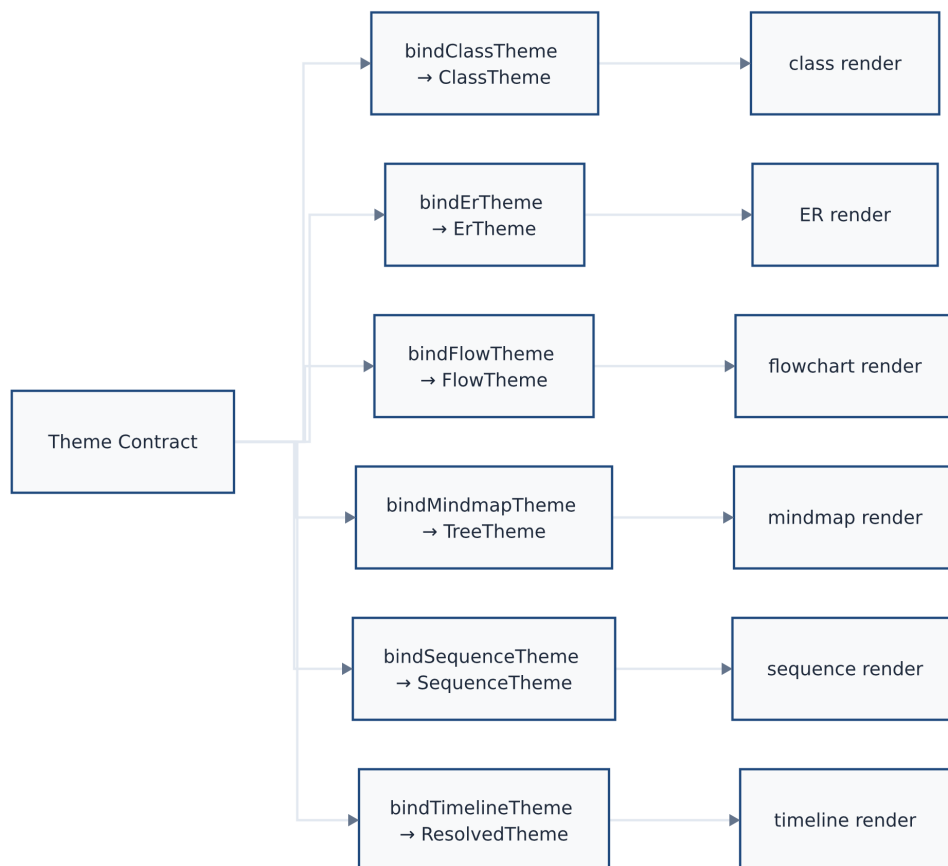


Figure 7: One Theme Contract, many component bindings. Each grammar (*flow*, *sequence*, *timeline*, ...) exposes a `bind{Grammar}Theme` function that translates the contract's general tokens into its own specific vocabulary. Adding a new component is one new binding; adding a new theme is one new contract definition.

Rendered by the diagram compiler this document describes (source: design/figures/src/theme-contract.mmd, built with make figures).

answer. No component-specific token may leak into the general contract. Components may *interpret* tokens in their own context; they may not *demand* tokens not in the contract.

The vocabulary is organized into six domains; the specific field names are subject to refinement but the domain coverage is fixed:

0.22.3.1 Palette

The general contract carries **two distinct palette systems**. Structural components consume the *role palette*; data and chart components consume the *data palette*. Both systems are part of the general contract.

Role palette (semantic/structural). Structural roles are expressed by name, not raw hex. Raw hex values are an implementation detail of a specific theme instance.

```

1 palette:
2   # Surface roles
3   surface:      "#FFFFFF"    # primary background
4   surface-raised: "#F8F8F8"  # cards, panels
5   surface-overlay: "#0A0A0A" # dark overlay / inverse background
6   surface-panel: "#F4F4F4"  # lane / panel / track-header
   background (distinct from surface)
7
8   # Content roles
9   ink:          "#111111"    # primary text
10  ink-muted:     "#666666"    # secondary / de-emphasized text
11  ink-inverse:   "#FFFFFF"    # text on dark backgrounds
12  ink-panel:     "#333333"    # text/ink rendered on panel or lane
   backgrounds
13
14  # Brand / accent
15  accent:        "#1F497D"    # primary brand colour
16  accent-muted:  "#6A92BF"    # lighter accent (inactive, planned)
17  accent-strong: "#0D2B4E"    # darker accent (active, in-progress)
18
19  # Border
20  border:         "#DDDDDD"    # default stroke / divider
21  border-strong:  "#999999"    # emphasized boundary
22
23  # Muted fill (background regions, disabled states)
24  muted:          "#F0F0F0"
25  muted-strong:   "#CCCCCC"
26
27  # Semantic status roles (components map these to IR status values)
28  status-success: { fill: "#4CAF50", stroke: "#388E3C" }
29  status-warning:  { fill: "#D97706", stroke: "#A35A04" }
30  status-error:    { fill: "#DC2626", stroke: "#A81919" }
31  status-info:     { fill: "#1F497D", stroke: "#0D2B4E" }
32
33  # IR workflow states (super-set of status roles above)
34  status-planned:  { fill: "#6A92BF", stroke: "#1F497D" }
35  status-active:   { fill: "#1F497D", stroke: "#0D2B4E" }
36  status-done:     { fill: "#888888", stroke: "#555555", opacity:
   0.85 }
```

```

37 status-cancelled: { fill: "#CCCCCC", stroke: "#999999", opacity:
    0.60 }
38 status-uncertain: { fill: "#CCCCCC", stroke: "#999999", opacity:
    0.70 }
39
40 # Optional fill pattern applied to workflow-state renders
41 status-pattern: "solid" # solid | hatch | dashed-border

```

Listing 45: General theme contract: role palette (decided)

Data palette (quantities and sequences). Chart and data components (pie, XY chart, kanban, gitGraph, radar, sankey) consume the data palette. It provides three sub-systems covering the full range of quantitative encodings [59].

```

1 data-palette:
2   # Categorical sequence – ordered colours for pie slices, chart
   series,
3   # kanban columns, branch colours. Use by index (0-based); themes
   may
4   # supply any length >= 6.
5   categorical:
6     - "#1F497D" # 0 – primary
7     - "#6A92BF" # 1
8     - "#4CAF50" # 2
9     - "#D97706" # 3
10    - "#9C27B0" # 4
11    - "#00BCD4" # 5
12
13   # Sequential ramp – for ordered quantitative encodings (sankey
14   # throughput, heat intensity, radar fill). Low → high.
15   sequential:
16     low: "#EEF4FB"
17     mid: "#6A92BF"
18     high: "#0D2B4E"
19
20   # Diverging ramp – for values around a meaningful midpoint
21   # (positive/negative, above/below target). Negative → zero →
   positive.
22   diverging:
23     negative: "#DC2626"
24     zero: "#F5F5F5"
25     positive: "#4CAF50"

```

Listing 46: General theme contract: data palette (decided)

0.22.3.2 Typography

Typography specifies family and fallback, a *type scale* (not a list of fixed sizes), and a weight set. Components consume the scale by step name, not by pixel value.

```

1 typography:
2   family: "Helvetica Neue"
3   fallback: "Arial, Helvetica, sans-serif"
4   # Embedded font files for deterministic metric computation.
5   files:

```

```

6   regular: "fonts/HelveticaNeue-Regular.woff2"
7   bold:    "fonts/HelveticaNeue-Bold.woff2"
8
9   # Type scale: step names → point sizes.
10  # Components reference steps by name, not by pt value.
11  scale:
12    xs:  8    # pt – axis ticks, dense labels
13    sm:  9    # pt – captions, secondary labels
14    base: 11   # pt – body text, activity labels
15    md:  13   # pt – subtitles, section headers
16    lg:  18   # pt – document title
17    xl:  24   # pt – hero / keynote headline
18
19  # Weight palette
20  weights:
21    regular: 400
22    medium:  500
23    semibold: 600
24    bold:    700

```

Listing 47: General theme contract: typography block (proposed shape)

0.22.3.3 Spacing / Rhythm Scale

A spacing scale defines the rhythm unit and derived named steps. Components consult the scale as *advice*, not as a binding pixel-identical contract: each component maps step names onto its own geometry as it sees fit, preserving layout judgement while sharing a common rhythm. The scale is not normalised across components.

```

1  spacing:
2    unit: 8          # px – base rhythm unit
3    # Named steps (advisory; components map these to their own
4      geometry)
5    steps:
6      xxs:  2    # 0.25u
7      xs:   4    # 0.5u
8      sm:   8    # 1u
9      md:  16    # 2u
10     lg:  24    # 3u
11     xl:  32    # 4u
12     xxl: 48    # 6u

```

Listing 48: General theme contract: spacing block (advisory)

0.22.3.4 Density

Density is a discrete named level that components use to select between compact and comfortable configurations. Three levels are defined—no more, no less. The layout engine consults density to choose row heights, inter-element gaps, and label truncation thresholds (e.g. whether secondary labels are shown at all).

```

1  density: "normal" # compact | normal | comfortable

```

Listing 49: General theme contract: density (decided)

A **compact** theme instructs every component to select its smallest safe row height, tightest inter-element gap, and most aggressive label truncation. A **comfortable** theme selects the most generous variants. Components map the three density levels to their own geometry constants; density is not a continuous scalar.

0.22.3.5 Shape Language

Shape language defines how geometric objects feel. Components use these tokens when deciding corner radius, node padding, stroke scale, and connector style.

```

1 shape :
2   corner-radius: 0      # px - 0 = square; >0 = rounded
3   node-padding:  8      # px - internal padding inside node shapes
4   stroke-scale:  1.0    # multiplier on all stroke widths (0.5 =
                        hairline; 2.0 = bold)
5   connector-style: "elbow" # straight | elbow | curved | orthogonal
6   marker-shape: "circle" # circle | diamond | square - milestone /
                        marker glyph

```

Listing 50: General theme contract: shape block (proposed shape)

0.22.3.6 Effects and Motion

Effects declare the fidelity tier and optional art-effect tokens. These gate which rendering backend capability profile is required (see Section 0.22.8).

```

1 effects :
2   fidelity: 1          # 0=Minimal | 1=Crisp | 2=Polished | 3=Showcase
3   drop-shadow: { enabled: false }
4   glow:         { enabled: false }
5   motion:       { enabled: false } # SMIL animation opt-in
                        (Section~14)

```

Listing 51: General theme contract: effects block (proposed shape)

0.22.3.7 The Timeline ResolvedTheme as the Generalisation Seed

The timeline’s `ResolvedTheme` (`packages/core/src/themes/types.ts`) is the closest existing artefact to the general contract. It already covers canvas, typography (with weights), axis geometry, track geometry, and layout-affecting flags (`spineSpacing`, `nodeWrap`, `maxSubLanes`). It demonstrates that a theme can—and must—drive layout decisions, not merely colour.

The migration plan is to *generalise upward*: extract the conceptual domains (palette roles, type scale, spacing rhythm, density, shape language) from `ResolvedTheme` into the shared general contract, then recast the timeline-specific fields (axis height, track header width, serpentine row spacing) as the timeline component’s interpretation of those general tokens [48].

0.22.4 Theme Drives Geometry, Layout, and Routing

Themes are not a post-processing colour filter. The layout engine queries the theme *during* layout computation. Representative examples from the existing codebase:

- **spineSpacing: 'time' vs. 'even'** — controls whether vertical-spine milestone nodes are placed proportionally to elapsed time or at uniform y-intervals. This changes node y-coordinates, not just colour.
- **nodeWrap: 'over-under'** — routes the horizontal timeline spine around each on-axis node as a semicircular arc (alternating above / below), producing entirely different path geometry compared to the default straight line.
- **density** — compact density reduces `row_height` and `sub_lane_height`, shrinking the canvas and shifting all track and milestone y-coordinates.
- **shape.corner-radius** — a flow diagram component may use this token to choose between rectangular and rounded node shapes, which affects label area computation and thus label wrap point.
- **connector-style: 'orthogonal'** — instructs the flow / sequence layout engine to use orthogonal edge routing rather than diagonal or curved, changing every edge path.

Layout-engine rule (binding): A layout engine **MUST** consult the active theme for any geometry decision that varies across themes. It **MUST NOT** hard-code values that the general contract expresses. Theme-driven geometry is still *closed-form*: the theme supplies constants; the engine computes coordinates deterministically from those constants. No iterative solver or random sampling is introduced.

0.22.5 Reach: Theme, Layout, and New Components

Drawing the right boundary between theme, layout argument, and new grammar is the architectural decision that keeps the system from proliferating components unnecessarily. The rule is stated at the IR boundary:

New component (grammar)

Only when the diagram requires *new semantic data* that the existing IR cannot carry. A new grammar is a last resort.

Layout argument

When the same data can be positioned differently without needing new IR fields. A layout mode is a named argument passed at render time, not a new grammar.

Theme tokens

Look-and-feel within a layout: colour, typography, spacing, shape, effects. A theme produces a visually distinct output from the same IR + same layout.

Worked example: the timeline as six diagrams. The timeline IR is one grammar. A single set of entries—each with a date, label, status, and optional icon—can be

Layout argument	Theme	Visual result
vertical-spine	consulting	Spine-and-card with navy accent, tidy whitespace
horizontal-arc	executive	Arc path with serif labels, subtle quarter shading
roadmap	product	Horizontal Gantt-style bars, dense information
serpentine	showcase	Gradient boustrophedon path, premium glow effects
gantt	release	Traffic-light Gantt, monospace labels, weekly grid
timeline-columns	minimal	Column-per-period grid, greyscale, LaTeX-safe

Table 44: One timeline IR; six visual outputs via `{layout, theme}` arguments. No new grammar is required for any of these. Same data, radically different look and behaviour.

rendered six ways that look and behave radically differently, purely via `{layout: L, theme: T}`:

The rule of thumb: if the data the IR already carries is sufficient to drive the new visual, use a layout argument and/or theme tokens. Only introduce a new grammar when the IR would need to grow new fields to support it (e.g., swimlane headers, dependency arrows, resource calendars—these are genuinely new semantic data, not layout variations).

0.22.6 Mermaid Configuration Surface

All user-facing surface is Mermaid or Mermaid-like. Theme selection, token overrides, layout selection, and density are expressed via Mermaid-native configuration mechanisms, so a user can remain in standard Mermaid syntax while accessing the extended vocabulary.

Frontmatter (`- ... -`).

```

1 ---
2 theme: executive           # named theme from the registry
3 layout: vertical-spine    # layout family for this diagram
4 density: compact         # compact | normal | comfortable
5 tokens:                  # inline token overrides (scoped to this
6   diagram)
7   palette.accent: "#8B0000"
8   typography.scale.base: 12
9 ---
10 timeline
11   2026-Q1 : Project kick-off
12   2026-Q2 : Beta release
13   2026-Q3 : GA

```

Listing 52: Theme and layout selection via Mermaid YAML frontmatter

Init directive (`%%{init: ... }%%`). For compatibility with tools that strip frontmatter, the same surface is available via Mermaid’s existing init directive hook:

```

1 %%{init: {"theme": "executive", "layout": "vertical-spine",
2   "density": "compact"}}%%
3 timeline
4   2026-Q1 : Project kick-off

```

Listing 53: Theme selection via Mermaid init directive

Surface contract. Mermaid is the floor: any standard Mermaid document is valid input, and the compiler respects Mermaid’s built-in theme names (`default`, `dark`, `neutral`, `forest`, `base`) as aliases to our nearest equivalent. Our named themes and token overrides are the ceiling—available but never required. A document with no theming directives renders under the default theme [31].

0.22.7 Determinism Under Theme-Driven Layout

Theme-driven geometry does not compromise determinism. The constraint is:

Determinism rule (binding): Themes supply *constants*; engines compute coordinates *closed-form* from those constants. Same IR + same theme + same layout argument $\rightarrow$ byte-identical output. Themes may change *which* constants are supplied; they may not introduce randomness, iteration-count sensitivity, or non-deterministic solver state [48].

This is already the case for the existing layout-driving tokens: `spineSpacing` and `nodeWrap` are constants supplied to the layout engine; the engine’s coordinate computation is a closed-form function of those constants. Extending this discipline to the general contract requires only that every new token be a scalar, enumerated value, or named step—not a runtime-computed value.

Effect tokens (`drop-shadow`, `glow`) degrade gracefully per backend; the *geometry* under any effect is always identical (see Section 0.22.8).

0.22.8 Fidelity Tiers and Backend Effect Profiles

Fidelity tiers define what class of visual richness a theme requests and therefore which rendering backend is sufficient to honour it fully. A backend that cannot satisfy the requested tier applies the fallback policies declared in each effect’s `fallback-policy` field and degrades gracefully; *geometry is never affected*.

Degradation policy. A Tier-3 cloud-layer effect targeting the SVG backend:

1. The SVG backend checks capability against `cloud-layer`: “not available.”
2. It reads the effect’s `fallback-policy`: `embed-raster`.
3. It invokes the Raster backend for the cloud-layer group only, receives a PNG, and embeds it as a `<image>` element. The rest of the diagram remains clean vector.
4. If the Raster backend is also unavailable, the policy chain falls to `omit`: the cloud layer is silently dropped; no geometry changes.

Geometry invariant under degradation. Fallback decisions never alter coordinates, dimensions, or label positions. The Scene geometry is the invariant shared across all backends and all fidelity levels.

Tier	Name	Effect classes	Min. backend	Examples
0	Minimal	No effects; solid fills, patterns, and opacity only	SVG backend	Minimal
1	Crisp	Linear/radial gradients, diagonal hatch, dashed borders, clip paths	SVG backend	Consulting, Release
2	Polished	Drop shadows, soft glow; native PPTX effects; SVG safe-filter set (determinism caveat)	SVG (caveat) or Raster	Executive, Product
3	Showcase	Bloom, cloud/atmosphere layers, noise textures, gradient meshes, volumetric compositing	Raster backend required	Showcase, ByteGo

Table 45: Fidelity tier definitions, effect classes, minimum required backend, and example themes

0.22.9 Theme Inheritance and Extension

Themes support single-level inheritance via the `extends` key. A child theme specifies only the tokens that differ from its parent; all others inherit. This enables branded variants (`acme-consulting` extending `consulting`) with minimal duplication.

```

1 theme-id:      "acme-corp"
2 display-name:  "ACME Corporate"
3 extends:      "consulting"
4
5 palette:
6   accent:      "#8B0000"    # ACME brand red replaces navy
7   accent-strong: "#5A0000"
8
9 typography:
10  family:      "Futura"
11  fallback:    "Century Gothic, sans-serif"
12  files:
13    regular:    "fonts/Futura-Book.woff2"
14    bold:       "fonts/Futura-Bold.woff2"

```

Listing 54: Custom theme extending Consulting (general-contract form)

Inheritance rules:

1. Properties not specified in the child are taken from the parent exactly.
2. `palette` and nested maps are merged entry-by-entry: a child overrides only the entries it declares; unspecified entries inherit from parent.
3. A font family change requires supplying the corresponding embedded font files. A theme that names a font without providing files is malformed.

4. Inheritance depth is exactly one level. Chains (A extends B extends C) are not supported.
5. The general contract must be fully satisfiable from the resolved (merged) theme. A child that leaves a required contract field unresolved is malformed.

0.22.10 Built-in Named Themes

The compiler ships six built-in named themes that address the primary use cases identified across David’s research and the existing timeline implementation. Each is a complete realisation of the general contract; each component’s implementation produces its own visual interpretation of the same theme tokens [33, 56].

consulting (Tier 1 — Crisp)

McKinsey/BCG-style. Black, white, single navy accent. Heavy typography, generous whitespace, square corners, no decorative chrome. **density:** `normal`. Optimised for A4/Letter print and A3 poster output.

executive (Tier 2 — Polished)

Corporate boardroom. Navy/slate on white, subtle quarter shading, 4 px corner radius on bars, serif heading font. Status communicated by colour + icon. **density:** `normal`. 16:9 slide target.

product (Tier 2 — Polished)

Developer-friendly; information-dense. Per-track colour coding. Category colours over status colours; status via small icon. **density:** `compact`. 1400 px canvas.

release (Tier 1 — Crisp)

Engineering release calendar. Traffic-light palette (green/amber/red). Monospace font. Today marker most prominent element. **density:** `compact`. Optimised for CI/CD dashboard display.

minimal (Tier 0 — Minimal)

Maximum whitespace, minimum decoration. Single grey fill regardless of status; pattern differentiates status. Print- optimised; renders correctly in greyscale. **density:** `comfortable`.

showcase (Tier 3 — Showcase)

Premium keynote aesthetic. Gradient fills, drop shadows on nodes, optional atmospheric cloud layer. Full fidelity requires Raster backend. **density:** `normal`. 1920 px canvas.

Cross-component example. Consider the consulting theme applied to three component types simultaneously in a composition poster:

- **Timeline component:** `spineSpacing: time`, accent drives activity bars, `corner-radius: 0` produces square bars, `scale.base = 11 pt` for activity labels.
- **Flow component:** accent drives node fills, `ink` drives edge labels, `corner-radius: 0` produces rectangular nodes, `scale.sm = 9 pt` for edge labels.
- **Sequence component:** accent drives participant header fills, `ink-muted` drives lifeline colour, `scale.base = 11 pt` for message labels.

All three diagrams share the same navy accent, the same type scale, the same square

shape language, and the same generous whitespace rhythm—because all three consume the same consulting general-contract tokens. The poster is visually coherent without any per-component style co-ordination.

0.22.11 The Three-Tier Token Architecture

Token vocabulary is organized into three tiers following the standard design-token pattern [59]. The tier boundaries are firm: references flow *upward only*; the general contract is never contaminated by component-specific concerns.

Tier 1 — Primitives

Raw, un-themed values: colour ramps (HSL ladders, raw hex families), typeface family names, the spacing base unit. These are the raw material from which Tier-2 tokens are assembled. Components *never* reference Tier-1 tokens directly; themes build Tier-2 values from them.

Tier 2 — Semantic tokens (the general contract)

Role palette, data palette, type scale and weights, spacing steps, density, shape language (corner radius, node padding, stroke scale, connector style), and effects/fidelity tier. **This is the interface every component implements against.** The full Tier-2 shape is specified in Section 0.22.3 and illustrated in the listings above.

Tier 3 — Component tokens

Namespaced components.name.token (e.g. components.serpentine.turnRadius, components.sankey.ribbonWidth, components.gitgraph.mergeCurveTension). Each Tier-3 token is *derived* from a Tier-2 token by the component’s theme-binding as its default. A concrete theme *may* override any Tier-3 token as an optional fine-tuning step; it is never required to do so.

Binding invariants (tier contract):

- The general contract (Tier 2) **NEVER** references a Tier-3 component token. The contract is component-agnostic by construction.
- Components reference **upward only**: Tier-3 tokens derive from Tier-2 defaults; a component may not require a Tier-2 token that does not exist in the contract.
- A theme **may** reach downward into a components.name namespace as an optional override. This is the escape hatch for per-component fine-tuning; it is never mandatory.

0.22.11.1 Tier-3 Example: serpentine turn radius

```

1 # Tier 2 (general contract, theme-authored)
2 shape:
3   corner-radius: 4          # px - general shape language
4
5 # Tier 3 (component binding, inside serpentine layout engine)
6 # Default derivation (computed at bind time, not stored in the theme
7   file):
8 #   components.serpentine.turnRadius = shape.corner-radius * 6
9 # →   4 * 6 = 24 px (sufficient arc to avoid a sharp hairpin)
```

```

10 # Optional Tier-3 override in a concrete theme (e.g. "showcase")
11 components:
12   serpentine:
13     turnRadius: 40      # override: generous arc for showcase
14     aesthetic
15   sankey:
16     ribbonOpacity: 0.55 # override: slightly more opaque ribbons
17   gitgraph:
18     mergeCurveTension: 0.4 # override: softer merge-curve

```

Listing 55: Tier-3 token deriving from Tier-2 with optional theme override

The `components` block is optional in every theme. A theme that omits it entirely is valid and fully correct: all Tier-3 tokens resolve to their Tier-2 derivation defaults. Only themes with a strong visual rationale for per-component fine-tuning need to supply it.

0.22.12 Contract Vocabulary Summary

The Tier-2 contract in full, showing all six domains together:

```

1 # Palette
2 palette:      # role palette – structural components
3   surface:    # see §12.3 for full role list
4   surface-panel: # lane / panel / track-header background
5   ink:
6   ink-panel:  # text rendered on panel backgrounds
7   accent:
8   border:
9   muted:
10  status-success: { fill: , stroke: }
11  status-warning: { fill: , stroke: }
12  status-error:   { fill: , stroke: }
13  status-info:    { fill: , stroke: }
14  # workflow states: status-planned, status-active, status-done,
15  #                   status-cancelled, status-uncertain
16  status-pattern: "solid" # solid | hatch | dashed-border
17
18 data-palette:  # data palette – chart/quantity components
19   categorical: [ , , , , , ] # min 6 entries, ordered
20   sequential:  { low: , mid: , high: }
21   diverging:   { negative: , zero: , positive: }
22
23 # Typography
24 typography:
25   family:
26   fallback:
27   files: { regular: , bold: }
28   scale: { xs: 8, sm: 9, base: 11, md: 13, lg: 18, xl: 24 } # pt
29   weights: { regular: 400, medium: 500, semibold: 600, bold: 700 }
30
31 # Spacing (advisory)
32 spacing:
33   unit: 8 # px – base rhythm unit
34   steps: { xxs: 2, xs: 4, sm: 8, md: 16, lg: 24, xl: 32, xxl: 48 }
35
36 # Density

```

```

37 density: "normal"    # compact | normal | comfortable
38
39 # Shape language
40 shape:
41   corner-radius: 0
42   node-padding: 8
43   stroke-scale: 1.0
44   connector-style: "elbow" # straight | elbow | curved | orthogonal
45   marker-shape: "circle"   # circle | diamond | square
46
47 # Effects / fidelity
48 effects:
49   fidelity: 1 # 0=Minimal | 1=Crisp | 2=Polished | 3=Showcase
50   drop-shadow: { enabled: false }
51   glow:        { enabled: false }
52   motion:      { enabled: false }

```

Listing 56: Full Tier-2 general contract skeleton (decided)

0.22.13 Contract Adoption Plan

0.22.13.1 Proof Set: Validate the Contract First

Before migrating any production component, the contract is validated by a spike: implement the **executive** theme against three components chosen to exercise the entire contract surface jointly.

flowchart

Node-link boxes, edge routing, orientation. Exercises: role palette, shape language (corner radius, connector style), type scale, density, spacing advisory steps.

sequence

Text/compartiment geometry, activation bars, fragments, lifelines. Exercises: role palette, type scale, density (row heights, secondary label visibility), shape language (node padding, corner radius).

xychart

Axes, gridlines, series bars/lines, tick labels. Exercises: the *data palette* (categorical sequence + sequential ramp), type scale, spacing rhythm. This is the critical third leg: the other two components do not touch the data palette, so xychart is required to prove the full contract surface.

The spike succeeds when all three diagrams render coherently under the single **executive** theme token set without any component-specific hacks. A passing spike commits the Tier-2 vocabulary as final [59].

0.22.13.2 Migration Order

After the proof spike, the 16 existing per-component `theme.ts` files are migrated in the following order:

1. **Generalise the timeline ResolvedTheme upward** into the Tier-2 contract (`packages/core/src/theme`). The timeline is the deepest existing layout-driven theme and serves as the reference implementation; all subsequent migrations align to the shape it establishes.
2. **Node-link family:** `class`, `state`, `ER`, `C4`, `requirement`, `block`, `architecture`. These share the same fundamental node-edge visual vocabulary and are highest value after the proof set.
3. **Remaining charts:** `pie`, `quadrant`, `radar`. These consume the data palette and validate categorical and diverging ramp usage beyond `xychart`.
4. **Timeline-family adoption of the contract.** The timeline component re-consumes the now-general Tier-2 contract, keeping its Tier-3 tokens (`components.serpentine.*`, `components.roadmap.*`) as derived overrides.
5. **Specialised components:** `sankey`, `gitGraph`, `journey`, `kanban`, `mindmap`, `packet`. These have the most Tier-3 component-specific tokens and are migrated last once the Tier-3 override mechanism is proven by earlier steps.

Migration invariant: Each step is independently shippable. No step requires a downstream step to complete. At every point the compiler is in a valid state: un-migrated components continue to consume their legacy `theme.ts`; migrated components consume the general contract. Both coexist until Step 5 completes.

0.22.14 Implementation Status (2026)

The contract described in this section is fully implemented. The Tier-2 `ThemeContract` interface and the `executive` reference theme reside in `packages/core/src/theme-contract/`. Every one of the 21 diagram components now adopts the contract via a per-component binding: for most grammars the binding is `grammars/<component>/contract-binding.ts`; the timeline uses `themes/contract-binding.ts`. Adoption is **opt-in**: a theme name registered in `CONTRACT_THEMES` resolves via the binding; all legacy per-component named themes are unchanged and their rendered output is byte-identical to pre-contract builds. The `executive` theme has been verified to render all 21 diagram types as one coherent design system (navy accent, Georgia serif, white surface).

- **Precedence rule:** Legacy component theme names always win. A contract theme name must not duplicate a name already claimed by a legacy per-component theme; the `CONTRACT_THEMES` resolver never overrides a legacy route.
- **Contract location:** `packages/core/src/theme-contract/` (`ThemeContract` Tier-2 interface + `executive` concrete theme).
- **Binding pattern:** `grammars/<component>/contract-binding.ts` (20 grammars) and `themes/contract-binding.ts` (timeline) — 21 bindings total.

0.23 The Aesthetic Bar: Beating Mermaid Visually

Aesthetics is not a finishing coat applied after architecture—it is a **first-class pillar** of the product. The explicit goal: every diagram produced by this compiler must look

demonstrably better than its Mermaid equivalent. “Pro,” “neat,” “coherent” are requirements, not aspirations.

0.23.1 Why Aesthetics Is Architecture

Mermaid is ubiquitous because of its ecosystem integration, not its visual quality. Users accept its output because it’s free and frictionless—not because it looks good. The aesthetic gap is the primary differentiation lever:

Mermaid

Documentation-grade. Fixed styles. Poor typography. No design system. Functional but never beautiful.

This compiler

Presentation-grade. Themeable. Proper typography. Coherent design language. Output you’d put in a board deck or technical blog.

This is not subjective polish—it is the **primary reason users choose this tool over Mermaid**. If the output doesn’t look materially better, there is no product.

0.23.2 The Design System

Every diagram shares a coherent visual language:

Typography

System-level font stack with proper weight hierarchy (title/heading/body/label/caption). Consistent sizing ratios across all diagram types. No all-caps labels. No tiny illegible text.

Spacing and rhythm

Consistent padding, margins, and gaps derived from a base unit. Generous whitespace. Visual breathing room. The “no clutter” principle (Section 0.3.7) is enforced structurally.

Color palette

Curated palettes with proper contrast ratios (WCAG AA minimum). Semantic color mapping: status colors, category colors, emphasis colors. Dark and light modes as first-class variants, not afterthoughts.

Shape language

Rounded corners, consistent border weights, soft shadows where appropriate. No harsh black-on-white wireframe aesthetic.

Edge/connector style

Smooth curves, proper arrowheads, consistent stroke widths. Routing that avoids overlaps where the layout algorithm permits.

0.23.3 Default Theme Craft

The default theme must be beautiful *out of the box*. Users who never configure a theme should still produce output that surpasses Mermaid. Design commitments:

- Modern, clean aesthetic inspired by technical publications (not wireframes)
- Light theme: warm grays, blue accent, Inter/system-ui stack
- Dark theme: deep navy/charcoal, blue-teal accent, proper contrast
- Professional appearance suitable for slides, blog posts, documentation
- No garish colors, no 1990s UML aesthetic, no pixelated edges

0.23.4 Brand Themes and Coherence

The theme system enables organization-specific brand themes:

- Themes can define complete visual identities (colors, fonts, shapes, spacing)
- Single inheritance: branded themes extend base themes, overriding selectively
- Cross-diagram coherence: a theme applied to a poster produces visually unified output across all constituent diagram types
- Built-in exemplars: professional-light, professional-dark, bytebytego (inspired by the popular technical illustration style)

0.23.5 Grammar = Semantics; Theme = Style

The architectural discipline that enables aesthetic excellence:

The Domain IR carries what to communicate. The theme carries how it looks. Never mix them. A theme change cannot alter diagram meaning. A content change cannot alter diagram style.

This separation means:

- The same diagram looks professional in professional-light and playful in a hypothetical sketch theme—without any IR changes.
- Agents author content without making aesthetic decisions.
- Designers craft themes without understanding grammar semantics.
- New grammars inherit the full theme system and look coherent from day one.

0.23.6 Aesthetic Verification

Beauty is hard to test, but we enforce baselines:

- **Golden-image regression:** visual output is diffed against known-good baselines. Any change to rendering requires explicit approval.
- **Contrast ratio checks:** automated WCAG AA verification for all text-on-background combinations in built-in themes.
- **Gallery examples:** every grammar ships with gallery outputs demonstrating the aesthetic bar. These serve as both documentation and regression targets.

- **Side-by-side comparisons:** marketing/documentation includes Mermaid vs. ours comparisons for every supported diagram type.

0.23.7 Implementation Status

Built: The theme system is fully implemented. Five timeline themes (including dark and ByteByteGo variants), two flow themes, two sequence themes, two tree themes, and two composition themes are shipped. All enforce the grammar=semantics/theme=style discipline.

In progress: Expanding the aesthetic standard to the new diagram types as they ship. The UML/software line (Tier-1) will establish the “modern UML” visual standard that differentiates from PlantUML’s dated look.

Part VI

Composition

0.24 Composition: Multi-Panel Posters

The Composition layer arranges multiple grammar outputs into a single poster or infographic. This is how the multi-section corpus images (10x-coding-playbook, archon-harness-builder, rag-vectorless-explainer) are assembled.

Implementation Status. Fully implemented in `packages/core/src/composition/` with grid-based layout and `scene-transform.ts` kernel helper (`translateAndScale`, `embedSceneInRect`). Supports content-driven row sizing, multi-cell posters, and inline grammar IR embedding. `CompositionTheme` with light and dark-poster variants. **Deferred:** `ir_file` external URI references (`pkg:`, `file:`, `http:`); currently inline embedding only.

0.24.1 Design Philosophy

Composition is a thin layer. The Composition layer does not define new visual primitives. It arranges existing grammar outputs (each a Scene IR) into a larger canvas.

Each panel is an independent grammar. A panel might be a timeline, a flow diagram, a sequence diagram, a tree, or a simple content element (stat callout, text block). Each panel has its own Domain IR and is compiled independently.

The composition specifies arrangement, not content. Content is defined in each panel's Domain IR. The Composition IR specifies panel positions, sizes, and inter-panel elements (dividers, titles).

Grammar = semantics; theme = style applies to composition too. The Composition IR describes WHAT is arranged WHERE (structure). HOW it looks (borders, backgrounds, title fonts, gap fills) is a `CompositionTheme` — never embedded in the IR.

0.24.2 Definitions

Composition

A document that holds one or more **cells** arranged in a deterministic grid layout. A Composition compiles to a single merged Scene IR.

Cell

A positioned slot in the grid. A cell either (a) references/embeds a sub-document of any grammar (timeline, sequence, tree, flow), or (b) holds a simple content element (title block, stat callout, text block, image).

Grid

The canonical layout model: rows $\times$ columns. A grid of R rows and C columns defines $R \times C$ unit slots. Cells may span multiple rows or columns. A *stack* is the special case $C = 1$ (vertical) or $R = 1$ (horizontal).

Panel Chrome

Visual decoration around/within a cell (border, background, title bar, caption) rendered by the composition engine from theme tokens.

Sub-Scene

The Scene IR produced by compiling a cell's grammar content through that grammar's layout engine.

0.24.3 Composition IR — Formal Shape

The Composition IR is a small, total, self-contained document. Every field has a well-defined type; optional fields state their defaults.

```

1 CompositionDocument:
2   version: "1.0"                                # REQUIRED; semver-major
3   metadata:                                       # REQUIRED
4     title: string                                # REQUIRED; poster title
5     subtitle?: string                             # default: absent
6     theme?: string                               # CompositionTheme name;
7       default: "default"
8   canvas:
9     width: number                                # REQUIRED; logical px
10    height?: number | "auto"                     # default: "auto" (computed)
11  grid:                                           # REQUIRED; canonical layout
12    columns: integer                             # >= 1; REQUIRED
13    rows?: integer | "auto"                      # default: "auto"
14      (ceil(cellCount / columns))
15    gap?: number                                  # inter-cell gap in px;
16      default: 0
17    padding?: number                             # outer canvas padding;
18      default: 0
19  cells: Cell[]                                  # REQUIRED; >= 1 cell;
20    row-major order
21
22 Cell:
23   id: string                                     # REQUIRED; unique within
24     composition
25   row?: integer                                 # 0-indexed; default: assigned
26     row-major
27   col?: integer                                # 0-indexed; default: assigned
28     row-major
29   rowSpan?: integer                             # default: 1
30   colSpan?: integer                             # default: 1
31   title?: string                               # cell title; rendered as
32     panel chrome
33   caption?: string                             # below-content caption
34   content: CellContent                         # REQUIRED; what this cell
35     holds
36
37 CellContent:                                     # discriminated union on "kind"
38   | GrammarContent                             # sub-diagram (any grammar)
39   | StatContent                                # stat callout (number +
40     caption)
41   | TextContent                               # rich text block
42   | TitleContent                              # section title / heading
43   | ImageContent                              # static image embed

```

```

33
34 GrammarContent:
35   kind: "grammar"
36   grammar: "timeline" | "sequence" | "tree" | "flow" # grammar name
37   ir?: object # inline Domain IR
38   (grammar-specific)
39   ir_file?: string # OR external file reference
40   # Exactly one of ir / ir_file REQUIRED.
41
42 StatContent:
43   kind: "stat"
44   value: string # e.g. "10x", "99.9%"
45   label: string # description beneath stat
46   trend?: "up" | "down" | "neutral" # optional trend indicator
47
48 TextContent:
49   kind: "text"
50   body: string # markdown-subset (bold,
51   italic, bullet)
52   align?: "left" | "center" | "right" # default: "left"
53
54 TitleContent:
55   kind: "title"
56   text: string
57   level?: 1 | 2 | 3 # heading level; default: 1
58
59 ImageContent:
60   kind: "image"
61   src: string # file path or data URI
62   alt?: string # accessibility text

```

Listing 57: Composition IR — complete type definition (pseudo-schema)

Identity. Each `Cell.id` is unique within a composition. `CompositionDocument.metadata.title` serves as the document identity for hashing and golden tests.

Row-major default ordering. If `row/col` are omitted from cells, they are assigned left-to-right, top-to-bottom in declaration order. Explicit placement overrides the default.

Invariants.

- All `Cell.id` values are unique.
- Cell placements must not overlap (no two cells occupy the same grid slot).
- Spans must not exceed grid dimensions: $\text{col} + \text{colSpan} \leq \text{columns}$, $\text{row} + \text{rowSpan} \leq \text{rows}$.
- Exactly one of `ir` or `ir_file` must be present in `GrammarContent`.

0.24.4 The Sub-Scene Embed Mechanism

This section specifies **precisely** how independently-compiled sub-scenes are embedded into one merged Scene. This is the core composition mechanism.

0.24.4.1 Step 1: Compile Each Cell's Content to a Sub-Scene

For each cell whose content is kind: "grammar":

1. Resolve the Domain IR (inline `ir` or load `ir_file`).
2. Validate via that grammar's schema.
3. Invoke the grammar's layout engine: e.g., `buildSequenceScene(ir, theme) → Scene`.
4. The resulting Scene has deterministic width, height, and `primitives[]`.

For content elements (`stat`, `text`, `title`, `image`): a small content-layout function produces a Scene of known dimensions (text-measured).

Each cell now owns a **sub-scene** S_i with dimensions (w_i, h_i) .

0.24.4.2 Step 2: Compute Grid Layout (Cell Rectangles)

The composition engine computes a deterministic rectangle (x, y, W, H) for each cell based on the grid, sub-scene sizes, gaps, and padding. The algorithm is specified in Section 0.24.5.

0.24.4.3 Step 3: Translate and Scale Sub-Scenes into Cell Rectangles

For each cell i with sub-scene S_i (dimensions $w_i \times h_i$) and allocated rectangle (x_i, y_i, W_i, H_i) :

1. **Compute scale factor** (uniform, aspect-preserving):

$$s_i = \min\left(\frac{W_i}{w_i}, \frac{H_i}{h_i}\right)$$

If $s_i \geq 1.0$ (sub-scene fits without scaling), set $s_i = 1.0$ (no upscale).

2. **Compute centering offset** within the cell rectangle:

$$\Delta x_i = x_i + \frac{W_i - s_i \cdot w_i}{2}, \quad \Delta y_i = y_i + \frac{H_i - s_i \cdot h_i}{2}$$

3. **Apply transform to every primitive** in S_i .primitives:

$$\text{translateAndScale}(p, \Delta x_i, \Delta y_i, s_i) \rightarrow p'$$

This is a pure function over Scene primitives (definition below).

4. **Collect** all transformed primitives p' into the output list.

The `translateAndScale` kernel helper. **FLAG FOR BARBARA/MARK:** The composition layer requires a pure utility function on Scene primitives:

```

1  /**
2   * Pure function: offsets and uniformly scales all coordinate fields
      of a
3   * ScenePrimitive (and recursively, GroupPrimitive children).
4   *
5   * Coordinates: (x,y) -> (x*s + dx, y*s + dy)
6   * Dimensions:  (w,h) -> (w*s, h*s)
7   * Font sizes:  fontSize -> fontSize * s
8   * Stroke widths: strokeWidth -> strokeWidth * s
9   * Radii: r, rx -> r*s, rx*s
10  *
11  * Path 'd' strings: all numeric coordinates in the path are
      scaled/translated.
12  * StrokeGradient coordinates: x1,y1,x2,y2 transformed identically.
13  * GroupPrimitive: recurse into children.
14  *
15  * Determinism: output rounded via rhu(2dp) per the determinism
      contract.
16  */
17  function translateAndScale(
18    primitive: ScenePrimitive,
19    dx: number, dy: number,
20    scale: number
21  ): ScenePrimitive;
22
23  /**
24   * Convenience: transform an entire Scene into a target rectangle.
25   * Returns a new primitives[] array (the Scene itself is not
      mutated).
26  */
27  function embedSceneInRect(
28    scene: Scene,
29    targetRect: { x: number; y: number; width: number; height: number }
30  ): ScenePrimitive[];

```

Listing 58: `translateAndScale` — kernel helper (specification)

This function lives in the kernel (e.g., `packages/core/src/scene-transform.ts`) because it operates on Scene IR primitives directly. It must handle *every* primitive kind defined in `scene.ts`: `Line`, `Rect`, `Circle`, `Text`, `MultiText`, `Path`, `Group`, `Image`.

0.24.4.4 Step 4: Render Panel Chrome

After sub-scenes are embedded, the composition engine emits additional Scene primitives for chrome:

- Cell borders (Rect primitives with stroke, no fill)
- Cell titles (Text primitives positioned above/inside cell rect)
- Cell captions (Text primitives below cell rect)
- Cell backgrounds (Rect primitives behind sub-scene content)
- Composition header/footer (Text, Image primitives at canvas top/bottom)

- Dividers between rows/cells (Line primitives)

All chrome styling (colors, fonts, border radii, etc.) comes from `CompositionTheme` tokens — never from the IR.

0.24.4.5 Step 5: Merge into Final Scene

```

1 function buildCompositionScene(doc: CompositionDocument): Scene {
2   const grid = computeGridLayout(doc);      // Step 2
3   const allPrimitives: ScenePrimitive[] = [];
4
5   // Background (from theme)
6   allPrimitives.push(backgroundRect(grid.canvasWidth,
7     grid.canvasHeight));
8
9   // Header chrome
10  allPrimitives.push(...renderHeader(doc.metadata, theme));
11
12  // Cells: chrome-behind, then sub-scene, then chrome-above
13  for (const cell of grid.cells) { // row-major order
14    allPrimitives.push(...renderCellBackground(cell, theme));
15    allPrimitives.push(...embedSceneInRect(cell.subScene,
16      cell.rect));
17    allPrimitives.push(...renderCellChrome(cell, theme)); //
18      borders, title
19  }
20
21  // Footer chrome
22  allPrimitives.push(...renderFooter(doc.metadata, theme));
23
24  return {
25    width: grid.canvasWidth,
26    height: grid.canvasHeight,
27    background: theme.canvasBackground,
28    primitives: allPrimitives,
29  };
30 }

```

Listing 59: Final merge (pseudo-code)

Determinism. The merged Scene is deterministic because: (a) each sub-scene is deterministic (grammar contract), (b) cell iteration is row-major (fixed order), (c) `translateAndScale` is a pure function with rhu-rounded output, (d) chrome rendering is theme-deterministic. The resulting Scene can be hashed via `sceneHash()` for golden testing.

0.24.5 Layout Contract — Deterministic Grid Sizing

The grid sizing algorithm computes exact cell rectangles from sub-scene dimensions, spans, gaps, and padding. It is fully deterministic (no iteration, no heuristics).

0.24.5.1 Inputs

- C = number of columns (from `grid.columns`)
- R = number of rows (explicit or $\lceil |\text{cells}|/C \rceil$)
- G = gap (from `grid.gap`, default 0)
- P = padding (from `grid.padding`, default 0)
- W_{canvas} = canvas width (from `metadata.canvas.width`)
- For each cell i : sub-scene dimensions (w_i, h_i) , placement (r_i, c_i) , spans (rs_i, cs_i)

0.24.5.2 Column Width Computation

Available width after padding and gaps:

$$W_{\text{avail}} = W_{\text{canvas}} - 2P - (C - 1) \cdot G$$

Column widths are computed as **content-driven maximum**:

$$\text{colWidth}[j] = \max_{\substack{i: c_i=j, \\ cs_i=1}} w_i \quad (\text{max natural width of single-span cells in column } j)$$

If the sum of column widths exceeds W_{avail} , columns are **proportionally scaled** to fit:

$$\text{colWidth}[j] \leftarrow \text{colWidth}[j] \cdot \frac{W_{\text{avail}}}{\sum_k \text{colWidth}[k]}$$

Multi-column-span cells: their natural width is distributed across spanned columns only if needed (max propagation).

0.24.5.3 Row Height Computation

Row heights are computed analogously:

$$\text{rowHeight}[r] = \max_{\substack{i: r_i=r, \\ rs_i=1}} h_i \quad (\text{max natural height of single-span cells in row } r)$$

Row heights are *not* constrained by canvas height (canvas height is derived from rows when `height: "auto"`).

0.24.5.4 Cell Rectangle Computation

For cell i at grid position (r_i, c_i) with spans (rs_i, cs_i) :

$$x_i = P + \sum_{j=0}^{c_i-1} (\text{colWidth}[j] + G) \quad (1)$$

$$y_i = P + H_{\text{header}} + \sum_{k=0}^{r_i-1} (\text{rowHeight}[k] + G) \quad (2)$$

$$W_i = \sum_{j=c_i}^{c_i+cs_i-1} \text{colWidth}[j] + (cs_i - 1) \cdot G \quad (3)$$

$$H_i = \sum_{k=r_i}^{r_i+rs_i-1} \text{rowHeight}[k] + (rs_i - 1) \cdot G \quad (4)$$

Where H_{header} accounts for the composition header chrome height (if present).

0.24.5.5 Canvas Height (auto)

$$H_{\text{canvas}} = 2P + H_{\text{header}} + H_{\text{footer}} + \sum_{k=0}^{R-1} \text{rowHeight}[k] + (R - 1) \cdot G$$

Determinism guarantee. All operations are max, sum, and proportional scaling — no iteration, no convergence, no randomness. The same input always produces the same rectangles.

0.24.6 Determinism and Theme/Semantics Split

0.24.6.1 What the IR Carries (Semantics)

- Grid structure: columns, rows, cell placement, spans
- Cell content: grammar type + domain IR (what to draw)
- Cell identity: id, title, caption (semantic labels)
- Composition metadata: title, subtitle

0.24.6.2 What the Theme Carries (Style)

```

1 interface CompositionTheme {
2   // Canvas
3   canvasBackground: string;           // CSS color
4
5   // Grid spacing (override IR defaults when theme wants control)
6   gap?: number;                       // overrides grid.gap if set
7   padding?: number;                   // overrides grid.padding if set
8
9   // Cell chrome
10  cellBorder: { color: string; width: number; radius: number };
11  cellBackground: string;              // per-cell fill (or "transparent")
12  cellTitleFont: { family: string; size: number; weight: number;
    color: string };

```

```

13  cellCaptionFont: { family: string; size: number; weight: number;
    color: string };
14  cellTitlePosition: "above" | "inside-top";
15
16  // Header/Footer
17  headerFont: { family: string; size: number; weight: number; color:
    string };
18  subtitleFont: { family: string; size: number; weight: number;
    color: string };
19  footerBadge: { fill: string; textColor: string; radius: number };
20
21  // Dividers
22  dividerColor: string;
23  dividerThickness: number;
24
25  // Scale policy
26  scalePolicy: "fit" | "clip" | "overflow"; // how oversized
    sub-scenes handled
27 }

```

Listing 60: CompositionTheme type (specification)

The boundary. The IR says “cell A1 holds a flow diagram titled ‘Pipeline.’” The theme says “render that title in Inter 18px bold white, with a 1px #333 border and 12px corner radius.” This separation ensures the same composition IR renders differently under different themes (dark-poster, light-dashboard, print) without IR modification.

0.24.7 Edge Cases

0.24.7.1 Sub-Scene Larger Than Cell (Scale-to-Fit)

When a sub-scene’s natural dimensions exceed its allocated cell rectangle:

- **Default policy (scalePolicy: "fit"):** The sub-scene is uniformly scaled *down* (aspect-preserving) to fit entirely within the cell. The scale factor $s = \min(W/w, H/h)$ from Section 0.24.4 handles this automatically. The sub-scene is centered within the remaining space.
- **Alternative (scalePolicy: "clip"):** The sub-scene is not scaled; overflow is clipped at the cell boundary (implemented via a clip-rect Group primitive).
- **Alternative (scalePolicy: "overflow"):** The sub-scene is rendered at natural size; overflow is visible (overlaps adjacent cells). Use only for artistic effect.

Decision: Default is "fit" (scale-to-fit, never upscale). This ensures every sub-scene is fully visible and the composition is self-contained.

0.24.7.2 Empty Cell

A cell with no content (or content that produces an empty Scene with $w = 0, h = 0$):

- The cell rectangle is allocated normally by the grid algorithm (it may still have non-zero size due to other cells in the same row/column).
- No sub-scene primitives are emitted for that cell.
- Cell chrome (border, background) is still rendered if theme specifies it.

0.24.7.3 Single-Cell Composition

A composition with one cell is valid. It produces the sub-scene embedded in the canvas with chrome (header, footer, border). This is useful for wrapping a single diagram in a poster frame.

0.24.7.4 Ragged Grid (Missing Cells)

If the grid has $R \times C$ slots but fewer cells are declared:

- Undeclared slots are treated as empty cells (no content, no chrome).
- Row heights and column widths are still computed from the cells that *are* present.
- The grid is NOT filled with phantom cells—only declared cells exist.

0.24.7.5 Nested Compositions

A cell's GrammarContent MAY reference grammar: "composition", embedding a nested Composition document. This is **allowed** but bounded:

- Nesting depth is limited to 3 levels (configurable; validation rejects deeper nesting).
- The nested composition compiles to a sub-Scene like any other grammar.
- No special handling is needed—the embed mechanism is grammar-agnostic.

Rationale: Nested compositions enable complex layouts (e.g., a 2×2 sub-grid within a cell of a larger grid) without introducing a richer layout primitive.

0.24.8 Worked Example: Multi-Grammar Poster

This example reproduces the structure of a corpus poster (inspired by the RAG-vectorless-explainer, sample-images/image copy 5): a 2×2 grid whose cells hold a flow pipeline, a tree hierarchy, a sequence diagram, and a stat callout.

0.24.8.1 Composition IR

```

1 version: "1.0"
2 metadata:
3   title: "RAG Architecture Deep Dive"
4   subtitle: "Retrieval-Augmented Generation Pipeline"
5   theme: dark-poster
6   canvas:
7     width: 1200

```

```

8     height: auto
9 grid:
10   columns: 2
11   rows: 2
12   gap: 24
13   padding: 32
14 cells:
15   - id: pipeline
16     row: 0
17     col: 0
18     title: "Ingestion Pipeline"
19     content:
20       kind: grammar
21       grammar: flow
22       ir:
23         direction: left-to-right
24         nodes:
25           - { id: docs, label: "Documents", category: input }
26           - { id: chunk, label: "Chunker", category: step }
27           - { id: embed, label: "Embedder", category: step }
28           - { id: store, label: "Vector DB", category: output }
29         edges:
30           - { from: docs, to: chunk }
31           - { from: chunk, to: embed }
32           - { from: embed, to: store }
33
34   - id: taxonomy
35     row: 0
36     col: 1
37     title: "Document Taxonomy"
38     content:
39       kind: grammar
40       grammar: tree
41       ir:
42         root:
43           id: corpus
44           label: "Corpus"
45           children:
46             - id: pdf
47               label: "PDFs"
48               children:
49                 - { id: papers, label: "Papers" }
50                 - { id: manuals, label: "Manuals" }
51             - id: web
52               label: "Web Pages"
53               children:
54                 - { id: docs, label: "Docs" }
55                 - { id: blogs, label: "Blogs" }
56
57   - id: retrieval
58     row: 1
59     col: 0
60     title: "Query Flow"
61     content:
62       kind: grammar
63       grammar: sequence
64       ir:
65         participants:

```



```

66     - { id: user, label: "User" }
67     - { id: api, label: "API" }
68     - { id: vdb, label: "VectorDB" }
69     - { id: llm, label: "LLM" }
70   messages:
71     - { from: user, to: api, label: "query", order: 1 }
72     - { from: api, to: vdb, label: "embed + search", order: 2 }
73     - { from: vdb, to: api, label: "top-k chunks", order: 3 }
74     - { from: api, to: llm, label: "prompt + context", order:
75       4 }
76     - { from: llm, to: api, label: "response", order: 5 }
77     - { from: api, to: user, label: "answer", order: 6 }
78 - id: accuracy
79   row: 1
80   col: 1
81   title: "Retrieval Accuracy"
82   content:
83     kind: stat
84     value: "94.2%"
85     label: "Top-5 recall on evaluation set"
86     trend: up

```

Listing 61: Worked example: RAG Architecture Poster

0.24.8.2 Compilation Walkthrough

1. **Parse & validate:** The composition IR is validated (4 cells, 2×2 grid, no overlaps, all grammar IRs valid).

2. **Compile sub-scenes:**

- Cell pipeline: $\text{buildFlowScene}(\text{ir}) \rightarrow \text{Scene } (w=480, h=120)$
- Cell taxonomy: $\text{buildTreeScene}(\text{ir}) \rightarrow \text{Scene } (w=360, h=280)$
- Cell retrieval: $\text{buildSequenceScene}(\text{ir}) \rightarrow \text{Scene } (w=520, h=340)$
- Cell accuracy: $\text{buildStatScene}(\text{ir}) \rightarrow \text{Scene } (w=200, h=100)$

3. **Grid sizing:**

- $W_{\text{avail}} = 1200 - 2(32) - 1(24) = 1112$ px
- Column widths (max of single-span cells): $\text{col0} = \max(480, 520) = 520$, $\text{col1} = \max(360, 200) = 360$
- $\text{Sum} = 880 < 1112$, so no scaling needed. Columns keep natural widths (centered in available space, or expanded proportionally — theme decision).
- Row heights: $\text{row0} = \max(120, 280) = 280$, $\text{row1} = \max(340, 100) = 340$

4. **Cell rectangles** (with padding $P=32$, gap $G=24$):

- pipeline: $(32, 32+H_{\text{hdr}}, 520, 280)$
- taxonomy: $(32+520+24, 32+H_{\text{hdr}}, 360, 280)$
- retrieval: $(32, 32+H_{\text{hdr}}+280+24, 520, 340)$
- accuracy: $(576, 32+H_{\text{hdr}}+280+24, 360, 340)$

5. **Embed sub-scenes:**

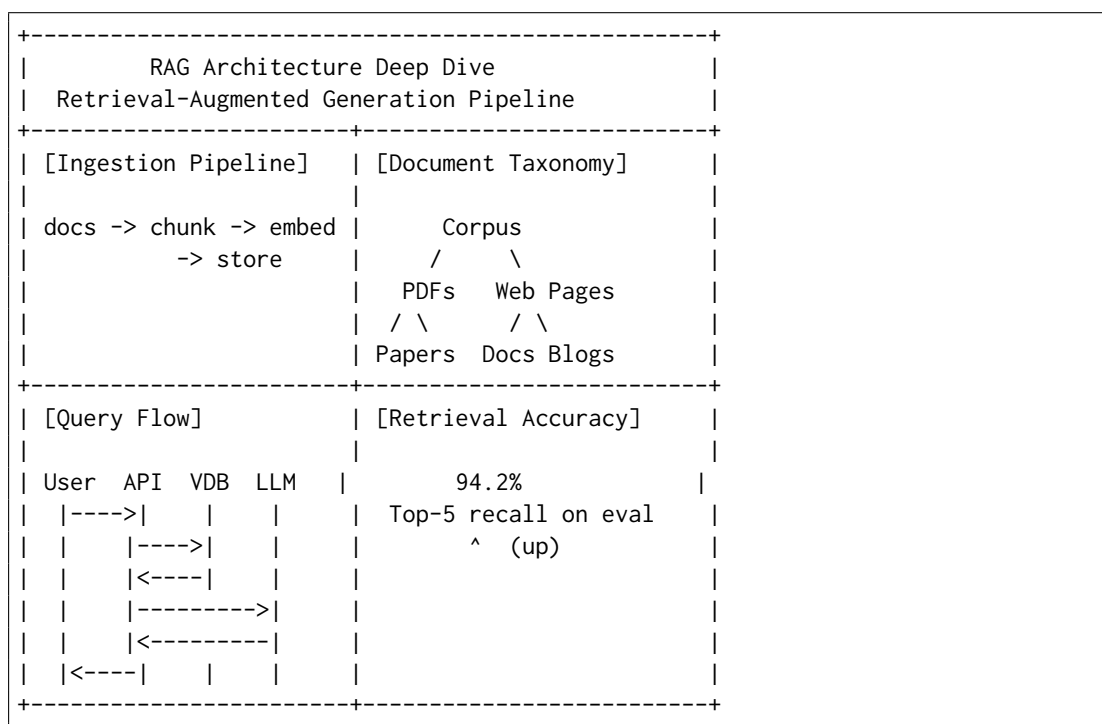
- pipeline (480×120 into 520×280): $s = \min(520/480, 280/120) = \min(1.08, 2.33) \rightarrow$

1.0 (no upscale). Centered: $\Delta x = 32 + (520 - 480)/2 = 52$, $\Delta y = 32 + H_{\text{hdr}} + (280 - 120)/2$.

- taxonomy (360×280 into 360×280): exact fit, $s = 1.0$, no offset.
- retrieval (520×340 into 520×340): exact fit.
- accuracy (200×100 into 360×340): $s = 1.0$, centered in larger rect.

6. **Panel chrome:** Borders, titles (“Ingestion Pipeline”, etc.), backgrounds emitted per theme.

7. **Final Scene:** $1200 \times (2 \cdot 32 + H_{\text{hdr}} + H_{\text{ftr}} + 280 + 340 + 24)$ — all primitives from 4 sub-scenes + chrome in a single Scene object. `sceneHash()` is deterministic.



0.24.9 Panel References

Panels can reference external IR files instead of inline definitions:

```

1 cells:
2   - id: timeline-panel
3     content:
4       kind: grammar
5       grammar: timeline
6       ir_file: "../roadmap.timeline.yaml" # external file

```

Listing 62: Panel with external IR reference

This enables:

- Reusing the same diagram in multiple compositions
- Separate version control for each panel’s IR
- Incremental updates (change one panel, not the whole poster)

0.24.10 Compilation Pipeline Summary

1. **Parse Composition IR:** Validate structure, resolve grammar references, check grid invariants.
2. **Compile each cell:** For each cell, invoke the appropriate grammar's layout engine to produce a sub-Scene.
3. **Compute grid layout:** Determine column widths, row heights, and cell rectangles (Section 0.24.5).
4. **Embed sub-scenes:** For each cell, `embedSceneInRect(subScene, cellRect)` produces translated/scaled primitives.
5. **Render chrome:** Emit background, borders, titles, captions, header, footer, dividers from `CompositionTheme`.
6. **Merge:** Concatenate all primitives in painter's-algorithm order (back to front) into a final Scene with computed width/height.
7. **Hash:** `sceneHash(finalScene)` for golden testing.

0.24.11 Limitations

Cross-panel connectors (specified, not yet implemented)

Edges that span from a node in one panel to a node in another are specified as a superset-only feature in Section 0.25. The implementation requires the node-anchor registry prerequisite described there. Panels remain visually independent until that prerequisite is shipped.

No responsive layout

Panel sizes are computed once; there is no responsive reflow for different canvas widths.

No interactive linking

Clicking a panel does not navigate or filter. Output is static.

No upscaling

Sub-scenes are never scaled up (only down or 1:1). Prevents pixelation of content designed for smaller viewports.

These constraints keep the Composition layer simple and deterministic. Cross-panel connections are addressed in Section 0.25; responsive layout remains out of scope.

0.24.12 Open Questions

For Mark (Schema):

- Final JSON Schema shape for `CompositionDocument` — should `CellContent` use a discriminated union (`kind` field) or separate top-level keys?
- Cell reference vs. inline embed: should `ir_file` support URI schemes (e.g., `pkg:` for package-internal references)?
- Validation: should the composition schema embed sub-grammar schemas or validate them separately (two-pass)?
- Should `grid.columns/grid.rows` allow named tracks (like CSS Grid's named lines)?

For Barbara (Rendering):

- **The `translateAndScale` kernel helper** (Section 0.24.4): implement as a pure function in `packages/core/src/scene-transform.ts`. Must handle Path d-string coordinate transformation and StrokeGradient coordinate transformation.
- Scale-to-fit policy: should the default clip oversized sub-scenes or scale them? (Spec recommends scale-to-fit; Barbara to confirm UX.)
- Panel chrome rendering: confirm that cell borders/titles use existing primitives (Rect, Text) and need no new Scene IR additions.
- How should `scalePolicy`: "clip" be implemented? Via a GroupPrimitive with a clip-path attribute (requires Scene IR extension) or via a post-processing step?

Cross-references:

- Grammar concept and two-IR model: Section 0.14
- Scene IR primitives: Section 0.10 and `packages/core/src/scene.ts`
- Flow Grammar: Section 0.18
- Sequence Grammar: Section 0.19
- Tree Grammar: Section 0.20
- Determinism contract: Section 0.12

0.25 Cross-Diagram Node Linking

0.25.1 Concept and Motivation

A poster assembled from independent grammar cells (Section 0.24) is, by default, a set of *panels side by side*: each cell compiles and renders autonomously with no structural relationship to its neighbours. Cross-diagram node linking removes that constraint. A link statement in the poster DSL draws a directed or undirected edge between a node in one cell's diagram and a node in another cell's diagram, turning the poster into a **linked multi-view system diagram**.

This section specifies two constructs built on the same underlying machinery:

Atomic link.

A single cross-diagram edge between a source node and a target node. This is the primitive from which all cross-diagram overlay drawing is built.

trace.

A *named, ordered, optionally-typed multi-hop path* of atomic links. A trace represents a logical thread — such as a user journey, a distributed request, or a requirements traceability chain — woven across the diagram layers of a poster. This is the flagship use case for cross-diagram linking: representing system traceability across abstraction levels. The `trace` construct is specified in Section 0.25.8.

Presentation-layer assertion.

A cross-diagram link does not modify either diagram’s Domain IR or Scene IR. It is an annotation on the composition itself, resolved after every cell has compiled independently.

Superset-only.

The link and trace statements are new top-level poster keywords with no Mermaid equivalent (Section 0.9). A standard Mermaid renderer will reject a poster document that contains these statements; this is the intended signal that the file requires our compiler.

Additive and non-fatal.

Unresolvable links (unknown cell, unknown node, non-linkable diagram type) emit a warning and are skipped. The poster and all valid links still render.

0.25.2 Link Syntax

A link statement is a poster-level declaration written alongside cell definitions. Its abstract grammar is:

```

1 link <cellAddr>.<nodeId> <edge> <cellAddr>.<nodeId> [: "label"]
2
3 <cellAddr>  ::= "[" row "," col "]"      (* bracket form, 0-indexed *)
4             | colLetters rowNumber    (* Excel form, e.g. A1, B2,
5             |                          AA3 *)
6
7 <nodeId>    ::= (* the node identifier as authored in that cell's
8             |   diagram source *)
9
10 <edge>      ::= "-->"                  (* directed arrow *)
11             | "---"                  (* plain undirected *)
12             | "-.-"                  (* dashed undirected *)
13             | "-.."                  (* dashed undirected,
14             |   alternate *)
15             | "-.->"|"-.->"          (* dashed directed arrow *)

```

Listing 63: Cross-diagram link statement — abstract grammar

Cell addresses reuse the dual addressing scheme introduced for the poster DSL and specified in Section 0.9.2: the bracket form [row,col] is 0-indexed (row first); the Excel form A1, B2, ... uses bijective base-26 column letters with 1-indexed row numbers. Both forms may be mixed freely within a single poster.

The optional : "label" suffix attaches a text label rendered at the midpoint of the overlay edge, styled by the composition theme’s overlayLabelFont token.

Worked example. The following poster specifies an Order Fulfilment system with four cells and three cross-diagram links that connect nodes across cells:

```

1 ---
2 theme: executive
3 layout: grid 2x2
4 ---
5 poster "Order Fulfilment System"

```

```

6
7   cell A1: flowchart LR
8       recv[Receive Order] --> valid[Validate]
9       valid --> pay[Payment]
10      pay --> pick[Warehouse Pick] --> ship[Dispatch]
11
12   cell B1: classDiagram
13       class OrderService
14       class PaymentGateway
15       class WarehouseSystem
16       OrderService --> PaymentGateway
17       OrderService --> WarehouseSystem
18
19   cell A2: stateDiagram-v2
20       [*] --> Pending
21       Pending --> Processing
22       Processing --> Shipped
23       Shipped --> [*]
24
25   cell B2: sequenceDiagram
26       Customer->>OrderService: placeOrder()
27       OrderService->>PaymentGateway: charge()
28       PaymentGateway-->>OrderService: receipt
29
30   link A1.pay --> B1.PaymentGateway : "handled by"
31   link A1.pick -. -> A2.Processing : "triggers"
32   link B1.OrderService --- A2.Pending : "creates"

```

Listing 64: Cross-diagram links in a four-cell poster

link A1.pay → B1.PaymentGateway reads: “draw a directed arrow from the node with id pay in cell A1 to the class named PaymentGateway in cell B1”. The cell addresses A1 and B1 are the Excel equivalents of $[0,0]$ and $[0,1]$.

0.25.3 The Node-Anchor Registry

The core engineering prerequisite. Grammar layout engines today compute precise node positions internally but **discard them**: the output of `buildFlowScene`, `buildTreeScene`, and their siblings is a bare `Scene` containing only flattened rendering primitives. For example, the flow layout engine builds a `PlacedNode` record for every node (carrying `x`, `y`, `w`, `h`, `cx`, `cy`, and port coordinates) but returns only the rendered `Scene`; there is no `nodeId` → position registry in the output type. **Making node positions addressable from the composition layer is the single prerequisite that must be resolved before any cross-diagram link can be drawn.**

0.25.3.1 Candidate Implementations

Two approaches exist for surfacing node anchors to the composition layer.

Candidate (a) — Populate `GroupPrimitive.id` and walk the placed scene. The `GroupPrimitive` in the Scene IR carries an optional `id?: string` field (see `packages/core/src/scene.ts`). Each grammar’s layout engine could wrap each node’s primitives in a `GroupPrimitive`

tagged with that node's id. At composition time the link layer would walk the placed (already translated-and-scaled) scene, compute the axis-aligned bounding box of each named group, and use it as the anchor.

Pros: No change to grammar function return types; `GroupPrimitive.id` already exists in the IR.

Cons:

- Recovering a bounding box requires walking the full scene tree for each node query ($O(n)$ primitives per lookup for a scene with n primitives).
- The walk must be performed after `translateAndScale`, meaning coordinate reconstruction happens at the composition layer, not the grammar layer.
- No port information (which side of the node to attach an edge to) is recoverable from a bounding box without additional metadata.
- Requires every linkable grammar to wrap each node in a named `GroupPrimitive`, which is not currently the case and is a fragile convention: a grammar that emits node primitives without a wrapping group would silently produce empty anchors.
- Not reusable for hit-testing or agent node-addressing without additional work.

Candidate (b) — Sidecar anchors record on the render result. Each grammar's layout function is extended to return a pair `{ scene: Scene, anchors: NodeAnchorRegistry }` in addition to the `Scene`. The registry maps each node id to its bounding box and optional connection ports, expressed in the cell's **local coordinate space** (i.e., before composition-level translation and scaling). Grammars that opt in populate the registry from data they already hold during layout (the flow grammar's `PlacedNode` is a direct source); grammars that have not yet opted in return an empty registry, and links referencing their nodes degrade gracefully.

```

1  /** Port positions on the four cardinal sides of a node's bounding
    box. */
2  type CardinalSide = 'N' | 'S' | 'E' | 'W';
3
4  /**
5   * Anchor for one node: bounding-box in local cell coordinates,
6   * plus optional explicit port attachment points.
7   * If ports is absent, the link layer derives port midpoints from
8   * (x,y,w,h) at rendering time.
9   */
10 interface NodeAnchor {
11   x: number;                // bounding-box
    top-left x
12   y: number;                // bounding-box
    top-left y
13   w: number;                // bounding-box width
14   h: number;                // bounding-box height
15   ports?: Partial<Record<CardinalSide, { x: number; y: number }>>;
16 }
17
18 /** Maps diagram-local node id -> anchor in local cell coordinates.
    */
19 type NodeAnchorRegistry = Record<string, NodeAnchor>;
20

```

```

21 /**
22  * Extended render result returned by linkable grammars.
23  * Non-linkable grammars continue to return Scene only; the
    composition
24  * layer treats an absent registry as {}.
25  */
26 interface RenderResult {
27   scene: Scene;
28   anchors: NodeAnchorRegistry; // empty ({}) for non-linkable
    grammars
29 }

```

Listing 65: NodeAnchorRegistry — type specification (pseudo-TypeScript)

Pros: Explicit, typed contract authored by the grammar that knows its node positions best; no scene walking; carries port information; each grammar opts in independently; composable independently of the scene.

Cons: Requires changing grammar function return types; migration is incremental (one grammar at a time).

0.25.3.2 Recommended Implementation: Sidecar Anchors (Candidate b)

The recommended approach is the sidecar NodeAnchorRegistry (candidate b).
The rationale:

1. The grammar layout engine already owns the exact node geometry. For the flow grammar, `PlacedNode.x`, `.y`, `.w`, `.h` and the port coordinates `.rx/.lx/.cx/.by` are computed during Phase 4 (coordinate assignment) and available before scene emission. Populating the registry is a two-line loop over the placed-node map — no new computation required.
2. The composition layer already tracks each cell's (`dx`, `dy`, `scale`) from the `translateAndScale` call in `layoutComposition` (see `packages/core/src/composition/layout.ts`). Transforming local anchors to poster coordinates is exact and trivial:

$$x_{\text{poster}} = x_{\text{local}} \cdot s + d_x, \quad y_{\text{poster}} = y_{\text{local}} \cdot s + d_y, \quad w_{\text{poster}} = w_{\text{local}} \cdot s, \quad h_{\text{poster}} = h_{\text{local}} \cdot s$$

where s is the uniform scale factor and (d_x, d_y) is the translation offset already computed by the grid layout pass.

3. Candidate (a) would require reconstructing the same geometry from the scene tree, at the cost of an $O(n)$ -walk on the flattened primitive list, after the coordinate transform has already been applied.
4. The opt-in migration path means zero risk to existing grammars: a grammar that has not yet emitted anchors returns `{}`; links referencing it degrade to a WARN and are skipped (Section 0.25.6).

Transformation to poster coordinates. After each cell's `RenderResult` is produced, the composition layer performs the anchor transformation:

```

1 function transformAnchors(
2   registry: NodeAnchorRegistry,

```



```

3  dx: number,    // cell's x offset in poster (from
    translateAndScale)
4  dy: number,    // cell's y offset in poster
5  scale: number, // uniform scale factor applied to the sub-scene
6 ): NodeAnchorRegistry {
7  const out: NodeAnchorRegistry = {};
8  for (const [id, anchor] of Object.entries(registry)) {
9      const pa: NodeAnchor = {
10         x: anchor.x * scale + dx,
11         y: anchor.y * scale + dy,
12         w: anchor.w * scale,
13         h: anchor.h * scale,
14     };
15     if (anchor.ports) {
16         pa.ports = {};
17         for (const [side, pt] of Object.entries(anchor.ports)) {
18             pa.ports[side as CardinalSide] = {
19                 x: pt.x * scale + dx,
20                 y: pt.y * scale + dy,
21             };
22         }
23     }
24     out[id] = pa;
25 }
26 return out;
27 }

```

Listing 66: Anchor transformation to poster coordinates (pseudo-code)

The resulting poster-coordinate registries from all cells are stored alongside the placed-cell rectangles for use by the overlay routing pass.

Reusability argument. The node-anchor registry is valuable well beyond cross-diagram linking and should be built regardless of whether the linking feature ships in the first increment:

- **Hit-testing:** An interactive SVG export can route mouse events to the correct diagram node by name, enabling click/hover handlers.
- **Tooltips:** Structured hover labels (from the Domain IR’s node metadata) can be attached to the exact node bounding box.
- **Agent node-addressing:** The agent integration surface (Section 0.29) currently addresses diagrams by their IR structure. A published anchor registry lets an agent reference “cell A1, node pay” by logical id rather than by pixel coordinates — a stable, round-trip-safe reference.
- **Accessibility:** ARIA role=“img” subtrees can carry aria-label attributes derived from each node’s anchor id and IR label, improving screen-reader navigation of complex diagrams.

0.25.4 Overlay Routing

Overlay layer. Cross-diagram edges are rendered as a synthetic pass executed *after* all cell sub-scenes have been embedded. The overlay primitives (line/path) are appended to

the poster’s primitive list last, placing them above all cell content in painter’s-algorithm order. No new Scene IR primitive type is required: overlay edges are `PathPrimitive` (directed, with arrowhead) or `LinePrimitive` (plain) constructed from the same kernel as intra-diagram edges.

Port selection. Given source anchor A (with cardinal ports A_N, A_S, A_E, A_W) and target anchor B , the link layer selects the port pair (p_A, p_B) that minimises the Euclidean distance $\|p_A - p_B\|$. Tie-breaking order: E before W before S before N (favouring left-to-right reading direction). When explicit ports are absent from the registry, the four cardinal midpoints are derived from the bounding box:

$$A_N = (x+w/2, y), \quad A_S = (x+w/2, y+h), \quad A_E = (x+w, y+h/2), \quad A_W = (x, y+h/2).$$

Phase 1 routing: straight and single-elbow. The initial implementation uses two routing strategies selected by the relative cell positions:

Same row, adjacent columns

Straight horizontal segment from p_A to p_B , passing through the inter-cell gap. This is the most common case (left-to-right flow) and produces clean, readable results with no routing logic.

Different rows or non-adjacent columns

Single-elbow (L-shaped) route: a horizontal segment from p_A to a midpoint column, then a vertical segment to p_B ’s row, then a horizontal segment to p_B . The elbow column is placed at the midpoint between the two cell centres. All segments run through the inter-cell gap space and do not overlap cell bodies for standard grid layouts.

Phase 2 hardening: orthogonal routing. For complex poster layouts where the straight/elbow route would cross a cell body, the composition engine can activate orthogonal routing:

1. Build a set of *forbidden rectangles* from each placed cell’s bounding box, expanded outward by the theme’s `gap / 2`.
2. Compute the rectilinear shortest path from p_A to p_B that avoids all forbidden rectangles, using the inter-cell gap channels as routing channels.
3. If no route exists (e.g., a cell is surrounded on all sides), emit a `WARNING` and fall back to the Phase 1 route drawn on top of the obstructing cell (readable but visually overlapping).

Phase 2 routing is a *hardening step*; Phase 1 covers the vast majority of practical poster layouts and is the only mode required for the first increment.

Dimension guard. The overlay routing pass applies dimension guards consistent with the project’s overall guard mindset (Section 0.24.7):

- If the route must cross more than three cell bodies (unreachable via gap channels), emit `WARNING: link A.nodeId → B.nodeId: route crosses N cell bodies; overlay may be unreadable. Still render best-effort.`

- If both endpoints resolve to the same cell, emit **WARNING: intra-cell link A.nodeId → A.nodeId2: use an intra-diagram edge instead.** The link is skipped (a cross-diagram link within one cell is semantically incoherent).
- If the computed route length (sum of segment lengths) exceeds three times the poster canvas diagonal, emit a **WARNING.** This guards against degenerate posters where two cells are placed at extreme corners of a very large canvas.

0.25.5 Linkable-Type Rules

A diagram type is **linkable** if and only if its layout engine can produce a non-empty `NodeAnchorRegistry` — that is, it authors stable ids for its visual nodes and can map those ids to bounding boxes.

Table 46: Linkable diagram types (v1)

Grammar	Addressable unit	Node id source
flowchart / graph	Node	Author-assigned id (e.g. A, pay)
classDiagram	Class	Class name as written
stateDiagram-v2	State	State name / explicit id
erDiagram	Entity	Entity name as written
C4Context / C4Container / C4Component	Person, System, Container, Component	Alias string
block-beta	Block	Block id
architecture-beta	Service, Group	Id as written
mindmap	Node	Label-derived kebab-slug
gitGraph	Commit, Branch	Commit id or branch name

Table 47: Non-linkable diagram types (v1) — excluded or deferred

Grammar	Reason excluded	Degradation
pie	No stable node ids; data is label+value pairs	WARN + skip link
sankey	Edge-based flow; no persistent node ids	WARN + skip link
quadrantChart	Continuous 2-D axes; data points have no ids	WARN + skip link
radar	Series labels, not structural nodes	WARN + skip link
packet-beta	Byte-field labels, not structural nodes	WARN + skip link
xychart-beta	Continuous axis; no node abstraction	WARN + skip link
timeline	Events are date-keyed, not node-keyed	WARN + skip link
sequenceDiagram	Participants are stable identifiers; deferred to v2	WARN + skip link
gantt	Bars are task-keyed; deferred to v2	WARN + skip link

Rule statement.

A link endpoint is resolvable if and only if (1) the referenced cell address is valid, (2) the cell’s diagram type appears in Table 46, and (3) the specified `nodeId` is present in that cell’s `NodeAnchorRegistry` after compilation. Any endpoint that fails any condition is **unresolvable**: the compiler emits one **WARNING** per unresolvable endpoint, and the entire `link` statement is omitted from the overlay. A link with one valid endpoint and one unresolvable endpoint is wholly omitted (a dangling edge has no sensible routing target).

0.25.6 Semantics and Degradation Contract

Cross-diagram link contract. Cross-diagram links are *presentation-layer assertions on the composition*. They do not modify, extend, or reference any individual diagram’s Domain IR or Scene IR; each cell compiles as if `link` statements did not exist. Cross-diagram links are *superset-only*: a standard Mermaid renderer will reject a poster document containing `link` statements (the keyword is outside Mermaid’s grammar), which is the intended portability signal. All link resolution — cell lookup, node anchor lookup, port selection, and route computation — occurs at composition time, after every cell has compiled independently. An unresolvable link (unknown cell address, unknown node id within a valid cell, non-linkable diagram type, or empty anchor registry) emits **exactly one WARNING per unresolvable endpoint** and the link is *silently omitted*. The poster renders in full; all other valid links render normally. **Cross-diagram links are never fatal.**

Mermaid portability. As with the `poster` keyword itself (Section 0.9), a file containing `link` statements cannot be rendered by a standard Mermaid renderer. This is intentional and clearly signals the superset requirement. Individual cell bodies (the diagram source embedded in each `cell` block) remain standard Mermaid syntax and can be extracted and rendered independently by any Mermaid renderer.

IR integrity. Because cross-diagram links are resolved entirely at composition time and rendered as overlay primitives, they require no changes to any grammar’s Domain IR schema, to the Scene IR primitive types, or to the determinism contract. The `sceneHash`

of any individual cell’s sub-scene is byte-identical to what it would be without the link statement.

Agent addressing. The `nodeId` token in a link statement is the same identifier an agent would use when addressing a node via the structured IR path (Section 0.29). This alignment is intentional: the anchor registry (Section 0.25.3) is the shared mechanism that makes both the cross-diagram link syntax and agent node-addressing stable and consistent.

0.25.7 Engineering Prerequisites

The following work items must be completed before any cross-diagram link can be drawn. They are listed in dependency order.

1. **Define `NodeAnchorRegistry` and `RenderResult` types** in `packages/core/src/scene.ts` (or a new `scene-anchors.ts`). This is a pure type addition; no behaviour changes.
2. **Extend `buildFlowScene`** to return `RenderResult`. The flow grammar already computes `PlacedNode` records; emitting the registry requires only a loop over the placed-node map at the end of Phase 4. This is the *reference implementation* that verifies the approach end-to-end.
3. **Extend linkable grammar layout functions** (one per grammar, per Table 46) to emit anchors. Each extension is independent and can be shipped incrementally. Non-extended grammars return `anchors: {}`.
4. **Extend the composition layer** (`layoutComposition` in `packages/core/src/composition/layout.ts`) to (a) collect anchor registries from each cell’s `RenderResult`, (b) apply the coordinate transform, and (c) build the poster-coordinate anchor map.
5. **Implement the link parser** in the poster DSL (`packages/core/src/frontend/mermaid/poster.ts`): recognise link statements and store them alongside the `cells` array in `PosterDocument`.
6. **Implement the overlay routing pass** after the cell embed loop in `layoutComposition`: resolve endpoints, select ports, compute routes, and append overlay primitives.
7. **Implement graceful degradation**: WARN on unresolvable endpoints, skip the link, continue.
8. **Extend the poster parser** to recognise trace statements (Section 0.25.8.2) and populate `PosterDocument.traces: TraceRecord[]`. Desugar each `TraceRecord` to the ordered sequence of atomic link pairs it implies, storing them as link entries tagged with their parent trace id. This is a pure addition to the poster parser; the link-resolution, routing, and overlay machinery from items 5–7 are reused without modification.
9. **Assign categorical-palette colours** to `TraceRecords` at composition time: iterate traces in declaration order and assign `categorical[i % n]` from the active theme’s data palette (Section 0.22.3.1). Store the resolved colour in `TraceRecord.color` for use by the overlay rendering pass.
10. **Emit the trace legend**: after all overlay edges are appended, generate a legend block below the poster canvas listing each named trace with its colour swatch and type pill. Suppress if `trace legend: off` is set in the poster frontmatter.

Items 1, 5, and 8 are pure additions. Items 2–4 and 6–7 are incremental extensions to

existing functions with no breaking changes to current callers (the `Scene` return type is replaced by `RenderResult` for linkable grammars; callers that ignore the `anchors` field are unaffected). Items 9–10 extend only the composition layer and overlay pass.

0.25.8 The trace Abstraction: Named, Typed, Ordered Paths

Headline use case. A poster assembled from layered system views — a C4 Context diagram, a Container diagram, a Component diagram, an ER schema, a test suite — is a natural representation of *system architecture across abstraction levels*. A single user story, requirement, or runtime request touches nodes in several of those layers simultaneously. The **trace** construct names, types, and orders that multi-layer thread, turning a poster from a side-by-side collection of views into a first-class **system traceability artefact**.

0.25.8.1 Concept

A **trace** is a *named, ordered, optionally-typed multi-hop path* of cross-diagram links. Where a single link is one atomic directed edge between two nodes in different cells, a **trace** is a path $A \rightarrow B \rightarrow C \rightarrow \dots$ stitching together a logical thread that runs across system layers or diagram views in a fixed order.

Three properties distinguish a trace from a loose collection of links:

Named.

A trace carries a human-readable name (a quoted string) that identifies the logical thread it represents — for example, "Login flow" or "REQ-12". The name is the stable handle used for highlight, filter, and legend display.

Ordered.

The hops of a trace form a sequence, not a bag of edges. Ordering encodes direction and traversal: from origin (e.g. a requirement or user action) through intermediate layers (components, services) to terminus (e.g. a test, a database table, or a deployed service). Ordering is what makes a trace a *path*, not a cluster.

Typed.

An optional type token drawn from a shared vocabulary (Section 0.25.8.3) annotates every hop in the trace with the same semantic relationship kind. Typing makes traceability uniform: a **satisfies** trace across a poster carries exactly the same meaning as a **satisfies** edge inside a `requirementDiagram` cell.

Relation to link: desugaring. The link statement remains the **primitive**. A trace *desugars* to an ordered sequence of atomic links *plus* a `TraceRecord` (name, type, ordered member link list). Specifically, a trace with n hops $h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_n$ lowers at composition time to the $n - 1$ atomic links $\{h_1 \rightarrow h_2, h_2 \rightarrow h_3, \dots, h_{n-1} \rightarrow h_n\}$ together with a `TraceRecord` that groups those links under the trace name, type, and ordinal index. The link-resolution, port-selection, and overlay-routing machinery from Sections 0.25.3–0.25.6 apply unchanged to each desugared hop.

```

1  -- Input (poster DSL):
2  trace "Login flow" : A1.Login -> B1.AuthAPI -> C1.UserService ->
   D1.users
3
4  -- Desugars to three atomic links:
5  link A1.Login      --> B1.AuthAPI      -- hop 1->2  (trace member 0)
6  link B1.AuthAPI    --> C1.UserService  -- hop 2->3  (trace member 1)
7  link C1.UserService --> D1.users      -- hop 3->4  (trace member 2)
8  -- PLUS a TraceRecord:
9  -- { name: "Login flow", type: none,
10 --   members: [ link[0], link[1], link[2] ] }

```

Listing 67: Desugaring a trace to atomic links + group record

A link statement that is not part of any trace continues to behave exactly as specified in Section 0.25.2: a standalone overlay edge with no trace-group affiliation and no categorical colour.

0.25.8.2 Syntax

The trace statement is a poster-level declaration written alongside cell and link definitions. Its abstract grammar is:

```

1  trace "<name>" [<type>] : <hop> <arrow> <hop> { <arrow> <hop> }
2
3  <name>  ::= quoted string (UTF-8; any character except unescaped "'")
4
5  <type>  ::= (* optional; drawn from the trace-type vocabulary,
6             Section~\ref{sec:cdl-trace-vocab} *)
7             "satisfies" | "derives" | "verifies" | "refines"
8             | "traces"   | "contains" | "copies"
9             | "calls"    | "flowsTo"  | "mapsTo"
10
11 <hop>    ::= <cellAddr> "." <nodeId>
12             (* cellAddr and nodeId same as link,
13                Section~\ref{sec:cdl-syntax} *)
13
14 <arrow>  ::= "-->"    (* solid directed --- default for most types *)
15             | "-.->"  (* dashed directed --- for weak or inferred
16                        links *)

```

Listing 68: Trace statement — abstract grammar

Two concrete examples:

```

1  -- Untyped trace: a named presentation-flow path across four cells
2  trace "Login flow" : A1.Login -> B1.AuthAPI -> C1.UserService ->
   D1.users
3
4  -- Typed trace: requirement traceability using the requirement
   vocabulary
5  trace "REQ-12" satisfies : A1.Req12 --> B1.AuthService -->
   C1.AuthTest

```

Listing 69: Trace syntax: untyped presentation-flow and typed requirement trace

Arrow forms in traces. Inside a trace path the arrow token determines the edge style of every desugared link in that trace. When the type token is present the compiler selects the canonical arrow form for that type per Table 48; the arrow token in the source may be omitted when a type is supplied. When omitted entirely the default is `->`.

0.25.8.3 Trace-Type Vocabulary

The trace type is drawn from two groups. The first group is the **requirement-relationship vocabulary** used by the `requirementDiagram` grammar, grounded in `packages/core/src/grammars/requirement` (`RequirementRelKind`). The Zod schema in `packages/core/src/grammars/requirement/schema.ts` enumerates the identical seven values. The second group is a small set of **presentation-flow types** for architectural and runtime diagrams.

Reusing the requirement-grammar vocabulary ensures that a traceability assertion is semantically identical whether expressed as a `requirementDiagram` edge inside one cell or as a typed trace across poster cells. An author can graduate a single-cell requirement diagram into a cross-view poster trace without changing the relationship kind. Tools that index or query the composition IR see uniform type tokens in both contexts.

Table 48: Trace-type vocabulary and default presentation

Type token	Group	Default arrow	IR source	Semantics
satisfies	Requirement	-->	<code>RequirementRelKind</code>	Component satisfies a requirement
derives	Requirement	-.>	<code>RequirementRelKind</code>	Requirement derived from another
verifies	Requirement	-->	<code>RequirementRelKind</code>	Test verifies a requirement
refines	Requirement	-.>	<code>RequirementRelKind</code>	Requirement refines a higher-level one
traces	Requirement	-.>	<code>RequirementRelKind</code>	Generic traceability link
contains	Requirement	-->	<code>RequirementRelKind</code>	Requirement contains a sub-requirement
copies	Requirement	-.>	<code>RequirementRelKind</code>	Requirement copied from another
calls	Presentation	-->	(new; poster-layer)	A service or component calls another
flowsTo	Presentation	-->	(new; poster-layer)	Data or control flows to a node
mapsTo	Presentation	-.>	(new; poster-layer)	A model element maps to a view element

When `<type>` is omitted, the trace is **untyped**: all desugared links receive the plain directed arrow (`->`) and no semantic label or pill is rendered. Untyped traces are valid and useful for pure presentation-flow paths where no traceability semantics are implied.

The three presentation-flow types (`calls`, `flowsTo`, `mapsTo`) are poster-layer additions with no counterpart in the `requirementDiagram` grammar. They are excluded from `RequirementRelKind` deliberately: they carry no formal requirements- engineering semantics and their home is the composition layer, not the domain grammar.

0.25.8.4 Rationale: Why Traces Beat Loose Links

Loose individual link statements can express the same set of edges as a trace, but without the structure that makes those edges useful as a traceability artefact. Three properties of traces over loose links:

Named and highlightable.

A trace can be emphasised *as a unit*. Hovering or selecting "Login flow" in the legend highlights every hop in that path simultaneously, dimming all other overlay edges and uninvolved cells. Filtering to a single trace hides all other links. A loose collection of links with the same topology has no identity, no unit of selection, and no filter target.

Typed and consistent.

A typed trace carries traceability semantics (e.g. *satisfies*, *verifies*) that are uniform across the poster because they come from the same `RequirementRelKind` vocabulary used inside `requirementDiagram` cells. A reader of a poster trace and a reader of a requirement-diagram edge see the same semantic label. Tools that consume the composition IR can index traceability uniformly regardless of which representation was used.

Ordered and directional.

A path $A \rightarrow B \rightarrow C$ encodes traversal direction and origin; a bag of edges $\{A \rightarrow B, B \rightarrow C\}$ does not, in isolation, identify that the path begins at A and terminates at C . For traceability, direction matters: *requirement REQ-12 is satisfied by AuthService which is verified by AuthTest* is different from the reverse, and the ordered trace makes that unambiguous.

0.25.8.5 Presentation Semantics

Per-trace colour from the data palette. Each named trace is assigned a distinct colour drawn from the theme's **categorical data palette** (Section 0.22.3.1). This is the same palette used by chart-family components (bar charts, XY charts, pie charts) to assign per-series colours. Reusing the categorical palette ensures that trace colours are:

1. **Perceptually distinct** from one another — the categorical palette is designed for discrimination across 6–12 concurrent categories.
2. **On-brand and theme-coherent** — the palette is defined by the active contract theme, so executive, pastel, terminal, and all other themes provide trace-appropriate categorical colours automatically and consistently with every other diagram on the poster.
3. **Non-clashing with diagram-internal colours** — diagram components consume the *semantic role palette* (surface, ink, accent, muted, border), while traces consume the *data palette*. The two sub-palettes are designed within each theme never to collide (Section 0.22.3.1).

Traces are assigned colours by declaration order: the first `trace` statement receives `categorical[0]`, the second `categorical[1]`, and so on. If the number of traces exceeds the categorical palette length n , the palette wraps modulo n with a 20% lightness offset applied on each cycle to maintain visual distinction.

Trace legend. A **trace legend** is automatically generated and appended at the bottom of the poster canvas when at least one named trace is present. Each legend entry shows:

- A colour swatch (small filled rectangle) in the trace’s assigned categorical colour.
- The trace name (the quoted string as authored).
- The type token, if present, rendered as a `ntypez` pill styled by the theme’s `overlayLabelFont` token — the same pill style used for `requirementDiagram` relationship labels.

The legend can be suppressed with a `trace legend: off` directive in the poster front-matter. When no named traces exist, no legend is emitted.

Highlight, dim, and filter modes. Traces introduce three presentation-layer interaction modes at the composition level. These are specified here at the design level; rendering detail (interactive SVG event handlers, static pre-rendered variants, CLI `-active-trace` flags) is deferred to the backend implementation.

Highlight mode (one trace active).

When a trace is activated (e.g. hover on a legend entry or programmatic selection via the agent API): all edges belonging to the active trace render at full opacity with a luminosity boost of the trace colour; all other overlay edges render at 30% opacity; cells containing no hop of the active trace render at 40% opacity; cells containing at least one hop render at full opacity.

Dim mode (any edge hovered, no specific trace selected).

All overlay edges render at full opacity. Bare link statements (not part of any trace) render at 60% opacity. No cells are dimmed.

Filter mode (trace selection).

Only selected trace(s) are visible; all other overlay edges and non-participating cells are hidden. Filter mode is toggled via the legend (click a legend entry) or programmatically via the agent API (Section 0.25.8.7). Multiple traces may be simultaneously active in filter mode.

Degradation contract for traces. Pathological cases in trace resolution follow the same “warn and render the resolvable portion” principle as atomic links, with refinements specific to the multi-hop structure:

One unresolvable hop (h_k fails).

The compiler emits `WARNING: trace "name" hop k: <reason> - hop omitted`. The trace renders as two sub-paths: $h_1 \rightarrow \dots \rightarrow h_{k-1}$ and $h_{k+1} \rightarrow \dots \rightarrow h_n$, both in the trace’s assigned colour. The trace name still appears in the legend.

All hops unresolvable.

The trace is wholly omitted. The compiler emits `WARNING: trace "name": no resolvable hops - trace skipped`. No legend entry is generated.

Duplicate hop (same node appears at positions i and j).

The compiler emits `WARNING: trace "name" hop j: node X.Y already appears at`

position *i* – cycle detected; loop arc rendered. A self-loop arc is drawn at the repeated node.

Fewer than two resolvable hops.

A trace cannot form an edge without two resolved endpoints. The compiler emits WARNING: trace "name": fewer than 2 resolvable hops – trace skipped.

Hop routing crosses cell bodies.

Follows the atomic-link routing degradation (Section 0.25.4): Phase 1 fallback is used per individual desugared hop; an additional warning is emitted if the hop crosses more than three cell bodies.

All trace degradation is non-fatal. The poster compiles and renders; only the affected hop segments are omitted or replaced by fallback arcs. Other traces and all standalone link statements are unaffected.

0.25.8.6 Worked Examples

Example 1 — C4 architecture drill-down (context to container to component to data). The classic C4 model organises a software architecture across four nested abstraction levels. A four-cell poster places one C4 view per cell and connects them with a single trace that represents one user journey through the stack: from the context-level user persona, through a container, down into a component, and into the data layer.

```

1  ---
2  theme: executive
3  layout: grid 2x2
4  ---
5  poster "E-Commerce Platform -- Login Journey"
6
7  cell A1: C4Context
8      Person(user, "Customer", "Web or mobile shopper")
9      System(webapp, "Web Application", "React SPA")
10     System_Ext(idp, "Identity Provider", "Auth0")
11     Rel(user, webapp, "Uses", "HTTPS")
12     Rel(webapp, idp, "Delegates auth", "OIDC")
13
14  cell B1: C4Container
15     Container(spa, "SPA", "React", "Browser client")
16     Container(api, "API Gateway", "Node/Express", "REST")
17     Container(authsvc, "Auth Service", "Go", "gRPC")
18     Rel(spa, api, "Calls", "REST/HTTPS")
19     Rel(api, authsvc, "Authenticates via", "gRPC")
20
21  cell A2: C4Component
22     Component(loginctrl, "LoginController", "React", "Login page")
23     Component(authclient, "AuthClient", "TS", "OIDC adapter")
24     Rel(loginctrl, authclient, "Invokes")
25
26  cell B2: classDiagram
27     class User
28     class Session
29     class Token
30     User "1" --> "*" Session : owns
31     Session --> Token : contains
32

```

```

33 trace "Login journey" calls :
34   A1.user --> B1.spa --> B1.api --> B1.authsvc --> B2.Token

```

Listing 70: C4 drill-down: one trace across four architecture abstraction levels

The `calls` trace renders in `categorical[0]` of the executive theme. A legend entry labelled “Login journey / calls” appears beneath the poster grid. The intra-cell sub-hop `B1.spa → B1.api` is within the same cell; the compiler emits a warning and renders a short intra-cell loopback arc styled in the trace colour, then resumes the cross-diagram routing for the next hop.

Example 2 — Distributed request trace across heterogeneous diagram types.

A distributed system poster combines a flowchart (entry-point logic), a sequence diagram (inter-service protocol, deferred/v2), an architecture diagram (deployed topology), and an ER diagram (data schema) in one poster. A single trace named for the operation represents one end-to-end request.

```

1  ---
2  theme: executive
3  layout: grid 2x2
4  ---
5  poster "Payment Processing -- Charge Request"
6
7  cell A1: flowchart LR
8    entry[API Entry] --> validate[Validate Payload]
9    validate --> auth[Authorise]
10   auth --> charge[Charge]
11
12  cell B1: sequenceDiagram
13    APIGateway->>PaymentService: POST /charge
14    PaymentService->>FraudEngine: check(payload)
15    FraudEngine-->>PaymentService: score
16    PaymentService->>Ledger: debit(account)
17
18  cell A2: architecture-beta
19    service APIGateway(server, "API Gateway", "us-east-1")
20    service PaymentService(server, "Payment Svc", "us-east-1")
21    service FraudEngine(server, "Fraud Engine", "eu-west-1")
22    service Ledger(database, "Ledger DB", "us-east-1")
23    APIGateway --> PaymentService
24    PaymentService --> FraudEngine
25    PaymentService --> Ledger
26
27  cell B2: erDiagram
28    ACCOUNT ||--o{ TRANSACTION : has
29    TRANSACTION }|--|| CHARGE_EVENT : records
30    CHARGE_EVENT }o--|| FRAUD_SCORE : scored_by
31
32  trace "Charge request" flowsTo :
33    A1.entry --> A1.charge --> A2.PaymentService --> B2.TRANSACTION

```

Listing 71: Distributed request trace across four diagram types

The hop `A1.entry → A1.charge` is intra-cell (both in the flowchart); the compiler warns and renders a loopback arc, then continues cross-diagram. The sequence diagram cell

B1 is deliberately not included in this trace because `sequenceDiagram` is non-linkable in v1 (Table 47); it remains visible on the poster as context. The trace renders two cross-diagram hops: A1 → A2 (flowchart → architecture) and A2 → B2 (architecture → ER), both coloured in `categorical[0]`.

Example 3 — Requirements traceability with typed traces and a trace legend. This is the flagship use case for typed traces. A requirement in one cell is satisfied by a component in a second cell, which is verified by a test in a third. Two separate `satisfies` traces make two independent traceability chains visible simultaneously, each in its own colour, with a generated legend.

```

1  ---
2  theme: executive
3  layout: grid 1x3
4  ---
5  poster "Authentication System -- Requirements Traceability"
6
7  cell A1: requirementDiagram
8    requirement REQ-12 {
9      id: "REQ-12"
10     text: "User must be authenticated within 500ms"
11     risk: high
12     verifymethod: test
13   }
14   requirement REQ-13 {
15     id: "REQ-13"
16     text: "Session token must expire within 24h"
17     risk: medium
18     verifymethod: test
19   }
20   REQ-12 - derives -> REQ-13
21
22  cell B1: C4Component
23    Component(authctrl, "AuthController", "Go", "JWT validation")
24    Component(tokensvc, "TokenService", "Go", "Token lifecycle")
25    Component(cache, "SessionCache", "Redis", "Active sessions")
26    authctrl --> tokensvc : "issues"
27    tokensvc --> cache : "stores"
28
29  cell C1: flowchart TD
30    T1[Test: auth_latency_test] --> A1r[Assert: p99 < 500ms]
31    T2[Test: token_expiry_test] --> A2r[Assert: exp = now+86400s]
32
33  -- REQ-12 satisfied by AuthController, verified by auth_latency_test
34  trace "REQ-12 trace" satisfies : A1.REQ-12 --> B1.authctrl --> C1.T1
35
36  -- REQ-13 satisfied by TokenService, verified by token_expiry_test
37  trace "REQ-13 trace" satisfies : A1.REQ-13 --> B1.tokensvc --> C1.T2

```

Listing 72: Multi-typed requirements traceability poster with trace legend

The two traces render in `categorical[0]` and `categorical[1]` of the executive theme respectively. The auto-generated legend beneath the poster grid reads (schematically):

```

[swatch] REQ-12 trace ńsatisfiesż
[swatch] REQ-13 trace ńsatisfiesż

```

A reviewer hovers “REQ-12 trace” in the legend to highlight only the first colour thread — from the requirement cell through `authctrl` to T1 — while all other cells and the REQ-13 trace dim to 40% opacity. The filter mode lets a reviewer isolate a single requirement’s complete traceability chain on a poster that may contain dozens of requirements and components. This is the core interaction model for poster-scale traceability review.

0.25.8.7 Agent Integration

Traces are first-class artefacts in the IR/composition API surface (Section 0.29). An agent does not need to manipulate poster DSL text to assert or query traceability; it operates on the structured `TraceRecord` type:

```

1  /** A single hop in a trace: canonical cell address + node id. */
2  interface TraceHop {
3      cell:    string;    // canonical Excel address ("A1", "B2", ...)
4      nodeId: string;    // node identifier within that cell's diagram
5  }
6
7  /**
8   * Trace-type vocabulary.
9   * Requirement group: identical to RequirementRelKind in
10  *   packages/core/src/grammars/requirement/types.ts
11  * Presentation-flow group: poster-layer only.
12  */
13  type TraceType =
14      | 'satisfies' | 'derives' | 'verifies' // RequirementRelKind
15      | 'refines'   | 'traces'   | 'contains' // RequirementRelKind
16      | 'copies'    // RequirementRelKind
17      | 'calls'     | 'flowsTo' | 'mapsTo'; // presentation-flow
18                                     (poster-layer)
19
20  /** A named, typed, ordered multi-hop trace: the composition IR
21      unit. */
22  interface TraceRecord {
23      name:    string;    // human-readable identifier; must be unique
24                          // per poster
25      type?:   TraceType; // omitted = untyped trace
26      hops:    TraceHop[]; // ordered list of hops; length >= 2
27      color?:  string;    // hex string; assigned at render time from
28                          // categorical palette
29  }

```

Listing 73: `TraceRecord` and `TraceHop` — IR types for the agent API (pseudo-TypeScript)

An agent can use this API to:

- **Assert traceability programmatically.** Construct a `TraceRecord` with a name, type, and ordered hops and push it to `PosterDocument.traces`. This is a structured traceability artefact that replaces ad-hoc comment annotations or manually maintained link lists. The artefact is reproducible, diffable across poster revisions, and queryable without parsing diagram source.
- **Query traces by type.** Filter `PosterDocument.traces` by type to enumerate all `satisfies` traces (requirement coverage) or all `verifies` traces (test coverage).

- **Activate filter mode.** Set `PosterRenderOptions.activeTraces` to a list of trace names to render a filtered view with only the specified traces visible. This lets an agent generate specialised traceability views without modifying the poster source.
- **Detect coverage gaps.** Walk the `requirementDiagram` Domain IR in cell *A* to enumerate all requirements, then check whether each has a corresponding `satisfies` trace hop as its origin. Flag requirements that have no such trace as uncovered — a gap report that drives the next authoring action.

A trace is therefore not merely a drawing command: it is a **structured, typed, agent-queryable traceability artefact**. An agent-generated traceability matrix is reproducible, queryable, and diffable across poster revisions — properties that no graphical or freeform annotation scheme can provide.

Cross-references

- Composition / Posters (poster DSL, cell addressing): Section [0.24](#) and Section [0.9.2](#)
- Dual cell addressing (bracket and Excel forms): Section [0.9.2](#)
- Scene IR primitives (`GroupPrimitive.id`, `PathPrimitive`): Section [0.10](#) and `packages/core/src/scene`
- `translateAndScale` coordinate helper: Section [0.24.4](#) (paragraph [0.24.4.3](#))
- Superset extension mechanisms (`poster` keyword, superset-only features): Section [0.9](#)
- Agent integration surface (IR-as-agent-API, node-addressing): Section [0.29](#)
- Dimension guard mindset: Section [0.24.7](#)
- Theme data palette / categorical sub-palette (trace colour source): Section [0.22.3.1](#) and Section [0.22](#)
- Requirement-relationship vocabulary (`RequirementRelKind`, seven values): `packages/core/src/grammars` and `packages/core/src/grammars/requirement/schema.ts`

Part VII

Architecture & Packaging

0.26 Layered Architecture

This section defines the layered architecture of the diagram compiler, emphasizing the clean separation between kernel, grammars, and composition.

0.26.1 The Three Layers

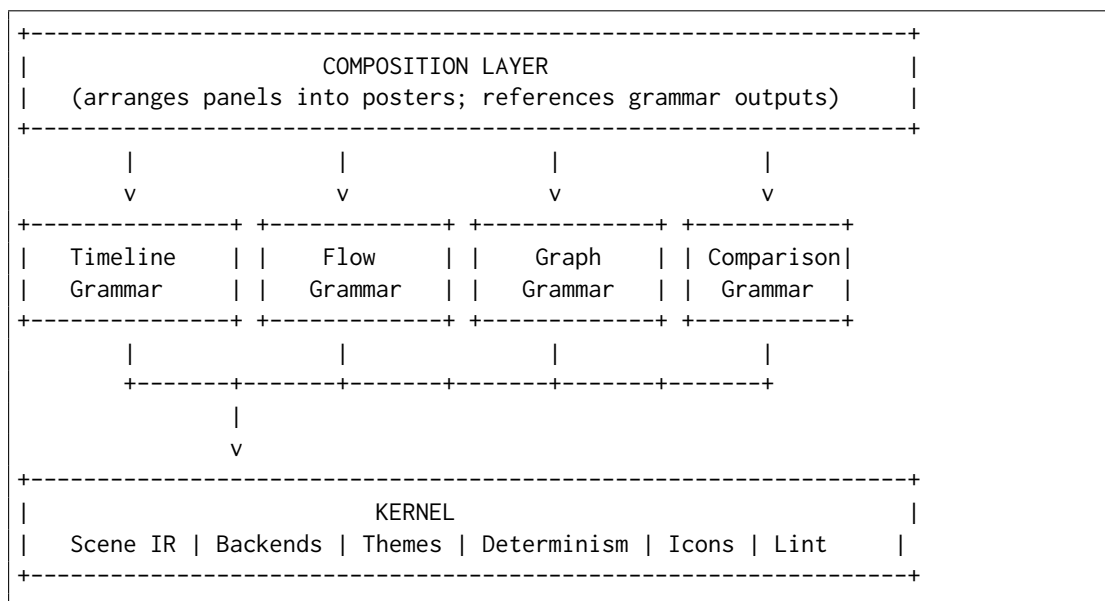


Figure 8: The three-layer architecture: Composition $\rightarrow$ Grammars $\rightarrow$ Kernel

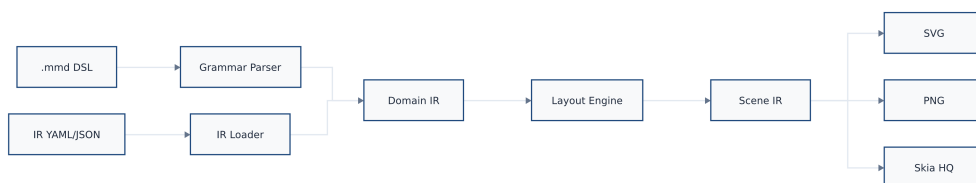


Figure 9: The two-frontend, two-IR compilation pipeline. Mermaid DSL (.mmd) and structured IR (YAML/JSON) both feed into a Domain IR, which the layout engine transforms into a Scene IR consumed by SVG, PNG, and Skia backends.

Rendered by the diagram compiler this document describes (source: design/figures/src/architecture.mmd, built with make figures).

0.26.2 Layer 1: The Kernel

The kernel is the shared infrastructure that all grammars depend on. It is grammar-agnostic: it knows nothing about timelines, flows, or graphs.

0.26.2.1 Kernel Components

Scene IR (§0.10)

The universal primitive set (Rect, Circle, Line, Path, Text, Group) and effect definitions. The render contract.

Rendering Backends (§0.11)

SVG, PNG (resvg), Skia, PPTX backends. Consume Scene IR, produce output.

Theme System (§0.22)

Theme schema, theme loading, inheritance, property resolution. Grammar-agnostic but grammars may extend.

Determinism Contract (§0.12)

Rounding rules, sort ordering, no-random, no-clock guarantees.

Deterministic Font Metrics

Embedded font metric tables for layout-critical text measurement.

Icon Registry

Named icons (SVG paths) available to all grammars.

Asset/Image Loader

Resolves and embeds external images as data URIs.

Layout Helpers

Common layout primitives: text measurement, box collision, orthogonal routing.

Lint/Quality-Gate Framework

Validation diagnostics, severity levels, suggestion generation.

Animation Hints (§0.13)

Declarative animation attached to Scene primitives.

0.26.2.2 Kernel Contract

The kernel makes the following guarantees to grammars:

1. **Grammar-agnosticism:** The kernel never imports from, or references, any grammar module.
2. **Determinism:** All kernel operations are deterministic given the same inputs.
3. **Backend uniformity:** All backends consume the same Scene IR interface.
4. **Theme uniformity:** Theme properties are resolved identically for all grammars.

0.26.3 Layer 2: Grammars

Grammars are peer modules that sit atop the kernel. Each grammar:

- Defines its own Domain IR schema
- Implements one or more layout engines (Domain IR $\rightarrow$ Scene IR)
- May extend the theme schema with grammar-specific properties
- Exposes validation and compilation entry points

0.26.3.1 Grammar Independence

Grammars are *independent* of each other:

- The Timeline grammar does not import from the Flow grammar
- Adding a new grammar does not modify existing grammars
- Each grammar can be versioned and released independently

0.26.3.2 Grammar-Kernel Dependency

All grammars depend on the kernel:

```

1 // packages/timeline/package.json
2 {
3   "dependencies": {
4     "@diagram-compiler/kernel": "^1.0.0"
5   }
6 }
7
8 // packages/flow/package.json
9 {
10  "dependencies": {
11    "@diagram-compiler/kernel": "^1.0.0"
12  }
13 }
```

Listing 74: Grammar dependency structure

0.26.4 Layer 3: Composition

The Composition layer sits above grammars. It:

- References panels, each specifying a grammar type and Domain IR
- Invokes grammar compilation for each panel
- Arranges panel Scenes into a unified poster Scene
- Adds editorial chrome (headers, footers, dividers)

0.26.4.1 Composition Dependencies

Composition depends on grammars (to compile panels) and kernel (for chrome rendering):

```

1 // packages/composition/package.json
2 {
3   "dependencies": {
4     "@diagram-compiler/kernel": "^1.0.0",
5     "@diagram-compiler/timeline": "^1.0.0",
6     "@diagram-compiler/flow": "^1.0.0",
7     "@diagram-compiler/comparison": "^1.0.0"
8     // ... all available grammars
9   }
10 }
```

```

9   }
10  }
```

Listing 75: Composition dependency structure

0.26.5 Dependency Rules

Module	→ Kernel	→ Grammars	→ Composition	→ Other Grammars
Kernel	—	×	×	×
Grammar (any)	✓	—	×	×
Composition	✓	✓	—	—

Key rule: The kernel must not import anything grammar-specific. This is enforced by linting.

0.26.6 Module Boundaries

0.26.6.1 What Lives in the Kernel

- Scene IR types and serialization
- All rendering backends
- Theme schema (core properties)
- Theme loader and resolver
- Icon registry
- Font metric tables
- Validation framework (DiagnosticSeverity, ValidationResult)
- Determinism utilities (rounding, sorting)
- Animation hint types

0.26.6.2 What Lives in Grammars

- Domain IR types (e.g., Activity, Track, Milestone for Timeline)
- Domain IR schema (JSON Schema)
- Validation rules (semantic validation beyond schema)
- Layout engine(s)
- Grammar-specific theme extensions

0.26.6.3 What Lives in Composition

- Composition IR types
- Composition IR schema
- Panel arrangement algorithms (stack, grid, custom)

- Chrome rendering (headers, footers, dividers)
- Grammar dispatcher (routes panel to correct grammar)

0.26.7 API Surface

Each layer exposes a clean API:

0.26.7.1 Kernel API

```

1 // Scene IR
2 export type Scene, DrawingPrimitive, EffectDefinition, AnimationHint;
3 export function sceneHash(scene: Scene): string;
4
5 // Backends
6 export function renderSvg(scene: Scene): string;
7 export function renderPng(scene: Scene): Buffer;
8 export function renderPdf(scene: Scene): Buffer;
9 export function renderPptx(scene: Scene): Buffer;
10
11 // Themes
12 export function loadTheme(name: string): Theme;
13 export function resolveTheme(base: Theme, overrides:
    Partial<Theme>): Theme;
14
15 // Validation
16 export type Diagnostic, ValidationResult;
17
18 // Icons
19 export function getIcon(name: string): SvgPath | undefined;
20
21 // Font metrics
22 export function measureText(text: string, font: FontSpec):
    TextMetrics;

```

Listing 76: Kernel public API

0.26.7.2 Grammar API

Each grammar exposes the same interface shape:

```

1 // @diagram-compiler/timeline
2 export type TimelineIR; // Domain IR type
3 export const schema: JsonSchema;
4 export function validate(ir: unknown): ValidationResult;
5 export function compile(ir: TimelineIR, theme: Theme): Scene;

```

Listing 77: Grammar public API (Timeline example)

0.26.7.3 Composition API

```

1 // @diagram-compiler/composition
2 export type CompositionIR;
3 export const schema: JsonSchema;
4 export function validate(ir: unknown): ValidationResult;
5 export function compile(ir: CompositionIR, theme: Theme): Scene;

```

Listing 78: Composition public API

0.27 Packaging & Repository Strategy

This section defines the packaging strategy for the diagram compiler: how modules are organized in the repository, how they are published, and the incremental path from the current timeline-only codebase to the full grammar family.

0.27.1 Strategic Principle

The diagram compiler lives **beside timeline in the same monorepo on a shared kernel**—not inside timeline’s IR (which would bloat it), not in a separate repo (which would duplicate the engine and drift).

packages/		
kernel/	@diagram-compiler/kernel	
timeline/	@diagram-compiler/timeline	
flow/	@diagram-compiler/flow	(future)
graph/	@diagram-compiler/graph	(future)
comparison/	@diagram-compiler/comparison	(future)
composition/	@diagram-compiler/composition	(future)
cli/	@diagram-compiler/cli	
mcp/	@diagram-compiler/mcp	

Figure 10: Target package structure

0.27.2 Compounding Payoff

This structure creates compounding value:

- **Animation** added to the kernel is inherited by all grammars
- **New backends** serve all grammars immediately
- **Theme improvements** apply everywhere
- **Determinism tests** cover all grammars
- **Icon additions** are available to all grammars
- **Security patches** to the kernel propagate to all grammars

A new grammar starts with the full feature set: determinism, theming, SVG/PNG/PDF/PPTX output, animation support, validation—all for free.

0.27.3 The Incremental Path

The current codebase (`packages/core`) is timeline-only. Extracting the kernel requires care to avoid a premature big-bang refactor. The recommended path:

0.27.3.1 Phase 0: Draw the Seam (Current)

Goal: Establish the kernel/grammar boundary *inside* `packages/core`, without restructuring packages.

Actions:

1. Create `src/kernel/` directory within `packages/core`
2. Move kernel components (Scene IR, backends, themes, icons, font metrics) into `src/kernel/`
3. Move timeline-specific components (Timeline IR, layout engine) into `src/timeline/`
4. Add lint rule: `src/kernel/` may not import from `src/timeline/`
5. Verify all tests pass

Outcome: The kernel/timeline boundary is enforced by convention and linting, but no packages are split. The monorepo structure is unchanged.

0.27.3.2 Phase 1: Prove Grammar-Agnosticism

Goal: Build the first new grammar (Flow) as a kernel-only consumer, proving the architecture.

Actions:

1. Create `src/flow/` within `packages/core`
2. Implement Flow grammar (IR, validation, layout) using only `src/kernel/` imports
3. Add Flow grammar tests
4. Verify the kernel imports nothing from `src/timeline/` or `src/flow/`

Outcome: Two grammars (Timeline, Flow) coexist, both consuming the same kernel. The architecture is validated.

0.27.3.3 Phase 2: Extract Packages

Goal: Split into separate npm packages when publishing/team boundaries justify it.

Actions:

1. Create `packages/kernel/` from `packages/core/src/kernel/`
2. Create `packages/timeline/` from `packages/core/src/timeline/`
3. Create `packages/flow/` from `packages/core/src/flow/`
4. Update `packages/cli/` and `packages/mcp/` to depend on the new packages

5. Publish as `@diagram-compiler/kernel`, `@diagram-compiler/timeline`, etc.
6. Deprecate the old `@timeline-compiler/core` package (or keep as umbrella re-export)

Trigger: Phase 2 is triggered when:

- A third grammar is added (justifies the extraction overhead)
- OR external contributors need to develop grammars independently
- OR the monorepo becomes unwieldy

Outcome: Clean package boundaries, independent versioning, clear ownership.

0.27.3.4 Phase 3: Grammar Marketplace (Future)

Goal: Enable community-contributed grammars outside the core monorepo.

Actions:

1. Publish kernel with stable API guarantees (semantic versioning)
2. Document grammar authoring guide
3. Create template repository for new grammars
4. Establish discovery mechanism (registry, awesome-list)

Outcome: Third-party grammars that interoperate with the kernel and can be used in compositions.

0.27.4 Package Naming

Package	npm Name
Kernel	<code>@diagram-compiler/kernel</code>
Timeline Grammar	<code>@diagram-compiler/timeline</code>
Flow Grammar	<code>@diagram-compiler/flow</code>
Graph Grammar	<code>@diagram-compiler/graph</code>
Comparison Grammar	<code>@diagram-compiler/comparison</code>
Stat Grammar	<code>@diagram-compiler/stat</code>
Composition	<code>@diagram-compiler/composition</code>
CLI	<code>@diagram-compiler/cli</code>
MCP Server	<code>@diagram-compiler/mcp</code>

Note: The existing `@timeline-compiler/*` packages may be retained as aliases or deprecated with migration guidance.

0.27.5 Versioning Strategy

Kernel

Follows semantic versioning strictly. Breaking changes require major version bump.

Grammars

Version independently. May have different version numbers.

Composition

Version tracks the highest grammar version it supports.

CLI/MCP

Version tracks the kernel version.

Compatibility matrix. A grammar at version X is compatible with kernel versions $[X, X+1)$. Major kernel versions may break grammars; grammars must be updated.

0.27.6 Monorepo Tooling

The monorepo uses:

pnpm workspaces

For package management and linking

TypeScript project references

For incremental builds and cross-package type checking

Changesets

For versioning and changelog generation

Turborepo or Nx

(optional) For build caching and task orchestration

0.27.7 CI/CD Considerations

- **Test matrix:** All grammars tested against kernel changes
- **Publish order:** Kernel publishes before grammars
- **Golden images:** Per-grammar golden test suites
- **Cross-platform:** macOS, Linux, Windows in CI

0.28 Graph Auto-Layout: The Hard Problem

Graph auto-layout is the hardest problem in the diagram compiler. This section is honest about the difficulty, prescribes practical solutions, and identifies risks.

0.28.1 Why Graph Layout Is Hard

General 2D graph layout is research-grade:

- **NP-hard subproblems:** Minimizing edge crossings in general graphs is NP-complete [19].

- **Aesthetic trade-offs:** No single algorithm optimizes all criteria (crossing minimization, edge length uniformity, node distribution, symmetry).
- **Non-determinism risk:** Many algorithms use randomized initialization or iterative convergence, which threatens the determinism contract.
- **Label placement:** Text labels add collision constraints that classic algorithms don't handle.

0.28.2 The Constraint as a Feature

The architecture embraces the constraint: **a small, opinionated grammar is both shippable AND reliably LLM-emittable**. Rather than attempting general graph layout, the compiler supports opinionated layouts per diagram sub-kind:

Sub-Kind	Algorithm	Constraints / Trade-offs
Tree	Reingold-Tilford [47]	Single root, no cycles. Clean, compact.
DAG (pipeline)	Sugiyama [52]	Directed acyclic. Layers + crossing minimization.
Hub-and-spoke	Radial custom	Single hub. Spokes arranged radially.
Left-to-right flow	Linear ranking	Nodes ranked by distance from sources.
Swimlane	Constrained grid	Nodes assigned to lanes; x by sequence.

Explicit layout type. The Graph grammar requires the author to specify the layout type:

```

1 graph:
2   layout: tidy-tree      # required: tidy-tree | dag | hub-spoke |
   swimlane
3   vertices: [...]
4   edges: [...]
```

Listing 79: Explicit layout type in Graph IR

This is a feature, not a limitation: the author (human or agent) declares the graph's structure, and the compiler uses an appropriate algorithm. Ambiguous “auto-detect” is out of scope.

0.28.3 Algorithm Details

0.28.3.1 Tidy-Tree (Reingold-Tilford)

For rooted trees: parent-child hierarchies with no cross-links.

Algorithm

Reingold-Tilford (1981) [47], extended by Walker (1990) [61] for arbitrary n -ary trees (Walker's claimed $O(n)$ has a known bug causing $O(n^2)$ worst case). Buchheim, Jünger & Leipert (2002) [6] corrected Walker to truly run in $O(n)$ by maintaining a thread pointer across subtrees; this is the state-of-the-art and the basis of D3's `d3.tree()` layout.

Complexity

$O(n)$ (Buchheim et al. corrected algorithm).

Determinism

Fully deterministic given node order.

Implementation

Well-documented; available in dagre, ELK [12].

0.28.3.2 DAG (Sugiyama Layered Layout)

For directed acyclic graphs: pipelines, dependency graphs, workflow stages.

Algorithm

Sugiyama, Tagawa & Toda (1981) [52]: four canonical phases:

1. **Cycle removal** — a minimal feedback arc set is reversed to produce a DAG. Greedy heuristics run in $O(|V| + |E|)$.
2. **Layer assignment** — network simplex (Gansner et al. [17]) minimises total edge span $\sum |\lambda(v) - \lambda(u)|$ in near-linear time.
3. **Crossing minimisation** — barycenter/median heuristic iterated over adjacent layer pairs (NP-complete optimum [19]); 24 sweeps is the standard practice.
4. **Coordinate assignment** — Brandes & Köpf [5] $O(n)$ algorithm constructs four candidate alignments and takes the median; guarantees at most two bends per edge.

Complexity

$O(|V| + |E|)$ for layers; $O(|V|^2)$ for crossing minimisation heuristics; $O(n)$ for coordinate assignment.

Determinism

Deterministic with fixed layer-assignment and barycenter ordering; tie-breaking by canonical node ID gives byte-identical output [7].

Implementation

dagre [7], ELK [12], Graphviz dot [14].

0.28.3.3 Hub-and-Spoke (Radial)

For star topologies: central hub with peripheral spokes.

Algorithm

Custom radial placement: hub at center, spokes at equal angular intervals.

Complexity

$O(n)$.

Determinism

Fully deterministic with sorted spoke order.

0.28.3.4 Force-Directed (General Graphs)

For graphs with no clear hierarchy or structure.

Algorithm

Fruchterman-Reingold (1991) [16] or Eades spring model (1984) [11]. For undirected networks where force-directed *aesthetics* are desired but determinism is required, stress majorization [18] (Gansner, Koren & North 2004) is the safer alternative: given a fixed canonical initial layout (e.g., nodes on a circle in alphabetical order), the SMACOF iteration is a pure function of the distance matrix — no randomness. This is the default in Graphviz `neato` and available in ELK [12] as `org.eclipse.elk.stress`. Kamada & Kawai (1989) [27] offer a complementary energy-minimization model connecting all node pairs by springs proportional to graph-theoretic distance (Graphviz `neato mode=KK`).

Complexity

$O(|V|^2 \times \text{iterations})$ for Fruchterman-Reingold; $O(n^3)$ for Kamada-Kawai.

Determinism

Risk: iterative simulation can converge to different local minima. Mitigations:

- Initial positions from sorted node IDs (deterministic starting point)
- Perturbations seeded by `scene_hash`
- Fixed iteration count (no convergence-based termination)
- Stress majorization with deterministic init (recommended)

Implementation

d3-force, sigma.js, WebCola [10] (constraint-aware), custom.

Force-directed caveat. Even with mitigations, force-directed layout is the least reliable for determinism. Small changes to nodes/edges can cause large layout changes. It is recommended for exploratory use, not production artifacts.

0.28.3.5 Orthogonal Layout (Architecture Diagrams)

For UML-style architecture diagrams and circuit schematics where all edges must be horizontal or vertical segments.

Framework

Topology–Shape–Metrics (TSM), introduced by Tamassia (1987) [53]: (1) find a planar embedding or planarize via dummy crossings; (2) assign edge orientations minimising total bends via minimum-cost network flow on the dual graph (polynomial time); (3) assign coordinates by compaction.

Determinism

Fully deterministic once the graph embedding is fixed.

Practical engine

ELK Layered with orthogonal routing mode; libavoid (Wybrow et al.) for edge-only routing around fixed nodes. Used in Inkscape, draw.io, and Graphviz’s orthogonal routing mode.

When to use

Nested container architecture diagrams, ER diagrams, and any diagram where right-angle edges reduce visual clutter.

0.28.3.6 Constraint-Based Layout (Swimlanes, Alignment)

For diagrams with hard spatial constraints: swimlane boundaries, node alignment, and non-overlapping placement.

Framework

IPSep-CoLa / WebCola [10] (Dwyer, Marriott & Stuckey): stress majorization combined with VPSC separation constraints (a quadratic program minimising displacement subject to $x_j - x_i \geq g_{ij}$). Prevents node overlap, enforces lane boundaries, and supports alignment constraints.

Determinism

Deterministic given a fixed initial layout. WebCola's monotone stress convergence avoids the oscillation of spring models.

When to use

Complex swimlane diagrams where Sugiyama alone produces lane-boundary violations; interactive diagrams where nodes drag within lane constraints.

0.28.4 Practical Layout Engines

Two battle-tested open-source layout engines are recommended:

0.28.4.1 ELK (Eclipse Layout Kernel)

Project

Eclipse Layout Kernel [12]

License

EPL-2.0

Algorithms

Layered (Sugiyama), tree, force, radial, and more

Integration

Java library; JavaScript port (elkjs) via emscripten

Determinism

Designed for determinism; used in production diagramming tools

0.28.4.2 dagre

Project

dagre [7]

License

MIT

Algorithms

Sugiyama layered layout only

Integration

Pure JavaScript; widely used (Mermaid uses dagre)

Determinism

Deterministic for DAGs

0.28.5 Build vs. Adopt Decision

Option	Pros	Cons
Build custom	Full control; no dependencies; tailored to Scene IR	High effort; bugs; maintenance burden
Adopt ELK	Battle-tested; multiple algorithms; deterministic	Large dependency (elkjs 2MB); EPL license
Adopt dagre	Small; MIT; widely adopted	DAG only; no tree/radial
Hybrid	dagre for DAGs; custom for simple cases	Two codepaths; integration complexity

Recommendation. Start with dagre for DAG/pipeline layouts (already proven in Mermaid). Implement custom radial layout for hub-and-spoke. Evaluate ELK for tree layout. Build custom force-directed only if needed.

0.28.6 Risk Assessment

Layout quality

Auto-layout may produce suboptimal results. Users may want manual position hints.

Determinism violations

Force-directed layout is risky. Recommend DAG/tree for production; force for drafts.

Label collisions

Standard algorithms don't handle labels. Post-processing required.

Performance

Large graphs (>1000 nodes) may be slow. Define reasonable limits.

0.28.7 Layout Hints (Escape Hatch)

For cases where auto-layout fails, the IR supports position hints:

```

1 graph:
2   layout: dag
3   vertices:
4     - id: start
5       position: { x: 0, y: 0 } # hint: override auto-layout for
6         this node
7     - id: end
8       position: { x: 400, y: 0 }
9   edges: [...]
```

Listing 80: Position hints in Graph IR

Position hints are optional. When present, auto-layout respects them; when absent, auto-layout computes positions.

0.29 Agent Integration

AI agents are first-class users of Timeline Compiler. They are not an afterthought wired to a human-facing CLI; they are a primary consumer of the IR, the primary beneficiary of a published JSON Schema, and the primary audience for the MCP server surface. This section designs the full agent integration path: from raw inputs through ingestion to a valid IR, through validation and repair, and through round-trip editing without loss of provenance.

The fundamental unit of agent work is the *ingestion task*: given a **data source** (work items, prose, structured export) and a **natural-language prompt**, produce a valid Timeline IR. The prompt is a first-class input—it determines what is selected, how it is framed, which entities are promoted to milestones, and which are dropped entirely. An agent that ignores the prompt and imports everything it can find is not doing its job.

0.29.1 The Ingestion Contract

The ingestion contract defines what the ingestion path is permitted to do with the inputs it receives. Table 49 captures the four categories of ingestion decision. Violations of this contract—particularly silent data loss or silent inference—are the most common source of incorrect or misleading timelines.

Category	Definition	Examples
Assumed	Facts taken from the prompt or schema without source evidence	Title derived from prompt instruction; version "1.0"
Inferred	Derived from source data by a deterministic rule	<code>axis_unit</code> from date span; <code>time_range</code> from min/max activity dates; track from <code>AreaPath</code>
Defaulted	Omitted from IR, relying on documented schema default	<code>status: planned</code> omitted when all items are future; <code>locale</code> omitted
Rejected	Source data discarded, reason logged	Bugs dropped when prompt says “features and milestones only”; items outside prompted time window; duplicate or zero-duration events with no label

Table 49: Ingestion decision categories

0.29.1.1 The Prompt as First-Class Input

The prompt directs every non-trivial ingestion decision. Table 50 lists what the prompt controls and what the ingestion layer must extract from it before touching the source data.

0.29.1.2 What Ingestion May Not Do

- **Must not compute dates.** If a source item has no date, it must use `tbd`, not invent one. Timeline Grammar does not compute schedules (see §0.4).

Prompt Signal	Ingestion Effect
Time horizon (“ <i>Q1–Q3 2026</i> ”)	Sets or constrains <code>time_range</code> ; items outside window are rejected
Entity filter (“ <i>features and milestones only</i> ”)	Restricts work item types included; bugs, tasks dropped
Track grouping (“ <i>group by team</i> ”)	Maps source field (e.g., <code>AreaPath</code>) to tracks
Emphasis (“ <i>highlight at-risk items</i> ”)	Promotes <code>status</code> field; may add annotation
Framing (“ <i>executive summary</i> ”)	Reduces label verbosity; promotes high-level items
Title / subtitle	Populates <code>metadata.title</code> / <code>metadata.subtitle</code>
Milestone promotion (“ <i>mark GA as a milestone</i> ”)	Converts matching activity to milestone entity

Table 50: Prompt signals and their ingestion effects

- **Must not collapse ambiguous status silently.** An ADO work item in state “On Hold” that has no clear IR equivalent must log a rejection warning, not silently map to planned.
- **Must not import the whole backlog.** The prompt defines the editorial scope. Importing everything that exists and hoping the renderer trims it is a contract violation.
- **Must not generate non-stable IDs.** IDs derived from sequential numbers (`act-1`, `act-2`) are permitted only as a last resort. IDs should be derived from the item’s title slug for git-friendliness (see §0.29.7).

0.29.2 Ingestion Workflows

The following subsections describe four concrete ingestion flows, each with a source excerpt, a prompt, the resulting IR YAML, and an explicit account of what the prompt selected, inferred, and rejected.

0.29.2.1 Azure DevOps Work Items

Source format. The ADO Work Items REST API [35] returns JSON objects. The fields relevant to timeline ingestion are listed in Table 51.

ADO Field	IR Target	Notes
<code>System.Title</code>	<code>activity.label</code> / <code>milestone.label</code>	Verbatim or prompt-shortened
<code>System.State</code>	<code>activity.status</code>	Via Table 52
<code>System.WorkItemType</code>	<code>activity.category</code> / entity type	Feature → activity; Milestone →
<code>System.IterationPath</code>	<code>span</code> or <code>start/end</code>	E.g. <code>Platform\2026\Q2</code> → <code>span</code>
<code>System.AreaPath</code>	<code>track</code>	Leaf segment becomes track label
<code>Microsoft.VSTS.Scheduling.TargetDate</code>	<code>milestone.date</code>	Used when promoting to milestone
<code>System.Tags</code>	<code>activity.tags</code>	Passed through
<code>System.Id</code>	<code>metadata.ado_id</code>	Provenance (not rendered)

Table 51: ADO work item field mapping

Source excerpt.

```

1 # ADO API response (workItems array, fields selected)
2 - id: 1024
3   fields:

```


ADO State	IR Status
New	planned
Active	in-progress
Resolved	done
Closed	done
Removed	cancelled
On Hold	blocked
(other)	planned (warn: unmapped state)

Table 52: ADO state to IR status mapping

```

4   System.Title: "API Gateway Migration"
5   System.State: "Active"
6   System.WorkItemType: "Feature"
7   System.IterationPath: "Platform\\2026\\Q2"
8   System.AreaPath: "Platform Team"
9   - id: 1025
10  fields:
11    System.Title: "Mobile App 3.0 GA"
12    System.State: "New"
13    System.WorkItemType: "Milestone"
14    System.AreaPath: "Mobile Team"
15    Microsoft.VSTS.Scheduling.TargetDate: "2026-09-15"
16  - id: 1031
17  fields:
18    System.Title: "Fix login timeout bug"
19    System.State: "Active"
20    System.WorkItemType: "Bug"
21    System.AreaPath: "Platform Team"

```

Listing 81: ADO work item JSON (abridged)

Prompt. *“Show the Q1–Q3 2026 Platform and Mobile roadmap. Include features and milestones only. Group by team. Drop bugs and tasks.”*

Resulting IR.

```

1  version: "1.0"
2  metadata:
3    title: Platform & Mobile Roadmap
4    subtitle: Q1-Q3 2026
5    time_range:
6      start: 2026-Q1
7      end: 2026-Q3
8    axis_unit: quarter    # inferred: 3-quarter window -> quarter
9                          granularity
10 tracks:
11   - id: platform
12     label: Platform Team
13   - id: mobile
14     label: Mobile Team
15
16 activities:
17   - id: api-gateway-migration
18     label: API Gateway Migration
19     track: platform

```

```

20     span: 2026-Q2          # inferred from IterationPath
      "Platform\2026\Q2"
21     status: in-progress   # mapped from ADO state "Active"
22     category: feature
23     metadata:
24       source: ado
25       ado_id: 1024
26       ado_iteration: "Platform\\2026\\Q2"
27
28 milestones:
29   - id: mobile-app-3-ga
30     label: Mobile App 3.0 GA
31     date: 2026-09-15      # from TargetDate field
32     track: mobile
33     status: planned       # mapped from ADO state "New"
34     metadata:
35       source: ado
36       ado_id: 1025

```

Listing 82: IR produced from ADO work items

What the prompt did.

- **Selected:** Items 1024 (Feature) and 1025 (Milestone). Item 1031 (Bug) was rejected because the prompt said “features and milestones only”.
- **Framed:** time_range set to Q1–Q3 2026 from prompt instruction. Item 1025’s target date (2026-09-15) falls within Q3, so it is included.
- **Inferred:** Tracks derived from AreaPath leaf segments. axis_unit inferred as quarter from the three-quarter window.
- **Dropped:** Bug work item 1031 (not in filter). Had the time range excluded any item, it would be dropped with a logged rejection.

0.29.2.2 Natural Language Prose

When the source is free-form text—a planning document, a briefing note, an email thread—the agent must extract temporal structure from prose. This is the most ambiguity-prone ingestion path; the contract requires explicit handling of every point of uncertainty.

Source text.

```

1 We are planning to migrate the data warehouse in the first half of
  2026.
2 The new customer portal is targeted for Q3. Both initiatives are
  owned
3 by the Engineering org. An executive review is scheduled for 15 July
  2026.
4 The legacy decommission date is TBD, pending the warehouse cutover.

```

Listing 83: Natural language source

Prompt. *“Show this as a two-track roadmap: Infrastructure and Product. Mark the executive review as a milestone. Include the decommission even though its date is not fixed.”*

Resulting IR.

```

1 version: "1.0"
2 metadata:
3   title: 2026 Engineering Roadmap
4   time_range:
5     start: 2026-H1
6     end: 2026-Q3
7   axis_unit: quarter # inferred: H1+Q3 window, quarterly detail
8
9 tracks:
10  - id: infrastructure
11    label: Infrastructure
12  - id: product
13    label: Product
14
15 activities:
16  - id: data-warehouse-migration
17    label: Data Warehouse Migration
18    track: infrastructure
19    span: 2026-H1
20    status: planned
21
22  - id: customer-portal
23    label: New Customer Portal
24    track: product
25    span: 2026-Q3
26    status: planned
27
28  - id: legacy-decommission
29    label: Legacy Decommission
30    track: infrastructure
31    start: tbd # explicit: date not yet determined
32    status: planned
33
34 milestones:
35  - id: executive-review
36    label: Executive Review
37    date: 2026-07-15
38    status: planned

```

Listing 84: IR produced from natural language prose

What the prompt did.

- **Structured:** Track assignments (Infrastructure vs. Product) came from the prompt. The source text only named “Engineering org”.
- **Promoted:** The executive review was elevated to a milestone (not an activity) by explicit prompt instruction.
- **Included with uncertainty:** Legacy decommission included despite no fixed date; start: tbd preserves the fact on the timeline without inventing a date.
- **Inferred:** time_range derived from the earliest (2026-H1 start) and latest (2026-07-15) concrete dates. axis_unit set to quarter as the coarsest granularity that fits the items.

Ambiguity resolution. “First half of 2026” is unambiguously 2026-H1 (ISO half, per the date model). “Q3” without a year is inferred as 2026-Q3 from context (all other

dates are 2026). If the year were ambiguous, the agent must surface a warning rather than guess.

0.29.2.3 GitHub Projects

GitHub Projects (ProjectV2) [21, 22] supports custom field types that carry roadmap-relevant data. The ingestion mapping is summarised in Table 53.

GitHub ProjectV2 Field	IR Target	Notes
title (built-in)	activity.label / milestone.label	Verbatim or shortened
status (select field)	activity.status	Label mapped to IR enum
DATE custom field	start / milestone.date	Named field determines role
ITERATION custom field	span	Iteration title maps to quarter
labels	activity.category (first) / activity.tags	First label to category
milestone (linked)	milestones entry	Promoted to IR milestone
repository	track	When prompt says “group by repo”
assignees	metadata.assignees	Not rendered; preserved for provenance
number (issue #)	metadata.github_issue	Provenance link

Table 53: GitHub ProjectV2 field mapping

The ITERATION field type is the most useful for quarterly roadmaps: iteration titles like “2026 Q2” map directly to span: 2026-Q2, making the ingestion path from GitHub Projects to a quarterly roadmap nearly lossless.

Gap note. GitHub Projects does not have a first-class concept analogous to “work item type” (Feature vs. Bug vs. Task). Filtering by type requires label or custom field conventions, which vary per team. The prompt must specify the filter criterion explicitly.

0.29.2.4 Mermaid Timeline Syntax

Mermaid’s timeline diagram [31, 33] is the most common structured timeline format already embedded in team documentation. Its syntax maps cleanly to IR sections and activities.

Source.

```

1 timeline
2   title Platform Evolution 2026
3   section H1 2026
4     API Gateway : Q1 : Migration complete
5     Auth Refresh : Q2 : New OAuth2 provider
6   section H2 2026
7     Mobile 3.0 GA : Q3
8     Legacy Cutover : Q4 : Final decommission

```

Listing 85: Mermaid timeline source

Resulting IR.

```

1 version: "1.0"
2 metadata:
3   title: Platform Evolution 2026
4   time_range:

```

```

5     start: 2026-Q1
6     end: 2026-Q4
7     axis_unit: quarter
8
9 tracks:
10  - id: platform
11    label: Platform          # default single-track when Mermaid has no
    swimlanes
12
13 sections:
14  - id: h1-2026
15    label: H1 2026
16    time_range:
17      start: 2026-Q1
18      end: 2026-Q2
19
20  - id: h2-2026
21    label: H2 2026
22    time_range:
23      start: 2026-Q3
24      end: 2026-Q4
25
26 activities:
27  - id: api-gateway
28    label: API Gateway
29    track: platform
30    span: 2026-Q1
31    description: "Migration complete"
32    status: planned
33
34  - id: auth-refresh
35    label: Auth Refresh
36    track: platform
37    span: 2026-Q2
38    description: "New OAuth2 provider"
39    status: planned
40
41  - id: mobile-3-ga
42    label: Mobile 3.0 GA
43    track: platform
44    span: 2026-Q3
45    status: planned
46
47  - id: legacy-cutover
48    label: Legacy Cutover
49    track: platform
50    span: 2026-Q4
51    description: "Final decommission"
52    status: planned

```

Listing 86: IR produced from Mermaid timeline syntax

Mapping notes. Mermaid timeline does not have a swimlane concept; all events are rendered in a single horizontal flow grouped by section. When promoting to IR, a default single track is created unless the prompt specifies a multi-track structure. Mermaid's colon-separated description field maps to `description`. Because Mermaid gantt [32] uses a different syntax (task dependencies, progress), Mermaid gantt ingestion requires

a separate mapping pass; that mapping is out of scope for the initial IR but is an extension point (see §0.4).

0.29.3 Reliable IR Generation by Agents

LLM-based agents generate IR reliably when three conditions hold: (1) the target schema is machine-readable and published, (2) the generation call is constrained to valid output shapes, and (3) the IR's structural choices minimise the surface area for generation errors.

0.29.3.1 JSON Schema Publication

The IR schema must be published as a JSON Schema document [41]. This is a binding requirement established by the positioning analysis (§0.5): without a machine-readable schema, agent generation is unreliable. The schema is a first-class deliverable of this project, versioned alongside the spec.

The top-level JSON Schema structure mirrors the IR document structure directly:

```

1 # $schema: https://json-schema.org/draft/2020-12/schema
2 # $id: https://timeline-compiler.dev/schema/v1/timeline.json
3 title: Timeline IR
4 type: object
5 required: [version, metadata, tracks, activities]
6 properties:
7   version:
8     type: string
9     description: "Specification version, e.g. \"1.0\""
10
11   metadata:
12     type: object
13     required: [title, time_range]
14     properties:
15       title: { type: string }
16       subtitle: { type: string }
17       time_range:
18         type: object
19         required: [start]
20         properties:
21           start: { "$ref": "#/$defs/Date" }
22           end: { "$ref": "#/$defs/Date" }
23       axis_unit: { "$ref": "#/$defs/AxisUnit" }
24       theme: { type: string }
25       locale: { type: string }
26
27   tracks:
28     type: array
29     minItems: 1
30     items: { "$ref": "#/$defs/Track" }
31
32   activities:
33     type: array
34     items: { "$ref": "#/$defs/Activity" }
35
```

```

36 milestones:
37   type: array
38   items: { "$ref": "#/$defs/Milestone" }
39
40 # groups, annotations, sections, legend all optional arrays
41
42 $defs:
43   ID:
44     type: string
45     pattern: "^[a-z][a-z0-9-]*$"
46
47   Date:
48     type: string
49     description: "ISO date, quarter (2026-Q2), half (2026-H1),
50                  relative (+3m),
51                  symbolic (now), or uncertain token (tbd, ongoing,
52                  unknown,
53                  or tilde-prefix ~2026-Q2)"
54
55   AxisUnit:
56     type: string
57     enum: [day, week, month, quarter, half, year]
58
59   Status:
60     type: string
61     enum: [planned, in-progress, done, at-risk, blocked, cancelled,
62           tentative]
63
64   Track:
65     type: object
66     required: [id, label]
67     properties:
68       id: { "$ref": "#/$defs/ID" }
69       label: { type: string }
70       color: { type: string }
71       order: { type: integer, minimum: 0 }
72
73   Activity:
74     type: object
75     required: [id, label, track]
76     oneOf:
77       - required: [start] # start + optional end
78       - required: [span] # span shorthand
79     properties:
80       id: { "$ref": "#/$defs/ID" }
81       label: { type: string }
82       track: { type: string } # bare string ref to Track.id
83       start: { "$ref": "#/$defs/Date" }
84       end: { "$ref": "#/$defs/Date" }
85       span: { "$ref": "#/$defs/Date" }
86       status: { "$ref": "#/$defs/Status", default: planned }
87       progress: { type: number, minimum: 0, maximum: 1 }
88       category: { type: string }
89       group: { type: string }
90       description: { type: string }
91       metadata: { type: object }
92
93   Milestone:

```

```

91   type: object
92   required: [id, label, date]
93   properties:
94     id:      { "$ref": "#/$defs/ID" }
95     label:   { type: string }
96     date:    { "$ref": "#/$defs/Date" }
97     track:   { type: string }
98     status:  { "$ref": "#/$defs/Status", default: planned }
99     category: { type: string }
100    icon:     { type: string }
101    description: { type: string }
102    metadata: { type: object }

```

Listing 87: Abbreviated JSON Schema for the IR (YAML representation)

The schema is published at a versioned URL. Minor version bumps to the IR may add optional fields; the schema handles this via `additionalProperties: true` for `metadata` maps and by allowing unknown top-level fields on minor revisions.

New cite key needed. The JSON Schema specification itself does not have a cite key in `references.bib`. A key `json-schema2020` pointing to the IETF JSON Schema draft (2020-12) should be added by David.

0.29.3.2 Structured Output Constraints

Platforms that support constrained generation [41] should use the published JSON Schema as the output schema for the IR generation call. This eliminates entire classes of generation errors:

- Required fields can never be missing when the schema enforces `required`.
- ID format is enforced by the `pattern` constraint on the ID definition, preventing spaces, uppercase, or other invalid characters.
- Status values are constrained to the enum; the model cannot hallucinate “completed” or “active”.
- References are just bare strings—the model emits `track: "platform"`, not a nested object, which is trivially correct.

When structured output is not available, the agent prompt must include the JSON Schema inline and instruct the model to validate its own output before returning it.

0.29.3.3 IR Design Choices That Help Agents

Several deliberate IR design choices reduce the error rate for LLM-generated documents:

Optional fields default to safe values.

`status` defaults to `planned`; `progress` defaults to `null`. An agent that omits these fields produces a valid document. Agents only need to emit fields they have evidence for.

span shorthand reduces reasoning load.

Instead of requiring an agent to compute both a `start` and `end` date for a quarter-length activity, it can emit `span: 2026-Q2` and let the compiler expand it. This avoids off-by-one boundary errors (is Q2 end June 30 or July 1?).

References are bare strings.

`track: platform` is far easier for an LLM to emit correctly than a nested reference object. The schema context (field name) determines the target type; no wrapper object is needed.

IDs are kebab-case slugs.

Agents derive IDs from labels: “API Gateway Migration” → `api-gateway-migration`. This is a simple and reliable transformation. UUIDs would be safe but opaque; sequential numbers would collide with re-generation.

No dependency scheduling.

Since the IR is a Timeline Grammar (not a Task Scheduling Grammar), there is no dependency resolution. Each activity is independently valid. Agents do not need to compute a topological order or check resource constraints.

0.29.4 Validation Pipeline

Validation is layered. Each layer is a gate: failure at an earlier layer produces errors that block later layers. This design prevents cryptic failures (e.g., a reference-resolution error caused by a silently malformed ID).

Layer	Name	What it checks	Failure mode
1	Syntactic parse	Valid YAML or JSON; no parse errors	Hard error: stop
2	Schema conformance	Required fields present; types match; enum values valid; ID regex; <code>oneOf</code> for start/span	Hard error: list all violations
3	Well-formedness	Referential integrity; ID uniqueness; date ordering; progress bounds; group acyclicity	Hard error: list all violations
4	Render-readiness	See §?? (Barbara)	Error or warning depending on severity
5	Semantic advisory	Degenerate documents; empty activities and milestones; items outside <code>time_range</code>	Warning only

Table 54: Validation pipeline layers

Layer 4 dependency. Render-readiness validation—checking that the IR can produce a legible visual output—is defined and owned by Barbara in §??. This includes checks such as overlapping activities on the same track that cannot be visually separated, labels that exceed track width at the chosen axis unit, and milestone dates that fall outside the document’s `time_range`. The validation pipeline calls Barbara’s render-readiness checks as Layer 4; this section does not duplicate them.

0.29.4.1 Errors vs. Warnings

Error

Prevents rendering. The document is invalid and the renderer must refuse to proceed. Errors come from Layers 1–3 and from Layer 4 severity “fatal”. The validator must return *all* errors in a single pass, not just the first.

Warning

The document is valid but the output may be suboptimal. Warnings come from Layer 4 (advisory) and Layer 5. Renderers proceed and may annotate the output (e.g., a clipped activity) or log the warning and continue.

0.29.5 Error Handling and Agent Repair Cycle

0.29.5.1 Error Message Format

Every validation error is **path-anchored**: it identifies the exact location in the document, the nature of the violation, and a suggested fix. This design enables an agent to mechanically apply repairs without re-reading the whole document.

The error message format is:

```

1 ERROR <path>: <description>
2     Expected: <constraint>
3     Found:    <actual value>
4     Fix:      <suggested correction>

```

Listing 88: Validation error message format

Concrete error examples:

```

1 # Missing required field
2 ERROR activities[0].id: required field 'id' is missing
3     Expected: string matching ^[a-z][a-z0-9-]*$
4     Found:    (absent)
5     Fix:      add id: api-gateway-migration
6
7 # Invalid ID format
8 ERROR activities[1].id: "API Migration" does not match
9     ^[a-z][a-z0-9-]*$
10    Expected: kebab-case slug
11    Found:    "API Migration"
12    Fix:      id: api-migration
13
14 # Unresolved track reference
15 ERROR activities[2].track: "eng" references a non-existent track
16    Expected: one of [platform, mobile, data]
17    Found:    "eng"
18    Fix:      track: platform (or add a track with id: eng)
19
20 # Invalid status value
21 ERROR activities[3].status: "active" is not a valid Status
22    Expected: one of [planned, in-progress, done, at-risk,
23                blocked, cancelled, tentative]
24    Found:    "active"

```

```

24         Fix:      status: in-progress
25
26 # Date ordering violation
27 ERROR   activities[4]: end precedes start
28         Expected: end >= start
29         Found:    start: 2026-Q3, end: 2026-Q1
30         Fix:      swap dates, or use span: 2026-Q3 if this is
31                   a single-quarter activity
32
33 # Progress out of range
34 ERROR   activities[5].progress: 1.4 is outside [0, 1]
35         Expected: number in [0, 1]
36         Found:    1.4
37         Fix:      progress: 1.0
38
39 # Duplicate ID
40 ERROR   milestones[1].id: "platform-launch" is already used by
         activities[2].id
41         Expected: globally unique identifier
42         Found:    duplicate
43         Fix:      id: platform-launch-ms (or another unique slug)
44
45 # Missing date source (neither start nor span present)
46 ERROR   activities[6]: no temporal anchor defined
47         Expected: exactly one of: (start [+ optional end]) or span
48         Found:    neither start nor span present
49         Fix:      add span: 2026-Q2 or start: 2026-04-01

```

Listing 89: Example validation errors with suggested fixes

0.29.5.2 Agent Repair Cycle

The validation-error-repair loop is the core of reliable agent IR generation. The cycle is:

1. **Generate.** Agent generates an IR document, optionally using structured output constraints (§0.29.3.2).
2. **Validate.** The IR is submitted to the `validate_timeline` MCP tool (§0.29.6.1). All layers are run; all errors collected.
3. **Report.** Errors are returned to the agent as a structured list with path, description, and suggested fix.
4. **Repair.** The agent applies targeted fixes. Because errors are path-anchored and include suggested fixes, repairs are surgical: the agent patches `activities[2].track` rather than regenerating the entire document.
5. **Re-validate.** The patched document is re-submitted. This loop repeats until validation passes with no errors (warnings may remain).
6. **Render.** Once validation passes, the agent calls `render_timeline` to produce output.

In practice, structured output constraints eliminate most Layer 2 errors on the first generation pass. Layers 3 (referential integrity, date ordering) are the most common source of residual errors for agents generating multi-track documents.

0.29.6 MCP Server

The MCP server [4] exposes Timeline Compiler as a tool-callable service for AI agents. It is a first-class distribution target, not an afterthought (see the distribution architecture in §0.31). The server hosts four tools.

0.29.6.1 Tool Contracts

validate_timeline Runs the full validation pipeline (Layers 1–5) against a provided IR document. Returns all errors and warnings in structured form.

```

1 tool: validate_timeline
2 description: >
3   Validate a Timeline IR document. Runs syntactic, schema,
4     well-formedness,
5     render-readiness, and semantic advisory checks. Returns structured
6     errors
7     and warnings with path anchors and suggested fixes.
8
9 input:
10   ir:
11     type: object | string    # IR as parsed object or raw YAML/JSON
12     string
13     required: true
14   stop_at_layer:
15     type: integer            # optional; 1-5; default: 5 (run all
16     layers)
17     required: false
18
19 output:
20   valid: boolean             # true iff zero errors (warnings do not
21   block)
22   errors:
23     - path: string           # e.g. "activities[2].track"
24       code: string           # machine-readable error code, e.g.
25         UNRESOLVED_REF
26       message: string        # human-readable description
27       fix: string            # suggested correction
28   warnings:
29     - path: string
30       code: string
31       message: string
32   layers_run: integer        # how many layers were executed before
33   stopping

```

Listing 90: validate_timeline tool contract

render_timeline Renders a valid Timeline IR to a visual output format. Returns the rendered output as a base64-encoded string or a file path (for local MCP deployments).

```

1 tool: render_timeline
2 description: >
3   Render a Timeline IR document to SVG, PNG, PDF, or PPTX. The IR
4     must
5     be valid (validate_timeline returns valid: true) before rendering.

```

```

5  Rendering is deterministic: identical IR + theme + format always
6  produces bit-identical output.
7
8  input:
9    ir:
10     type: object | string    # IR as parsed object or raw YAML/JSON
11     string
12     required: true
13   format:
14     type: string
15     enum: [svg, png, pdf, pptx]
16     default: svg
17     required: false
18   theme:
19     type: string              # theme identifier, e.g. "default",
20     "corporate"
21     default: "default"
22     required: false
23   width:
24     type: integer             # output width in pixels (PNG/PDF only)
25     required: false
26
27 output:
28   format: string              # echo of requested format
29   data: string                # base64-encoded output bytes
30   mime_type: string           # e.g. "image/svg+xml"
31   warnings:                   # render-layer warnings (does not block
32     output)
33   - message: string

```

Listing 91: render_timeline tool contract

describe_schema Returns the full JSON Schema for the IR and field-level documentation. Agents call this to bootstrap IR generation or to recover from schema errors.

```

1  tool: describe_schema
2  description: >
3    Return the JSON Schema for the Timeline IR, plus human-readable
4    documentation for each field. Use this to understand what fields
5    are
6    required, their types, allowed values, and defaults.
7
8  input:
9    version:
10     type: string              # IR version to describe; default: latest
11     required: false
12   entity:
13     type: string              # optional: limit to one entity, e.g.
14     "Activity"
15     required: false
16
17 output:
18   version: string             # IR version described
19   schema: object              # full JSON Schema document
20   docs:
21     - entity: string          # entity name
22     fields:

```

```

21     - name:      string
22       type:      string
23       required:  boolean
24       default:   string | null
25       description: string
26       example:   string

```

Listing 92: describe_schema tool contract

suggest_time_range A convenience tool for agents that need to derive a sensible `time_range` and `axis_unit` from a collection of candidate activities before composing the full document. This is especially useful when ingesting data from sources that lack explicit time windows.

```

1  tool: suggest_time_range
2  description: >
3    Given a list of activity date hints (start, end, span values),
4    suggest
5    appropriate time_range and axis_unit values for the IR metadata.
6    Does not
7    require a complete IR document.
8
9  input:
10   dates:
11     type: array          # list of date strings, e.g. ["2026-Q1",
12       "2026-09-15"]
13     items: { type: string }
14     required: true
15   prefer_unit:
16     type: string         # optional hint: "quarter", "month", etc.
17     required: false
18
19  output:
20   time_range:
21     start: string
22     end: string
23   axis_unit: string      # recommended AxisUnit
24   rationale: string      # explanation of the choice

```

Listing 93: suggest_time_range tool contract

0.29.6.2 MCP Server Deployment

The MCP server is deployed in two modes:

Local server

Runs as a subprocess alongside the agent environment. Started via the CLI (`timeline mcp-server -port 3000`) or as an npm/pip package. Suitable for development, CI, and agent runtimes on the same host.

Hosted endpoint

A cloud-deployed instance of the MCP server. Suitable for cloud-native agent runtimes where a local subprocess is impractical.

Both modes expose identical tool contracts. The local server operates on the local filesystem (output file paths are local); the hosted server returns base64-encoded output in all cases.

0.29.7 Round-Trip and Iterative Editing

A Timeline IR document lives in version control alongside the source data it was derived from. Humans and agents edit it directly. Re-syncing against an updated source must not silently overwrite editorial decisions made in the IR. This section designs the mechanisms that make round-trip editing safe.

0.29.7.1 Source Provenance via Metadata

Every entity produced by an ingestion workflow includes a `metadata` block that records the source of record:

```

1 activities:
2   - id: api-gateway-migration
3     label: API Gateway Migration
4     track: platform
5     span: 2026-Q2
6     status: in-progress
7     metadata:
8       source: ado
9       ado_id: 1024
10      ado_revision: 42          # ADO ETag for change detection
11      ado_area: "Platform Team"
12      ado_iteration: "Platform\\2026\\Q2"
13      ingested_at: 2026-06-09

```

Listing 94: Provenance metadata block (ADO example)

```

1 activities:
2   - id: customer-portal-redesign
3     label: Customer Portal Redesign
4     track: product
5     span: 2026-Q3
6     status: planned
7     metadata:
8       source: github
9       github_project_id: PVT_kwD0A12345
10      github_issue: 214
11      github_url: "https://github.com/acme/portal/issues/214"
12      ingested_at: 2026-06-09

```

Listing 95: Provenance metadata block (GitHub Projects example)

The `source` key in `metadata` is a reserved provenance marker. The ingester writes it; the renderer ignores it. Re-sync logic uses it to correlate IR entities back to their source records.

0.29.7.2 Re-Sync Strategy

When source data is updated (e.g., a work item's state changes from Active to Resolved), the re-sync operation must:

1. **Match by source ID.** Use `metadata.ado_id` or `metadata.github_issue` as the stable foreign key, not the IR id. The IR id is a human-edited slug that may have been renamed.
2. **Update only source-mapped fields.** Fields that the ingestion workflow maps from the source (e.g., `status`, `span` derived from `IterationPath`) are overwritten. Fields the human may have edited in the IR (`label`, `category`, `description`, `color`, `progress`) are *not* overwritten.
3. **Preserve the IR id.** The kebab-case slug must not be regenerated on re-sync. Once set, it is stable.
4. **Log what changed.** The re-sync operation must produce a structured change log (entity id, field, old value, new value) before writing, enabling human review.
5. **Reject destructive deletions.** If a source item is deleted or removed from scope, the re-sync must flag it as a warning rather than silently removing the IR entity. A human or agent must confirm removal.

0.29.7.3 Stable IDs and Git-Friendly YAML

The combination of stable kebab-case IDs and line-oriented YAML serialisation produces IR documents that diff well under version control:

```

1  activities:
2    - id: api-gateway-migration
3      label: API Gateway Migration
4      track: platform
5      span: 2026-Q2
6    - status: in-progress
7    - progress: 0.4
8    + status: done
9    + progress: 1.0
10   metadata:
11     source: ado
12     ado_id: 1024

```

Listing 96: Git diff of an IR update (status change + progress update)

Key properties of the YAML serialisation that preserve git-friendliness:

- Fields within each entity are emitted in a **canonical order** (required fields first, optional fields in schema definition order). Reordering fields does not change semantics but does generate spurious diffs; canonical order prevents this.
- **No auto-generated IDs or timestamps** in non-metadata fields. The IR ID is derived from content and frozen; `ingested_at` in `metadata` only changes when the entity is re-ingested from source.
- **Consistent quoting conventions.** Strings that are safe as bare YAML scalars are not quoted; strings with special characters use double quotes. Serialisers must apply this consistently to avoid quote-flip diffs.

- **Lists maintain insertion order**, not alphabetical. New entities appended to the end of a list produce a single-line diff at the bottom.

0.29.7.4 Human Edits and Incremental Refinement

A human author may open the IR YAML in any editor, add annotations, adjust labels, add a milestone the ingester missed, or change a `status` value to reflect a decision made after ingestion. These edits are valid IR operations and must survive re-sync (per the strategy above).

An agent may similarly refine an IR: for example, an editorial agent might re-read the document, notice that five activities in one track are unlabelled, and add descriptions. Because the agent is editing a YAML file with stable IDs and a well-defined schema, this is a safe, auditable operation. The MCP `validate_timeline` tool can be called after any edit to confirm the document remains valid.

The editing workflow is:

```

1 1. ingest(source, prompt) -> roadmap.yaml # initial
   ingestion
2 2. validate(roadmap.yaml) -> valid, 0 errors # confirm valid
3 3. edit(roadmap.yaml) -> roadmap.yaml (modified) # human/agent
   edit
4 4. validate(roadmap.yaml) -> valid, 0 errors # re-confirm
5 5. render(roadmap.yaml, svg) -> roadmap.svg #
   deterministic output
6 6. (commit roadmap.yaml + roadmap.svg to git)
```

Listing 97: Iterative refinement loop (human or agent)

Step 6 commits both the IR source and the rendered output. Because rendering is deterministic, the SVG in the repository is always reproducible from the YAML. The YAML is the source of truth; the SVG is a derived artefact that can be regenerated on demand.

0.29.8 IR Gaps and Open Questions

During the design of this section, three gaps or ambiguities in the IR contract were identified. These are flagged here for Leslie (reviewer) and Mark to reconcile; they have *not* been silently resolved in this section’s examples.

1. **today field missing from metadata.** Leslie’s scope decisions specify that the now symbolic date must be evaluated from an explicit `today` field in the IR (not from the system clock), in order to preserve determinism. However, the `metadata` table in Mark’s IR contract does not include a `today` field. Until this is resolved, the annotation type `today-marker` with `date: now` is non-deterministic. Suggested resolution: add `today: date?` to the `metadata` schema; if absent, render-time clock is used and a warning is emitted.
2. **Track sort field: index vs. order.** Mark’s binding contract (well-formedness invariant 14) refers to a `track.index` field with the rule “track index order values unique and non-negative”. The `04-ir.tex` specification defines the same field as

`order` (type `int?`). The canonical field name should be resolved to one term; this section uses `order` to match the spec, but the contract should be updated to match.

3. **span vs. start+end mutual exclusion.** The IR allows either `span` (shorthand) or `start` plus optional `end`, but does not specify the behaviour when both are present (e.g., `span: 2026-Q2` and `start: 2026-05-01` coexist in the same activity). Should this be a schema error, or should one take precedence? For agent generation, structured output can prevent this from occurring, but the validation specification needs an explicit ruling. Suggested resolution: treat co-presence of `span` and `start/end` as a schema error.

Part VIII

Roadmap & MVP

0.30 Coverage Roadmap

The product roadmap is organized around the **tiered coverage strategy**: achieving full Mermaid compatibility in priority order, then extending beyond Mermaid.

0.30.1 Strategy: Mermaid-Complete First, Net-New After

The sequencing discipline:

1. Cover all 22 Mermaid diagram types (Tiers 0–3).
2. Establish the aesthetic bar for each type (beat Mermaid’s rendering).
3. Only then invest in net-new diagram types with no Mermaid equivalent.

This ensures the migration story (“drop-in replacement with better output”) is complete before pursuing differentiation that requires user education.

0.30.2 Tier 0: Wire Existing Grammars to Mermaid Syntax

Types: flowchart, sequence, gantt, timeline, mindmap (5 types).

Work required:

- Mermaid syntax parsers for each type (PEG/Nearley grammars)
- CST → Domain IR lowering for each parser
- Compatibility test suite (parse Mermaid examples → verify output)
- Frontmatter/directive handling in parser layer

Kernel work: None. All four layout engines are built and tested.

Exit criteria: Standard Mermaid examples from `mermaid.js.org` documentation parse and render with our compiler, producing visually superior output.

0.30.3 Tier 1: The UML/Software Line

Types: class, state, erDiagram, C4Context, C4Container, C4Component, C4Dynamic (7 types).

Work required:

- New Domain IRs: ClassDiagram IR, StateDiagram IR, ER IR
- C4 IR (extends Flow IR with stereotype/boundary semantics)
- Layout: class/state use layered layout (reuse Sugiyama kernel); ER uses force-directed or layered
- Mermaid syntax parsers for all UML types
- UML-specific theme extensions (stereotypes, compartments, cardinality labels)

- “Modern UML” visual standard (beating PlantUML’s dated rendering)

Kernel work: Minor extensions to node rendering (compartmented boxes, stereotype headers, cardinality decorators on edges).

Exit criteria: Real-world UML diagrams from popular open-source projects render beautifully. Side-by-side with PlantUML shows clear aesthetic superiority.

0.30.4 Tier 2: The Chart Family

Types: pie, xychart, quadrant, radar (4 types).

Work required:

- New Chart Domain IR and JSON Schema (Section 0.21)
- Scale computation library (linear, band, radial)
- Chart layout engine compiling to Scene IR
- Mermaid parsers for all four chart syntaxes
- Chart-specific theme extensions (axis styling, gridlines, palettes, legends)

Kernel work: The chart layout engine is genuinely new architecture—the only family that requires a new kernel rather than reusing an existing one.

Exit criteria: All four chart types render from Mermaid syntax with proper axes, legends, and responsive sizing. Output quality comparable to simple Vega-Lite charts.

0.30.5 Tier 3: Remaining Types

Types: sankey, requirement, gitGraph, block, architecture, treemap, kanban, journey, packet (9 types).

Work required per type:

- Mermaid syntax parser
- Domain IR definition (most are minor variants of existing IRs)
- Layout integration (most reuse node-link, tree, or timeline kernels)
- Type-specific rendering (sankey flow widths, git branch colors, etc.)

Kernel work: Minimal. Sankey requires weighted-edge rendering; treemap requires squarified rectangle packing; others reuse existing layout families.

Exit criteria: All 22 Mermaid diagram types render correctly.

0.30.6 MVP Definition (Reframed)

The MVP is defined as:

Tier 0 complete

All five existing grammars accessible via Mermaid syntax parsers.

Aesthetic bar met

Output demonstrably superior to Mermaid for all Tier-0 types.

Agent IR path working

Structured IR input for all Tier-0 types with published JSON Schemas.

CLI + library

`timeline render input.mmd -o output.svg` works.

Composition

Multi-diagram posters from superset DSL.

The MVP answers: *Can users drop in Mermaid files and get better output, with zero migration cost, plus the agent IR and composition capabilities Mermaid lacks?*

0.30.7 Post-MVP: Net-New Diagram Types

After Mermaid-complete coverage, new diagram types with no Mermaid equivalent:

- **Architecture Decision Records** (ADR visualization)
- **Dependency graphs** (package/module dependency visualization)
- **Infrastructure diagrams** (cloud architecture beyond C4)
- **Data pipeline diagrams** (ETL/streaming topology)

These are post-Tier-3 and driven by user demand, not pre-planned.

0.30.8 Implementation Status Summary

Table 55: Current implementation status vs. roadmap tiers

Component	Tier	Status	Tests
Timeline grammar + themes	0	Built	551
Flow grammar + themes	0	Built	33
Sequence grammar + themes	0	Built	37
Tree grammar + themes	0	Built	26
Composition layer	—	Built	25
Animation (SMIL)	—	Built	incl. in flow
Schema validation	—	Built	13
Mermaid DSL parsers	0	Planned	—
UML/software line	1	Planned	—
Chart family	2	Planned	—
Remaining types	3	Planned	—

Total tests passing: 795 (all grammars + composition + schema + CLI). All existing goldens byte-identical across runs.

0.31 Distribution Strategy

This section evaluates packaging and distribution models for Timeline Compiler, recommending an architecture that serves human authors, AI agents, and embedding use cases.

0.31.1 Distribution Requirements

Timeline Compiler must be accessible to multiple audiences:

CLI Users

- Developers, DevOps engineers, and technical users who invoke rendering from terminal or scripts.

Application Embedders

- Tools that integrate Timeline Compiler as a library (documentation generators, planning tools, dashboards).

AI Agents

- LLM-based agents that need to render timelines as a tool capability.

Non-Technical Users

- Product managers and executives who want live preview while authoring (via IDE integration).

CI/CD Pipelines

- Automated rendering in build processes.

Key requirements:

- Zero-friction installation for common platforms
- Embeddable as a library
- Invocable by AI agents (MCP server or similar)
- Deterministic output regardless of invocation method
- Self-contained — minimal or no external dependencies at runtime

0.31.2 Packaging Options

0.31.2.1 Core Library

Concept: A core rendering library with programmatic API, distributed as a package in the target ecosystem’s package manager.

Options:

@timeline-compiler/core (npm)

Node.js/TypeScript library. Pros: large ecosystem, easy embedding in JS/TS tools, npm distribution is well-understood. Cons: Node runtime dependency, potentially slow for large renders, binary output (PDF, PNG) may require native dependencies.

Go Module

Single-binary compilation, embeddable via Go’s module system. Pros: no runtime dependency, fast, easy cross-compilation. Cons: embedding in non-Go tools requires FFI or subprocess.

Rust Crate

Similar to Go — single binary, high performance. Smaller ecosystem adoption than Go for CLI tools. WASM compilation possible.

Python Package (pip)

Broad adoption in data/ML communities. Cons: runtime dependency, packaging complexity, slower execution.

Recommendation: Implementation language is not decided in this spec, but the core library should expose:

- A stable programmatic API (`render(ir, theme, options) -> output`)
- Schema validation functions
- Theme loading and composition

0.31.2.2 Command-Line Interface

Concept: A CLI tool for rendering from terminal.

```

1 # Basic rendering
2 timeline render roadmap.yaml -o roadmap.pdf
3
4 # With theme
5 timeline render roadmap.yaml --theme corporate-blue -o roadmap.svg
6
7 # Validate only
8 timeline validate roadmap.yaml
9
10 # Watch mode (optional)
11 timeline watch roadmap.yaml -o roadmap.svg

```

Listing 98: CLI usage examples

Distribution:**npm global install**

`npm install -g @timeline-compiler/cli` — Easy for Node.js users, but requires Node runtime.

Standalone Binary

Distributed via GitHub Releases, Homebrew, apt, etc. No runtime dependency. Cross-platform (macOS, Linux, Windows).

Docker Image

`docker run timeline-compiler render ...` — Isolated environment, good for CI/CD.

npx

`npx @timeline-compiler/cli render ...` — Zero-install execution for one-off use.

Recommendation: Provide both:

- npm package for Node.js ecosystems
- Standalone binaries for users without Node.js

0.31.2.3 VS Code Extension

Concept: IDE integration providing:

- Syntax highlighting for `.timeline.yaml` files
- Live preview panel (re-renders on save or keystroke)
- Export commands (PDF, PNG, SVG, PPTX)
- Schema validation and error highlighting
- Theme selection

Value: Dramatically improves authoring experience for non-CLI users. Live preview is critical for iterative refinement.

Implementation: The extension embeds the core library (via WASM, bundled Node module, or subprocess call to CLI).

Distribution: VS Code Marketplace.

Recommendation: High-priority for adoption. Authors should see their timeline update as they type.

0.31.2.4 MCP Server (Model Context Protocol)

Concept: A server exposing Timeline Compiler as a tool for AI agents.

```

1 tools:
2   - name: render_timeline
3     description: "Renders a timeline IR to visual output"
4     inputSchema:
5       type: object
6       properties:
7         ir:
8           type: object
9           description: "Timeline IR conforming to schema"
10        theme:
11          type: string
12          description: "Theme name or inline theme object"
13        format:
14          type: string
15          enum: [svg, png, pdf]
16      returns:
17        type: string
18        description: "Base64-encoded output or file path"

```

Listing 99: MCP tool definition (conceptual)

Value: Agents can render timelines as a native capability. The agent generates IR, invokes the tool, receives visual output.

Deployment:

- Local MCP server (runs alongside the agent environment)
- Hosted MCP endpoint (cloud-deployed rendering service)

Recommendation: Essential for the agent use case. MCP server should be a first-class distribution target, not an afterthought.

0.31.2.5 NPM Package for Embedding

Concept: The core library packaged for use in JavaScript/TypeScript applications.

```

1 import { render, loadTheme, validate } from
  '@timeline-compiler/core';
2
3 const ir = loadYAML('roadmap.yaml');
4 const errors = validate(ir);
5 if (errors.length > 0) throw new ValidationError(errors);
6
7 const theme = await loadTheme('corporate-blue');
8 const svg = await render(ir, theme, { format: 'svg' });
9
10 res.setHeader('Content-Type', 'image/svg+xml');
11 res.send(svg);

```

Listing 100: Embedding in a Node.js application

Use Cases:

- Documentation sites that render timelines dynamically
- Internal tools with timeline visualization
- SaaS products embedding Timeline Compiler

Recommendation: Core distribution path for the JavaScript ecosystem.

0.31.2.6 Standalone Go Binary

Concept: If the implementation uses Go, the CLI is a single statically-linked binary.

Pros:

- No runtime dependencies
- Fast startup
- Easy cross-compilation (macOS, Linux, Windows, ARM)
- Simple distribution (download and run)

Cons:

- Embedding in non-Go applications requires subprocess or FFI
- Community contribution may be lower than TypeScript (smaller Go population among target users)

Recommendation: Attractive for CLI distribution. If TypeScript is the core language, a Go wrapper or WASM compilation may achieve similar benefits.

0.31.3 Implementation Language Trade-offs

The distribution strategy depends on implementation language. Key considerations:

Table 56: Language Trade-offs for Distribution

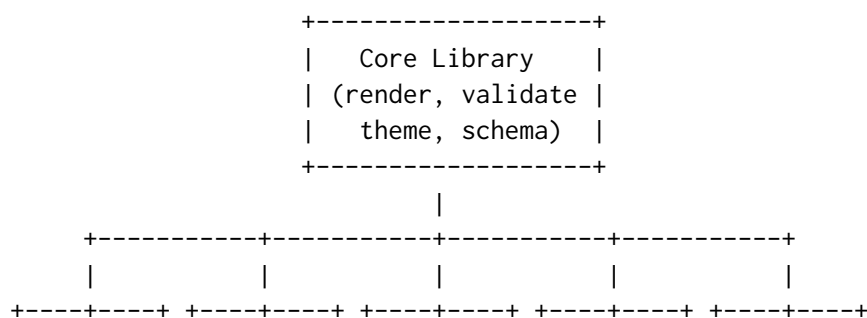
Factor	TypeScript/Node	Go
CLI standalone binary	Requires pkg/nexe or similar	Native
npm embedding	Native	Requires WASM or subprocess
WASM (browser/edge)	Via esbuild or similar	Native compilation
MCP server	Native (Node ecosystem)	Native
VS Code extension	Native embedding	Subprocess or WASM
Community contributions	Larger pool (frontend devs)	Smaller but growing
PDF generation	Needs native dep (puppeteer, etc.)	Native libraries available
Startup time	Slower (V8 init)	Fast
Binary size	Larger (bundled runtime)	Smaller

Recommendation: This spec does not mandate implementation language. However, the distribution architecture should support:

1. A core library embeddable in JavaScript/TypeScript contexts (npm)
2. A standalone CLI that does not require a runtime install
3. An MCP server for agent integration
4. A VS Code extension with live preview

A TypeScript core with WASM or native compilation for CLI may satisfy all requirements. Alternatively, a Go core with a thin Node wrapper for npm distribution is viable.

0.31.4 Recommended Architecture



	CLI			npm			MCP			VS Code			Docker	
	Binary			Package			Server			Ext			Image	
+	-----+	+	-----+	+	-----+	+	-----+	+	-----+	+	-----+	+	-----+	+

Core Library: Single implementation of IR parsing, validation, theme loading, layout, and rendering. All distribution targets wrap this core.

CLI Binary: Wraps core library. Distributed via:

- GitHub Releases (tarballs for each platform)
- Homebrew (`brew install timeline-compiler`)
- npm global install (if Node.js-based)
- curl one-liner for quick install

npm Package: The core library published to npm for JavaScript/TypeScript embedding.

MCP Server: Wraps core library, exposes `render_timeline` and `validate_timeline` tools. Distributed as:

- npm package (for local MCP server)
- Docker image (for hosted deployment)

VS Code Extension: Embeds core library (via bundled WASM or Node module). Published to VS Code Marketplace.

Docker Image: Contains CLI binary. For CI/CD pipelines and isolated execution.

0.31.5 Versioning and Compatibility

Semantic Versioning: All packages follow semver. Breaking changes to IR schema or theme format require major version bump.

Schema Versioning: The IR declares its schema version (`version: "1.0"`). The compiler reports errors if the IR version is unsupported.

Theme Versioning: Themes declare compatibility with IR/compiler versions.

CLI Version Reporting: `timeline -version` reports compiler version, supported IR versions, and supported output formats.

0.31.6 Distribution Priorities

For MVP, distribution priority order:

1. **CLI Binary** — Core functionality, scriptable, CI/CD compatible
2. **npm Package** — Embedding in JS/TS tools

3. **VS Code Extension** — Authoring experience
4. **MCP Server** — Agent integration
5. **Docker Image** — Isolated execution

Post-MVP:

- Homebrew formula
- apt/yum packages
- GitHub Action for rendering in workflows
- Hosted rendering API (optional commercial offering)

0.32 Open-Source Strategy

0.32.1 Is This a Viable Open-Source Project?

The short answer is yes, but with conditions. The diagram-as-code category has demonstrated commercial and community viability at scale: Mermaid [33] has over 88,000 GitHub stars, raised \$7.5M in seed funding in 2024 [30], and has been natively integrated into GitHub, GitLab, Notion, Obsidian, and Azure DevOps. The lesson from Mermaid is that a plain-text format with zero-install rendering, embedded in documentation ecosystems, can achieve massive adoption.

Timeline Compiler’s gap analysis (Section 0.5) identifies a cell in the tool landscape that is genuinely unoccupied: an *open-source, presentation-quality, agent-generation-friendly* timeline compiler. This gap is both an opportunity and a risk; the following sections analyse both.

0.32.2 Closest Existing Projects and the Gap They Leave

- **Mermaid** (§0.5) leaves the presentation-quality gap. Its `timeline` and `gantt` types are documentation tools, not slide-production tools. No PPTX target. No theme engine. Layout is viewport-dependent, not deterministic.
- **TimelineJS** [28] (Knight Lab, GPL) is the best-known open-source timeline tool, but it is a web-storytelling widget (iframes, Google Sheets), not a document/slide compiler. No SVG/PPTX/PDF output; no CLI; no agent path.
- **Frappe Gantt** [15] and **vis-timeline** [58] are interactive browser widgets. No declarative text format; no static artefact output.
- **think-cell** [56] produces the closest visual quality but is closed-source, expensive, and PowerPoint-locked. It is the benchmark for what Timeline Compiler output should look like, not a substitute for it.
- **Plotly.js** [44] can produce high-quality timeline charts, but requires programming and has no declarative timeline grammar or agent-generation story.

The gap is real: the market has diagram-as-code (low quality, wrong idiom) or presentation tools (high quality, closed, manual). Nothing sits in between.

0.32.3 Potential Adopters

Four adopter personas have been identified based on real-data patterns observed during the research phase:

Developer-Relations and Technical Writers. Dev-rel teams already live in Mermaid and Markdown. They need to produce roadmaps and architecture-evolution timelines for conference talks, blog posts, and product docs. The barrier to adoption is near-zero if Timeline Compiler has a CLI, a Markdown code-fence syntax, and SVG output. This group provides early adopters and community feedback.

Product Managers and Programme Leads. PMs need quarterly roadmaps and programme timelines for steering committees and executive reviews. They currently use PowerPoint or think-cell manually. A compiler that takes a YAML/CSV definition and emits a PPTX with think-cell-quality layout would replace hours of work per roadmap cycle. This group provides the strongest word-of-mouth growth vector.

Management Consultants (Freelance / Boutique). Large firms use think-cell. Freelancers and boutique consultancies cannot justify \$27.30/user/month per project. An open-source tool with a polished PPTX target has a credible value proposition here.

AI Agent Toolchains. The emergent user is not a human at all. LLM-based planning agents (GitHub Copilot, OpenAI Assistants, Anthropic Claude with tools) generate project plans, architecture proposals, and transformation roadmaps. Today there is no standard open-source *target format* for those plans. Timeline Compiler’s IR, if published as a JSON Schema and exposed as an MCP tool [4], becomes the target format AI agents compile to. This persona drives long-tail adoption as agent toolchains proliferate.

0.32.4 Ecosystem Fit

0.32.4.1 Mermaid / Markdown / CI Ecosystem

Timeline Compiler should position itself as the “next step” after Mermaid: use Mermaid in your README; use Timeline Compiler when you need a slide. Concretely:

- A Mermaid-syntax ingester (accepting `timeline` and `gantt` blocks) provides a zero-friction migration path.
- A Markdown code-fence processor (similar to Mermaid’s own fence renderer) enables embedding timelines in MkDocs, VitePress, and Quarto [33].
- A GitHub Actions step for CI-driven rendering integrates into the workflow teams already use for diagram-as-code.
- SVG output embeds directly in Slidev [3] and Obsidian [40] — both are Mermaid-native environments.

0.32.4.2 MCP / Agent Ecosystem

The Model Context Protocol [4] is the emerging standard for LLM tool invocation. An MCP server that exposes Timeline Compiler as a tool would allow any MCP-compatible agent (GitHub Copilot, Claude, etc.) to:

1. Accept a natural-language plan from the user.
2. Generate a valid Timeline IR (JSON/YAML, constrained by the published JSON Schema).
3. Invoke the Timeline Compiler MCP tool.
4. Receive back an SVG, PNG, or PPTX and embed it in the agent’s response.

OpenAI’s Structured Outputs feature [41] (JSON Schema-constrained generation) enables agents to reliably produce valid IR without hallucinating schema fields. The design implication is that the Timeline IR schema *must* be published as a JSON Schema document alongside the compiler.

0.32.4.3 PPTX Ecosystem

python-pptx [49] is the leading open-source Python library for programmatic PPTX generation. It is MIT-licensed, well-maintained, and can produce slides that open natively in PowerPoint and Keynote. Building the PPTX output target on python-pptx avoids a dependency on LibreOffice or headless Chrome, making the compiler pip-installable with minimal system requirements.

0.32.5 Extension Opportunities

The compiler-IR architecture creates natural extension points:

Ingesters.

New data sources can be added without touching the renderer: CSV/Excel, Jira API, Linear API, Notion database, ADO work items [35], GitHub Projects [22], markdown plans, Mermaid blocks. Ingesters are the highest-traffic contribution path for community members.

Themes.

A typed theme schema (fonts, colour palettes, swimlane heights, milestone shapes) allows community contribution of brand themes without touching rendering code. McKinsey-style, BCG-style, GitHub-style, and corporate-brand themes are natural community contributions.

Output Targets.

Additional renderers (HTML interactivity, Keynote, Google Slides API, LaTeX beamer) can be added without changing the IR or ingesters. The Vega-Lite model [48] demonstrates that a clean IR/renderer separation enables this pattern reliably.

Grammar Extensions.

The IR can be versioned and extended (new event types, annotations, dependency arrows) without breaking existing themes or renderers, following established IR versioning practice from compiler design [1].

0.32.6 Strongest Differentiators

Three differentiators are assessed as strongest, in decreasing order of defensibility:

1. **The only open-source, agent-generation-friendly timeline compiler with PPTX output.** No existing open-source tool combines a stable text-format IR, deterministic rendering, and a PPTX output target. This is the clearest gap in the landscape.
2. **Visual-communication grammar vs. task-scheduling grammar.** Positioning the tool as a “timeline grammar for human communication” rather than a “project management tool” is a genuine conceptual distinction. It attracts users who are currently underserved by Mermaid (insufficient quality) and repelled by MS Project (wrong abstraction).
3. **AI-agent-first design.** Publishing a JSON Schema for the IR and an MCP tool interface makes Timeline Compiler the natural compiler target for any LLM that generates plans or roadmaps. As agent-driven workflows proliferate [4], this positions Timeline Compiler at the centre of a growing distribution channel with no human adoption friction.

0.32.7 Community Contribution Model

The Mermaid model [30] is instructive: open-source core (MIT), commercial SaaS layer for hosted rendering, team features, and enterprise integrations. Timeline Compiler should follow a similar open-core pattern:

- **Core compiler** (MIT): IR schema, renderer, CLI, standard output targets, standard ingesters. All of this is open-source. Community contributions to ingesters and themes are accepted here.
- **Theme marketplace** (community-driven): A GitHub-hosted registry of community themes, analogous to the npm registry for packages or the VS Code extension marketplace. Themes are independent packages; they do not require merging into the core.
- **Hosted service** (future, optional): Web UI for non-technical users, CI integration hooks, and enterprise SSO. This funds maintenance without restricting the open-source core.

The contribution surface most likely to attract community contributions (in descending order of probability) is: ingesters, then themes, then additional output targets, then IR schema extensions. Core renderer changes require higher trust and should require maintainer review.

0.32.8 Adoption Risk Assessment

No adoption analysis is honest without a candid risk section.

Risk 1: The presentation-quality bar is high. think-cell is the quality benchmark. Matching it with an open-source, text-driven renderer is a significant engineering challenge. If the initial PPTX output is visually inferior to think-cell, the “presentation quality” differentiator collapses, and Timeline Compiler is merely a less capable Mermaid. *Mitigation:* Define a strict visual quality bar in the design spec and validate against real executive roadmaps before launch.

Risk 2: The IR may be too narrow or too wide. An IR that is too narrow (only covering the simplest roadmaps) will be inadequate for real-world use. An IR that is too wide (attempting to model every possible timeline idiom) will be complex to understand, hard for agents to generate reliably, and fragile to maintain. *Mitigation:* Scope the IR to the five canonical timeline types identified in the scope section, and use real data crawls (ADO exports, GitHub Projects exports) to validate coverage before freezing the schema.

Risk 3: Mermaid is a gravity well. Mermaid has 88,000 stars, is embedded everywhere, and is improving. If Mermaid’s timeline type adds PPTX export and better theming, the core differentiator weakens. *Mitigation:* The visual-communication grammar thesis and the agent-generation design are harder to retrofit into Mermaid than a PPTX renderer. The IR schema and MCP interface are the durable moats.

Risk 4: Agent toolchains may standardise on Mermaid instead. LLMs already reliably generate Mermaid syntax. If the agent ecosystem converges on Mermaid + post-processing as the standard agent-to-timeline pipeline, the agent-first positioning loses force. *Mitigation:* Publish the JSON Schema early; build the MCP tool before the ecosystem settles. Structured Outputs [41] are demonstrably more reliable than free-text Mermaid generation for complex schemas.

Risk 5: PPTX format instability. The OOXML PPTX specification is controlled by Microsoft. python-pptx [49] may lag new PowerPoint features. *Mitigation:* Treat PPTX as an output layer, not the IR. SVG and PDF are canonical; PPTX is a convenience export. Degrade gracefully when PPTX features are unavailable.

0.33 Target Outputs: Worked Examples and Coverage Analysis

This section validates the Timeline Compiler design against five concrete end-results provided by the project owner. For each target we embed the reference image, characterise the visual, prove representability with a worked Timeline IR in YAML using Mark’s field names (Section ??), and state the layout family, theme, effects, and backend tier required. Section 0.33.6 then synthesises the findings: new layout families the targets demand, a consolidated coverage table, a structured gap list, and a prioritised verdict.

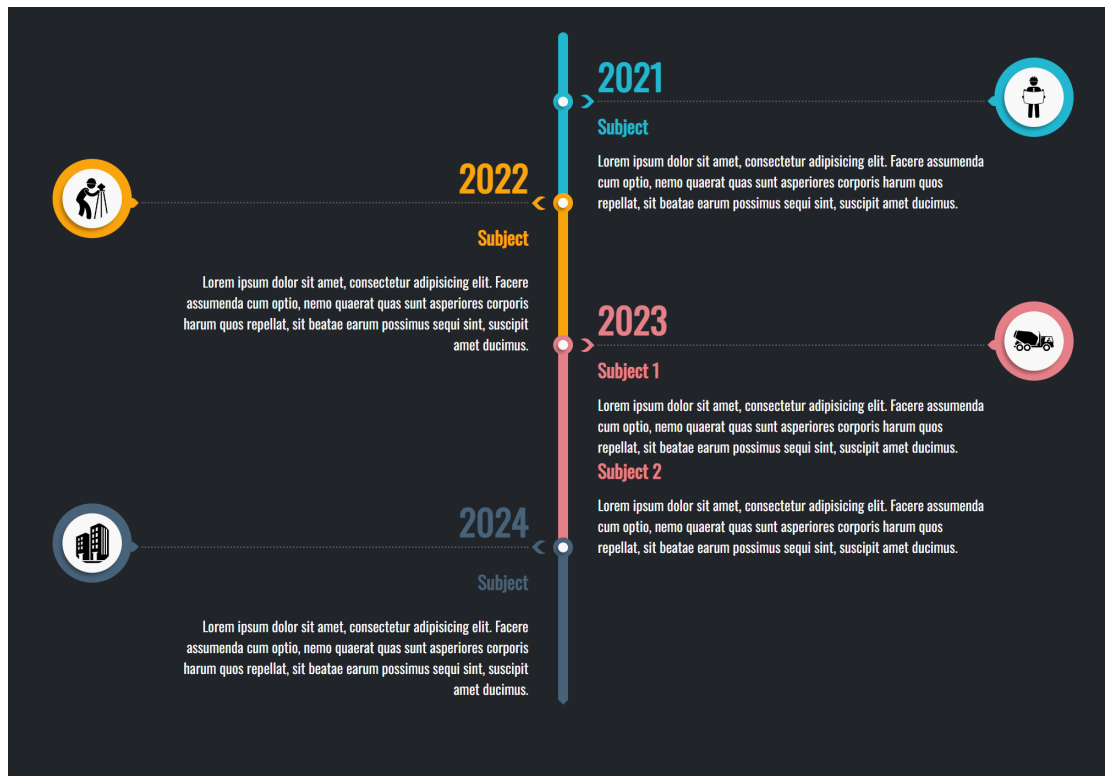


Figure 11: T1: Vertical central-spine timeline, dark charcoal background, colour-coded year segments, alternating left/right text entries, and circular icon badge callouts with dashed leader lines. (Reference image provided by project owner.)

0.33.1 T1 — Vertical Spine, Dark Theme, Icon Badges

Visual characterisation.

- **Layout family:** Vertical central-spine; time axis runs top to bottom; year nodes (small circles) sit on the spine; entries alternate right/left by year.
- **Spine segmentation:** Each year-to-year segment is coloured distinctly: cyan (2021), amber (2022), pink-red (2023), slate-blue (2024).
- **Entry style:** Coloured “Subject” heading plus body paragraph, stacked left or right of the node. Year 2023 has *two* stacked subjects under one node.
- **Icon badges:** Circular white disc with coloured ring and soft drop shadow, holding pictograms (hard-hat worker, surveyor, cement truck, office building). Each badge sits at the far canvas edge, connected to its year node by a horizontal dashed leader line with a small arrowhead.
- **Theme/mood:** Dark charcoal background, clean sans-serif, soft shadows, high contrast colour accents.

Worked Timeline IR. The year nodes map to milestones; each subject entry maps to an activity with span equal to the relevant year. Two subjects in 2023 share a group. Per-node colour is expressed via `category` resolved through the theme’s `category_map` (the IR has no direct `color` field on milestones or activities—see Gap IR-2 in Section 0.33.6.3). The icon pictogram is carried by `milestone.icon`. Alternating side placement and the badge-leader-line visual are render concerns, not IR data.

```

1 version: "1.0"
2 metadata:
3   title: "Company Journey 2021-2024"
4   today: 2026-06-10
5   time_range:
6     start: 2021
7     end: 2024
8   axis_unit: year
9   theme: dark-executive          # GAP: theme does not yet exist
    (see Sec. 14.6c)
10 tracks:
11   - id: spine
12     label: ""
13     index: 0
14 groups:
15   - id: y2023-entries
16     label: "2023"
17     members: [y2023-s1, y2023-s2]
18 milestones:
19   - id: y2021
20     label: "2021"
21     date: 2021-01-01
22     track: spine
23     icon: hard-hat-worker        # pictogram hint; renderer resolves
    to pictogram
24     category: year-cyan          # theme category_map: { year-cyan:
    {fill: "#00BCBE"} }
25   - id: y2022
26     label: "2022"
27     date: 2022-01-01
28     track: spine
29     icon: surveyor
30     category: year-amber         # theme category_map: { year-amber:
    {fill: "#F5A623"} }
31   - id: y2023
32     label: "2023"
33     date: 2023-01-01
34     track: spine
35     icon: cement-truck
36     category: year-red           # theme category_map: { year-red:
    {fill: "#E8314A"} }
37   - id: y2024
38     label: "2024"
39     date: 2024-01-01
40     track: spine
41     icon: office-building
42     category: year-slate         # theme category_map: { year-slate:
    {fill: "#4A6FA5"} }
43 activities:
44   - id: y2021-entry
45     label: "Subject"
46     track: spine
47     span: 2021
48     description: "Body text describing the 2021 period."
49     category: year-cyan
50     metadata:
51       entry_side: right          # render hint: vertical-spine

```

```

    renderer reads this
52 - id: y2022-entry
53   label: "Subject"
54   track: spine
55   span: 2022
56   description: "Body text describing the 2022 period."
57   category: year-amber
58   metadata:
59     entry_side: left
60 - id: y2023-s1
61   label: "Subject 1"
62   track: spine
63   span: 2023
64   group: y2023-entries
65   description: "Body text for 2023 Subject 1."
66   category: year-red
67   metadata:
68     entry_side: right
69 - id: y2023-s2
70   label: "Subject 2"
71   track: spine
72   span: 2023
73   group: y2023-entries
74   description: "Body text for 2023 Subject 2."
75   category: year-red
76   metadata:
77     entry_side: right
78 - id: y2024-entry
79   label: "Subject"
80   track: spine
81   span: 2024
82   description: "Body text describing the 2024 period."
83   category: year-slate
84   metadata:
85     entry_side: left
86 # Icon badge leader lines: the vertical-spine renderer draws these
87   automatically
88 # from each milestone icon badge to its spine node; badge position
89   is a
90 # layout-family concern and requires no separate annotation in the
91   IR.

```

Listing 101: T1 worked IR — vertical spine, dark, icon badges (abbreviated)

IR field mapping summary. `milestones[].icon` carries the pictogram hint. `milestones[].category` resolves to per-year spine colour via `theme.category_map`. `activities[].group` clusters the two 2023 subjects. The `entry_side` key within `activities[].metadata` is a render hint read by the vertical-spine layout module (open extension map; no IR schema change needed). Multiple subjects under one node are fully expressible using `group.members`.

Layout, theme, effects, and tier.

- **Layout family:** Vertical central-spine with alternating card entries (Section [0.33.6.1](#)).
Gap Render-1: not in current §5 pipeline.

- **Theme:** Dark-executive — deep charcoal background, coloured spine segments, drop-shadow icon badges. **Gap Theme-1:** no dark theme in the current five; see Section 0.33.6.3.
- **Effects:** Drop shadow on icon badges (Tier 2 DropShadow); dashed-leader connector style (annotation connector_style vocabulary extension, Gap Render-5). Already supported by the effect registry (§??).
- **Fidelity tier:** Tier 2 (Polished); SVG backend with safe-filter caveat or Raster backend. PPTX native a:outerShdw satisfies the drop-shadow.

0.33.2 T2 — Horizontal Linear, Light, Numbered Nodes



Figure 12: T2: Horizontal single-line timeline, white background, three large numbered circular nodes (01/02/03), date above each node, title below. Corporate minimal aesthetic with generous whitespace. (Reference image provided by project owner.)

Visual characterisation.

- **Layout family:** Horizontal single-line; a single axis with no track swimlanes; milestones only, evenly spaced.
- **Node style:** Large filled circles in dark teal, with a bold ordinal number centred (01, 02, 03); the middle node is visually emphasised.
- **Labels:** Date string above each node (e.g., “15th May 2021”); title below (Application Deadline / Qualifying Exam / Training Starts).
- **Theme/mood:** Light/white background; “Our Timeline” header with logo; minimal, corporate, generous whitespace; no decorative chrome.

Worked Timeline IR. This is the most directly representable target. A single empty track carries three milestones; `metadata.title` is the header text. The ordinal number (01, 02, 03) is *purely a render concern*—the vertical-spine renderer assigns sequential numbers by milestone sort order (`date_ordinal`, `id`), so no IR field is needed. The logo is a theme-level image asset. Emphasis on the middle node is a `tags` hint or a theme-defined category.

```

1 version: "1.0"
2 metadata:
3   title: "Our Timeline"
4   today: 2026-06-10
5   time_range:
6     start: 2021-03
7     end: 2021-11
8   axis_unit: month
9   theme: light-minimal-corporate # GAP: theme does not yet exist
10   (Sec. 14.6c)
11   locale: en
12 tracks:
13   - id: main
14     label: ""
15     index: 0
16 milestones:
17   - id: app-deadline
18     label: "Application Deadline"
19     date: 2021-05-15
20     track: main
21     status: done
22     category: standard-node
23   - id: qualifying-exam
24     label: "Qualifying Exam"
25     date: 2021-06-20
26     track: main
27     status: in-progress
28     tags: [emphasized] # render hint: larger/filled node
29     treatment
30     category: standard-node
31   - id: training-starts
32     label: "Training Starts"
33     date: 2021-09-01
34     track: main
35     status: planned
36     category: standard-node
37 # Ordinal node numbers (01, 02, 03) are assigned by the renderer in
38 # milestone sort order; no IR field required.
39 # Logo in header: a theme asset, not IR data.

```

Listing 102: T2 worked IR — horizontal linear, numbered nodes (abbreviated)

IR field mapping summary. `milestones[].label` becomes the title below the node. `milestones[].date` is formatted by the renderer as “15th May 2021”. `milestones[].tags: [emphasized]` is the render hint for the filled/prominent middle node. `metadata.title` is the “Our Timeline” header. All content is fully expressible with current IR fields.

Layout, theme, effects, and tier.

- **Layout family:** Horizontal single-line (Section 0.33.6.1). This is an *edge case* of the current horizontal pipeline (one track, zero activities, no axis header band) rather than a wholly new family. The current §5 pipeline can produce it with a zero-height track-header and suppressed axis band. **Gap Render-2:** numbered circular node rendering and date-above-label-below placement rule are not in the current milestone geometry.
- **Theme:** Light-minimal-corporate. The closest existing theme is Consulting (Tier 1, clean, no gridlines) but it lacks the large numbered circle node style. **Gap Theme-2:** a dedicated corporate-minimal theme variant is needed.
- **Effects:** None beyond solid fills. Tier 1 (Crisp); SVG backend sufficient.

0.33.3 T3 — Dense Vertical Infographic, AI Timeline

Visual characterisation.

- **Layout family:** Vertical central-spine; 12+ year nodes, high density, alternating short text blurbs (bold year label + one sentence).
- **Node style:** Small colourful connector dots on the spine, each with a distinct accent colour (red, orange, teal, purple, etc.).
- **Content:** Per-node: bold year number as heading; one descriptive sentence.
- **Theme/mood:** Light background, title “THE AI TIMELINE” in large uppercase; gradient/coloured accents; high information density.

Worked Timeline IR. Twelve year milestones each carry a one-sentence description. The bold year label is `milestone.label`. Per-node colour uses `category` resolved through `theme.category_map`. The “dense” layout mode (tight vertical pitch, short alternating blurbs) is a render/theme parameter, not IR data.

```

1 version: "1.0"
2 metadata:
3   title: "The AI Timeline"
4   today: 2026-06-10
5   time_range:
6     start: 1967-01-01
7     end: 2023-12-31
8   axis_unit: year

```

```

9   theme: colorful-infographic      # GAP: theme does not yet exist
   (Sec. 14.6c)
10  tracks:
11    - id: spine
12      label: ""
13      index: 0
14  milestones:
15    - id: y1967
16      label: "1967"
17      date: 1967-01-01
18      track: spine
19      category: accent-red
20      description: "ELIZA demonstrates natural language processing at
   MIT."
21    - id: y1972
22      label: "1972"
23      date: 1972-01-01
24      track: spine
25      category: accent-orange
26      description: "PROLOG logic programming language is introduced."
27    # ... eight further year milestones omitted for brevity ...
28    - id: y2015
29      label: "2015"
30      date: 2015-01-01
31      track: spine
32      category: accent-blue
33      description: "AlphaGo defeats professional Go players for the
   first time."
34    - id: y2023
35      label: "2023"
36      date: 2023-01-01
37      track: spine
38      category: accent-purple
39      description: "Large language models achieve broad public
   deployment."
40    # No activities required: this is a pure milestone spine.
41    # Per-node colour: theme category_map entry per accent-* category.
42    # Alternating left/right blurb placement: vertical-spine layout
   concern.
43    # Dense pitch: theme parameter (entry_row_height, spine_node_gap).

```

Listing 103: T3 worked IR — dense AI timeline infographic (abbreviated to four nodes)

IR field mapping summary. `milestones[].label` is the bold year heading. `milestones[].description` is the one-sentence blurb. `milestones[].category` resolves to node accent colour. High entry count (12+) is supported without limit; the IR places no cap on list lengths. All content is expressible. The dense-pitch visual is a theme parameter.

Layout, theme, effects, and tier.

- **Layout family:** Vertical central-spine with alternating text blurbs (same family as T1). **Gap Render-1** applies.
- **Theme:** Colorful-infographic — light background, multi-colour `category_map` entries, bold oversized title, tight spine pitch. **Gap Theme-3:** not in current five

themes.

- **Effects:** Gradient/colour accents via `category_map` fills; no art effects required. Tier 1 (Crisp); SVG backend sufficient.

0.33.4 T4 — Serpentine Glowing Path

Visual characterisation.

- **Layout family:** Serpentine winding path; the time axis is encoded as distance along an S-curve, not a straight line. Milestone dots are placed at parametric arc positions.
- **Node style:** Small dots along the winding green path; icon at path start (repository host pictogram) and end (clock-in-ring pictogram).
- **Art effect:** Prominent soft glow–bloom halo around the entire path; rendered via per-pixel compositing (Bloom effect, Raster backend).
- **Theme/mood:** Light background; the path is the hero element; minimal supplementary text; “roadmap as a journey” metaphor.

Worked Timeline IR. The milestone positions along the curve are represented by their dates; the serpentine geometry is the renderer’s responsibility. The icon field on boundary milestones carries pictogram hints. The glow–bloom is declared in the theme’s fidelity block and consumed by the Raster backend.

```

1 version: "1.0"
2 metadata:
3   title: "Project Roadmap"
4   today: 2026-06-10
5   time_range:
6     start: 2026-01-01
7     end: 2026-12-31
8   axis_unit: month
9   theme: showcase                # existing Showcase theme (Tier-3
10    glow supported)
11   # GAP: serpentine spine geometry not in Sec. 5 pipeline; see Gap
12   Render-3
13 tracks:
14   - id: journey
15     label: ""
16     index: 0
17 milestones:
18   - id: start-point
19     label: "Project Start"
20     date: 2026-01-01
21     track: journey
22     icon: github-octocat         # pictogram hint: repository host
23     icon
24     status: done
25   - id: checkpoint-a
26     label: "Alpha Release"
27     date: 2026-04-01
28     track: journey
29     status: in-progress

```

```

27 - id: checkpoint-b
28   label: "Beta Release"
29   date: 2026-08-01
30   track: journey
31   status: planned
32 - id: end-point
33   label: "GA Launch"
34   date: 2026-12-01
35   track: journey
36   icon: clock-ring           # pictogram hint: clock in ring
37   status: planned
38 # GAP Render-3: serpentine spine geometry requires a dedicated module
39 # (Bezier/spline S-curve); the Sec. 5 straight-axis pipeline does
   not apply.
40 # Glow/bloom: Showcase theme fidelity.effects.glow + Bloom; Raster
   backend.
41 # Theme category_map: single green fill for path and milestone dots.

```

Listing 104: T4 worked IR — serpentine path (abbreviated)

IR field mapping summary. `milestones[].icon` carries the boundary pictogram hints. `milestones[].date` positions each dot at its parametric arc position. `milestones[].status` drives the dot fill (done/in-progress/planned). The Bloom effect and glow halo are theme-declared (existing Showcase theme, Section ??); the effect registry already contains `Bloom{radius_px, threshold, intensity, fallback_policy}`. *The sole fundamental gap is the serpentine spine geometry itself.*

Layout, theme, effects, and tier.

- **Layout family:** Serpentine winding path. **Gap Render-3** (high priority, future scope): requires a new `spine_geometry: serpentine` layout module with Bézier/spline S-curve parametrics. This is the most architecturally novel layout in the target set and cannot reuse the straight-axis pipeline.
- **Theme:** Existing Showcase theme covers glow, bloom, and the Raster backend. Only the serpentine geometry is missing from the pipeline.
- **Effects:** Glow (Tier 3 Bloom effect, Raster backend required). The Bloom primitive is already defined in the Scene effect registry (§0.10) and the Showcase theme already activates it. No new effect type is needed.
- **Fidelity tier:** Tier 3 (Showcase); Raster backend required for per-pixel bloom compositing.

0.33.5 T5 — Gitline: Card Timeline, Dark Textured, Data-Driven

Visual characterisation.

- **Layout family:** Vertical central-spine with alternating rounded cards; same family as T1 and T3.
- **Card anatomy:** Bold repo title (activity label), date with clock icon (activity start), description paragraph, “VIEW REPOSITORY” outline CTA button (activity url). One card carries a small warning note.

- **Data source:** GitHub username/org input drives activity generation (this is the project’s core data-ingestion flow; the IR result is independent of the source).
- **Theme/mood:** Dark navy background (#0D1117 range), subtle swirl/radial background texture (Tier-3 art effect), rounded cards with soft border-shadow, pagination control.
- **Alternate view (out of scope):** The “YEARLY CHART” tab shows a per-year histogram of the same data. This is a non-timeline chart view and is *explicitly out of scope* (see §0.4); it is noted here only to delimit the boundary, not as a supported render target.

Worked Timeline IR. Repository entries map directly to activities. The “VIEW REPOSITORY” CTA uses `activity.url` (already in the IR schema). The warning note maps to an annotations callout. The org avatar and web UI chrome (search input, tabs, pagination) are render/shell concerns, not IR data. The dark textured background is declared in the theme’s fidelity block (Showcase dark variant).

```

1 version: "1.0"
2 metadata:
3   title: "Gitline: mui-org"
4   author: "mui-org"
5   today: 2026-06-10
6   time_range:
7     start: 2020-06
8     end: 2022-06
9   axis_unit: month
10  theme: showcase-dark          # GAP: dark variant of Showcase
11    (Sec. 14.6c)
12  description: "Repository timeline for the mui-org GitHub
13    organisation"
14 tracks:
15   - id: repos
16     label: "Repositories"
17     index: 0
18 activities:
19   - id: react-tech-challenge
20     label: "react-technical-challenge"
21     track: repos
22     span: 2021-01
23     description: "A technical challenge project for React
24       developers."
25     url: "https://github.com/mui-org/react-technical-challenge"
26     status: done
27     category: repo-card
28   - id: material-ui-v5
29     label: "material-ui v5"
30     track: repos
31     start: 2021-03
32     end: 2021-09
33     description: "Major version release with full redesign of
34       component APIs."
35     url: "https://github.com/mui-org/material-ui"
36     status: done
37     category: repo-card
38   - id: mui-x-grid
39     label: "mui-x-grid"

```

```

36   track: repos
37   start: 2021-06
38   end: 2022-01
39   description: "Advanced data grid component with enterprise
40     features."
41   url: "https://github.com/mui-org/mui-x"
42   status: in-progress
43   category: repo-card
44 annotations:
45   - type: callout
46     target: react-tech-challenge
47     text: "Repository archived; no recent commits since 2021."
48     style: warning # theme maps 'warning' to amber
49     callout style
50 # activity.url --> "VIEW REPOSITORY" CTA button (renderer generates
51   button label)
52 # Org avatar header: theme asset / web-app shell; not IR data
53 # YEARLY CHART tab: out of scope (non-timeline aggregate view); see
54   Scope Boundaries
55 # Swirl background texture: theme fidelity.effects.noise_texture
56   (Tier-3 Raster)
57 # Pagination: HTML backend concern; no IR field needed

```

Listing 105: T5 worked IR — Gitline card timeline (abbreviated)

IR field mapping summary. `activities[].label` is the bold card title. `activities[].start` / `span` renders as the clock-icon date. `activities[].description` is the card body paragraph. `activities[].url` is the “VIEW REPOSITORY” CTA link target; the button label is a theme-level string constant (render concern, not IR data). `annotations[].style: warning` maps to the warning callout via `theme.annotation` styling. The “YEARLY CHART” tab is a non-timeline aggregate view and is *out of scope* (see §0.4); the timeline card view is the target reproduced here.

Layout, theme, effects, and tier.

- **Layout family:** Vertical central-spine with alternating rounded cards. **Gap Render-1** applies (same as T1/T3). Additionally, the card rendering anatomy (title–date–description–CTA-button within a rounded-rect card shape) requires a dedicated card-entry renderer. **Gap Render-4.**
- **Theme:** Showcase dark variant — deep navy background, swirl/radial texture via `noise_texture` / `cloud_layer` effects, rounded card shapes with soft borders and shadows. **Gap Theme-4:** the existing Showcase theme has a dark palette option (`#0A1628` background) but does not define card-entry shape rendering or the CTA-button primitive. A showcase-dark child theme is needed.
- **Effects:** Background swirl/radial texture (`NoiseTexture` or `CloudLayer`, Tier 3); drop shadows on cards (Tier 2 `DropShadow`, already in effect registry). Raster backend required for the texture layer; SVG backend handles card geometry with `embed-raster` fallback for texture.
- **Fidelity tier:** Tier 3 (Showcase); Raster backend required for background texture; HTML backend for interactive CTA links and pagination.

0.33.6 Synthesis

0.33.6.1 A: Layout Families Finding

The current §5 rendering model implements a single layout family: **horizontal swimlane Gantt** (orientation: horizontal; spine geometry: straight; entry placement: parallel track bands). The five targets expose three additional families:

1. **Vertical central-spine with alternating entries** (T1, T3, T5): orientation rotated 90°; a single central line runs top-to-bottom; entries (text blurbs, subject headings, or rounded cards) alternate left and right of year/date nodes; icon badges may anchor at the canvas edges with leader lines.
2. **Horizontal single-line with numbered nodes** (T2): orientation horizontal; but the track-band structure is collapsed to a single zero-height line; milestones are rendered as large numbered circles; date appears above the node, title below; no activity bars. This is an edge case of the existing pipeline achievable by suppressing the track header column and axis band and overriding the milestone shape—no separate layout engine is strictly required, but the milestone geometry (Phase 4, §0.17.5.4) needs a “numbered circle” shape variant.
3. **Serpentine winding path** (T4): spine geometry is a parametric Bézier S-curve; time is encoded as arc-length position along the curve; the date-to-coordinate function $x(T)$ is replaced by a curve parametrisation $\mathbf{p}(t(T))$. This is architecturally the most novel family and cannot share Phase 1–6 geometry with the straight-axis pipeline.

Layout family is a render/theme concern; the IR stays layout-agnostic. The Timeline IR (§4) makes no assumption about axis orientation or spine geometry. Field semantics (date, track, milestones) are spatial only in the sense that a renderer maps them to a canvas. Swapping layout family requires only a theme/renderer change; the same IR document is valid for any family.

Recommendation. Add an explicit `layout_family` property to the *theme schema* (Section ??) with the structure:

- `orientation`: horizontal (default) | vertical
- `spine_geometry`: straight (default) | serpentine
- `entry_placement`: swimlane (default) | alternating | card | numbered-single-line

This property lives in the theme, not the IR. The layout engine dispatches to the appropriate Phase-1–6 pipeline variant based on `layout_family.orientation` and `spine_geometry`. The IR contract is unchanged.

0.33.6.2 B: Consolidated Coverage Table

0.33.6.3 C: Gap List

(a) IR gaps — flagged for Mark (Section ??)

IR-1: Milestones lack metadata extension field.

Activities carry metadata: `map<string,any>` for arbitrary render hints and extension data; milestones do not. As T1 and T3 show, milestones (year nodes) are the primary semantic entity in infographic layouts and benefit equally from open extension. Workaround: tags: `[list<string>]` can carry string flags but not structured key–value data. *Recommendation for Mark: add metadata: map<string,any> to the Milestone schema.*

IR-2: Activities and milestones lack a direct color field.

Neither entity type has `color: string?` (tracks do). Per-entity colour today requires defining a separate category and a corresponding `category_map` entry in the theme. For dense infographic layouts (T1: 4 year colours; T3: 12 accent colours) this is ergonomically burdensome: 12 theme category definitions just to express 12 distinct dot colours. The color hint on tracks shows the precedent. *Recommendation for Mark: add color: string? to Activity and Milestone (as a direct colour override hint, applied after category_map and before status_map).*

IR-3: annotation.callout has no icon field.

In T1, the circular icon badge at the canvas edge is semantically “the milestone’s icon rendered in a badge callout”. The current design resolves this by having the vertical-spine layout module read `milestone.icon` and auto-generate the badge—no separate annotation or IR change is needed. Confirmed not an IR gap; documented for clarity.

IR-4: Multiple subjects under one year node.

T1 shows two stacked subjects under 2023. Representable today via `groups[].members` containing two activities both with `span: 2023`. The renderer clusters group members at the same node. Confirmed not an IR gap.

IR-5: CTA link label is not separately stored.

T5’s “VIEW REPOSITORY” button text is not in the IR; the renderer uses a theme-level default label derived from `activity.url`. Authors who want a custom CTA label can use `activity.metadata.cta_label: "..."`. No IR schema change needed; documented for theme implementers.

(b) Render and layout additions — owned by Barbara**Render-1: Vertical central-spine layout module (Priority 1).**

Covers T1, T3, T5 (three of five targets). New `layout_family` property in the theme schema (`orientation: vertical, entry_placement: alternating | card`). The Phase-1 axis computation maps dates to the *y*-axis; Phases 2–4 compute entry positions left/right of the spine. The Phase-1–6 pipeline structure is preserved; phases are parameterised, not replaced.

Render-2: Numbered circular node shape for milestones (Priority 2).

Covers T2. A new milestone shape variant `shape: numbered-circle` in the theme milestone block. The renderer assigns ordinal numbers by milestone sort order; no IR field needed.

Render-3: Serpentine spine geometry (Priority 3 / future scope).

Covers T4. Replaces the straight-axis date-to-coordinate function with a parametric Bézier S-curve module. The date-to-arc-length mapping must be monotone and deterministic. Architecturally the most complex addition; recommended for a post-MVP release.

Render-4: Card-entry rendering for vertical-spine (Priority 2).

Covers T5. Within the vertical-spine layout, entries rendered as rounded-rect cards (title text, date–icon row, description paragraph, CTA-button primitive). The card shape and CTA-button are theme-defined; data comes from `activity.{label, start, description, url}`.

Render-5: Dashed-leader annotation connector style (Priority 2).

Covers T1 icon-badge leader lines. Extend the `theme.annotation.connector_style` vocabulary with `dashed-leader-arrow` (dashed stroke with arrowhead terminus). The annotation type: `connector` is already in the IR; only the visual style is missing.

Render-6: “YEARLY CHART” aggregate view — out of scope.

The T5 reference includes a “YEARLY CHART” tab (a per-year histogram). This is a non-timeline aggregate visualization and is *out of scope* (§0.4); it is not a deferred feature but an explicit boundary.

(c) Theme additions — owned by Barbara**Theme-1: dark-executive (Priority 1).**

Deep charcoal–navy background, coloured spine segments per category, drop-shadow icon badges, clean sans-serif. Fidelity Tier 2. Maps to T1 (dark spine, year nodes) and the dark shell of T5 (Gitline). Extends the existing Showcase base: `extends: showcase`; overrides `canvas.background_color`, `layout_family.orientation: vertical`, `fidelity.tier: 2`.

Theme-2: light-minimal-corporate (Priority 2).

White background, generous whitespace, numbered circular milestone nodes, no grid-lines, large milestone labels. Fidelity Tier 1. Maps to T2. Extends Consulting: `extends: consulting`; overrides `milestone.shape: numbered-circle`, `layout_family.entry_placement: numbered-single-line`.

Theme-3: colorful-infographic (Priority 2).

Light background, multi-accent `category_map` (12 colours pre-defined), dense vertical pitch (tight `spine_node_gap`), bold oversized title typography. Fidelity Tier 1. Maps to T3. New standalone theme; no suitable parent among the current five.

Theme-4: showcase-dark child theme (Priority 1).

Extends `showcase`; overrides background to deep navy (`#0D1117`), activates `noise_texture` effect (swirl/radial), sets `layout_family.orientation: vertical`, `layout_family.entry_placement: card`. Maps to T5 Gitline aesthetic. Fidelity Tier 3; Raster backend required for texture; HTML backend for interactive CTA links.

(d) Effects validation against effect registry All effects required by the five targets are already defined in the Scene effect registry (§0.10) and theme fidelity schema (§??):

- **Glow / Bloom (T4):** `Bloom{radius_px, threshold, intensity, fallback_policy}` and `Glow{color, radius_px, intensity}` are registered effects. Showcase theme activates them. Raster backend renders per-pixel; SVG backend emits `feGaussianBlur+feComposite` with determinism caveat; PPTX maps to native `a:glow`.
- **Background swirl texture (T5):** `NoiseTexture{seed, scale, turbulence_type, opacity}` and `CloudLayer{seed, scale, octaves, opacity, color_stops}` are

both registered. The seed is derived from `scene_hash + effect_id` (deterministic). Raster backend renders natively; SVG backend applies `embed-raster` fallback policy.

- **Drop shadows (T1, T5):** `DropShadow{offset_x, offset_y, blur_radius, color, opacity, fallback_policy}` is registered and active in the Showcase theme. PPTX maps to native `a:outerShdw`; SVG backend uses `feDropShadow` with determinism caveat at Tier 2.
- **Dashed-leader connector style (T1):** This is a *stroke style* extension on the existing Line Scene primitive (the annotation connector), not a new effect type. Add `dashed-leader-arrow` to the `theme.annotation.connector_style` vocabulary. No new Scene effect definition required.

0.33.6.4 D: Verdict

The Timeline IR can already represent the *data content* of all five targets without schema changes, with two exceptions flagged for Mark: the missing `metadata` field on Milestone (Gap IR-1) and the absence of a direct `color` hint on Activity and Milestone (Gap IR-2). All other requirements are render, layout, theme, or effect concerns that Barbara owns.

The current render-theme architecture *can* produce all five targets given the following additions, in priority order:

1. **Vertical central-spine layout family** (Render-1): unlocks T1, T3, T5 simultaneously; highest return on investment.
2. **Dark-executive theme** (Theme-1) and **showcase-dark child theme** (Theme-4): cover T1 and T5 dark aesthetics.
3. **Card-entry renderer** (Render-4) and **numbered-circle milestone shape** (Render-2): complete T5 and T2 respectively.
4. **Light-minimal-corporate** (Theme-2) and **colorful-infographic** (Theme-3) themes: complete T2 and T3.
5. **Dashed-leader connector style** (Render-5): completes T1 icon callout fidelity.
6. **Serpentine layout module** (Render-3): completes T4; recommended for a post-MVP release given its architectural novelty.

The effect infrastructure (glow, bloom, drop shadow, noise texture, cloud layer) is *already designed and sufficient*. No new Scene effect types are required. The fidelity-tier and backend model (§0.22.8) correctly classifies T4 and T5 as Tier 3 (Raster backend required) and T1–T3 as Tier 1–2 (SVG backend sufficient). The design is sound; the gap is in the layout family and theme inventory, not in the core architecture.



Figure 13: T3: “The AI Timeline” — dense vertical infographic, light background, (1967-2023) with key milestones and dates, timeline style.

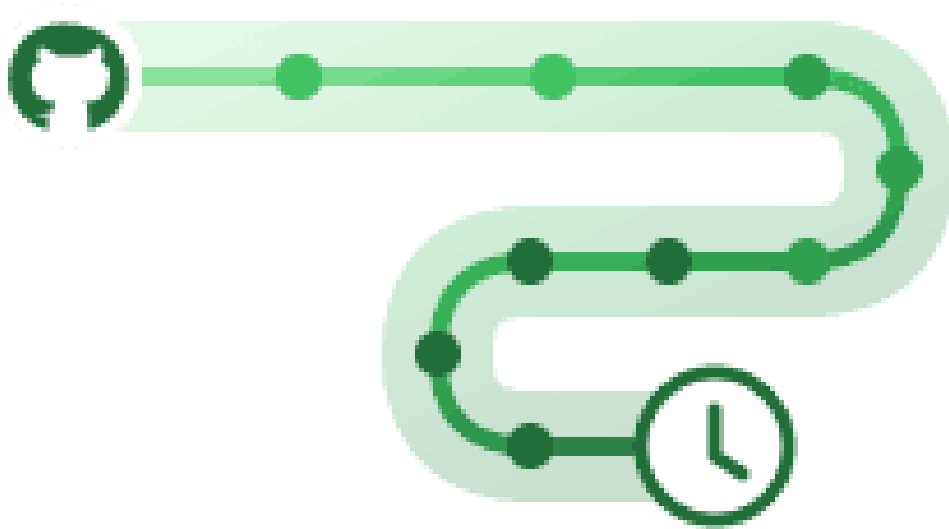


Figure 14: T4: Serpentine/winding S-curve path in green with a soft glow-bloom halo; milestone dots along the path; a repository-host icon at the start and a clock icon (in a ring) at the end. Tier-3 art-effect target. (Reference image provided by project owner.)



Figure 15: T5: Gitline web app — dark navy background with subtle swirl/radial tex-

Target	IR-Expressible?	Layout Family Supported?	Theme Available?	Effects Tier	/	Notes
T1 Vertical spine, dark	Partial	No	No	Tier (Drop-Shadow)	2	Vertical-spine layout gap; dark-executive theme gap; icon-badge leader-line render gap; per-node colour via <code>category_map</code> workaround
T2 Horizontal numbered	Yes	Partial	Partial	Tier (Crisp)	1	Data fully representable; numbered-circle node shape not in current milestone geometry; light-minimal-corporate theme variant needed
T3 Dense AI infographic	Yes	No	No	Tier (Crisp)	1	All data via <code>milestones[].description + category</code> ; vertical-spine layout gap; colorful-infographic theme gap
T4 Serpentine glow	Partial	No	Partial	Tier (Bloom, Raster)	3	Data representable; serpentine spine geometry is fundamental layout gap; Showcase theme covers glow/bloom effect
T5 Gitline cards, dark	Yes	No	Partial	Tier (Noise-Texture + Drop-Shadow)	3	<code>activity.url</code> covers CTA link; <code>annotation.callout</code> covers warning; vertical-spine + card-entry render gap; showcase-dark theme variant gap

Table 57: Coverage analysis: five targets versus current design (IR, layout, theme, effects)

Bibliography

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, 2 edition, 2006. ISBN 978-0-321-48681-3. The “Dragon Book”. Chapter 5 covers intermediate representations. Informs the argument that a stable, typed IR is preferable to direct rendering from raw input.
- [2] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi: 10.1145/182.358434. Establishes the 13 binary interval relations (before, meets, overlaps, starts, during, finishes, equals, and converses) that underpin OWL-Time and qualitative temporal reasoning. Validates open/ongoing interval semantics: an activity with no end date “starts” the document time range without requiring a concrete bound.
- [3] Anthony Fu and contributors. Slidev: Presentation slides for developers. <https://sli.dev/>, 2024. MIT-licensed Markdown-first slide tool with Mermaid integration. Relevant as a downstream consumer of Timeline Compiler SVG output.
- [4] Anthropic. Model context protocol — specification. <https://spec.modelcontextprotocol.io/>, 2024. Open protocol for LLM tool/resource access. Defines how agents invoke external tools and consume structured outputs. Timeline Compiler can expose MCP tools for plan-to-timeline generation.
- [5] Ulrik Brandes and Boris Köpf. Fast and simple horizontal coordinate assignment. In *Proceedings of the 9th International Symposium on Graph Drawing (GD 2001)*, volume 2265 of *Lecture Notes in Computer Science*, pages 31–44. Springer, 2001. doi: 10.1007/3-540-45848-4_3. $O(n)$ horizontal coordinate assignment for Sugiyama-style layouts. Constructs four candidate alignments (left-top, right-top, left-bottom, right-bottom) and takes the median. Guarantees at most two bends per edge. Used in Graphviz dot, dagre, and ELK Layered. Closes Phase 4 of the Sugiyama pipeline deterministically.
- [6] Christoph Buchheim, Michael Jünger, and Sebastian Leipert. Improving Walker’s algorithm to run in linear time. In *Proceedings of the 10th International Symposium on Graph Drawing (GD 2002)*, volume 2528 of *Lecture Notes in Computer Science*, pages 344–353. Springer, 2002. doi: 10.1007/3-540-36151-0_32. Corrects Walker (1990) to true $O(n)$ using a thread pointer that prevents quadratic re-traversal. State-of-the-art tree layout algorithm; the basis of D3’s `d3.tree()` layout.
- [7] Chris Pettitt and contributors. dagre: Directed graph layout for JavaScript. <https://github.com/dagrejs/dagre>, 2024. MIT license. Pure JavaScript implementation of Sugiyama layered layout for DAGs. Small footprint (50KB). Used by Mermaid for flowchart and state diagram layout. Deterministic for DAGs.
- [8] Ed. Desruisseaux, B. RFC 5545: Internet calendaring and scheduling core object specification (iCalendar). <https://datatracker.ietf.org/doc/html/rfc5545>, 2009. IETF Standards Track. Defines VEVENT with fields DTSTART, DTEND, SUMMARY, DESCRIPTION, CATEGORIES, STATUS (TENTATIVE, CONFIRMED, CANCELLED), and URL.

The field naming is the dominant vocabulary donor for Timeline IR. The ICS wire format is not git-friendly and has no swimlane or visual-status concept.

- [9] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models, 2024. URL <https://arxiv.org/abs/2411.15100>. Context-free grammar engine for LLM inference. Separates context-independent from context-dependent tokens to achieve up to $100\times$ speedup over naive per-step CFG execution. Integrated into vLLM and SGLang. Supports JSON Schema directly, making the diagram Domain IR schema directly actionable for grammar-constrained generation.
- [10] Tim Dwyer. WebCoLa: JavaScript constraint-based layout. <https://ialab.it.monash.edu/webcola/>, 2024. URL <https://github.com/tgdwyer/WebCola>. MIT license. JavaScript reimplement of libcola: stress majorization combined with VPSC separation constraints. Prevents node overlap, enforces alignment, and supports directed layout hints. Deterministic given fixed initial positions. Useful for swimlane and constraint-heavy layouts.
- [11] Peter Eades. A heuristic for graph drawing. In *Congressus Numerantium*, volume 42, pages 149–160, 1984. Early spring-embedder algorithm for force-directed graph layout. Simpler than Fruchterman-Reingold but foundational to the field.
- [12] Eclipse Foundation. ELK — eclipse layout kernel. <https://www.eclipse.org/elk/>, 2026. EPL-2.0 license. Java library (JavaScript port: elkjs) providing multiple layout algorithms: layered (Sugiyama), tree, force, radial, and more. Designed for determinism. Used in production diagramming tools including Eclipse-based IDEs.
- [13] ECMA International. ECMA-376: Office open XML file formats. <https://ecma-international.org/publications-and-standards/standards/ecma-376/>, 2026. Standard for Office Open XML, including DrawingML shapes and effects (a:glow, a:outerShdw, etc.) embedded in PPTX. Informs Timeline Compiler’s PPTX shape generation and styling fidelity.
- [14] Ellson, John and Gansner, Emden and Koutsofios, Lefteris and North, Stephen C. and Woodhull, Gordon. Graphviz — graph visualization software. <https://graphviz.org/>, 2024. Eclipse Public License. The classic graph visualization tool. DOT language for graph description; multiple layout engines (dot, neato, fdp, sfdp, circo, twopi). Excellent algorithms, dated visual output. Prior art for diagram-as-code.
- [15] Frappe Technologies. Frappe gantt: A modern, configurable gantt library for the web. <https://github.com/frappe/gantt>, 2024. MIT license. SVG-based interactive Gantt; used in ERPNext. No declarative text-format input; requires JavaScript task objects. No static file export built-in.
- [16] Thomas M. J. Fruchterman and Edward M. Reingold. Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991. doi: 10.1002/spe.4380211102. Influential force-directed algorithm treating edges as springs and nodes as repelling charges. $O(|V|\log E)$ iterations). Used in d3-force and many visualization libraries. Non-deterministic without careful seeding.
- [17] Emden R. Gansner, Eleftherios Koutsofios, Stephen C. North, and Kiem-Phong Vo. A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19(3):214–230, 1993. doi: 10.1109/32.221135. URL <https://www>.

- graphviz.org/documentation/TSE93.pdf. The “dot” paper. Introduces network simplex for layer assignment (Phase 2 of Sugiyama) and a crossing-minimisation improvement; the coordinate assignment in the current implementation was later superseded by Brandes & Köpf (2001). Canonical reference for the Graphviz dot algorithm.
- [18] Emden R. Gansner, Yehuda Koren, and Stephen C. North. Graph drawing by stress majorization. In *Proceedings of the 12th International Symposium on Graph Drawing (GD 2004)*, volume 3383 of *Lecture Notes in Computer Science*, pages 239–250. Springer, 2004. doi: 10.1007/978-3-540-31843-9_25. URL <https://graphviz.org/documentation/GKN04.pdf>. Stress majorization (SMACOF) for graph layout. Guaranteed monotone convergence; deterministic given fixed initial positions. Default algorithm in Graphviz neato. Safer than Kamada-Kawai for the determinism contract.
- [19] Michael R. Garey and David S. Johnson. Crossing number is NP-complete. *SIAM Journal on Algebraic and Discrete Methods*, 4(3):312–316, 1983. doi: 10.1137/0604033. Proves that minimizing edge crossings in graph drawings is NP-complete, motivating heuristic approaches in practical layout algorithms.
- [20] ggml-org contributors. GBNF guide: Grammar-based constrained generation in llama.cpp, 2024. URL <https://github.com/ggml-org/llama.cpp/blob/master/grammars/README.md>. GGML BNF grammar format for constraining LLM output in the llama.cpp runtime. Auto-converts JSON Schema to GBNF, enabling diagram IR schema to be used as a grammar constraint for local on-device LLM authoring.
- [21] GitHub, Inc. ProjectV2 — GitHub GraphQL API reference. <https://docs.github.com/en/graphql/reference/objects#projectv2>, 2024.
- [22] GitHub, Inc. About GitHub projects — GitHub documentation. <https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>, 2024. ProjectV2 GraphQL API fields include DATE, ITERATION, SINGLE_SELECT custom fields for roadmap bars. Roadmap view became GA in March 2023.
- [23] Google, Inc. Skia: An open source 2d graphics library. <https://skia.org>, 2026. Open source 2D graphics engine. Powers Chrome, ChromeOS, Android, Flutter, and many other products. Candidate rendering backend for deterministic, high-quality SVG and raster timeline output.
- [24] ISO/TC 154. ISO 8601-1:2019 — date and time — representations for information interchange. <https://www.iso.org/standard/70907.html>, 2019. The dominant standard for date/time strings in structured data. Timeline IR should support ISO-8601 dates natively.
- [25] ITU-T. Recommendation Z.120: Message sequence chart (MSC). <https://www.itu.int/rec/T-REC-Z.120>, 2011. ITU-T formal language for describing interactions between system components. Predates and influenced UML sequence diagrams. Defines instances (participants), messages, conditions, timers, and inline expressions (loop, alt, par, opt).
- [26] John MacFarlane. Pandoc: A universal document converter. <https://pandoc.org/>, 2024. MIT-licensed document converter. Pandoc filters could be used to embed Timeline Compiler output in converted documents.

- [27] Tomihisa Kamada and Satoru Kawai. An algorithm for drawing general undirected graphs. *Information Processing Letters*, 31(1):7–15, 1989. doi: 10.1016/0020-0190(89)90102-6. Energy-minimization algorithm connecting all node pairs by springs whose ideal length is proportional to graph-theoretic distance. $O(n^3)$ complexity; highly aesthetic for undirected networks. Graphviz `neato mode=KK`.
- [28] Knight Lab, Northwestern University. TimelineJS: Beautifully crafted timelines that are easy, and intuitive to use. <https://timeline.knightlab.com/>, 2024. Open source (GPL). Input via Google Sheets or JSON. Interactive web output; no native SVG/PDF/PPTX export. Storytelling-oriented, not presentation-oriented.
- [29] mark-when Contributors. Markwhen: An interactive text-to-timeline tool. <https://github.com/mark-when/markwhen>, 2024. MIT license (parser, view container, VS-Code extension). Web editor (markwhen.com) is proprietary. Markdown-inspired text format: `date: label` events grouped into `group:/section:` blocks. Swimlanes supported; no PPTX/PDF export; no stable machine-verifiable schema; no theme/render separation.
- [30] Mermaid Chart, Inc. Mermaid chart: AI-powered diagramming for teams. <https://www.mermaidchart.com/>, 2024. Commercial SaaS layer on top of open-source Mermaid. \$7.5M seed funding (March 2024). Demonstrates the open-core commercial model for diagram tooling.
- [31] Mermaid Contributors. Timeline diagram — mermaid documentation. <https://mermaid.js.org/syntax/timeline.html>, 2024. Official documentation for the timeline diagram type, which graduated from beta in 2023. Sections group events; time axis accepts year, quarter, or ISO-8601 strings.
- [32] Mermaid Contributors. Gantt diagrams — mermaid documentation. <https://mermaid.js.org/syntax/gantt.html>, 2024. Documents the gantt diagram type with `dateFormat`, `section`, and `task` syntax. Export quality is SVG/PNG only; no native PPTX or PDF.
- [33] Mermaid-js Contributors. Mermaid: Generation of diagrams like flowcharts or sequence diagrams from text in a similar manner as markdown. <https://github.com/mermaid-js/mermaid>, 2023. MIT license. Over 88,000 GitHub stars as of 2026. Raised \$7.5M seed round (Sequoia/M12) in March 2024.
- [34] Microsoft Corporation. Microsoft project XML data interchange file format. <https://learn.microsoft.com/en-us/office-project/xml-reference/introduction-to-the-microsoft-project-xml-data-interchange-file-format>, 2023. Official schema reference for Project XML export format.
- [35] Microsoft Corporation. Work items — Get — Azure DevOps REST API reference. <https://learn.microsoft.com/en-us/rest/api/azure/devops/wit/work-items/get?view=azure-devops-rest-7.1>, 2024. Work items export fields include `System.IterationPath`, `System.State`, `System.Title`, `Microsoft.VSTS.Scheduling.Effort`, etc. Quarter/sprint granularity captured via `IterationPath`.
- [36] Microsoft Research. Timeline storyteller: An expressive visual storytelling tool for presenting timelines. <https://github.com/Microsoft/timelinestoryteller>, 2019. Open-sourced 2019; MIT license. Research prototype accepting CSV or JSON array (`start`, `end`, `label`). No swimlanes, no status, no PPTX output, no active maintenance. Not adoptable.

- [37] Arpit Narechania, Arjun Srinivasan, and John Stasko. NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. In *Proceedings of the IEEE Conference on Visualization (IEEE VIS)*, volume 27, pages 369–379, 2021. doi: 10.1109/TVCG.2020.3030378. NL query $\rightarrow$ Vega-Lite JSON spec pipeline. The Vega-Lite grammar’s compact, schema-validated JSON proved tractable for programmatic generation. Key insight: targeting a well-specified intermediate grammar (not a general-purpose API) dramatically simplifies NL-to-visualization translation.
- [38] Object Management Group. Unified Modeling Language (UML) version 2.5.1. <https://www.omg.org/spec/UML/2.5.1>, 2017. OMG formal specification. Chapter 17 (Interactions) defines sequence diagrams: lifelines, messages, combined fragments (loop, alt, opt, par, critical, break), execution specifications, and interaction operands.
- [39] Observable, Inc. Observable plot: The javascript library for exploratory data visualization. <https://github.com/observablehq/plot>, 2024. ISC license. Concise JavaScript charting API inspired by Vega-Lite grammar. No native swimlane roadmap type; requires imperative mark composition. Not suitable for adoption or declarative text-format authoring.
- [40] Obsidian. Obsidian: A second brain, for you, forever. <https://obsidian.md/>, 2024. Popular Markdown knowledge-management tool with native Mermaid rendering. Potential embedding target for Timeline Compiler output.
- [41] OpenAI. Structured outputs — OpenAI API documentation. <https://platform.openai.com/docs/guides/structured-outputs>, 2024. JSON Schema-constrained generation from LLMs. A well-specified Timeline IR with JSON Schema enables agents to generate valid IR directly.
- [42] PlantUML Contributors. PlantUML: Open-source tool to draw UML diagrams, using a simple and human readable text description. <https://plantuml.com/>, 2024. Outputs PNG, SVG, ASCII art, LaTeX, PDF via GraphViz/Ditaa. Gantt support (@startgantt) is experimental as of 2024. GPL/LGPL/Commercial licenses.
- [43] PlantUML Contributors. Gantt diagram — PlantUML documentation. <https://plantuml.com/gantt-diagram>, 2024.
- [44] Plotly Technologies Inc. Plotly.js: Open source graphing library. <https://github.com/plotly/plotly.js>, 2024. MIT license. Supports Gantt/timeline charts via horizontal bar charts. High-quality SVG export. Requires programming; no declarative text format.
- [45] Prabhu Murthy and contributors. react-chrono: A modern timeline component for React. <https://github.com/prabhuignoto/react-chrono>, 2024. MIT license. JavaScript items array: title, cardTitle, cardDetailedText. Date fields are plain strings with no semantic granularity. Browser-only; no swimlanes; no static SVG/PDF/PPTX export. Not adoptable.
- [46] Jaideep Ray. The constraint tax: Measuring validity–correctness tradeoffs in structured outputs for small language models, 2026. Preprint, May 2026. Hard schema-decoding raises syntactic validity 61.5% $\rightarrow$ 100% but *lowers* semantic accuracy for sub-3B parameter models. Recommends “reason free, constrain late”: unconstrained chain-of-thought reasoning followed by constrained structured output. Informs the two-stage authoring path for on-device models.

- [47] Edward M. Reingold and John S. Tilford. Tidier drawings of trees. *IEEE Transactions on Software Engineering*, SE-7(2):223–228, 1981. doi: 10.1109/TSE.1981.234519. Linear-time algorithm for aesthetic tree layout. Produces compact, centered trees. Extended by Walker (1990) for n-ary trees. Standard tree layout in most graph visualization libraries.
- [48] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-Lite: A grammar of interactive graphics. In *IEEE Transactions on Visualization and Computer Graphics (Proc. InfoVis)*, volume 23, pages 341–350, 2017. doi: 10.1109/TVCG.2016.2599030. High-level declarative JSON grammar for interactive statistical graphics; BSD-3-Clause open source. Demonstrates how declarative IRs enable deterministic, composable rendering.
- [49] Scanny and contributors. python-pptx: Create and update PowerPoint files. <https://github.com/scanny/python-pptx>, 2024. MIT license. Python library for programmatic PPTX generation. Candidate for the PPTX output target of Timeline Compiler.
- [50] Schema.org Community Group. Event — schema.org. <https://schema.org/Event>, 2024. Schema.org type for events, defining properties name, startDate, endDate, description, url, organizer, and eventStatus. Open vocabulary maintained by W3C community. Not a visual-communication format, but confirms canonical web naming for timeline entity fields.
- [51] Simon Brown. Structurizr lite: C4 model architecture tooling. <https://github.com/structurizr/structurizr-lite>, 2024. Apache 2.0. Domain-specific to software architecture (C4 model); not a general-purpose timeline tool.
- [52] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981. doi: 10.1109/TSMC.1981.4308636. The foundational layered graph drawing algorithm. Four phases: cycle removal, layer assignment, crossing minimization, horizontal coordinate assignment. Basis for DAG layout in dagre, Graphviz dot, and ELK.
- [53] Roberto Tamassia. On embedding a graph in the grid with the minimum number of bends. *SIAM Journal on Computing*, 16(3):421–444, 1987. doi: 10.1137/0216030. Topology–Shape–Metrics (TSM) framework for orthogonal graph drawing. Bend minimisation formulated as minimum-cost network flow on the dual graph, solvable in polynomial time. Foundational for UML-style and circuit-schematic diagram layouts.
- [54] TaskJuggler Contributors. TaskJuggler: A free and open source project management tool. <https://taskjuggler.org/>, 2024. GPL v2. Plain-text .tjp DSL with project, task, resource, depends keywords. Full scheduling grammar: dependency resolution, resource levelling, critical path. Out of scope for visual-communication timelines; no vocabulary worth borrowing beyond what iCalendar and schema.org already provide.
- [55] Terrastruct, Inc. D2: A modern diagram scripting language that turns text to diagrams. <https://d2lang.com/>, 2024. MPL-2.0 license. Minimalist syntax, auto-layout. No native timeline/gantt type.

- [56] think-cell Software GmbH. think-cell: Intelligent charting and slide automation software for PowerPoint. <https://www.think-cell.com/>, 2024. Proprietary PowerPoint add-in. Approx. \$27.30/user/month (2026 pricing). Industry standard for McKinsey/BCG-style Gantt and waterfall charts. Not open source; not source-control friendly.
- [57] Yuan Tian, Weiwei Cui, Dazhen Deng, Xinjing Yi, Yurun Yang, Haidong Zhang, and Yingcai Wu. ChartGPT: Leveraging LLMs to generate charts from abstract natural language, 2023. URL <https://arxiv.org/abs/2311.01902>. Identifies LLM failure modes for chart spec generation: syntactically valid but semantically wrong specs; ambiguous field names; missing required fields. Mitigated by: grammar constraints (eliminate syntactic failures), validation layer (semantic failures), decomposed generation. Validates the compiler’s validate-before-render step.
- [58] visjs Contributors. vis-timeline: Create fully customizable, interactive timelines and 2d graphs with items and groups. <https://github.com/visjs/vis-timeline>, 2024. MIT license. Browser-only interactive timeline; no native static SVG/PDF export. Successor to the original vis.js timeline component.
- [59] W3C Design Tokens Community Group. Design tokens format module. <https://tr.designtokens.org/format/>, 2024. Community Group specification defining a JSON-based interchange format for design tokens—named values representing design decisions (colours, typography, spacing, etc.) that are shared across tools and platforms. Introduces the concepts of token type, alias, and composite token. Foundational reference for the general theme contract vocabulary.
- [60] W3C OWL Time Ontology Working Group. OWL-Time: Time ontology in OWL — W3C recommendation. <https://www.w3.org/TR/owl-time/>, 2022. W3C Recommendation (updated 2022). Defines `TemporalEntity`, `Instant`, `Interval`, and properties `hasBeginning`, `hasEnd`, `hasTemporalDuration`. Omitting `hasEnd` denotes an open/ongoing interval, validating the IR’s end: ongoing semantics. Implements Allen’s 13 interval relations.
- [61] II Walker, John Q. A node-positioning algorithm for general trees. *Software—Practice and Experience*, 20(7):685–705, 1990. doi: 10.1002/spe.4380200705. Extension of Reingold–Tilford to n-ary trees; claimed $O(n)$ but contains a bug causing $O(n^2)$ worst-case behaviour (corrected by Buchheim et al. 2002).
- [62] Bailin Wang, Zi Wang, Xuezhi Wang, Yuan Cao, Rif A. Saurous, and Yoon Kim. Grammar prompting for domain-specific language generation with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2305.19234>. Grammar constraints derived from domain-specific knowledge enable LLMs to generalize from few examples on complex DSL generation tasks. Key finding: a *minimal grammar fragment* for a specific instance is more reliable than the full schema. Directly motivates keeping domain IRs small.
- [63] Hadley Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, 2010. doi: 10.1198/jcgs.2009.07098. Formalizes the Grammar of Graphics as a layered system (data → aesthetics → geom → stat → coord → facet). Powers ggplot2.
- [64] Leland Wilkinson. *The Grammar of Graphics*. Springer, New York, 2 edition, 2005. ISBN 978-0-387-24544-7. Foundational theory decomposing statistical graph-

ics into data, aesthetics, geometry, statistics, scales, coordinates, and facets. Basis for ggplot2 and Vega-Lite.

- [65] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023. URL <https://arxiv.org/abs/2307.09702>. Reformulates LLM generation as transitions in a finite-state machine derived from a formal grammar or regex. Vocabulary index pre-computed at grammar-compilation time; per-step overhead is negligible. Open-source: Outlines library (<https://github.com/dottxt-ai/outlines>). Foundational result for grammar-constrained diagram IR generation.